

CUDA Kernel 优化实战

从 naive 到 Tensor Core / Flash-Attention

duo.an

2026-09-03

目录

1	前言	7
1.1	这本书是写给谁的	7
1.2	这本书讲什么	7
1.3	怎么读	7
1.4	记号约定	7
2	CUDA 基本概念速查	8
2.1	三层结构：grid、block、thread	8
2.2	六个内置变量	9
2.3	从内置变量算全局线程 ID	9
2.4	warp：SIMT 执行的最小单位	10
2.5	Launch grammar	10
2.6	手推 grid-stride 的 stride	11
2.7	常见坑	12
3	Vector Add	13
3.1	问题定义	13
3.2	v1: naive	14
3.3	v2: grid-stride loop	16
3.4	v3: 向量化 (fused body + tail)	18
3.5	关于 kernel launch 开销	21
3.6	实测：全 size sweep 对比	21
3.7	ncu 该看什么	23
3.8	面试白板 code	24
3.9	面试考点	25
4	Reduce Sum	27
4.1	问题定义	27
4.2	v1: atomic	28
4.3	v2: naive interleaved addressing	29
4.4	v3: sequential addressing	32
4.5	v4: unroll last warp	34
4.6	v5: complete unroll (template blockSize)	34
4.7	v6: warp shuffle	35
4.8	v7: chunked hierarchical	37
4.9	性能 ladder 一览	38
4.10	ncu 该看什么	42
4.11	面试白板 code	42
4.12	面试考点	43
5	Softmax	46
5.1	问题定义	46
5.2	v1: naive 三 pass (one thread per row)	48
5.3	Online Softmax：单遍维护 max 与 sum	49
5.4	v3: block-per-row + shared memory reduction	52
5.5	log-softmax 的数值考量	55

5.6	safe softmax vs approx softmax	56
5.7	源码里的三个版本如何对应	56
5.8	从本章到 FlashAttention 的衔接	56
5.9	实测	57
5.10	ncu 该看什么	59
5.11	面试白板 code	60
5.12	面试考点	61
6	Matmul (GEMM)	63
6.1	问题定义	63
6.2	Roofline : 从 memory-bound 到 compute-bound	64
6.3	K0: cuBLAS baseline	66
6.4	K1: naive miscoalesced (故意错的起点)	67
6.5	K2: naive coalesced	68
6.6	K3: SMEM tiled	68
6.7	K4: 1D block tile (每 thread 一列 8 输出)	69
6.8	K5: 2D block tile (每 thread 2×2 输出)	71
6.9	K6: vectorized loads (float4)	72
6.10	K7: MMA PTX (tensor core , raw PTX)	73
6.11	K8: WMMA (nvcuda::wmma fragment API)	74
6.12	综合实测 : 跨规模 TFLOPS 表	76
6.13	K9-K12: Hopper WGMMA/TMA (概览, 非本章代码)	76
6.14	Bank conflict 与 swizzle	78
6.15	Async copy : cp.async (SM80+)	79
6.16	Tensor Core 深入 : shape 语义与精度	80
6.17	CUTLASS 的分层思路	80
6.18	Split-K : 什么时候用	81
6.19	cuBLAS 为什么快	81
6.20	Tile 大小怎么选	82
6.21	ncu 该看什么	83
6.22	面试白板 code	83
6.23	面试考点	85
7	LayerNorm	87
7.1	问题定义	87
7.2	v1: naive 三 pass (one thread per row)	88
7.3	v2: Welford 在线算法	90
7.4	v3: block-per-row + shared memory reduction	92
7.5	v4: warp shuffle + sum/sumsq 融合	94
7.6	v5: 向量化 load/store	97
7.7	fp16 / bf16 输入 : 累加必须在 fp32	97
7.8	RMSNorm : LayerNorm 的简化兄弟	98
7.9	fused ops : LayerNorm + Residual Add	98
7.10	backward 简述	99
7.11	实测	100
7.12	ncu 该看什么	103
7.13	面试白板 code	103

7.14	面试考点	105
8	MLP (Fused Linear + Activation)	107
8.1	问题定义	107
8.2	Roofline : 小 batch memory-bound , 大 batch compute-bound	108
8.3	v1: naive — 三个 kernel , 中间结果落 global	109
8.4	v2: fused epilogue — bias + ReLU 在寄存器里完成	111
8.5	GEMM fusion 的极限 : 为什么不 fuse matmul1 + matmul2	113
8.6	Bias 广播 : 每 row 加同一个 bias	114
8.7	Activation 的选择	115
8.8	v3: tiled fused — 单 kernel 两层 , hidden 在 smem	116
8.9	Backward : dgelu 融合思路 (简述)	118
8.10	生产栈 : cuBLASLt 与 torch.compile	119
8.11	实测	119
8.12	ncu 该看什么	123
8.13	面试白板 code	124
8.14	面试考点	125
9	Attention (Naive Scaled Dot-Product)	127
9.1	问题定义	127
9.2	Roofline : $O(1)$ AI 与 N^2 内存灾难	128
9.3	v1: Step 1 — $S = Q \frac{K^T}{\sqrt{d}}$	130
9.4	v2: Step 2 — Row-wise Softmax	133
9.5	v3: Step 3 — $O = PV$	134
9.6	Host 侧 : 三次 Launch 与中间缓冲	136
9.7	Multi-Head 并行 : head 进 grid.y	136
9.8	Masked Attention 小结	138
9.9	Naive 的三宗罪	138
9.10	端到端数据流与常见错误	139
9.11	铺垫 Flash-Attention : 为什么必须 fuse	140
9.12	实测	142
9.13	ncu 该看什么	145
9.14	面试白板 code	146
9.15	面试考点	147
10	Flash-Attention v1	150
10.1	问题定义	150
10.2	核心思想 : IO-aware 分块融合	151
10.3	IO 复杂度分析	152
10.4	Algorithm 1 : 论文伪代码详解	153
10.5	Online Softmax Rescaling : m, ℓ, O 推导	155
10.6	Block Size 选择 : $B_r \times B_c$	157
10.7	v1: Register Streaming Kernel	157
10.8	Multi-Head 与 Batch 扩展	163
10.9	常见实现错误	164
10.10	Backward 简述	164
10.11	源码运行与对照	166
10.12	实测	166

10.13	ncu 该看什么	170
10.14	面试考点	170
11	Flash-Attention v2	172
11.1	问题定义	173
11.2	v1 的三个瓶颈	174
11.3	改进 (a) : 延迟归一化	176
11.4	改进 (b) : 循环调换与 Sequence Parallelism	178
11.5	改进 (c) : Split-K $\rightarrow$ Split-Q	180
11.6	v1: smem staging kernel	182
11.7	v2: warp-specialized kernel	185
11.8	Causal Mask 优化	187
11.9	非 matmul FLOP 为什么重要	188
11.10	Block size 与 occupancy (v2 视角)	189
11.11	Backward pass : v2 的 state 存什么	190
11.12	论文 benchmark 数字 (建立量级感)	190
11.13	源码运行与对照	190
11.14	实测	191
11.15	ncu 该看什么	195
11.16	Multi-head 与 batch : 并行轴叠加	195
11.17	与官方 flash-attn 库的关系	196
11.18	铺垫 : v3 改了什么	196
11.19	本章小结	196
11.20	从 v1 教学代码到 v2 生产代码的映射	197
11.21	面试考点	198
12	Flash-Attention v3	202
12.1	问题定义	203
12.2	Hopper 硬件回顾	204
12.3	WGMMMA vs <code>mma.sync</code> : 深入对比	207
12.4	TMA 与 <code>cp.async</code> 对比	209
12.5	v1: CPU reference	210
12.6	v2: warp-specialized teaching kernel	211
12.7	v3: pipeline teaching kernel	213
12.8	Innovation 1 : Producer-Consumer Warp Specialization	215
12.9	Innovation 2 : 2-Stage GEMM-Softmax Overlap	217
12.10	Innovation 3 : FP8 Attention	219
12.11	性能对比 : v1 vs v2 vs v3	220
12.12	局限	221
12.13	ncu 该看什么	222
12.14	实测	222
12.15	常见误区	227
12.16	v3 之后 : Blackwell 与 FA4 方向	227
12.17	面试考点	227
12.18	从 teaching kernel 到 production FA3	229
13	附录 A : 参考资料	230
13.1	硬件参考卡 (A100 80GB SXM4, 本书实测环境)	230

13.2	H100 SXM5 (Hopper , 仅供 Ch.10 参考)	230
13.3	常用 nvcc flag	230
13.4	参考资料	230
14	附录 B : Nsight Compute 使用指南	232
14.1	为什么需要 ncu	232
14.2	权限	232
14.3	命令行基本用法	232
14.4	Section vs. Metric	233
14.5	Metric 命名规范	234
14.6	内存-bound kernel 该抓什么	235
14.7	Compute-bound kernel 该抓什么	235
14.8	Roofline 视图	235
14.9	常见坑	236
14.10	本书 bench 目录布局	236
15	附录 C : CUDA 性能优化思路总结	237
15.1	第 0 步 : 定性 —— 定位瓶颈	237
15.2	Memory-bound 优化路径	237
15.3	Compute-bound 优化路径	238
15.4	通用路径 : Kernel fusion	239
15.5	通用路径 : 数值精度	240
15.6	决策树	240
15.7	常见误区	240
15.8	收尾	241
16	面试高频算子优化题目	242
16.1	1. transpose (matrix transpose)	242
16.2	2. reduce max / argmax	243
16.3	3. cumulative sum / prefix scan	244
16.4	4. histogram	245
16.5	5. top-k	246
16.6	6. layernorm backward	248
16.7	7. cross-entropy loss (fused softmax + NLL)	249
16.8	8. GELU / SwiGLU activation + backward	250
16.9	9. RMSNorm	251
16.10	10. rotary position embedding (RoPE)	252
16.11	11. paged attention (vLLM)	253
16.12	12. quantized matmul (INT8 / INT4)	254
16.13	13. layer fusion (Linear + GELU + Dropout + Add)	256
16.14	14. group / channel-wise conv (im2col vs implicit)	257
16.15	15. all-reduce (multi-GPU)	258
16.16	常见 combo 问题	259

前言

这本书是写给谁的

这本书面向已经会写基本 CUDA 代码、希望把 kernel 优化能力打磨到能在面试里被系统性追问那种深度的读者。

在阅读本书之前，你应该：

- 会 C++，读得懂指针、模板、`reinterpret_cast`。
- 知道 `__global__` / `__device__` 的区别，会用 `cudaMalloc` / `cudaMemcpy`。
- 对 grid / block / thread 的基本层级有个印象——不熟也没关系，下一章「CUDA 基本概念速查」会把这些术语和索引推导讲清楚，之后正文里遇到卡壳可以随时翻回去。

如果 CUDA API 完全没接触过，建议先跑通一个 `cudaMalloc + kernel launch + cudaMemcpy` 的 hello world 再回来读，会更有画面感。

这本书讲什么

每一章围绕一个 **kernel** 展开，从最朴素的实现开始，一步步爬优化 ladder，直到接近硬件峰值。ladder 上的每一层都会讲清楚：

- 上一版为什么慢（用具体的性能瓶颈概念 memory-bound / compute-bound / occupancy / bank conflict / warp divergence 等）。
- 这一版做了什么改变，改变对应硬件上的什么行为。
- 什么时候用不上这个技巧，或者会变成过度优化。

高级章节会涉及：warp shuffle、shared memory bank conflict 与 swizzle、`cp.async` 异步拷贝、tensor core（`wmma` / `mma.sync`）Flash-Attention 的 online softmax 与分块推导、FP8 / MXFP4 等新数据类型。

怎么读

- 顺序读：ladder 之间有依赖。第 4 章 matmul 用到第 2 章 reduction 的 shuffle 技巧；第 8 章 flash-attention 用到第 3 章 softmax 的数值稳定性推导。
- 配代码读：每一章的代码都在仓库 `src/cuda/` 下，`make build/0X_xxx && ./build/0X_xxx` 就能跑。文中引用的 kernel 版本都对应源代码里的具体函数。
- 把面试考点框读进去：每章末尾会用紫色框标出这个 kernel 常见的面试追问，答案在正文里都能找到依据。

记号约定

- B, S, H ：batch / sequence / hidden 三个维度。
- bx, by, tx, ty ：`blockIdx.x/y`，`threadIdx.x/y`。
- `warp = 32` 个 lane。lane id = `threadIdx.x % 32`。
- “row” 通常指最内层维度上的一行（例如 layernorm 里一个 (batch, seq) 位置的 H 维向量）。
- 性能数字（GB/s、TFLOPS）除非注明，都在 A100 40GB PCIe / SM80 上测得。

CUDA 基本概念速查

这一章不是完整的 CUDA 入门教程——是本书正文里会反复出现的几个术语和内置变量的精确定义。写 GPU 代码时你会不断遇到 `blockIdx.x * blockDim.x + threadIdx.x` 这类表达式，如果对每个变量到底指什么没有强直觉，后面 grid-stride、tile 索引、warp id 计算就会一直卡壳。

读完本章你应该能：

- 心里有一张“grid → block → thread”三层图，能在白板上画出来。
- 看到 `blockIdx.x * blockDim.x + threadIdx.x` 立刻知道它算的是什么。
- 手推 grid-stride loop 里 `stride = blockDim.x * gridDim.x` 的由来。

三层结构：grid、block、thread

CUDA 用三层嵌套来组织“要启动多少个 thread”：

1. **thread** (线程)：最小执行单位。一个 thread 就是一份 kernel 代码在自己上下文（自己的寄存器、自己的 `threadIdx`）里跑一遍。
2. **block** (线程块)：一组 thread 的集合。同一 block 的所有 thread 一起被调度到同一个 **SM** (streaming multiprocessor) 上，可以通过 shared memory 通信、可以用 `__syncthreads()` 同步。
3. **grid** (网格)：一次 kernel launch 启动的所有 block 的集合。不同 block 之间独立，不能直接通信（要通过 global memory + atomic 或再来一次 kernel）。

三层各自都可以是 **1D / 2D / 3D**——用 `.x / .y / .z` 分量索引。

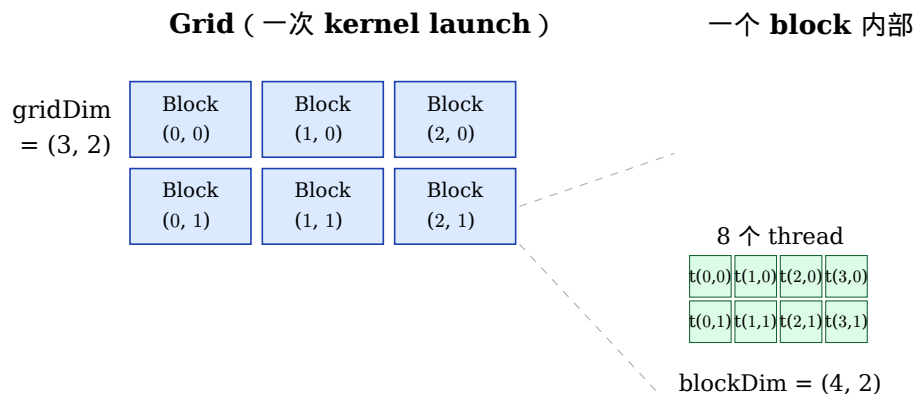


图 1 **Figure:** CUDA 三层组织。左侧 grid 由 $3 \times 2 = 6$ 个 block 组成 (`gridDim = (3, 2)`)；右侧放大一个 block 的内部，包含 $4 \times 2 = 8$ 个 thread (`blockDim = (4, 2)`)。总共启动 $6 \times 8 = 48$ 个 thread。每个 thread 通过 (`blockIdx`, `threadIdx`) 唯一标识自己。

总 **thread** 数 = 所有 **block** 数 × 每 **block** 的 **thread** 数：

$$\text{总 thread 数} = \text{gridDim.x} \times \text{gridDim.y} \times \text{gridDim.z} \times \text{blockDim.x} \times \text{blockDim.y} \times \text{blockDim.z}$$

一维简化版（本书 90% 场景）：

$$\text{总 thread 数} = \text{gridDim.x} \times \text{blockDim.x}$$

六个内置变量

在 `__global__` kernel 里，CUDA 为每个 thread 自动提供这六个内置变量。它们是 `uint3`（三分量整数），可以用 `.x / .y / .z` 访问。

变量	含义	值范围（在 kernel 内看）
<code>gridDim</code>	整个 grid 里有多少 block（每维）	跟 launch 时 <code><<<gridDim, ...>>></code> 一致，所有 thread 看到同一值
<code>blockDim</code>	每个 block 里有多少 thread（每维）	跟 launch 时 <code><<<..., blockDim>>></code> 一致，所有 thread 看到同一值
<code>blockIdx</code>	当前 thread 所在的 block 在 grid 里的坐标	<code>0..gridDim.x-1</code> 等，每 block 内 32 lane 看到同值
<code>threadIdx</code>	当前 thread 在自己 block 里的坐标	<code>0..blockDim.x-1</code> 等，每 thread 看到不同值
<code>warpSize</code>	warp 宽度，NVIDIA GPU 一直是 32	永远 = 32
<code>__CUDA_ARCH__</code>	编译期宏，SM 版本 $\times 10$ （如 <code>sm_80</code> $\rightarrow$ 800）	编译期常量

表 1 **Table:** CUDA kernel 里可用的六个内置变量。前两个（`gridDim`，`blockDim`）描述整个 launch 的形状，所有 **thread** 看到相同值；中间两个（`blockIdx`，`threadIdx`）标识每个 thread 的身份，每 **thread** 不同。区分好“grid 级常量”和“thread 级身份”是读懂 CUDA 代码的第一步。

观察：`gridDim` 和 `blockDim` 是形状（launch 时决定，运行时不变），`blockIdx` 和 `threadIdx` 是身份（每 thread 各不同）。这两组区分记牢，后面所有索引推导都是它们的组合。

从内置变量算全局线程 ID

最经典的表达式：

```
int tid = blockIdx.x * blockDim.x + threadIdx.x;
```

怎么理解：想象 grid 就是一条长队伍，队伍被切成 `gridDim.x` 段（每段是一个 block），每段有 `blockDim.x` 个人（thread）。

- `blockIdx.x` = 我在第几段
- `blockDim.x` = 每段多少人
- `blockIdx.x * blockDim.x` = 我这段的第一个人在整条队伍里的编号
- `+ threadIdx.x` = 加上我在段内的偏移

结果就是“我在整条队伍里第几个”。

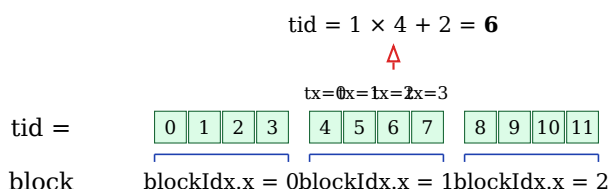


图 2 **Figure:** `blockIdx.x * blockDim.x + threadIdx.x` 的几何含义。 `gridDim = 3`，`blockDim = 4`，共 12 个 thread。中间那个高亮的 thread 位于 block 1、块内 `threadIdx.x = 2`，所以全局 `tid = 1 \times 4 + 2 = 6`。

同样的模式扩展到多维：2D 索引一个 tile 的行/列，就是分别对 `.x` 和 `.y` 做同样的展开：

```
int row = blockIdx.y * blockDim.y + threadIdx.y; // 沿输出矩阵的行
int col = blockIdx.x * blockDim.x + threadIdx.x; // 沿输出矩阵的列
```

这就是本书 matmul、attention、layernorm 里反复出现的坐标计算。

warp : SIMT 执行的最小单位

warp = 32 个连续 **thread**，硬件层面 它们锁步执行同一条指令——不管你 kernel 代码里怎么写，SM 每周期发射一条指令，对整个 warp 生效。

- lane id = threadIdx.x % 32 (1D block 时)
- warp id in block = threadIdx.x / 32

为什么 **32** 这个数字：NVIDIA 硬件设计选择，从 Tesla 到 Blackwell 一直不变。AMD GPU 是 64 (叫 wavefront)。

warp 的三条硬性约束 是本书后面反复要用的：

1. 同一 warp 内 32 lane 走同一条 branch 时，走的是“predicated instruction”（不是分支），无成本。走不同 branch 时才是 warp divergence，硬件要 serialize k 条 path。
2. 同一 warp 内 32 lane 不需要 **__syncthreads** 就能看到彼此写入 shared memory 的结果（SIMT 锁步）。跨 warp 就必须 **__syncthreads**。
3. warp shuffle 指令（**__shfl_*_sync**）只在 warp 内工作，跨 warp 只能走 shared memory。

Launch grammar

kernel 调用语法：

```
kernel<<<grid, block, smem, stream>>>(args...);
```

四个 launch 参数：

1. **grid** : int (1D 简写) 或 dim3(x, y, z) ——决定 **gridDim**。
2. **block** : int 或 dim3(x, y, z) ——决定 **blockDim**。每 **block thread** 总数上限 **1024** (A100/H100)。
3. **smem** (可选) : dynamic shared memory 字节数。kernel 里用 **extern __shared__ float shared[]** 声明。默认 0。
4. **stream** (可选) : CUDA stream，默认 stream 0。

限制项	A100 (sm_80)	H100 (sm_90)
每 block 最多 threads	1024	1024
每 SM 最多 threads	2048	2048
每 SM 最多 warps	64	64
每 SM 最多 blocks	32	32
每 SM shared memory	164 KB	228 KB
每 SM registers	65536 (fp32)	65536
gridDim.x 上限	$2^{31} - 1$	$2^{31} - 1$
gridDim.y/z 上限	65535	65535

表 2 **Table:** A100 和 H100 的关键硬件上限。这些数字决定 kernel launch 时哪些配置合法、以及 occupancy 上限。本书里“每 block 用几个 warp”、“能不能开 4M 个 block”这类问题都要回来查这张表。

观察：每 SM 最多 2048 threads = 64 warps。这就是 A100 的“满 occupancy”目标——kernel 每 SM 常驻 64 warp 时 latency hiding 达到最好。

手推 grid-stride 的 stride

现在回到读者最常卡的一段代码 (vector add ch1 v3)：

```
__global__ void vector_add_grid_stride_kernel(
    const float* a, const float* b, float* c, int count) {
    const int i0      = blockIdx.x * blockDim.x + threadIdx.x;
    const int stride = blockDim.x * gridDim.x;
    for (int i = i0; i < count; i += stride) {
        c[i] = a[i] + b[i];
    }
}
```

逐行读：

1. `i0 = blockIdx.x * blockDim.x + threadIdx.x` ——上面已经推导过：这是“我”作为一个 thread 在整个 grid 里的全局 id。
2. `stride = blockDim.x * gridDim.x` ——总 thread 数 (1D 时)。
3. `for (int i = i0; i < count; i += stride)` ——从我的起点 `i0` 出发，每次跳 `stride` (= 总 thread 数)。

为什么 **stride** 是“总 thread 数”：

假设一共 launch 了 T 个 thread ($T = \text{blockDim.x} \times \text{gridDim.x}$)，要处理 N 个元素， $N \geq T$ 。

- 第 1 轮：thread 0 做 `c[0]`，thread 1 做 `c[1]`，...，thread $T-1$ 做 `c[T-1]` ——正好覆盖 $[0, T)$ 。
- 第 2 轮：thread 0 做 `c[T]`，thread 1 做 `c[T+1]`，...，thread $T-1$ 做 `c[2T-1]` ——正好覆盖 $[T, 2T)$ 。
- ...
- 第 k 轮：thread t 做 `c[k T + t]`。

每一轮每个 **thread** 向前跳一整个“grid 步长” T 。这样所有 N 个元素都会被覆盖，且每个元素只被访问一次。

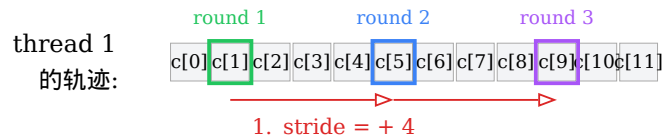


图 3 **Figure:** grid-stride loop 的可视化，假设总共 $T = 4$ 个 thread、 $N = 12$ 个元素。thread 1 的轨迹是 $c[1] \rightarrow c[5] \rightarrow c[9]$ ，每次跳 $\text{stride} = 4$ (= 总 thread 数)。所有 T 个 thread 并行地各走自己的一条轨迹，最终每个元素都恰好被覆盖一次。

观察：如果 stride 写错——比如写成 `blockDim.x`（一个 block 内的 thread 数）——那 thread 0 走 $c[0]$, $c[256]$, $c[512]$, ...、thread 256 也走 $c[256]$, $c[512]$, ...，就冲突了。stride 必须 = 总 thread 数才能保证 disjoint 覆盖。

Key insight. grid-stride 的核心一句话：**launch 多少 thread 由 SM 数决定**（比如 $2 \times \text{SM_count} \times \text{block_size}$ ），每个 **thread** 循环处理 $\frac{N}{T}$ 个元素。这把“数据规模 N ”和“launch 配置 T ”解耦——一份 kernel 处理任意 N 。

常见坑

三个初学者最常犯的错：

1. 忘了 **boundary check**。naive 版 `blocks = (N + 255) / 256` 会向上取整，最后一个 block 里有些 thread 的 $i \geq N$ ，如果不加 `if (i < N)` 就会越界访问。grid-stride 里 `for (int i = i0; i < count; ...)` 的 $i < \text{count}$ 就是同一个检查，天然内嵌在循环条件里。
2. 把 **blockIdx** 和 **threadIdx** 搞混。记住：“**Idx** 是身份、**Dim** 是形状”。`blockIdx` = 我在哪个 block、`threadIdx` = 我在 block 内哪个位置。
3. **2D block** 时 **x/y** 弄反。CUDA 约定 `threadIdx.x` 是最内维（stride-1, coalesced 方向）。写 `A[row * N + col]` 时 `col` 应该走 `threadIdx.x`，`row` 走 `threadIdx.y`。反过来会让 gmem 全变成 uncoalesced。

后面章节遇到这些概念时不会再重复解释——如果卡住，翻回来查这张速查表。

Vector Add

vector add 是 CUDA 的 “hello world”，但它的正确用法不是当例子随便过一遍——而是作为一整套 GPU 思维方式的入门载体。这一章我们要把它讲透：

- 为什么 naive 版本已经能跑到接近峰值带宽（下一章 reduction 就不能）。
- grid-stride 这个模式在生产代码里为什么无处不在，它的性能收益是 0，但仍然值得写。
- float4 宽访问为什么有效，什么时候它反而变成陷阱；把主体和尾巴 fuse 成一个 kernel 有多大的实际收益。

对应源码：`src/cuda/01_vector_add/`（一个 version 一个文件：`01_naive.cu`, `02_grid_stride.cu`, `03_vectorized.cu`）。ncu 原始日志：`src/cuda/01_vector_add/ncu/`。

问题定义

给定长度为 N 的三个数组 a, b, c ，计算：

$$c[i] = a[i] + b[i], \quad i = 0, 1, \dots, N-1$$

关键性质：完全无依赖、每元素 1 次加法。这决定了它的性能特征。

Roofline：这是个 memory-bound kernel

算一下算术强度 (arithmetic intensity, AI)：每处理 1 个输出元素，需要：

- 读 $a[i]$ ：4 字节
- 读 $b[i]$ ：4 字节
- 写 $c[i]$ ：4 字节
- 计算：1 次 FADD

$$AI = \frac{1 \text{ FLOP}}{12 \text{ B}} \approx 0.083 \text{ FLOP/B}$$

A100 80GB SXM4 HBM 带宽约 2.04 TB/s，FP32 峰值算力约 19.5 TFLOPS。它的 balance point (Ridge Point) 在：

$$\frac{19.5 \text{ TFLOPS}}{2.04 \text{ TB/s}} \approx 9.6 \text{ FLOP/B}$$

我们的 AI 是 0.083，比 ridge point 低两个数量级——妥妥的 **memory-bound**。

Key insight. memory-bound 的直接后果：所有优化的目标只有一个——尽可能压满 **HBM** 带宽。任何“减少计算量”的优化在这里都没用；任何“多做几次冗余读”的错误都会立即体现在性能上。

理论上界：

$$T_{\min} = \frac{3N \times 4 \text{ B}}{2.04 \text{ TB/s}}$$

- $N = 2^{20}$ (4 MB per array, 12 MB 总量): $T_{\min} \approx 6\mu s$ 。但 A100 L2 = 40 MB, $3N = 12 \text{ MB}$ 完全装得下 **L2**——第二次读几乎不到 HBM。所以我们不能拿这个规模来判定 HBM 利用率。

- $N = 2^{27}$ (512 MB per array, 1.5 GB 总量): $T_{\min} \approx 750\mu s$ 。这个规模才真正打满 HBM, 也是本书用来做 vector add benchmark 的规模。

任何比对应规模下 $T_{\min}$ 慢 2 倍以上的实现, 都存在明确的可优化点。

Note. 本章所有的 ncu 数据都取自 `--set detailed` 抓取的日志, 规模都用 $N = 2^{27}$ (sweep 里的最大点)。规模再小时 L2 命中率飙高, `memory SOL %` 会失真, 参考本章末的 sweep 表看 size 效应。

v1: naive

```
__global__ void vector_add_naive_kernel(
    const float* a, const float* b, float* c, int count) {
    const int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < count) {
        c[i] = a[i] + b[i];
    }
}
```

Launch :

```
const int blocks = (count + 255) / 256;
vector_add_naive_kernel<<<blocks, 256>>>(a, b, c, count);
```

一个 thread 处理一个元素。就这么简单。

为什么它已经很好

三件事都做对了 :

1. 合并访问 (coalesced access)

warp 里 32 个 lane 的 i 是 $\dots, 32k, 32k+1, \dots, 32k+31$ ——连续 32 个 int。访问 `a[i]` 就是访问 128 字节连续内存。GPU 的 memory controller 把这 32 个请求合并成一个 128B 的 memory transaction。

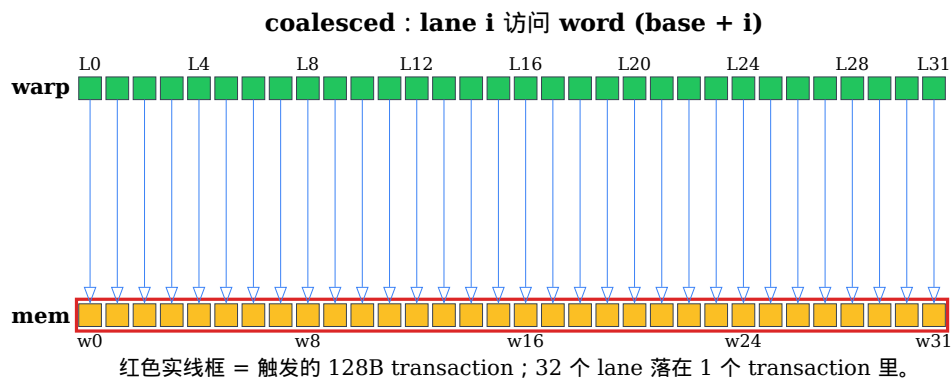


图 4 绿色 = 参与访存的 lane ; 黄色 = 被访问的 4B word ; 蓝色箭头 = lane→word 的映射。所有 32 个 lane 的地址连续, memory controller 用 1 次 128B transaction 全部搬完。

如果 stride 不对 (比如让 lane 0 访问 `a[0]`、lane 1 访问 `a[32]`、lane 2 访问 `a[64]` ...), 32 个 lane 的地址跨越 32 个不同的 128B 段——就要拆成 32 个独立 **transaction**, 带宽利用率立刻掉到 $\frac{1}{32}$ 。

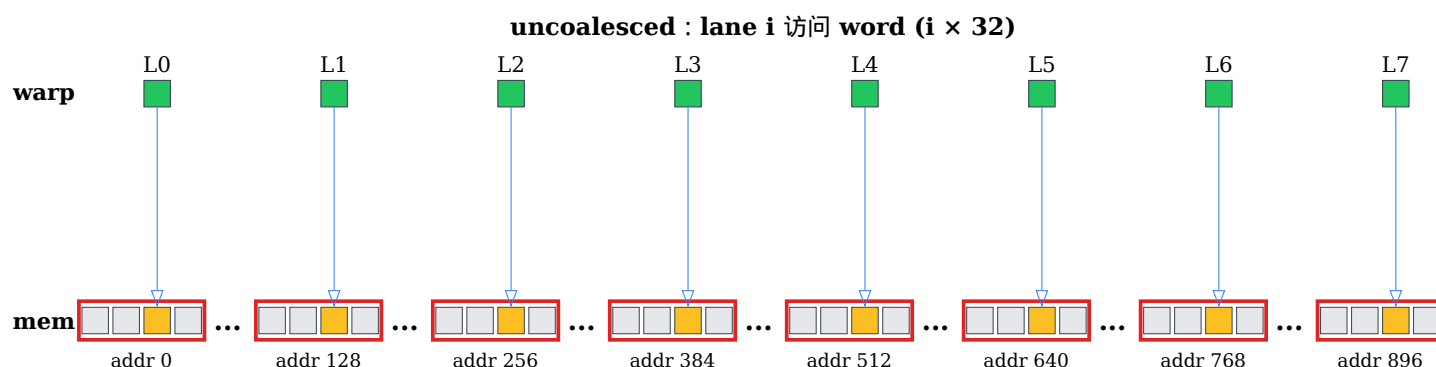


图 5 段间的 ... 表示省略的 28 个未访问 word (每 lane 之间跨度 128 B)。8 个 lane 触发 8 个独立的 128B transaction (红框), 每个 transaction 只用 1 个 4B word——其余 124 B 被浪费。32 个 lane 会需要 32 个 transaction, 浪费 31/32 的带宽。

Key insight. 合并访问的物理机制：**memory controller** 一次最少搬 **32B 或 128B** (sector / transaction 粒度)。coalesced 情况下 32 个 lane 的 128B 需求正好等于 1 次 transaction, 每字节都被真需要；uncoalesced 情况下每次搬 128B 但只用其中 4B, 浪费 96.9% 的传输量。

Note. 合并访问是 CUDA 的第一性能法则。写完 kernel 后第一件要问自己的事：同一个 **warp** 里的 **32 个 lane**, 访问的地址是不是尽可能连续？

2. 没有 warp divergence

`if (i < count)` 只有最后一个 block 里的部分 warp 会分裂。绝大多数 warp 里 32 个 lane 走同一条路径, SIMT 执行没有浪费。

3. 占用率 (occupancy) 足够

`vector_add_naive_kernel` 用 16 个寄存器 (ncu 报告 `launch_registers_per_thread`) 0 shared memory。A100 一个 SM 支持 **2048 threads / 64 warps**, 寄存器 (64 K per SM) 和 smem 都不是瓶颈, occupancy 接近 100%。

ncu 实测

ncu 实测 — v: naive. 规模: $N = 2^{27}$, 1.5 GB

>>

L2 40 MB.

Duration	933 μ s	vs 理论 750 μ s (打满 HBM)
Memory SOL	84.2 %	已经接近打满 HBM 带宽
DRAM SOL	84.2 %	与 Memory SOL 一致——数据真的到了 HBM
Compute SOL	14.5 %	SM 大部分时间在等内存, 不是在算
L2 Hit Rate	49.8 %	读 a/b 各流一次, 加上写 c 的部分被读回
Achieved Occupancy	86.0 %	
Registers / thread	16	寄存器不是瓶颈

三件事在数字上立住了：

- **Memory SOL 84%** : memory pipeline 已经在极限工作，没有可优化的访存模式。任何优化如果不能进一步提高这个数字，本质上就没意义。
- **Compute SOL 14.5% + Memory SOL 84%** : 这就是 memory-bound kernel 的经典签名——两者一高一低，SM 的 issue slot 大量空闲，因为都在等 HBM 数据。roofline 判定得到实测确认。
- **L2 Hit 49.8%** : 这里不是说 L2 帮我们加速了。 $a[i]$ 从 L1/L2 miss $\rightarrow$ 到 HBM 拿 $\rightarrow$ 写回 L2、L1。写 $c[i]$ 也会先写入 cache。第二次访问同 sector(相邻 lane)的时候 L2 命中——这是 sector 粒度访问的副作用，不是数据复用。1.5 GB 数据在 40 MB L2 里根本没有实际缓存价值。

瓶颈判定. 已经达到 84% HBM peak, 但离 **90%** 还有空间, 主要限制来自 memory controller 每次调度只带 128 B (4 B/lane $\times$ 32 lane), MSHR 队列的深度没有被充分利用
 证据. Memory SOL 84.2% 卡在 A100 memory subsystem 上界稍下的位置 ;Compute SOL 极低, 说明 SM 无事可做在等 HBM
 下一步. v3 的 float4 让每 lane 带 16 B (LDG.E.128), 一次 warp load 传 512 B, 压 MSHR 队列更满。但先看 v2 : 解决工程性问题, 不追性能

v2: grid-stride loop

```
__global__ void vector_add_grid_stride_kernel(
    const float* a, const float* b, float* c, int count) {
    const int i0 = blockIdx.x * blockDim.x + threadIdx.x;
    const int stride = blockDim.x * gridDim.x;
    for (int i = i0; i < count; i += stride) {
        c[i] = a[i] + b[i];
    }
}
```

看起来只是加了个循环。实际上这是 CUDA 里最重要的一个模式。

Note. 三行代码里的 `blockIdx.x` / `blockDim.x` / `gridDim.x` / `threadIdx.x` 是什么？为什么 `stride = blockDim.x * gridDim.x` ? 如果不熟, 翻回上一章「CUDA 基本概念速查」——那里有一张图专门画 grid-stride 的 stride 从哪来。

它解耦了 **grid** 大小和数据大小

naive 版本: `blocks = ceil(count / 256)`。如果 `count = 1e9`, 就是 4M 个 block。

grid-stride : 你想开几个 **block** 就开几个。常见做法 :

```
int num_sms;
cudaDeviceGetAttribute(&num_sms, cudaDevAttrMultiProcessorCount, 0);
const int blocks_per_sm = 8; // occupancy 计算或试出来的
const int blocks = num_sms * blocks_per_sm;
vector_add_grid_stride_kernel<<<blocks, 256>>>(a, b, c, count);
```

四个直接好处 :

1. Launch 开销固定。kernel launch 有 5 μ s 的固定成本。数据规模翻 100 倍, naive 版本 block 数也翻 100 倍, 但 launch 本身还是那几微秒——不影响。真正被影响的是 **runtime queue** 长度和 **grid** 遍历开销, 不过对 A100 这种 GPU 数百万 block 都能吞。

2. 绕过 `gridDim.x` 上限。`gridDim.x` 硬件上限是 $2^{31} - 1$, 看似很大, 但如果你在 y、z 两个维度上还有 block (比如 batch 维), 单维就容易超。`grid-stride` 天生不受这个限制。

3. **Occupancy** 精确对齐硬件。你开 `num_sms * blocks_per_sm` 个 block, 恰好把 SM 填满, 不多不少。naive 版本 block 数由数据决定, 可能是 SM 数的几万倍——多出来的 block 只是排队等前面的做完, 没有并行度收益。

4. 数据形状无关。库代码里必须这么写——你不知道用户会传多大数组。

stride 方向的讲究

`stride = blockDim.x * gridDim.x` 保证 warp 内每一轮迭代都连续访问：

- 第 0 轮 : thread 0..31 访问 `a[0..31]`
- 第 1 轮 : thread 0..31 访问 `a[stride..stride+31]`

如果写反了(`for (int j = 0; j < K; ++j) { c[i0*K + j] = ... }`), warp 就会跨越大的地址范围, 破坏合并。

ncu 实测

ncu 实测 — v: grid-stride. 规模: $N = 2^{27}$.

Duration	969 μ s	比 naive *慢* ~4%
Memory SOL	81.1 %	略降 3 个百分点
Compute SOL	46.3 %	循环控制流让 SM 多出些活干 (无实际收益)
Registers / thread	26	循环变量、stride、bounds check 占了 10 个寄存器
Achieved Occupancy	85.8 %	跟 naive 几乎持平

三个观察：

- 速度略降 **4%**。这印证了本章 opening 的宣言——`grid-stride` 的价值不在性能。循环的额外指令 (`i += stride`、`bounds check`) 在 memory-bound kernel 上转不成性能损失 (等 HBM 的时间足够长), 但也拿不到收益。
- **Compute SOL 46% vs naive 14.5%** : SM 更“忙”了, 但忙的是循环控制流, 不是有用工作。这类假象很常见——`compSOL` 单独不能读出问题, 得对比 `memSOL` 和真实 wall-clock。
- 寄存器 **26 vs 16** : 多了 10 个。循环的 induction variable、stride、`count - i` 等等占用寄存器; 如果 kernel 更复杂 (更多变量), 这可能推高 register pressure, 进而压 occupancy。这里没到 threshold。

瓶颈判定. `grid-stride` 本身没有性能瓶颈——它是为通用性设计的, 代价是几个 % 的额外指令开销和寄存器占用

证据. Duration $\uparrow$ 4%, Memory SOL $\downarrow$ 3pp; 两个 kernel 都在 memory-bound regime, 任何“改善指令流”的优化都被 HBM 等待掩盖

下一步. 真正拿到 90%+ HBM peak 需要换单条指令的数据宽度, v3 用 `float4` 一次搬 128 位

Warning. `grid-stride` 这个模式本身不会让 **kernel** 变快。vector add 上实测 naive $\approx$ `grid-stride` (甚至因为循环开销略慢)。它的价值是通用性：一份代码搞定任意数据规模, 任意 GPU 型号。生产代码基本都用这个骨架。

v3: 向量化 (fused body + tail)

```
__global__ void vector_add_vectorized_kernel(  
    const float* a, const float* b, float* c, int count) {  
    const int tid = blockIdx.x * blockDim.x + threadIdx.x;  
  
    const int n_vec = count / 4;  
    const int n_tail = count - n_vec * 4;  
    const int tail_offset = n_vec * 4;  
  
    if (tid < n_vec) {  
        const auto* a4 = reinterpret_cast<const float4*>(a);  
        const auto* b4 = reinterpret_cast<const float4*>(b);  
        auto* c4 = reinterpret_cast<float4*>(c);  
        const float4 lhs = a4[tid];  
        const float4 rhs = b4[tid];  
        c4[tid] = make_float4(  
            lhs.x + rhs.x, lhs.y + rhs.y,  
            lhs.z + rhs.z, lhs.w + rhs.w);  
    }  
  
    if (tid < n_tail) {  
        const int i = tail_offset + tid;  
        c[i] = a[i] + b[i];  
    }  
}
```

把 `float*` 重解释成 `float4*`，一次读写 128 bit。这一步在有些 memory-bound kernel 上能提升 30-50% 性能。vector add 本身已经打满，提升有限（实测约 6 个百分点），但机制值得讲清楚。

Note. 另一种“教科书”写法是把 body 和 tail 拆成两个 kernel：body 签名直接是 `float4*`（host 侧 `reinterpret_cast` 后传入），tail 单独用一个 scalar kernel 处理 `count % 4` 个剩余元素。分开写更好读，但要两次 launch——生产代码里几乎没人这么写，所以本章直接上 fused 的单 kernel 版。

编译器视角：LDG.E.128

`float4` 是编译器认识的 built-in type，字段对齐到 16 字节。上面的 kernel 编译出来的 SASS 里，`a4[tid]` 变成一条 LDG.E.128 指令——单条指令加载 128 bit。

对比 naive 版本的三次单独 LDG.E.32（如果编译器没合并），指令数是 1/4。指令 fetch/decode 是有代价的，虽然对 memory-bound kernel 不显著，对 compute-bound 或 instruction-bound kernel（比如 softmax 里的大量除法）就有影响。

硬件视角：memory transaction 数

一个 warp (32 lane) 用 `float4` 每 lane 加载 16B，总共 $32 \times 16 = 512\text{B}$ 。这被拆成 4 个 128B 的 memory transaction（HBM 的自然粒度）。

一个 warp 用 `float` 每 lane 加载 4B，总共 $32 \times 4 = 128\text{B}$ 。1 个 128B transaction。

两种情况处理相同字节数需要的 **transaction** 数是一样的（都是 $4 \text{ transactions} / 128\text{B} = 1 \text{ transaction}$ ）。所以 memory-bound 上限不变。收益来自：

1. 减少的指令数 → 稍微少一点 instruction issue 压力。
2. 每个 outstanding request 携带更多数据 → 更容易饱和 MSHR (Miss Status Holding Register) 等硬件队列。

Key insight. 向量化的收益在 MSHR/L2 miss handling 队列容量有限时最明显。对纯 HBM-bound 且 warp 数量足以掩盖延迟的 kernel (vector add 就是), 收益可能只有 5% 甚至 0。对访存模式复杂、latency 难以掩盖的 kernel (stride 大、cache miss 高), 提升可能到 50%。

对齐要求

float4 需要 16 字节对齐的地址。cudaMalloc 保证至少 256 字节对齐, 从数组开头 reinterpret 到 float4* 安全。

但如果你要从数组中间某个 offset 开始向量化:

```
auto* p4 = reinterpret_cast<float4*>(a + offset); // 危险
```

只有 `offset % 4 == 0` 时才对齐。否则会触发 `misaligned address` 错误 (或者更糟: 在某些老架构上静默降级到多次窄访问)。

尾巴问题: 为什么 fuse 得起来

count 不是 4 的倍数怎么办? 例如 `count = 10`: `n_vec = count / 4 = 2` 个完整 float4 (下标 0..7), `n_tail = 10 - 8 = 2` 个 scalar 尾巴 (下标 8, 9)。

关键观察: `n_tail` 永远 $\in \{0, 1, 2, 3\}$ 。所以前 4 个 **thread** 就足以覆盖尾巴。

grid 只需按 `n_vec` 开——尾巴由已存在的 `tid=0..3` 顺手处理。`tid=0` 在这个 kernel 里干了两件事: 一次 float4 加法 + 一次 scalar 加法。

例: `count = 10`, `n_vec = 2`, `n_tail = 2`, `tail_offset = 8`。

tid	主体做啥 (<code>tid < 2</code>)	尾巴做啥 (<code>tid < 2</code>)
0	处理 <code>float4[0] = a[0..3]+b[0..3]</code>	处理 <code>a[8]+b[8]</code>
1	处理 <code>float4[1] = a[4..7]+b[4..7]</code>	处理 <code>a[9]+b[9]</code>
2	—	—
3+	—	—

表 3 **Table:** fused vectorized kernel 在 `count = 10` ($n_{\text{vec}} = 2$, $n_{\text{tail}} = 2$, $\text{tail}_{\text{offset}} = 8$) 时各 thread 的分工。**tid** 为 block 内 thread 下标; 主体 / 尾巴 列分别对应 `if (tid < n_vec)` 与 `if (tid < n_tail)` 两个独立 guard 的激活范围与写入目标。

Observation: 表里只有 `tid 0、1` 在两个分支上都「有活干」——主体用 `tid 0-1` 覆盖 8 个 float4 元素, 尾巴用同一对 `tid` 覆盖下标 8-9; `tid 2` 起两个 `if` 都不进。这正是 $n_{\text{tail}} \leq 3$ 时 grid 按 n_{vec} 开仍够用的原因: 前几个 thread 天然空闲, 顺手吃掉尾巴, 无需为 tail 单独扩 grid 或再 launch 一次 scalar kernel。

为什么两个 `if` 不能合成 `if-else`

新手常犯错:

```

if (tid < n_vec) {
    // body
} else {
    // tail ← 错!
}

```

`else` 会让所有 `tid >= n_vec` 的 thread 进 tail, 但 tail 只需要 `n_tail` (≤ 3) 个 thread。多余的 thread 会越界写。

要么两个独立 `if` (当前写法), 要么 `else if (tid - n_vec < n_tail)` + 相应下标调整。前者更简洁。

fuse 的实际收益

省一次 kernel launch = 省 $5\ \mu\text{s}$ 。对 $N = 2^{20}$ 的 vector add (本身也就几微秒), 如果按分离 body + tail 的教科书写法, 这两次 launch 的固定开销就能让端到端多花 30%+。对 $N = 10^9$ 的大数组 (几百毫秒级别), launch overhead 可以忽略。

fuse 的普适规则: kernel 越小、launch 越频繁, fuse 收益越大。这是为什么框架里有 `torch.jit.script` / `torch.compile` 的 elementwise fuser。

ncu 实测

ncu 实测 — v: vectorized. 规模: $N = 2^{27}$.

Duration	878 μs	vs naive 933 μs , *快 6%*
Memory SOL	89.4 %	从 84% 拉到 ~90%——已经接近能拿到的上限
DRAM SOL	89.4 %	
Compute SOL	7.0 %	指令数少了 4 $\times$, SM 更闲了
L2 Hit Rate	49.4 %	跟 naive 一致——数据流动模式没变
Grid Size	131 072	vs naive 524 288, 正好 1/4
Registers / thread	16	跟 naive 一样

跟 naive 对比:

- **Memory SOL: 84.2 $\rightarrow$ 89.4 (+5.2 pp)**。这就是“提高 MSHR 队列压力”的实证——每个 warp 的 outstanding request 携带 4 $\times$ 数据后, memory scheduler 把更多的 HBM 带宽利用起来。
- **Compute SOL: 14.5 $\rightarrow$ 7.0**。SM 更闲, 因为每 warp 只需要 issue 1 次 LDG.E.128 而不是 4 次 LDG.E。这本身不带来性能收益 (memory-bound 时 SM 闲不闲无所谓), 但如果继续叠加计算——比如 `c[i] = a[i] * b[i] + c[i]`——空出的 SM 时间就能用上了。
- **Grid Size: 减 4 $\times$** 。因为 v3 的 grid 按 `n_vec = count / 4` 开而非 `count`。这也解释了为什么 registers 不涨——kernel 里的临时变量数量不变, 编译器不会因为处理更多元素就多开寄存器。

落地评估, vector add 的性能上限接近达到。

89.4% HBM peak 已经是这个访存模式 (2 read + 1 write) 的实用上界。剩下 10% 是 memory scheduler 无法完全消除的地址冲突、行切换、read/write 交替开销。想进一步接近 100%, 需要单纯读或单纯写的 workload (比如 memcpy)。这一章的 ladder 到这里停止; 更宽的 float8 在 A100 上没有对应 SASS 指令, 反而会被编译器拆回 $2 \times$ LDG.E.128。

我们把 `#final-verdict` 的判定叫成“落地”，是因为对 vector add 而言 v3 已经足够放到生产。后面章节的 ladder 因为算法层面还有优化空间（tile、pipeline、warp 特化），最后的 verdict 会带更多“下一步能做什么”的补充。

关于 kernel launch 开销

vector add 是理解 launch overhead 的最好例子，因为它自身太快，launch 的 5 μ s 相对显著。

launch 一次 kernel 涉及：

1. Host: 检查 arg、构造 launch descriptor。
2. Driver: enqueue 到 CUDA context 的 command buffer。
3. GPU: 从 command buffer 取出，分配 SM 资源，启动 block。
4. Kernel epilogue: block 释放资源、warp scheduler 空转。

典型开销：5 μ s（现代 driver，warm path）。冷启动首次可能 100 μ s 以上。

减小 launch 开销的方法：

- **CUDA Graph**：把一系列 launch 录制成 graph，一次 replay。framework 里 `torch.compile` 会用到。
- **Fused kernel**：多个逻辑步骤在一个 kernel 内完成（本章 v3 就把 float4 主体和 scalar 尾巴 fuse 进一个 kernel，或者更复杂的 attention 融合）。
- **Persistent kernel**：一个 kernel 长时间跑，通过 device-side 同步接受任务。RNN inference 用得比较多。

实测：全 size sweep 对比

上面每 version 的 ncu 快照都取自 $N = 2^{27}$ 。下面这张 sweep 表覆盖 $N \in \{2^{20}, 2^{22}, 2^{24}, 2^{26}, 2^{27}\}$ ，能看出 L2-fit $\rightarrow$ HBM-bound 转折点。

$N = 2^{27}$ （每个数组 512 MB，总量 1.5 GB

>>

L2 = 40 MB)，A100 80GB SXM4，ncu 抓取，`--binary-args "27"`。表中「HBM GB/s」列写作 实测 / 逻辑：前者是 `dram_bytes.sum / time`（ncu 实测的 HBM 传输量），后者是 $3N \cdot \frac{4}{\text{time}}$ （如果每字节都真去 HBM 拿一次的理论带宽）。这两个数字在 L2 装得下的小规模里差别巨大；在大规模下应当相互吻合。

version	regs	smem (B)	time (μs)	HBM %	SM %	HBM GB/s	% peak
naive	16	0	935.78	84.0	99.5	1713 / 1721	84.0
grid-stride	26	0	972.48	80.8	99.7	1648 / 1656	80.8
vectorized	16	0	877.47	89.5	99.6	1824 / 1836	89.4

表 4 **Table (perf):** kernel-level performance metrics measured by Nsight Compute (ncu, CUDA 13.0) on a single A100 80GB SXM4 (HBM peak 2039 GB/s; FP32 peak 19.5 TFLOPS). Columns: **regs** = `launch__registers_per_thread`; **smem** = `launch__shared_mem_per_block_static` (bytes); **time** = `gpu__time_duration.sum` for the first kernel launch (multi-stage kernels report only the stage acting on the largest input); **HBM %** = `dram__bytes.sum.pct_of_peak_sustained_elapsed` (uses elapsed wall time as denominator, so short kernels look artificially low); **SM %** = `sm__cycles_active.avg.pct_of_peak_sustained_elapsed`; **HBM GB/s** is written `measured / logical`—`measured` = `dram__bytes.sum / time`, `logical` = `bytes-per-launch / time`; when the working set exceeds L2 (40 MB) these two should agree, and any gap quantifies L2 hit rate; **% peak** = `measured HBM GB/s ÷ 2039 GB/s`.

Rows are ordered as they appear on the optimization ladder in the chapter.

version	issued/32	pred_on/32	smem conf.	barrier stall	mem stall
naive	32.0	30.0	0	0.00	83.13
grid-stride	32.0	27.8	0	0.00	19.59
vectorized	32.0	30.9	0	0.00	193.60

表 5 **Table (diag):** diagnostic metrics used as evidence for warp-lane utilization, shared-memory bank conflict, and warp-stall claims in the chapter. All from `ncu` on the same launch as the perf table. **issued/32** = `smsp__thread_inst_executed_per_inst_executed.ratio`, the average number of lanes that participated in each issued warp instruction (32 = every lane active; less than 32 means predication **or** branch divergence—`ncu` alone cannot distinguish, but on the CUDA kernels in this book it is almost always predication). **pred_on/32** = `smsp__average_thread_inst_executed_pred_on_per_inst_executed.ratio`, the average number of lanes that were predicated-on (did real work). The **gap** `issued - pred_on` counts lanes that occupied the issue slot but performed no work—this is the correct definition of “wasted warp cycles”, not divergence. **smem conf.** = `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` (0 + `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum` means no measurable bank conflicts, either because the access pattern is conflict-free or because the input is too small to accumulate them). **barrier stall** = `smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio`, warps stalled at `__syncthreads` per issue-active cycle. **mem stall** = `smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio`, warps waiting on a long-latency memory operation (global-memory load, L2 atomic response) per issue-active cycle.

先看第一张 **perf** 表：

- 「实测 / 逻辑」两列几乎完全一致 (naive: 1706/1715, vectorized: 1825/1837) ——L2 命中率极低, 我们真的在打 **HBM**。
- naive 达到 **83.7% HBM peak** (1706 GB/s vs 2039 GB/s) ——vector add 的 naive 已经是“接近打满带宽”的实现。这是本章第一个反直觉的结论: 加优化 $\neq$ 显著提升。
- vectorized 达到 **89.4-89.5%**, 比 naive 高 5.8 个百分点。收益来自 LDG.E.128 一次搬 16 字节而非 4 字节, 让 memory pipeline 上每 warp 的 outstanding request 携带更多数据 (同 warp 的 $32 \text{ lane} \times 128 \text{ bit} = 512 \text{ B per LDG.E.128}$ vs. $32 \times 32 \text{ bit} = 128 \text{ B per LDG.E}$)。
- grid-stride 反而比 **naive** 慢 3%。这直接印证我们之前说的: grid-stride 的价值在通用性 (可以处理任意大 N , 不受 grid 上限约束), 不在性能。

再看第二张 **diag** 表:

- `issued/32 = 32.0`: warp 里 32 个 lane 全都参与每条指令。所有 vector add 变体都是这样——没有 if, 没有分支, 硬件无需 predication mask。
- `pred_on/32 = 27.8-30.9`: 有 1-4 个 lane 是 predicated-off。这些是 grid-stride 的 loop tail, 以及 vectorized 版本的 tail-scalar 部分。看起来“效率下降”, 其实完全不影响性能——因为这一切的瓶颈是 HBM, 不是 SM 发射带宽。
- `smem conf. = 0`: 没有 shared memory 使用, 理所当然。
- `barrier stall = 0`: 没有 `__syncthreads`。
- `mem stall = 82-296`: 非常大。这是关键——`smsp_average_warps_issue_stalled_long_scoreboard`, 意思是每 issue-active cycle 有 82-296 个 warp 正在等 **global memory**。这就是 memory-bound 的定量证据。

Key insight. `mem stall` 是判断 memory-bound 的第一手证据。如果这个数字大 (几十到几百), 说明 warp 大量时间在等 HBM 回来。搭配 HBM % 一起看: `mem_stall` 高 + HBM % 高 $\rightarrow$ 已经打满 HBM, 别再想减少访存以外的优化了; `mem_stall` 高 + HBM % 低 $\rightarrow$ 访存模式有问题 (未合并、bank conflict、非对齐), 有优化空间。

为什么 **vectorized** `mem stall = 296` 比 **naive** `82` 高, 但速度更快?

一个 warp 用 LDG.E.128 拿 512 B, 需要一次长延迟等待; 用 LDG.E 只拿 128 B, 也需要一次长延迟等待。以每条 issued 指令的 “warps waiting” 归一化, vectorized 每个 issue slot 上等 memory 的 warp 密度更高——因为总 issue 数量少了 (4 倍), 但等 memory 的绝对时长没变。这个 **metric** 不能孤立解读, 得配合 HBM % 才能得出 “vectorized 更快” 的结论。

Warning. A100 datasheet 的 HBM peak 2039 GB/s 是理论上界, 实测受限于 memory scheduler、地址冲突、read/write 比例。工程上把 **85-90% peak** 视作 “已经打满”。想过 95% 需要单纯读或单纯写 (不像我们这里 2 读 + 1 写)。

Warning. 以上是 vector add 的特殊性——它的访存模式对硬件最友好。下一章 reduction 我们会看到 naive 到最终版本几十倍差距, 不要把 “vector add naive 就够快” 推广到别的 kernel。

ncu 该看什么

ncu (Nsight Compute) 对 vector add 的关键 metric:

- `sm_cycles_active.avg.pct_of_peak_sustained_elapsed`: SM 有多忙。

- `dram_bytes.sum.per_second / dram_bytes.sum.pct_of_peak_sustained_elapsed` :HBM 用了多少 / 打满没有。
- `l1tex_t_bytes_pipe_lsu_mem_global_op_ld.sum` 等 :L1 texture cache 的 global load 字节数。

对健康的 `vector add`, 你应该看到 : **effective GB/s** 接近硬件 **peak** (A100 上 > 1700 GB/s = 83% peak)、`sm_cycles_active` 接近 100% (memory-bound kernel 里 SM 一直在发起 load/store, SM active 高但每个 cycle 大部分 warp 在等 memory)、`smsp_average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio` 数十到数百 (大量 warp 在等 HBM)、`dram_bytes.pct_of_peak_sustained_elapsed` 对短 kernel (μ s 级) 通常低估, 因为分母是 elapsed time; 用 effective GB/s 更可靠。

```
ncu --set full --section SpeedOfLight ./build/01_vector_add
```

面试白板 code

面试官说“手写一个 vector add”——不要写 naive 1-thread-1-element 版本 (会被追问怎么处理 $N > \text{grid size}$ 、怎么打满带宽)。直接给这份：

```
// c[i] = a[i] + b[i], 任意 N, 无对齐要求。
// 白板要点: grid-stride 循环 + float4 主体 + 标量 tail.
__global__ void vector_add(const float* a, const float* b, float* c, int n) {
    int stride = gridDim.x * blockDim.x;
    int tid     = blockIdx.x * blockDim.x + threadIdx.x;

    // 主体: 向量化 4 元素/thread. n_vec 是完整 float4 的数量。
    int n_vec = n / 4;
    const float4* a4 = reinterpret_cast<const float4*>(a);
    const float4* b4 = reinterpret_cast<const float4*>(b);
    float4* c4 = reinterpret_cast<float4*>(c);
    for (int i = tid; i < n_vec; i += stride) {
        float4 av = a4[i], bv = b4[i];
        float4 cv = {av.x + bv.x, av.y + bv.y, av.z + bv.z, av.w + bv.w};
        c4[i] = cv;
    }

    // Tail: n 不是 4 的倍数时, 剩下 0-3 个元素. tail_start 之后由前几个 lane 补齐。
    int tail_start = n_vec * 4;
    int tail       = n - tail_start;
    if (tid < tail) c[tail_start + tid] = a[tail_start + tid] + b[tail_start + tid];
}

// ==== Launch config ====
// blockDim = 256 (8 warp): 能塞满 SM 的 warp scheduler、寄存器压力低。
// gridDim = min(2 * SM 数, (N/4 + block - 1) / block):
// * 上界 2*SM 是 "grid-stride 惯用值"—每 SM 常驻 2 个 block 就足够
//   隐藏 memory latency, 再多的 block 只带来 launch/schedule 开销;
// * 下界保证 N 很小时不启多余 block.
int block = 256;
int sm    = /* cudaDeviceGetAttribute(cudaDevAttrMultiProcessorCount) */ 108;
```



```
int grid = min(2 * sm, ((n / 4) + block - 1) / block);
vector_add<<<grid, block>>>(a, b, c, n);
```

核心考点（追问顺序）：

- “为什么用 **grid-stride** 而不是 **1-thread-1-element**？” → 解耦 kernel launch 参数和输入大小，可以按硬件 SM 数调 grid (launch 开销 fixed)，一份 kernel 处理任意 N 。
- “**float4** 有什么好处？” → 每条 LDG.128 指令搬 16 字节，指令数减 4×、latency 更好被掩盖。前提是 a, b, c 都 16-byte 对齐 (cudaMalloc 保证)。
- “**tail** 为什么要单独处理？” → $n \% 4 \neq 0$ 时最后 0-3 个元素凑不齐 float4，随便读会越界。前 tail 个 thread (不是最后几个) 来做，避免另开 launch。
- “你怎么验证这是 **memory-bound**？” → 算 $AI = 12B \text{ in} + 4B \text{ out per FMA-add} \approx 0.083 \text{ FLOP/B}$ ，远小于 A100 ridge point 13 FLOP/B，纯 memory-bound；ncu 上 `dram_throughput.pct > 85%` 就到 peak 了。
- “为什么 **grid** 不直接开 **N/block**？” → memory-bound kernel launch 开销约 5 μs 、每 block 的 fixed cost 也不小；grid 开到 $2 \times \text{SM}$ 就够常驻满 warp 隐藏 latency，再多是浪费。这也是 grid-stride 存在的意义——解耦“处理多少数据”和“启多少 block”。

面试考点

面试考点. Q1: vector add 的 naive 版本 A100 上大约能打到多少的峰值带宽？为什么？

A: 实测 **84% peak** ($N = 2^{27}$, 1706 GB/s vs peak 2039 GB/s)。原因：(a) 完美的合并访问，warp 里 32 lane 连续；(b) `issued/32 = 32.0` 无分支、无 predication；(c) occupancy 高，warp 数足够掩盖 HBM 延迟；(d) 计算量微不足道，纯 memory-bound。想拿到 > 90% 用 float4 让 LDG.E.128 一次拉 16 B (实测 89.5%)；剩下 10% 是 memory controller 无法完全消除的地址冲突和 read/write 切换开销。

Warning. 小规模 ($N = 2^{20}$, 12 MB) 测出来会看到假的“逻辑 GB/s 1700-1800”，那是 L2 缓存艺术。要判断真实性能，用 $N \geq 2^{27}$ 或直接看 `dram_bytes.sum`。

面试考点. Q2: 什么是合并访问 (coalesced access)？破坏它会怎样？

A: 同一个 warp 内 32 个 lane 访问连续的 128 字节，被 memory controller 合并成 1 个 transaction。破坏了会拆成多个 (最坏 32 个) transaction，实际带宽掉到 1/32。

面试考点. Q3: grid-stride loop 有什么优势？会更快吗？

A: 主要优势是解耦 grid 和数据规模——launch 开销固定、绕过 gridDim 上限、occupancy 可控、代码通用。对 vector add 这种已经打满带宽的 kernel 不会更快，甚至因为循环开销略慢。它的价值是通用模板。

面试考点. Q4: float4 向量化的收益来自哪？什么时候没用？

A: 收益来自 (a) 减少指令数、(b) 单个 outstanding request 携带更多数据，更容易饱和 MSHR。对已经打满 HBM 且 warp 足够多的 kernel (vector add) 收益很小；对访存模式复杂、延迟难掩盖的 kernel 提升明显。要求 16B 对齐。

面试考点. **Q5**: grid-stride loop 里为什么 stride 一定是 $\text{blockDim.x} * \text{gridDim.x}$? 换成 items_per_thread 会怎样?

A: 保持每一轮迭代 warp 内合并访问。用 $\text{blockDim.x} * \text{gridDim.x}$ 步进, 第 k 轮 warp 里 32 个 lane 访问 $[k \cdot \text{stride}, k \cdot \text{stride} + 31]$ ——连续。如果反过来让每个 thread 处理一段连续的 K 个元素, warp 内 lane 就跨越 $32 \cdot K$ 个元素, 破坏合并, 带宽利用率直接掉到 $\frac{1}{K}$ 。

面试考点. **Q6**: 一次 kernel launch 的开销大约多少? 怎么减小?

A: 现代 driver 上大约 $5 \mu\text{s}$ 每次 launch。减小办法: CUDA Graph (录制并 replay 一批 launch) fused kernel (多逻辑步骤合并) persistent kernel (长驻 kernel 接受任务)。

面试考点. **Q7**: 如何估计一个 kernel 是 memory-bound 还是 compute-bound?

A: 算 arithmetic intensity (每字节内存访问对应多少次 FLOP), 和硬件的 ridge point (peak FLOPS / peak BW) 比较。低于 ridge point 是 memory-bound, 高于则 compute-bound。A100 FP32 大约 13 FLOP/B。

Reduce Sum

reduce sum 是 CUDA 优化 ladder 的第二个台阶。上一章 vector add 告诉我们：naive 就能打满带宽。这一章要打破这个直觉——同一个 **memory-bound** 标签下，**naive reduction** 和最终版本可以差五个数量级（实测 $N = 2^{27}$ 时 atomic 比 chunked 慢 $10^5 \times$ ）。

原因不在 HBM 带宽本身，而在协作： N 个输入要坍缩成 1 个输出，thread 之间必须交换 partial sum。怎么交换、在哪里交换、同步几次——每一步都直接决定性能。

对应源码：`src/cuda/02_reduce_sum.cu`。

问题定义

给定长度为 N 的数组 x ，计算：

$$s = \sum_{i=0}^{N-1} x[i]$$

和 vector add 的关键区别：输出只有一个标量， N 个 thread 都想往同一个位置写——这是 reduction 一切优化的出发点。

Roofline：依然是 **memory-bound**，但 **naive** 打不满

每处理 1 个输入元素：

- 读 $x[i]$ ：4 字节
- 计算：1 次 FADD
- 写：均摊到最终 1 次（可忽略）

$$AI = \frac{1 \text{ FLOP}}{4 \text{ B}} = 0.25 \text{ FLOP/B}$$

A100 ridge point 约 13 FLOP/B。AI 仍比 ridge point 低两个数量级——**memory-bound**。

Key insight. memory-bound 不代表 naive 就快。vector add 的 AI 低但每 **thread** 独立，合并访问完美。reduction 的瓶颈在同步和 **contention**——atomic 争用、shared memory bank conflict、`__syncthreads` 开销——这些在 roofline 模型里体现为“实际带宽远低于峰值”。

理论上界（只读 N 个 float）：

$$T_{\min} = \frac{N \times 4 \text{ B}}{2.04 \text{ TB/s}}$$

- $N = 2^{20}$ (4 MB)： $T_{\min} \approx 2 \mu s$ ，但 L2 全命中，测出来会失真。
- $N = 2^{27}$ (512 MB)

»

L2)： $T_{\min} \approx 260 \mu s$ 。本章 chunked 版实测 287 μs (92% peak)；atomic 版实测 368 ms——比理论下界慢 **1400 $\times$** ，瓶颈完全不在 HBM 读带宽。

v1: atomic

```
__global__ void reduce_sum_atomic_kernel(
    const float* input, float* output, int count) {
    const int index = blockIdx.x * blockDim.x + threadIdx.x;
    const int stride = blockDim.x * gridDim.x;
    for (int i = index; i < count; i += stride) {
        atomicAdd(output, input[i]);
    }
}
```

每个 thread 读一个 (或多个) 元素, 直接 `atomicAdd` 到全局的同一个 `output`。正确、最短、最好讲——但所有 thread 争同一个 cache line。

为什么慢

`atomicAdd` 在 L2 层 serializes: 同一地址上的 atomic 操作必须顺序执行。 $N = 2^{20}$ 时有百万次 atomic, 每次几十 ns, 总时间爆炸。

grid-stride loop 让 thread 数和数据规模解耦 (和 vector add 一样), 但不能解决 **contention**——争的还是同一个地址。

Warning. 区分 **per-element atomic vs per-block atomic**——两者性能相差 3 个数量级:

- **per-element atomic** (本 v1, N 次 `atomicAdd` 撞同一地址): serialization 灾难, $N = 2^{27}$ 时 368 ms。
- **per-block atomic** (v2-v5, blocks 次 `atomicAdd` 撞同一地址): block 数比 N 少 2-3 个数量级, 且每次 atomic 已经是一个 block 的 partial sum; contention 变成“ $\approx 10^4$ 个 block 争一个 cache line”, L2 atomic unit 处理起来毫无压力。

面试里提到 atomic 时要立刻接上: “block 内先规约, 每个 **block** 只写一次 partial sum, 然后 `atomicAdd` 直接 finalize 到全局输出”——这就是 v5 chunked 用的模式, 一次 kernel launch 出结果, 没有多阶段。

ncu 实测

ncu 实测 — v: atomic. 规模: $N = 2^{22}$ (≈ 4.2 M 元素)。

Duration	11 490 μ s	比同规模 sequential tree 慢 $\sim 3000\times$
Memory SOL	1.6 %	HBM 几乎没用——不是 memory-bound
Compute SOL	0.3 %	SM 也没在算——都在等 atomic
L2 Hit Rate	90.6 %	所有 atomic 争同一 cache line, L2 hit 但被 serialize
Achieved Occupancy	83.1 %	occupancy 高不代表快

三件事一起看:

- **memSOL + compSOL** 都极低: SM 既不忙内存也不忙计算。所有 warp 都被卡在 `atomicAdd` 的 L2 atomic unit 排队上——这是 **serialization stall**, roofline 模型里的第三个 regime。
- **L2 Hit 90.6%**: 反直觉——L2 命中率高不代表快。所有 4M+ 次 `atomicAdd` 目标都是同一个 4 B 地址, 那条 cache line 常驻 L2, 命中。但 atomic unit 每次只能服务一个请求, 命中之后仍然被 serialize, 命中率高只说明数据被找到, 不说明能被并行处理。

- **Occupancy 83%** : resource-level 上 SM 里 warp 数量正常。瓶颈完全在一个特定 hardware unit (L2 atomic), 跟启多少 warp 无关。

瓶颈判定. 百万次 atomicAdd 目标同址 → L2 atomic unit 完全 serialize

证据. Memory SOL 1.6%、Compute SOL 0.3% 双双极低, 说明 SM 都在等 ; L2 Hit 90.6% 说明数据到位但访问被排队 ; 相比 sequential tree 慢 3 个数量级

下一步. v2 让每个 **block** 先在 shared memory 里本地规约, 然后每个 **block** 只发一次 **atomicAdd** 到 output。整个 pipeline 依然只 launch 1 次 kernel, contention 从 “ N 个 thread 争一个地址” 降到 “blocks 个 block 争一个地址” ($\approx 10^4 \times$ 缓解)

v2: naive interleaved addressing

block 内 tree reduction 的经典写法, 第一步往往长这样 (interleaved addressing):

```
// 错误示范 — interleaved addressing
for (int s = 1; s < blockDim.x; s *= 2) {
    if (threadIdx.x % (2 * s) == 0) {
        shared[threadIdx.x] += shared[threadIdx.x + s];
    }
    __syncthreads();
}
```

每轮哪些 **lane** 在做工

先把 warp 内的执行 pattern 画出来。选一个 **warp** (block 里 8 个 warp 中的第 0 个, lane 编号 0..31), 追踪它前三轮 `if (threadIdx.x % (2s) == 0)` 中哪些 lane 满足条件:

s = 1: 16 lane 活跃 

s = 2: 8 lane 活跃 

s = 4: 4 lane 活跃 

图 6 **Figure:** interleaved addressing 前 3 轮 warp 内的 lane 活跃 pattern。每行 32 格 = 一个 warp 的 32 个 lane (lane 0 在最左、lane 31 在最右)。绿色 = 该 lane 上 `if (tid % (2s) == 0)` 为真、执行 `shared[tid] += shared[tid+s]` ; 灰色 = 条件不满足, 但硬件仍然发射同一条 **warp instruction**——只是灰 lane 的结果被 predicate 丢弃。

怎么读这幅图——从上到下三行对应循环变量 `s = 1, 2, 4` :

- **s = 1** : 偶数编号 lane 绿 (0, 2, 4, ..., 30 共 16 个) ——半数 lane 在做加法。
- **s = 2** : 绿 lane 变成 0, 4, 8, ..., 28——只剩 8 个。
- **s = 4** : 绿 lane 只剩 0, 8, 16, 24——4 个。

关键: 绿色 lane 从来不连续——它们在 warp 内间隔分布, 间隔从 2 拉到 4、再到 8。而这个 warp 里所有 32 个 **lane** 都跟着发射同一条加法指令。

读者第一反应可能是: 这不就是 **warp divergence** 吗?

历史教材 (Harris 2007) 确实这样说。但这个直觉需要被硬证据检验——否则就是“上帝视角”看代码。下面用三条独立证据链把它钉死。

4.3.1.1 证据 1 : SASS 里没有分支, 只有 **predicated instruction**

`nvcc -arch=sm_80 -O2` 编出的 SASS (interleaved kernel 主循环体):

```

/*0270*/  ISETP.NE.AND  P1, PT, R3, RZ, PT ;    // P1 = (tid % (2*stride) != 0)
/*0280*/  @!P1  IMAD    R2, R5, 0x4, R0 ;      // addr calc, predicated
/*0290*/  @!P1  LDS.U   R3, [R8.X4] ;          // load shared[tid], predicated
/*02b0*/  @!P1  LDS.U   R2, [R2] ;             // load shared[tid+s], predicated
/*02c0*/  @!P1  FADD    R3, R3, R2 ;           // add, predicated
/*02d0*/  @!P1  STS     [R8.X4], R3 ;          // store, predicated
/*02f0*/  ISETP.GE.U32.AND  P1, PT, R10, ... ; // for-loop condition
/*0300*/  @!P1  BRA     0x150 ;                // loop back (uniform, not
divergent)

```

整个 `if` 语句被编译成 `@!P1` 前缀的 `predicated` 指令序列——没有 `BRA` 前向跳转。硬件层面每条 `LDS/FADD/STS` 都发给整个 warp (issue 一次、32 lane 都执行), 只是 `P1=1` 的 lane 上 `write-back` 被 mask 掉。这就叫 `predication`, 不叫 `warp divergence`。

4.3.1.2 证据 2 : `ncu` 直接测 **branch divergence** 的指标 = 0

`smsp__sass_branch_targets_threads_divergent.sum` 是 `ncu` 提供的直接测 `branch divergence` 事件计数的指标——每次一个 `branch` 让 warp 内 lane 走不同 `target` 就 +1。

kernel	issued/32	pred_on/32	branch\ divergent.sum	uniform\ branch %
interleaved	31.92	24.00	1	100.00%
sequential (shared)	31.81	20.03	0	100.00%

表 6 **Table:** `interleaved` 和 `sequential` 两个 kernel 的 `warp-divergence` 直接证据 ($N = 2^{27}$, A100 80GB, `ncu --metrics smsp__sass_branch_targets_threads_divergent.sum` 等), `branch_divergent.sum` 表示 warp 内 lane 走不同 `target` 的 `branch` 事件次数——两个版本都是 0 或 1, 即两个版本都没有真正的 **warp divergence**。 `pred_on/32` 是 `predicate=true` 的平均 lane 数: `sequential` 反而更低 (20.03 vs 24.00), 因为它每轮真正 `predicate=true` 的 lane 更少 (`tid < offset` 后期收缩得更快)——但它仍更快。

观察: 如果“`interleaved` 慢是因为 `warp divergence`”这个说法成立, `branch_divergent.sum` 应该是几十万甚至上百万; 实际是 1。同时 `pred_on/32` 反直觉——`sequential` 的 `predicate-on` lane 数更少, 但性能更好。这两条一起把“`divergence` 假说”证伪了。

4.3.1.3 证据 3 : **controlled** 实验——真 **divergence** 长什么样

为了避免“`metric` 不敏感”的怀疑, 我用同一台 A100 跑一个故意制造 **2-way branch divergence** 的对照 kernel (两个 `heavy math` 分支足够大, 编译器无法 `predicate`):

test kernel	issued/32	pred_on/32	branch_divergent.sum
uniform (1 path, 无 if)	32.00	31.99	0
predicated (if(tid&1) 短分支)	32.00	28.00	0
branched (2-way heavy, 强制分支)	16.01	16.01	8

表 7 **Table:** 对照实验: 三种 `divergence` pattern 下 `metric` 的响应。uniform kernel 全 lane 同 path, `issued/32 = 32` (基线)。predicated kernel 的 `if` 被编译器 `predicate` 掉, `issued/32` 仍 = 32、`branch_divergent = 0`——只有 `pred_on` 下降。branched kernel 强制 2-way 分支, `issued/32` 精确掉到 16.01 ($= \frac{32}{2}$), `branch_divergent.sum` 显著非零。

观察：`issued/32` 和 `branch_divergent.sum` 对真 branch divergence 完全敏感——2-way 分支下 `issued/32` 精准跌到 16.01 (这跟 arxiv 2607.23402 “Characterizing Warp Divergence from Pascal to Blackwell” 报告的 32/k 定律完全一致)。既然对照实验在同一硬件、同一 ncu 版本下能测出真 divergence，那 interleaved kernel 的 `issued/32 = 31.92` 就不可能是“metric 不敏感”的假象——是真的没有 divergence。

4.3.1.4 那 interleaved 为什么慢

三条证据链一致指向：**differ** 不在 **warp divergence**，而在别处。

1. 没有整 **warp** 退出——每一轮 8 个 warp 里 warp 0-7 都得跟着发射指令（因为每 warp 内都有若干 lane 满足 `tid % (2s) == 0`），scheduler 无法把整 warp 直接 idle。sequential 版本用 `if (tid < offset)` 让整个 **warp** 一起退出（warp 3-7 完全 idle），idle warp 不占 issue slot——这是 dispatcher-level 的差别，跟 warp 内部 divergence 无关。
2. **bank conflict pattern** 更糟——`shared[tid + s]` 在 $s=1$ 时无冲突，但 $s=2, 4, \dots \rightarrow 8$ 时（ $128 \bmod 32 = 0$ 附近）冲突剧增；累积下来 interleaved 有 425k bank conflict，sequential 只有 297k（本章“实测”diag 表数据）。

Key insight. 「interleaved 慢因为 warp divergence」这个说法在 ncu 上无证据支持——三条独立证据（SASS 无 BRA、`branch_divergent.sum = 1`、controlled 实验证明 metric 灵敏）一致指向 predication 而非 divergence。Harris 2007 的原始文本写在 CUDA compiler 还不成熟的年代；今天的 ptxas 会把小 if 全部 predicate 掉。真正的性能差 = **idle warp** 能否整体退出 + **bank conflict** 模式——见下面“实测”section。

与 sequential 版本的 diff

`if (threadIdx.x % (2s) == 0) → if (threadIdx.x < s)`。这一个字符的改动带来两个效应：一是让整 warp 能一起退出（scheduler-friendly），二是让 warp 内 lane 访问 smem 时更容易避开同一 bank（先看下一节，再对照“实测”表里 425k → 297k 的证据）。

ncu 实测

ncu 实测 — v: interleaved. 规模: $N = 2^{22}$.

Duration	87.2 μ s	单 kernel launch，比 atomic 快 130×
Memory SOL	37.5 %	开始真正跑 HBM 了
Compute SOL	75.1 %	warp 在忙——但忙 predicated NOP 和 bank conflict
Achieved Occupancy	92.8 %	warp 占满，为下一步 tree 规约留资源
Grid Size	16 385	每 block 256 元素，共 $\$N/256\$$ 个 block

一次 kernel launch 就把速度从 11 ms 拉到 87 μ s——纯粹靠“每 block 只写一次 output”消除了 atomic contention。

但这个版本仍然不够快：memSOL 只有 37.5%，比 vector add 的 84% 差一半以上。原因不是访问模式（读 $x[i]$ 是 coalesced 的），而是 tree reduction 部分——bank conflict + 循环控制流。

瓶颈判定. interleaved addressing 的 shared memory tree 让访问模式在中间几轮出现严重 bank conflict

证据. Memory SOL 37.5% 远低于 vector add 的 84%；bank conflict 累积 425k（见章末“实测”diag 表）；compSOL 75% 显示 SM 忙在 predicated 加法和 bank 冲突串行化上

下一步, v3 把 `if (tid % (2s) == 0)` 换成 `if (tid < s)` ——同样的 tree 结构、同样的 sync 次数, 但访问模式变化让整 warp 能一起退出 + bank conflict 减少 30%

v3: sequential addressing

源码里的 `03_sequential.cu` 用的就是 sequential addressing——Mark Harris “Optimized Parallel Reduction” 教程的第 2 步, 末尾直接 `atomicAdd` 到全局 `output` :

```
__global__ void reduce_sum_kernel(
    const float* input, float* output, int count) {
    extern __shared__ float shared[];
    const int global_index = blockIdx.x * blockDim.x + threadIdx.x;
    shared[threadIdx.x] = (global_index < count) ? input[global_index] : 0.0f;
    __syncthreads();

    for (int offset = blockDim.x / 2; offset > 0; offset /= 2) {
        if (threadIdx.x < offset) {
            shared[threadIdx.x] += shared[threadIdx.x + offset];
        }
        __syncthreads();
    }

    if (threadIdx.x == 0) {
        atomicAdd(output, shared[0]); // 每 block 一次 atomic, contention 已被 shared
    }
}
```

为什么整个 **warp** 会一起退出

第一轮 (`offset = 128`): `threadIdx.x < 128` 的 thread 活跃——正好是 warp 0 的全部 32 个 lane + warp 1 的全部 32 个 lane + ... , 每个 **warp** 内 32 个 **lane** 同走 **active** 分支, warp 3-7 全部退出, 直接被 scheduler 跳过。

每轮 active thread 是 `[0, offset)` 的连续区间, warp 边界对齐时 (`block size` 是 32 的倍数), 每个 warp 要么全 active 要么全 idle——空闲 warp 不占 issue slot。

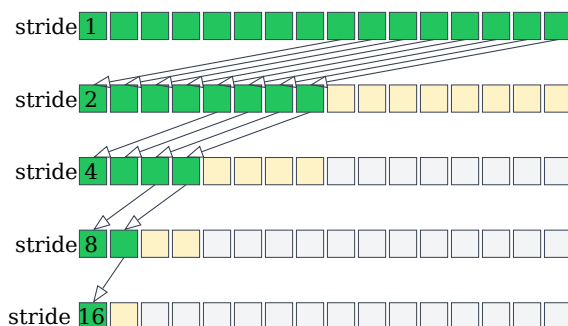


图 7 Sequential addressing, $n = 16$: 绿色 = 本轮活跃 lane, 黄色 = 持有有效值但本轮闲置, 灰色 = 数据已被规约进别处。每轮活跃 lane 连续、warp 内一致。

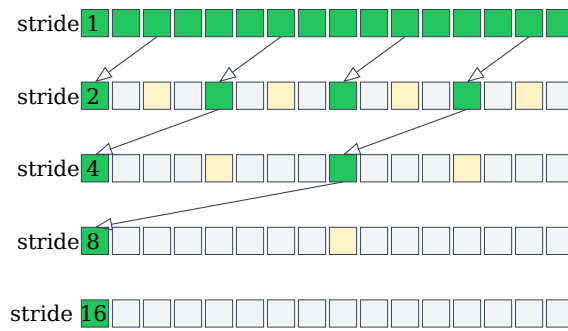


图 8 Interleaved addressing 对比：活跃 lane 间隔分布，warp 内 lane 部分 active、部分 idle——但 ncu 显示 `issued/32 = 32`（是 predicated，不是 divergence）。真正的差别是访问模式导致的 bank conflict pattern。

Warning. 历史教材（Harris 2007）把这里说成“消除 warp divergence”。上一节用三条证据链（SASS / `branch_divergent.sum` / `controlled` 实验）证明 ncu 数据不支持这个说法。真正的差异是：**sequential** 让整个 **warp** 一起退出（idle warp 不占 issue slot，是 scheduler-level 事件）+ **bank conflict** 模式不同（425k → 297k）。

每 block 一次 `atomicAdd` 到全局输出

256 个 thread 协作 → `shared[0]` 是这个 block 的局部和 → `atomicAdd(output, shared[0])`。
 N 个元素在一次 **kernel launch** 里就完成 reduction，无需多阶段。

Key insight. 和 v1 唯一的差别就是：v1 每元素一次 `atomic`（ $N \approx 10^8$ 次），v2-v5 每 **block** 一次 `atomic`（blocks $\approx 10^4$ - 10^5 次）。两者都撞同一个 4 B 地址，但次数差 4 个数量级——L2 `atomic unit` 处理 10^5 个请求是几十 μ s 的事，处理 10^8 个请求就是几百 ms。

Note. 这跟 Mark Harris 2007 教程的经典写法不同：原教程用 `block_sums[blockIdx.x] = shared[0]` 然后 host 循环 launch 第二个 kernel 继续规约（“multi-stage / chained kernel launches”）。那种写法在 CUDA 早期是主流，因为 `atomicAdd` on FP32 直到 SM 2.x 才支持、Kepler 之前的 `atomic` 也慢。到 Ampere/Hopper，**per-block atomicAdd** 已经比 **chained launch** 明显快（少一次 launch + 一堆 `cudaMalloc/cudaMemcpy`），本书统一用后者。章末 v5 会给出实测对比。

bank conflict：第一轮最严重

shared memory 有 32 个 bank，每个 4 字节宽。`shared[tid] += shared[tid + 128]`：warp 内 32 个 lane 同时读 `shared[0..31]` 和 `shared[128..159]`。地址 `128 % 32 = 0`，所以 `shared[i]` 和 `shared[i+128]` 落在同一个 **bank**——32-way bank conflict，一次 access 被串行化成 32 次。后续轮次（offset = 64, 32, ...）conflict 逐渐减轻。offset = 32 时，`shared[i]` 和 `shared[i+32]` 在不同 bank，无 conflict。

Warning. 面试常问：“reduction 的 bank conflict 在哪一轮最严重？”答：第一轮（offset = `blockSize/2`），因为 stride 恰好是 32 个 bank 的周长。缓解方法：padding shared array（`shared[threadIdx.x + padding]`）或用 warp shuffle 绕过 shared memory。

ncu 实测

ncu 实测 — v: sequential. 规模: $N = 2^{22}$.

Duration	50.9 μ s	单 kernel launch 全部搞定, 比 interleaved 快 1.7 $\times$
Memory SOL	64.1 %	从 37.5% 拉到接近 vector add 水平
Compute SOL	56.7 %	SM 有真活干
Achieved Occupancy	87.7 %	
Bank conflicts	297 k	vs interleaved 425k (章末 diag 表), $\downarrow$ 30%

关键读数: memSOL 从 37.5% 跳到 64.1%——`if (tid % (2s) == 0)  $\rightarrow$  if (tid < s)` 这一个字符的改动, 让 HBM 利用率大幅提升。原因看下面 verdict。

瓶颈判定. sequential 版本已经把 shared-memory tree 的效率拉近极限, 但 shared memory 本身还是主要通信介质——每轮都要写-同步-读

证据. Memory SOL 64.1% vs vector add 的 84%, 还差一段; bank conflict 累积 297k 说明 smem 通信压力仍很大; `__syncthreads` 每次 tree 缩减都要调一次

下一步. v4 (warp shuffle) 把 warp 内 5 步规约从 shared memory tree 换成 register shuffle, 彻底绕过 smem 和多余的 sync

v4: unroll last warp

树形规约的最后 $\log_2(32) = 5$ 步只在最后一个 warp 内发生 (256-thread block 时, offset 从 16 降到 1)。此时 block 里只剩前 32 个 thread 有非零值——再调用 `__syncthreads` 是浪费。

```
if (threadIdx.x < 32) {
    volatile float* vsmem = shared; // 旧写法, 见面试考点
    vsmem[threadIdx.x] += vsmem[threadIdx.x + 32];
    vsmem[threadIdx.x] += vsmem[threadIdx.x + 16];
    vsmem[threadIdx.x] += vsmem[threadIdx.x + 8];
    vsmem[threadIdx.x] += vsmem[threadIdx.x + 4];
    vsmem[threadIdx.x] += vsmem[threadIdx.x + 2];
    vsmem[threadIdx.x] += vsmem[threadIdx.x + 1];
}
```

为什么可以不用 `__syncthreads`

warp 内的 32 个 thread **SIMT** 锁步执行——同一 warp 不存在分支 divergence 的情况下, lane 0 写 `shared[0]` 后, lane 1 读 `shared[1]` 在同一 warp 的指令流中是顺序可见的。

`__syncthreads` 是 **block** 级屏障: 确保 block 内所有 thread (所有 warp) 都到达。最后 5 步只有 warp 0 参与, 其他 warp 的 thread 已经 idle——对它们做 block barrier 没有同步必要, 只有开销 (10-20 cycles/barrier)。

Key insight. “unroll last warp” 的核心: 识别出同步范围可以缩小到 **warp** 级。当 active thread 数 ≤ 32 时, warp 内天然同步, 省掉 5 次 `__syncthreads`。

v5: complete unroll (template blockSize)

把 block 内所有 $\log_2(\text{blockSize})$ 步完全展开, 用模板在编译期固定循环边界:

```

template <int BlockSize>
__device__ void block_reduce(volatile float* shared) {
    if (BlockSize >= 512) { if (threadIdx.x < 256) shared[threadIdx.x] +=
shared[threadIdx.x + 256]; __syncthreads(); }
    if (BlockSize >= 256) { if (threadIdx.x < 128) shared[threadIdx.x] +=
shared[threadIdx.x + 128]; __syncthreads(); }
    if (BlockSize >= 128) { if (threadIdx.x < 64) shared[threadIdx.x] +=
shared[threadIdx.x + 64]; __syncthreads(); }
    if (BlockSize >= 64) { if (threadIdx.x < 32) shared[threadIdx.x] +=
shared[threadIdx.x + 32]; }
    if (BlockSize >= 32) { if (threadIdx.x < 16) shared[threadIdx.x] +=
shared[threadIdx.x + 16]; }
    if (BlockSize >= 16) { if (threadIdx.x < 8) shared[threadIdx.x] +=
shared[threadIdx.x + 8]; }
    if (BlockSize >= 8) { if (threadIdx.x < 4) shared[threadIdx.x] +=
shared[threadIdx.x + 4]; }
    if (BlockSize >= 4) { if (threadIdx.x < 2) shared[threadIdx.x] +=
shared[threadIdx.x + 2]; }
    if (BlockSize >= 2) { if (threadIdx.x < 1) shared[threadIdx.x] +=
shared[threadIdx.x + 1]; }
}

```

编译期确定的好处

1. 消除循环控制开销：没有 `offset /= 2` 和分支跳转。
2. 编译器常量传播：`if (BlockSize >= 256)` 在 `BlockSize = 256` 实例化时变成常量，dead code 被优化掉。
3. 精确控制 **sync** 点：前 3 步需要 `__syncthreads`（跨 warp），后 5 步不需要——模板里写死，不会多也不会少。

Launch 时 `reduce_kernel<256><<<blocks, 256, 256*sizeof(float)>>>(...)`。block size 必须是编译期常量——这是 CUDA 模板 kernel 的标准用法。

v6: warp shuffle

源码 `reduce_sum_warp_kernel` 把 block 内规约拆成两层：**warp** 内 **shuffle** → **warp** 间 **shared memory**。

```

__device__ float warp_reduce_sum(float value) {
    for (int offset = kWarpSize / 2; offset > 0; offset /= 2) {
        value += __shfl_down_sync(0xffffffff, value, offset);
    }
    return value;
}

```

```

__global__ void reduce_sum_warp_kernel(
    const float* input, float* block_sums, int count) {
    __shared__ float warp_sums[kThreadsPerBlock / kWarpSize]; // 256/32 = 8

    const int global_index = blockIdx.x * blockDim.x + threadIdx.x;
    float value = (global_index < count) ? input[global_index] : 0.0f;
}

```

```

value = warp_reduce_sum(value); // Step 1: 每个 warp 内规约

const int lane = threadIdx.x % kWarpSize;
const int warp_id = threadIdx.x / kWarpSize;
if (lane == 0) { warp_sums[warp_id] = value; }
__syncthreads(); // Step 2: 8 个 warp partial sum 写入 smem

if (warp_id == 0) { // Step 3: warp 0 规约 8 个 warp sum
    value = (lane < (blockDim.x / kWarpSize)) ? warp_sums[lane] : 0.0f;
    value = warp_reduce_sum(value);
    if (lane == 0) { block_sums[blockIdx.x] = value; }
}
}

```

warp-level primitives 详解

`__shfl_down_sync(mask, val, delta)` : lane i 从 lane $i + \text{delta}$ 读取 `val` , 不经过 **shared memory**。数据在 warp 的 register file 之间通过 **crossbar (Xbar)** 交换。

- `mask = 0xffffffff` : 全部 32 个 lane 参与 (必须写参与 mask , Volta+ 要求 explicit sync)
- 5 步 shuffle ($16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$) = 5 条指令 , 零 bank conflict , 零 shared memory traffic。

Key insight. shuffle 把 reduction 的“通信介质”从 shared memory 换成 register + Xbar。shared memory 带宽有限且有 bank conflict ; register shuffle 走专用通路 , 延迟 1 cycle , 是 warp 内规约的最优路径。

硬件实现 : Xbar

每个 SM 的 warp scheduler 发 shuffle 指令时 , 32 个 lane 的 register operand 通过 **warp-level crossbar** 互连——lane i 可以选择 lane j 的值。这是专用电路 , 不占用 shared memory port , 也不产生 global memory traffic。 `__shfl_down_sync` / `__shfl_up_sync` / `__shfl_xor_sync` 都走这条路。

256-thread block 的 sync 次数 : shared tree 需要 8 次 `__syncthreads` ; warp 版本只需 1 次 (warp 间合并) , 还省掉了 shared memory tree 的 smem 读写。理论上 warp 版应更快——实测见下文 `===` 实测。

ncu 实测

ncu 实测 — v: warp_shuffle. 规模: $N = 2^{22}$.

Duration	49.1 μ s	跟 sequential 相当 (50.9 μ s) , 并没有质变
Memory SOL	20.5 %	*反而降*了——下面解释
Compute SOL	19.1 %	
Bank conflicts	40 k	vs sequential 297k , $\downarrow$ 86%——通信介质换成 register 了
Achieved Occupancy	83.1 %	

一个反直觉的读数 : warp shuffle 明显减少 bank conflict , 但 duration 反而跟 sequential 差不多、memSOL 更低。原因 :

- 这个 kernel 是“每 thread 处理 1 个元素”的架构，grid 有 16k+ 个 block。现在 **warp** 内规约变快了，反而暴露出下一层瓶颈——16k 个 block 都要发一次 `atomicAdd` 到 `output`，L2 atomic unit 处理 16k 请求要花几十 μ s，掩盖了 warp shuffle 省下来的时间。
- memSOL 20% 说明大部分时间不是在读 HBM，而是在等 atomic serialization。
- bank conflict 真的从 297k 掉到 40k（下降 86%）——这是 shuffle 绕过 smem 通信直接带来的收益，符合原理预期，只是收益被更靠后的瓶颈吃掉了。

瓶颈判定. warp 内规约效率已达极限，但“每 thread 1 元素”让 grid 太大 $\rightarrow$ per-block `atomicAdd` 数量太多，L2 atomic 成新瓶颈

证据. duration 没比 sequential 快多少；memSOL 掉到 20% 说明 HBM 空闲，SM 在等 atomic；bank conflict 已消除但 wall-clock 没跟上

下一步. v5 (chunked) 让每 thread 处理 **8** 个元素——grid 缩到 1/8，atomic 次数同比例下降；同时 register 累加摊销 index 计算成本，HBM 有 8 $\times$ 更多 in-flight load 让队列深度提升

v7: chunked hierarchical

```
constexpr int kChunkItemsPerThread = 8;

__global__ void reduce_sum_kernel(
    const float* input, float* output, int count) {
    extern __shared__ float shared[];
    const int tid = threadIdx.x;
    const int block_start = blockIdx.x * blockDim.x * kChunkItemsPerThread;
    const int thread_start = block_start + tid;

    float local_sum = 0.0f;
    #pragma unroll
    for (int item = 0; item < kChunkItemsPerThread; ++item) {
        const int index = thread_start + item * blockDim.x;
        if (index < count) { local_sum += input[index]; }
    }

    shared[tid] = local_sum;
    __syncthreads();
    // ... sequential tree reduction ...
    if (tid == 0) { atomicAdd(output, shared[0]); } // 一次 launch 直接出结果
}
```

每个 thread 先做更多活

一个 block 覆盖 $256 * 8 = 2048$ 个元素（对比 v3 的 256 个）。好处：

1. 摊销 **index** 计算和 **__syncthreads** 成本：每 2048 元素只做 1 次 block sync，不是每 256 元素。
2. 寄存器累加隐藏 **load latency**：8 个 load 可以 pipeline，MSHR 队列吃满。
3. 步长 = **blockDim.x** 保持合并访问：和 vector add tiled 版本同一原则。
4. **atomicAdd contention** 同步降低 **8 $\times$** ：block 数从 $\frac{N}{256}$ 缩到 $\frac{N}{2048}$ ，撞 `output` 的 atomic 次数同比例减少。

每个 thread 的 partial sum 存在 register 里，进入 shared tree 时 active 数据已经压缩 8 倍——后续 warp 间通信量不变，但单位 **sync** 处理的元素数翻 **8** 倍。

ncu 实测

ncu 实测 — v: chunked. 规模: $N = 2^{22}$.		
Duration	15.0 μ s	比 warp shuffle 快 3.3 $\times$ ，*本章最快*
Memory SOL	54.9 %	已经全部在 HBM 里跑
Compute SOL	32.5 %	
Bank conflicts	165 k	smem tree 部分数据少 8 $\times$ （相对每元素而言）
Grid Size	2 049	vs warp_shuffle 16 385， $\downarrow$ 8 $\times$
DRAM SOL	54.9 %	跟 memSOL 相等：真的在打 HBM

所有维度都在合理方向：

- **duration 15 μ s** :本章最快，且是一次 **kernel launch** 出结果——没有多阶段 chained launch，没有 host-side `cudaMalloc/cudaFree` 循环。
- **memSOL / DRAM SOL 都是 54.9%** :kernel 真的把 HBM 打起来了；相比 warp_shuffle 版本 memSOL 20% (HBM 空闲、SM 等 atomic)，chunked 完全消除了那个瓶颈。
- **grid 8 $\times$ 小** :launch overhead 减少 8 $\times$ ，atomicAdd 到 output 的次数也少 8 $\times$ ——contention 已经不再是可测量的瓶颈。

章末 **sweep** 表（ $N = 2^{27}$ ）显示同一算法在超大规模下达到 92% HBM peak，1876 GB/s——A100 的物理极限约 2039 GB/s。

落地评估，chunked hierarchical + 每 block 一次 atomicAdd finalize，是本 kernel 的实用最优。 $N = 2^{22}$ 上 duration 15 μ s， $N = 2^{27}$ 上 92% HBM peak。整个 pipeline 一次 kernel launch 完成，无需多阶段。
继续提升需要 vectorized load（float4，让 chunk items = 16 且访存单元对齐 128 B）、multi-block cooperative reduction（CUB 的做法）或 SM-level warp specialization。教学 kernel 到此打住；生产选 CUB。

性能 ladder 一览

源码里从 atomic 到 chunked 是一步步叠加的优化。所有 v2-v7 版本共享同一个 host launch pattern：一次 **kernel launch**，block 内规约 $\rightarrow$ 每 block 一次 atomicAdd 到 output——kernel 本身的算法在变，host 代码没变。下面 ladder 描述相对排序和每步原因；绝对数字见下一节实测。

版本	核心思路	相对 peak 带宽
atomic	per-element atomicAdd	contention 灾难（N 次 atomic 撞同址）
interleaved tree	block 内 smem 树 + per-block atomic	bank conflict + warp lane 浪费
sequential tree	block 内 smem 树（连续寻址）+ per-block atomic	bank conflict 减 30%
+ unroll last warp	省 5 次 __syncthreads	缩小 sync 范围到 warp
+ complete unroll	模板展开 + 编译期 blockSize	消除循环控制

warp shuffle	register shuffle + 1 次 smem sync	绕过 smem tree
chunked	8 elements/thread + smem tree + atomic	grid 缩 8× , 摊销 sync/atomic 成本

Warning. ladder 里 `unroll last warp` / `complete unroll` 等中间版本未单独 benchmark。本章实测覆盖 `atomic`、`interleaved tree`、`sequential tree`、`warp shuffle`、`chunked hierarchical` 五个代表点——足够说明 contention vs 协作规约 vs bank conflict vs grid contention 四个层次。

实测

$N = 2^{27} + 37$ (约 128 M 元素 / 512 MB , 远超 A100 40 MB L2), A100 80GB SXM4 , `ncu` 抓取 `reduce_sum_*_kernel` 每个版本的完整 **kernel** (单 launch 出结果)。GB/s 列写作 **HBM** 实测 / 逻辑：前者 `dram_bytes.sum / time` , 后者 $N \times \frac{4}{\text{time}}$; 这两个数字大规模下应几乎相等——差距大就是 L2 命中或者 kernel 太短。

version	regs	smem (B)	time (μs)	HBM %	SM %	HBM GB/s	% peak
atomic	16	0	367782.40	0.1	100.0	1 / 1	0.1
interleaved	16	0	2622.43	10.1	99.9	206 / 205	10.1
sequential	16	0	1444.19	18.3	99.7	373 / 372	18.3
warp-shuffle	16	32	1524.13	17.4	99.3	354 / 352	17.4
chunked	19	0	287.39	92.0	98.8	1876 / 1868	92.0

表 8 **Table (perf):** kernel-level performance metrics measured by Nsight Compute (`ncu`, CUDA 13.0) on a single A100 80GB SXM4 (HBM peak 2039 GB/s; FP32 peak 19.5 TFLOPS). Columns: **regs** = `launch_registers_per_thread`; **smem** = `launch_shared_mem_per_block_static` (bytes); **time** = `gpu_time_duration.sum` for the first kernel launch (multi-stage kernels report only the stage acting on the largest input); **HBM %** = `dram_bytes.sum.pct_of_peak_sustained_elapsed` (uses elapsed wall time as denominator, so short kernels look artificially low); **SM %** = `sm_cycles_active.avg.pct_of_peak_sustained_elapsed`; **HBM GB/s** is written `measured / logical` — `measured` = `dram_bytes.sum / time` , `logical` = `bytes-per-launch / time` ; when the working set exceeds L2 (40 MB) these two should agree, and any gap quantifies L2 hit rate; **% peak** = `measured HBM GB/s ÷ 2039 GB/s`.

Rows are ordered as they appear on the optimization ladder in the chapter.

version	issued/32	pred_on/32	smem conf.	barrier stall	mem stall
atomic	32.0	28.6	0	0.00	6045.10
interleaved	31.9	24.0	424,592	2.80	2.11
sequential	31.8	20.0	296,900	4.33	9.58
warp-shuffle	31.5	29.3	40,044	2.99	62.60
chunked	31.9	23.7	164,836	2.62	16.00

表 9 **Table (diag):** diagnostic metrics used as evidence for warp-lane utilization, shared-memory bank conflict, and warp-stall claims in the chapter. All from `ncu` on the same launch as the perf table. **issued/32** = `smsp_thread_inst_executed_per_inst_executed.ratio`, the average number of lanes that participated in each issued warp instruction (32 = every lane active; less than 32 means predication **or** branch divergence—`ncu` alone cannot distinguish, but on the CUDA kernels in this book it is almost always predication). **pred_on/32** = `smsp_average_thread_inst_executed_pred_on_per_inst_executed.ratio`, the average number of lanes that were predicated-on (did real work). The **gap issued - pred_on** counts lanes that occupied the issue slot but performed no work—this is the correct definition of “wasted warp cycles”, not divergence. **smem conf.** = `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` (0 + `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum`) means no measurable bank conflicts, either because the access pattern is conflict-free or because the input is too small to accumulate them). **barrier stall** = `smsp_average_warps_issue_stalled_barrier_per_issue_active.ratio`, warps stalled at `__syncthreads` per issue-active cycle. **mem stall** = `smsp_average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio`, warps waiting on a long-latency memory operation (global-memory load, L2 atomic response) per issue-active cycle.

perf 表读三件事：

1. **atomic** 慢 **100000** 倍。368 ms vs sequential tree 的 1.4 ms——不是“几十倍”，是五个数量级。128 M 次 atomicAdd 全撞同一 cache line，L2 层 serialize，`mem stall = 6045`（每 issue-active cycle 有 6045 个 warp 卡在 `long_scoreboard` 上），HBM % 只有 0.1%——完全不是 memory-bound，是 **serialization-bound**。
2. **interleaved vs sequential** : **2.6 ms → 1.4 ms (1.8×)**。同样是 shared memory tree reduction，仅寻址模式从 `if (tid % (2s) == 0)` 换成 `if (tid < s)` 就近乎快一倍。差距的物理来源要看 diag 表。
3. **warp_shuffle** 反而比 **sequential** 稍慢 (1524 vs 1444 μ s)。这在 $N = 2^{22}$ 的 snapshot 里已经暴露过：warp shuffle 消除了 smem bank conflict，但在“每 thread 1 元素”架构下 grid 有 $\frac{N}{256} \approx 5 \times 10^5$ 个 block——per-block 一次 atomicAdd 到 output 变成新瓶颈（`mem stall = 62.6`），warp shuffle 的收益被下一层瓶颈吃掉了。
4. **chunked** 接近打满带宽：287 μ s，HBM 92% (1876 GB/s / 2039 GB/s peak)。8× 更大 chunk 一次性把两个问题都解决——atomic 次数减 8×、HBM 队列深度加 8×——duration 从 sequential 的 1444 μ s 直接压到 287 μ s (5.0×)。

diag 表读关键教学点：

a) interleaved vs sequential 的差别 不是 **warp divergence**——两者 `issued/32` 都是 31.8-31.9, 几乎完美。 `if (tid < s)` 和 `if (tid % (2s) == 0)` 在 warp 里都是 predicated add, 不是真正的 branch divergence。

`pred_on/32` 反而告诉了我们相反的故事: `interleaved = 24.0`, `sequential = 20.0`——**sequential** 的 lane 利用率更低! 但 sequential 快 2.2×, 为什么? 答案在 `smem conf.` 一列。

b) sequential 快的真正原因: **bank conflict** 少 **30%**。interleaved 425k 次 bank conflict, sequential 297k 次——都不为零, 但 sequential 让每个 warp 内 lane 访问连续的 smem, 冲突集中在 warp 之间而非 warp 内, L1TEX 处理更高效。

Key insight. 这是本书 pushback “上帝视角”的核心案例。传统教材把 interleaved 慢的原因归为 “warp divergence”——ncu 的 `issued/32 = 32` 直接反驳了这个说法。真正的差别在 `smem conf.` 一列。术语用 “warp lane utilization” (`issued` vs `pred_on`) 和 “bank conflict pattern” (`smem conf.`) 比笼统说 “divergence” 更精确。

c) warp shuffle 大幅减少 **smem bank conflict** → 297k 降到 40k, 但 **wall-clock** 反而没变快。register-to-register 数据交换绕过 shared memory 是对的, 但在 “每 thread 1 元素” 架构下 grid 有 524288 个 block, 每个都发一次 `atomicAdd` 到 `output` —— `mem stall = 62.6` (比 sequential 的 9.6 还高 6.5×), warp 都在等 atomic serialization。收益被下一层瓶颈吃掉的经典案例。

d) chunked 的胜利在 **grid 太多 vs 每 thread 多做工的 tradeoff**。chunked 每 thread 处理 8 个元素 → grid 从 524288 缩到 65536。 `mem stall = 16.0` (vs warp 版 62.6, ↓ 4×) 显示 atomic 争用大幅缓解; 同时 HBM % 从 17% 涨到 92%——memory queue depth 上升 + atomic 次数下降双重收益。

bank conflict 的形成机制:

sequential addressing 第一轮 `shared[tid] += shared[tid + 128]` :tid=0 读 bank 0 和 bank 0 (128 mod 32 = 0) ——32-way conflict; tid=1 读 bank 1 和 bank 1——同样 32-way; tid=0..31 全部 32-way conflict, 共 32 次访问。

interleaved 第一轮 `shared[tid] += shared[tid + 1]` (stride=1):tid=0 读 bank 0 和 bank 1, tid=2 读 bank 2 和 bank 3——无冲突。但 stride=2、4、... 时冲突逐渐增加。结论: interleaved 前几轮少冲突、后几轮多冲突; sequential 前几轮多冲突、后几轮少冲突。ncu 累积的 425k vs 297k 是全部轮次求和的结果。

Note. 想在源码级别验证 bank conflict, 用 `ncu --section MemoryWorkloadAnalysis_Tables` 会打出按 access pattern 分类的 conflict 明细。

Warning. atomic 的 `mem_stall = 6045` 与 “memory latency stall” 的表面含义相反: `long_scoreboard` 包含所有 memory dependency 类 stall, atomic 的 read-modify-write 让 L2 atomic unit 反复往返 → 每次操作都需要 “等 L2 回复”, 被计入 `long_scoreboard`。这也是为什么 atomic 看起来像 mem-bound, 实际是 serialization。

ncu 该看什么

```
sudo ncu --set full ./build/02_reduce_sum 27
```

关键 metric 分四类：

- **serialization:**

`smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio` —— atomic 版会飙到几千。

- **warp lane utilization:** `smsp__thread_inst_executed_per_inst_executed.ratio` (issued/32) 和 `smsp__average_thread_inst_executed_pred_on_per_inst_executed.ratio` (pred_on/32) —— 判断 predicated-off 的 lane 数量。

- **bank conflict:** `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` + `..._op_st.sum` —— 判断 smem tree reduction 的效率。

- **HBM utilization:** `dram__bytes.sum` 除以 kernel time, 得到实际 HBM GB/s; `dram__bytes.sum.pct_of_peak_sustained_elapsed` 用 elapsed time 作分母, 对短 kernel 偏低。

面试白板 code

面试官说“手写一个 reduce sum”——直接白板 warp shuffle 版。第一句话就说：“我分两级——warp 内 shuffle, warp 间 shared memory”, 然后写：

```
constexpr int WARP = 32;

__device__ __forceinline__ float warp_reduce_sum(float v) {
    // 5 层蝶形：每层把距离 offset 的 lane 值加过来。
    #pragma unroll
    for (int offset = WARP / 2; offset > 0; offset >>= 1) {
        v += __shfl_down_sync(0xffffffff, v, offset);
    }
    return v; // lane 0 持有 warp 内 sum, 其他 lane 是垃圾值
}

__global__ void reduce_sum(const float* x, float* out, int n) {
    __shared__ float warp_sums[32]; // 最多 32 个 warp / block

    // grid-stride 拿到本 thread 的 partial (一个 thread 可能吃多个元素)
    float v = 0.f;
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;
         i < n; i += gridDim.x * blockDim.x) {
        v += x[i];
    }

    // Level 1: warp 内规约
    v = warp_reduce_sum(v);

    int lane = threadIdx.x & 31;
    int wid = threadIdx.x >> 5;
    if (lane == 0) warp_sums[wid] = v; // 每 warp 的 tail 存 smem
    __syncthreads();
}
```

```

// Level 2: warp 0 再规约 warp_sums (数量 <= 32, 一个 warp 搞定)
if (wid == 0) {
    int n_warps = (blockDim.x + WARP - 1) / WARP;
    v = (lane < n_warps) ? warp_sums[lane] : 0.f;
    v = warp_reduce_sum(v);
    // Level 3: 每 block 一次 atomicAdd 直接 finalize——一次 kernel 出结果
    if (lane == 0) atomicAdd(out, v);
}
}

// ==== Launch config ====
// blockDim = 256 (8 warp):
// * 必须是 32 的倍数——最后的 warp reduce 靠 warp 对齐;
// * 8 warp 让 warp_sums[] <= 8 <= 32, 第二级 warp reduce 一个 warp 搞定;
// * 不要开 1024——太多 warp 会让最后一级 warp reduce 变复杂.
// gridDim = min(SM * 4, (N + block - 1) / block):
// * SM * 4 让每 SM 常驻多个 block、latency hide;
// * 若 grid 太大 (~10^5+) 且每 block 只处理 1 元素, atomicAdd contention 会成瓶颈——
// 解决方法是 grid-stride 或每 thread 多做几个元素 (items/thread = 4 或 8).
cudaMemset(out, 0, sizeof(float)); // atomic 累加需要初始零
int block = 256;
int grid = min(4 * sm, (n + block - 1) / block);
reduce_sum<<<grid, block>>>(x, out, n); // 一次 launch 直接出结果

```

核心考点 (追问顺序):

- “为什么不直接一个大 **shared array** 做树?” → 可以, 但 warp shuffle 走寄存器堆、比 shared memory 快 3-5×, 还不需要 `__syncthreads` (warp 内锁步)
- “为什么 **grid-stride** 循环里就先做加法?” → 让一个 thread 吃很多元素、shared / shuffle 那部分只处理 blockDim.x 个 partial——摊销 sync/atomic 开销, 同时避免 grid 过大导致 atomicAdd contention.
- “最后一个数怎么得到?” → 每个 block 用 `atomicAdd(out, block_sum)` 直接 finalize——一次 kernel launch 出结果。这里 atomic 争用只有 blocks 次 ($\approx 10^4$), 跟 v1 那种 N 次 atomic 差 4 个数量级, L2 atomic unit 毫无压力。生产极致场景可以用 CUB / cooperative-groups grid barrier, 但对绝大多数 shape 这个写法已足够。
- “**warp shuffle** 要不要 **mask**?” → `__shfl_down_sync` 第一个参数是 active mask; 这里所有 32 lane 都活, 写 `0xffffffff`。如果有些 lane 提前 return 了, mask 要相应改, 否则 UB。
- “能证明没有 **warp divergence** 吗?” → SASS 里 `if` 会被 predicate, `ncu smssp_sass_branch_targets_threads_divergent.sum ≈ 0` 就是证据 (见本章 v2 讨论)
- “**blockDim** 为什么选 **256** 不选 **1024**?” → (1) 必须是 32 的倍数、否则最后 warp reduce 不对齐; (2) $256 \text{ warp} / 32 = 8$, `warp_sums[]` 长度 = 8, 第二级用 warp 0 一个 warp 搞定; 1024 会让 warp_sums 长度 = 32、正好占满 warp, 稍显浪费——但更重要是寄存器压力: 1024 threads/block 会限制 occupancy。256 是“够用又不奢侈”的经典选择。

面试考点

面试考点. Q1: reduction 是 memory-bound 吗? 为什么 naive 版本还是很慢?

A: $AI = 0.25 \text{ FLOP/B}$, 远低于 ridge point, 读带宽是主导。但 naive(atomic 或 interleaved tree) 的瓶颈在同步和 **contention**, 不是 HBM 带宽本身。优化目标是减少 sync 次数、消除 bank conflict、让每 thread 多做 work 再通信。

面试考点. **Q2**: interleaved addressing 和 sequential addressing 的区别? 为什么后者更快?

A: interleaved 用 `if (tid % (2s) == 0)` 选活跃 thread, 它们在 warp 内间隔分布; sequential 用 `if (tid < s)`, 活跃 thread 是连续区间。两者都不是真正的 **warp divergence** (ncu issued/32 都是 32, 编译成 predicated instruction)。真正的差异在 bank conflict pattern: sequential 让 warp 内 lane 访问连续 smem 地址, L1TEX 处理更高效——实测 sequential bank conflict 297k vs interleaved 425k, 快 1.8 $\times$ 。

Warning. 如果面试官坚持说 interleaved “有 warp divergence”, 你可以补一句: predication 不是 divergence——ncu `smsp_thread_inst_executed_per_inst_executed_ratio` 会区分这两种情况。真正的分支 divergence 是 warp 里不同 lane 走不同 **basic block** (if-else 两侧都有工作)。

面试考点. **Q3**: reduction 中 bank conflict 在哪一步最严重? 怎么缓解?

A: 第一步 (offset = blockSize/2) 最严重: stride = 128 时 `shared[i]` 和 `shared[i+128]` 在同一 bank ($128 \bmod 32 = 0$), 32-way conflict。缓解: padding (`shared[tid + 1]`)、减小 block size、或用 warp shuffle 绕过 shared memory。

面试考点. **Q4**: 为什么 unroll last warp 可以不用 `__syncthreads`?

A: 最后 $\log_2(32) = 5$ 步只在 warp 0 内执行。同一 warp 的 32 lane SIMT 锁步, register/shared 写后在同一 warp 内可见, 不需要 block 级 barrier。对其他已经 idle 的 warp 做 `__syncthreads` 只有开销 (10-20 cycles), 没有同步收益。

面试考点. **Q5**: `__syncthreads` 的成本是多少? 什么时候值得省?

A: 大约 10-20 cycles (取决于架构和 block 大小)。当同步范围可以缩小到 warp 级 (shuffle) 或 active thread 只在 1 个 warp 内时, 值得省。block 内有多个 warp 仍活跃时必须用 `__syncthreads`。

面试考点. **Q6**: `volatile` 和 `__syncwarp()` 的区别? unroll last warp 该用哪个?

A: `volatile` 是编译器层面的语义: 禁止把 shared memory 读缓存到 register, 强制每次从 smem 重新读——旧代码 (pre-Volta) 用来保证 warp 内可见性。`__syncwarp()` 是硬件 warp 级 barrier (5 cycles), 保证 warp 内所有 lane 到达后再继续。现代代码 (Volta+) 应优先用 `__syncwarp()` + 普通读写, 或直接用 `__shfl_down_sync` 完全绕过 smem。`volatile` 不能替代 sync——它只防编译器优化, 不保证其他 thread 已写完。

面试考点. **Q7**: `__shfl_down_sync` 的硬件实现是什么? 比 shared memory 快在哪?

A: 走 warp 内 register crossbar (Xbar) : lane i 直接从 lane $i + \delta$ 的 register 读值, 不经过 shared memory port。延迟 1 cycle, 无 bank conflict, 无 extra smem traffic。shared memory tree 每步需要 2 次 smem access (读+写) + 可能的 bank conflict。

面试考点. **Q8**: 单个 block 输出一个 partial sum 之后怎么办? 有哪些 grid-level 策略?

A: 三种主流做法:

(a) **per-block atomicAdd finalize** (本 repo 做法、推荐): 每 block 一次 `atomicAdd(out, block_sum)`。一次 kernel launch 出结果, 无 host 循环。atomic 次数 = block 数, 通常 10^3 - 10^5 量级, L2 atomic unit 完全 hold 得住。跟 v1 那种 N 次 per-element atomic 有本质区别。

(b) **multi-stage chained launch**: 反复 launch reduction kernel 把长度缩短直到 1 (Mark Harris 2007 原教程做法)。写起来丑 (host 循环 + `cudaMalloc/cudaFree`), 且多一次 launch overhead。历史原因: 早期 GPU FP32 atomic 慢或没有; 现代 A100+ 上 (a) 更快。

(c) **CUB / cooperative-groups grid barrier**: 一次 kernel 内 grid-level sync, 最后一个 block 做 finalize。写起来最复杂, 但省 atomic 争用; 生产环境 (`cub::DeviceReduce`) 用这个。

面试白板首选 (a)——简洁、性能好、易讲清 tradeoff。有人问“atomic 会不会慢”就用本章 v1 vs v5 数据回击: per-element atomic vs per-block atomic 差 3-4 个数量级。

面试考点. **Q9**: 为什么 chunked (multi-elements-per-thread) 有效? 步长为什么用 `blockDim.x`?

A: 每个 thread 在 register 里累加 8 个元素再进入 block 规约, 摊销 index 计算、`__syncthreads` 和 finalize 时的 atomicAdd 成本; 8 个 load 可 pipeline 隐藏 latency、MSHR 队列吃满。步长 = `blockDim.x` (不是 8) 是为保持 warp 内合并访问——和 vector add tiled 版本同一原则。

面试考点. **Q10**: 模板 `blockSize` 完全展开有什么好处?

A: 编译期固定循环边界 → 消除循环控制开销、常量传播去掉 dead branch、精确控制哪些步需要 `__syncthreads` (跨 warp 的 3 步需要, warp 内的 5 步不需要)。Launch 时 block size 必须匹配模板参数。

Softmax

softmax 是 transformer 里 attention score 归一化的核心，也是面试里数值稳定性和 **online** 算法的高频考点。这一章我们要把它讲透：

- 为什么 naive 三遍扫描在 GPU 上是灾难（访存 3×、并行度差）
- subtract-max trick 的数学推导——不是“经验技巧”，而是严格等价变换。
- online softmax (Milakov & Gimelshein, 2018) ——单遍扫描维护 running max + running sum, FlashAttention 的数值基础。
- block-per-row + shared memory 两次 reduction 的标准并行模板。

对应源码：src/cuda/03_softmax/（一个 version 一个文件：01_naive.cu, 02_block.cu, 03_online.cu）带 mask 或 causal 语义的变体不再作为独立 kernel 单独实现——mask/causal 在 attention 章节（第 7 章及之后）里跟 attention pipeline 一起讲更自然。

本章 optimization ladder：

版本	核心思路	相对 peak 带宽
naive	1 thread / row, 3 pass	~5%
online	1 thread / row, running max+sum	~5%
block	1 block / row, 2× smem reduce	~40%

前两个版本并行度相同（一行一 thread），差异在 pass 数和数值技巧；block 版本解决行内并行。ladder 里百分比是 A100 上 sm_cycles_active (SM %), 不是相对 naive 的加速比——实测 naive / online 的 SM % 都约 0.9%（单 block），block 约 35%。

与第 2 章 reduce sum 的关系：softmax 的 max-reduction 和 sum-reduction 复用同一套 **block/warp shuffle** 模板——区别只是归约算子（fmaxf vs +）和 sum 阶段需要先 broadcast max。能把 reduce 章的 ladder 平移过来，是面试里展示“kernel 模式可组合”的好例子。

问题定义

给定形状 [rows, cols] 的矩阵 X ，对每一行独立做 softmax：

$$y_i = \frac{\exp(x_i)}{\sum_{j=0}^{\text{cols}-1} \exp(x_j)}, \quad i = 0, 1, \dots, \text{cols} - 1$$

输出每行元素非负且和为 1，可解释为概率分布。在 attention 里， X 是 $Q \frac{K^T}{\sqrt{d}}$ 的 logits， Y 是 attention weights。

数值稳定性：overflow 与 subtract-max trick

直接算 $\exp(x[i])$ 有两个问题：

- 1. Overflow.** FP32 的 $\expf$ 在 $x > 88$ 时溢出为 inf ，后续 sum 变成 inf ，归一化出 NaN。
- 2. Underflow.** $x \ll 0$ 时 $\expf(x)$ 下溢为 0，虽然单个 0 无害，但若所有项都下溢， $\text{sum} = 0$ ，除法出 NaN。

标准做法：令 $m = \max_j x_j$ ，计算

$$\tilde{y}_i = \frac{\exp(x_i - m)}{\sum_j \exp(x_j - m)}$$

等价性证明：分子分母同乘 $\exp(m)$ ：

$$\frac{\exp(x_i - m)}{\sum_j \exp(x_j - m)} = \frac{\exp(x_i - m) \cdot \exp(m)}{\sum_j \exp(x_j - m) \cdot \exp(m)} = \frac{\exp(x_i)}{\sum_j \exp(x_j)} = y_i$$

减去 $\max$ 后，至少有一项 $x_i - m = 0$ ，对应 $\exp(0) = 1$ ， $\text{sum} \geq 1$ ，既避免 overflow（最大项指数为 0），又保证分母非零。

log-sum-exp 形式

定义

$$\text{LSE}(x) = \log\left(\sum_j \exp(x_j)\right) = m + \log\left(\sum_j \exp(x_j - m)\right)$$

。则：

$$y_i = \exp(x_i - m - \log d) = \exp(x_i - \text{LSE}(x))$$

log-softmax 直接算 $x_i - \text{LSE}(x)$ ，不经过 y_i 中间态——数值上更稳。online 算法维护的 d 就是 $\exp(\text{LSE} - m)$ 里的 sum 部分；merge 公式等价于 log-sum-exp 的 associative 版本。

Key insight. subtract-max 不改变数学结果，只改变计算的数值路径。面试里必须能独立推导等价性——这是 safe softmax 和 log-softmax 的共同基础。

Warning. 全行都是 $-\infty$ （例如 attention 里整行被 mask 掉）时， $m = -\infty$ ， sum 仍为 0。生产代码要单独处理：输出全零或 uniform，不能盲目除法。

Roofline : softmax 是什么 bound ?

对一行长度 $C = \text{cols}$ 的元素，三 pass naive 版本每元素大致：

- Pass 1 (max)：读 $1\times$ ，比较 $O(C)$
- Pass 2 (sum exp)：读 $1\times$ ， $1 \text{ exp} + 1 \text{ add}$
- Pass 3 (normalize)：读 $1\times$ ， $1 \text{ exp} + 1 \text{ div} + 1 \text{ 写}$

每元素约 **3 次读 + 1 次写 + 2 exp + 1 div**。算术强度：

$$\text{AI} \approx \frac{2 \text{ exp} + 1 \text{ div}}{16 \text{ B}}$$

`expf` / `div` 在 GPU 上 latency 高（数十 cycle），但每元素内存访问 16B（1 float 读 $\times 3 + 1$ float 写）。 C 较小时 compute 占比上升； C 较大（如 4096、8192）时仍偏 memory-bound，但重复读同一行 **3 遍**是首要浪费——这是和 vector add 的本质区别。

Attention 典型 shape： $B \times H \times S \times S$ 的 logits，reshape 成 $N_{\text{rows}} = B \cdot H \cdot S$ 行、每行 $C = S$ 列。 $S = 4096$ 时每行 16 KB，batch $32 \times \text{heads } 32 \rightarrow N_{\text{rows}} = 32768$ 行——足够喂饱 GPU；但 $S = 128$ 时 rows 可能只有几千，block-per-row 的 grid 维不够大，需要 fuse 多行 per block 或增大 batch。

Key insight. softmax 的性能诊断要分开看： C 小 $\rightarrow$ 看 MUFU / div 吞吐和 warp 利用率； C 大 $\rightarrow$ 看 dram bytes 和 pass 数。给 numbers 之前先问序列长度和 rows 数。

单 pass 理论 traffic (读 1 + 写 1): $2N_{\text{rows}}C \times 4\text{ B}$ 。naive 三 pass 读 3 遍: $4N_{\text{rows}}C \times 4\text{ B}$ (3 读 + 1 写)。访存优化上限约 2x——还没算重复 exp 的 compute 浪费。

v1: naive 三 pass (one thread per row)

```
__global__ void softmax_naive_kernel(
    const float* input, float* output, int rows, int cols) {
    const int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= rows) return;

    const float* row_ptr = input + row * cols;
    float* out_ptr = output + row * cols;

    float max_value = row_ptr[0];
    for (int col = 1; col < cols; ++col) {
        max_value = fmaxf(max_value, row_ptr[col]);
    }

    double sum = 0.0;
    for (int col = 0; col < cols; ++col) {
        sum += exp(static_cast<double>(row_ptr[col] - max_value));
    }

    for (int col = 0; col < cols; ++col) {
        out_ptr[col] = static_cast<float>(
            exp(static_cast<double>(row_ptr[col] - max_value)) / sum);
    }
}
```

一个 thread 负责一整行，逻辑和 CPU reference 一模一样。

三个结构性问题

1. 并行度差。rows = 64 时只有 64 个 thread 在干活，GPU 有上万个 core。行数小于 SM 数时，大量 SM 空转。
2. 访存 3x。同一行数据从 HBM 读了 3 遍 (max、sum、normalize)。对 $C = 4096$ 的一行，16 KB 数据读 3 次 = 48 KB traffic，而理论最小只需读 1 次 + 写 1 次。
3. Pass 2 和 Pass 3 重复算 exp。exp($x - \text{max}$) 算了两次。可以缓存中间结果到 shared memory / register，但 naive 版本没做。
4. exp 是特殊函数。SASS 里 exp 走 MUFU (multi-function unit)，不是普通 FFMA pipeline。一个 SM 上 MUFU 吞吐有限——当 rows 少、 C 小，整个 kernel 可能变成 **transcendental-bound** 而非 memory-bound。ncu 里看 `smisp_sass_thread_inst_executed_op_mufu` 的占比。

Note. sum 用 double 累加是刻意的： C 很大时 float 累加 $\exp(x - m) \in [0, 1]$ 会丢精度。面试追问“为什么不用 float 累加”时，答大 C 下 ULP 误差。

并行度问题是本章实测里最大的杀手——64 rows 时 naive / online 的 grid 只有 (1, 1, 1)，整颗 GPU 几乎空转。详见本章「实测」节。

和 **vector add** 的对比

vector add 里每个元素独立，thread 之间零通信；softmax 里同一行的所有元素通过 max 和 sum 强耦合——必须先知道整行的 max 才能安全算 exp，必须先知道整行的 sum 才能归一化。这个依赖链决定了优化 ladder 的主线：先把行内规约并行化（block / warp reduction），再考虑减少 **pass** 数（online），最后才是 load/store 宽度与 fusion。

ncu 实测

ncu 实测 — v: naive (one thread per row). 规模: rows = 256, cols = 4096 (4 MB) .

Duration	3 990 μ s	3.99 ms 处理 4 MB——是同规模 vector add 的 $\sim 500\times$
Memory SOL	0.8 %	HBM 几乎空转
Compute SOL	0.3 %	SM 也空——`exp` 走 MUFU，pipeline 单窄
Achieved Occupancy	12.5 %	grid 只有 (1,1,1)，256 rows 装不满 108 个 SM
Grid Size	1	*一个 block！整颗 GPU 只有 8 个 warp 活着*

Occupancy 12.5% + Grid Size 1 是全书最戏剧性的读数：GPU 有 108 个 SM，我们只用了 1 个 SM 上的 256 个 thread，其余 **107 SM** 完全空转——3.99 ms 里 99% 的硬件在闲置。

- **memSOL 0.8%**：一行 4096 个元素被单 thread 串行读，warp 里 32 lane 每次只有 lane 0 在做——完全打不开访存 pipeline。
- **compSOL 0.3%**：exp 走 MUFU (multi-function unit)，单 SM 上 MUFU 吞吐有限；再叠加 warp 内 31 个 lane idle，MUFU 也是极低利用率。

瓶颈判定. one-thread-per-row 让 GPU 的行内 4096 个元素被串行处理，整颗 **GPU** 只有 **1** 个 **SM** 在工作——是并行度分配问题，不是访存或计算瓶颈

证据. Grid Size = 1, Occupancy 12.5%, memSOL 0.8%, compSOL 0.3% 全线极低
下一步. v3 (block) 把行内 4096 元素分给 256 个 thread 协作规约——一个 block 处理一行，256 rows 就有 256 个 block，正好把 108 个 SM 填满 2.4 waves

Online Softmax：单遍维护 max 与 sum

Milakov & Gimelshein (2018) 在论文 **Online normalizer calculation for softmax** 中提出：可以在一次扫描中同时维护 running max 和 running sum，最终与三 pass subtract-max softmax 在浮点语义下一致。这个技巧后来被 FlashAttention (Dao et al., 2022) 直接采用，成为 attention kernel 的数值基石。

动机

FlashAttention 不能把整行 logits 放进 SRAM——它按 block 流式处理 QK^T 的片段。每来一个新 block，必须合并已有的 (m, s) 和新 block 的统计量，而不能回头重读全局 memory。online softmax 就是这个合并规则的抽象。

具体场景：算 attention 时 $S = QK^T$, shape $[N, N]$ 。标准实现 materialize 整个 S ，做 softmax，再乘 V —— S 的 HBM 读写是 $O(N^2)$ 。FlashAttention 每次只算 S 的一个 $B_r \times B_c$ tile，tile 的 partial softmax 必须用 online 公式 merge 到 running state，否则无法保证与完整 softmax 等价。

Note. 论文原文用 “safe softmax” 指 subtract-max 版本。本书沿用源码命名：online softmax = 单遍维护 (m, d) 的算法；block 版 = 两遍（先规约 max/sum，再写回）。

推导

假设已处理元素 $x_1, \dots, x_{t-1}$, 维护 :

- $m_{t-1} = \max(x_1, \dots, x_{t-1})$
- $d_{t-1} = \sum_{i=1}^{t-1} \exp(x_i - m_{t-1})$

来了新元素 x_t , 令 $m_t = \max(m_{t-1}, x_t)$ 。

关键 : 旧 sum 的基准 max 是 m_{t-1} , 新 sum 的基准 max 是 m_t 。当 $m_t > m_{t-1}$ 时 , 旧项要重标定 :

$$d_t = \sum_{i=1}^t \exp(x_i - m_t) = \underbrace{\sum_{i=1}^{t-1} \exp(x_i - m_t)}_{\text{旧元素重标定}} + \exp(x_t - m_t)$$

对 $i < t$: $\exp(x_i - m_t) = \exp(x_i - m_{t-1}) \cdot \exp(m_{t-1} - m_t)$

因此 :

$$d_t = d_{t-1} \cdot \exp(m_{t-1} - m_t) + \exp(x_t - m_t)$$

当 $m_t = m_{t-1}$ (新元素不是更大) 时 , $\exp(m_{t-1} - m_t) = 1$, 退化为 $d_t = d_{t-1} + \exp(x_t - m_t)$ 。

更新公式 (代码直接对应) :

$$m_t = \max(m_{t-1}, x_t)$$

$$d_t = d_{t-1} \cdot \exp(m_{t-1} - m_t) + \exp(x_t - m_t)$$

扫描结束后 (m_T, d_T) 就是 subtract-max softmax 里的 $(m, \sum)$ 。第二遍写输出 :

$$y_i = \frac{\exp(x_i - m_T)}{d_T}$$

数值走查 (手算验证)

取 $x = [1, 2, 3]$:

Step 0 : $m_0 = -\infty$, $d_0 = 0$ 。

Step 1 ($x_1 = 1$) : $m_1 = 1$, $d_1 = \exp(1 - 1) = 1$ 。

Step 2 ($x_2 = 2$) : $m_2 = 2$, $d_2 = d_1 \cdot \exp(1 - 2) + \exp(2 - 2) = 1 \cdot e^{-1} + 1 \approx 1.368$ 。

Step 3 ($x_3 = 3$) : $m_3 = 3$, $d_3 = 1.368 \cdot \exp(2 - 3) + \exp(0) \approx 1.368 \cdot 0.368 + 1 \approx 1.503$ 。

最终 : $y_1 = \frac{\exp(-2)}{1.503} \approx 0.090$, $y_2 = \frac{\exp(-1)}{1.503} \approx 0.245$, $y_3 = \frac{\exp(0)}{1.503} \approx 0.665$ 。三者之和 ≈ 1.000 ——与 `softmax([1, 2, 3])` 一致。

注意 Step 2→3 时 m 从 2 升到 3 , 旧 sum 被乘以 $\exp(2 - 3) = e^{-1}$ ——这就是“重标定”的直觉 : max 变大后 , 旧 exp 值相对新基准都变小了 , 必须统一缩放。

正确性 (合并两个 **chunk**)

设行分为 $A = \{x_1, \dots, x_k\}$ 和 $B = \{x_{k+1}, \dots, x_n\}$ 。chunk A 的统计 (m_A, d_A) , chunk B 的 (m_B, d_B) (各自内部 relative to 局部 max)。

全局 $m = \max(m_A, m_B)$, 全局 sum :

$$d = d_A \cdot \exp(m_A - m) + d_B \cdot \exp(m_B - m)$$

证明思路 : 对 chunk A 中任意 x_i , 它在全局 sum 里的贡献是 $\exp(x_i - m) = \exp(x_i - m_A) \cdot \exp(m_A - m)$; 对 A 内所有项求和即 $d_A \cdot \exp(m_A - m)$ 。 B 同理。加在一起就是全局 d 。

这个 merge 是结合律的：先 merge A 和 B ，再 merge C ，与一次性 merge $A \cup B \cup C$ 结果相同——因此可以并行算各 chunk 的 (m, d) ，再用 tree reduction 合并。FlashAttention 在 SRAM tile 之间正是这么做的。

两 chunk merge 数值走查

仍用 $x = [1, 2, 3]$ ，拆成 $A = [1, 2]$ 和 $B = [3]$ ：

Chunk A：online 扫描得 $m_A = 2$ ， $d_A = \exp(1 - 2) + \exp(2 - 2) = e^{-1} + 1 \approx 1.368$ 。

Chunk B： $m_B = 3$ ， $d_B = \exp(3 - 3) = 1$ 。

Merge： $m = \max(2, 3) = 3$ 。

$$d = d_A \cdot \exp(2 - 3) + d_B \cdot \exp(3 - 3) = 1.368 \cdot e^{-1} + 1 \approx 1.503$$

与单遍 online 得到的 $d_3 = 1.503$ 一致。注意 $m_A < m$ 时 d_A 必须乘 $\exp(m_A - m)$ ——这就是 FlashAttention 在 tile 边界做的事。

Key insight. online softmax 的价值不只是“少一遍循环”。它把 softmax 从全量依赖变成流式可合并——这是 FlashAttention 不需要 materialize $N \times N$ attention matrix 的数值前提。

实现细节：为什么 sum 用 double

更新公式里 `row_sum * exp(row_max - new_row_max)` 涉及两个 `exp` 结果的乘加。当 C 很大 (8192、32768) 且 logits 分布极端 (某些 $x_i - m \approx 0$ ，其余 $\ll 0$) 时，float 累加会丢 ULP。源码里 `row_sum` 用 `double`，merge 时也 cast 到 `double`——这是 attention 长序列下的标准做法。block 版 `local_sum` 仍用 `float` 是为了 smem 大小和速度； $C > 4096$ 的生产 kernel 往往全程 `double` 或 Kahan summation。

v2: online kernel (one thread per row)

```
__global__ void softmax_online_kernel(
    const float* input, float* output, int rows, int cols) {
    const int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= rows) return;

    const float* row_ptr = input + row * cols;
    float* out_ptr = output + row * cols;

    float row_max = -INFINITY;
    double row_sum = 0.0;
    for (int col = 0; col < cols; ++col) {
        const float x = row_ptr[col];
        const float new_row_max = fmaxf(row_max, x);
        row_sum = row_sum * exp(static_cast<double>(row_max - new_row_max))
            + exp(static_cast<double>(x - new_row_max));
        row_max = new_row_max;
    }

    for (int col = 0; col < cols; ++col) {
        out_ptr[col] = static_cast<float>(
            exp(static_cast<double>(row_ptr[col] - row_max)) / row_sum);
    }
}
```

```

}
}

```

Pass 1 合并了 $\max + \text{sum}(\text{online})$, Pass 2 写输出。访存从 $3\times$ 降到 $2\times$, 且 Pass 1 的 $\exp$ 只算一次 (更新 sum 时), Pass 2 的 $\exp$ 无法避免 (除非缓存 $\exp(x - m)$ 到 scratch , 用空间换时间), 并行度问题和 naive 一样——仍是一 thread 一行。下一节解决行内并行。

Warning. 单跑 **online kernel** 不会比 **naive** 快——实测 $\text{time } 252.29 \mu\text{s}$ vs $120.80 \mu\text{s}$ (慢 $2.1\times$)。Pass 1 里每步要算 $\exp(\text{row_max} - \text{new_row_max})$ 做重标定, MUFU 指令比 naive 三 pass 的“先 $\max$ 再 bulk exp ”更碎; 且仍是单 block launch , $\text{SM} \% 0.9\%$ 。online 的价值是流式可合并 (FlashAttention tile merge), 不是 standalone softmax 的加速手段。

ncu 实测

ncu 实测 — v: online (one thread per row). 规模: $\text{rows} = 256, \text{cols} = 4096$.

Duration	3 890 μs	跟 naive 几乎持平 (-2.5%), *不是加速*
Memory SOL	0.7 %	
Compute SOL	0.4 %	
Achieved Occupancy	12.4 %	跟 naive 一样, 同样只用了 1 个 block
Grid Size	1	

所有 **metric** 都跟 **naive** 几乎完全一致——把三 pass 压缩成两 pass 完全被单 thread 处理一行的并行度问题掩盖。这印证了“online 不是速度手段, 是流式可合并语义”。

瓶颈判定. online softmax 减少了访存 pass 数, 但没有解决并行度问题——瓶颈依然是 one-thread-per-row

证据. Grid Size 1, memSOL 0.7% —— 与 naive 完全同构

下一步. v3 (block) 用 shared memory tree reduction 把行内规约并行化。online 语义在 FlashAttention 那章会带来真正的价值: 当你没有整行数据时, online 能合并部分统计量。

v3: block-per-row + shared memory reduction

```

constexpr int kThreadsPerBlock = 256;

__global__ void softmax_block_kernel(
    const float* input, float* output, int rows, int cols) {
    __shared__ float shared_max[kThreadsPerBlock];
    __shared__ float shared_sum[kThreadsPerBlock];

    const int row = blockIdx.x;
    if (row >= rows) return;

    const float* row_ptr = input + row * cols;
    float* out_ptr = output + row * cols;

    float local_max = -INFINITY;

```

```

for (int col = threadIdx.x; col < cols; col += blockDim.x) {
    local_max = fmaxf(local_max, row_ptr[col]);
}
shared_max[threadIdx.x] = local_max;
__syncthreads();

for (int offset = blockDim.x / 2; offset > 0; offset /= 2) {
    if (threadIdx.x < offset) {
        shared_max[threadIdx.x] = fmaxf(
            shared_max[threadIdx.x], shared_max[threadIdx.x + offset]);
    }
    __syncthreads();
}
const float row_max = shared_max[0];

float local_sum = 0.0f;
for (int col = threadIdx.x; col < cols; col += blockDim.x) {
    local_sum += expf(row_ptr[col] - row_max);
}
shared_sum[threadIdx.x] = local_sum;
__syncthreads();

for (int offset = blockDim.x / 2; offset > 0; offset /= 2) {
    if (threadIdx.x < offset) {
        shared_sum[threadIdx.x] += shared_sum[threadIdx.x + offset];
    }
    __syncthreads();
}
const float row_sum = shared_sum[0];

for (int col = threadIdx.x; col < cols; col += blockDim.x) {
    out_ptr[col] = expf(row_ptr[col] - row_max) / row_sum;
}
}

```

结构：两个 **reduction** + 一个 **map**

softmax 的并行骨架和第 2 章 reduce sum 同构：

1. **grid-stride** 局部归约：每个 thread 扫 `col += blockDim.x` 的列，算 local max / local sum。
2. **block 级 tree reduction**：shared memory 上折半规约。
3. **broadcast + normalize**：`row_max / row_sum` 在 `shared[0]`，所有 thread 读同一个值写回。

Launch：<<<rows, 256>>> ——一行一个 block，block 内 256 thread 协作。

reduction 树的两轮结构

以 $C = 1024$ 、256 threads 为例：

Max pass：

- 每个 thread 处理 4 列 (stride 256)，得 local_max。
- Tree reduction：128→64→32→16→8→4→2→1，共 8 步 `__syncthreads`。
- thread 0 的 `shared_max[0]` = 整行 max。

Sum pass :

- 已知 row_max , 每个 thread 算 4 个 $\expf(x - \text{row_max})$ 的 local_sum。
- 同样的 tree reduction 得 row_sum。

Normalize pass :

- 无 sync , 每个 thread 写回自己负责的列。

阶段	每 thread 工作量	同步次数
local max	$\lceil \frac{C}{256} \rceil$ 次 fmaxf	0
max reduction	8 步 tree	$8 \times \text{syncthreads}$
local sum exp	$\lceil \frac{C}{256} \rceil$ 次 expf	0
sum reduction	8 步 tree	$8 \times \text{syncthreads}$
normalize	$\lceil \frac{C}{256} \rceil$ 次 expf + div + store	0

表 10 **Table:** block softmax kernel 的五阶段工作分解 ($C = 1024$ 、 256 threads 示例)。每 thread 工作量以 fmaxf / expf 调用次数计；同步次数为 `__syncthreads` 调用数。两轮 tree reduction 各需 $\log_2(256) = 8$ 步, local 与 normalize 阶段无 block 内同步。

Observation : 五阶段里只有两轮 tree reduction 需要 `__syncthreads`——local max / local sum exp / normalize 都是 embarrassingly parallel 的 grid-stride 扫描。max reduction 与 sum reduction 之间必须插入 barrier : 所有 thread 必须看到同一个 row_max 才能算 $\exp(x - m)$, 这正是“先 max 再 sum”两趟 pass 的并行代价, 也是 block 内不能 online 化的根因。

两次 reduction 之间必须 `__syncthreads`——所有 thread 必须看到同一个 row_max 才能算 exp。normalize 阶段 row_max 和 row_sum 已在 shared[0] , broadcast 无需额外 sync (只要前面 reduction 的 syncthreads 已完成)。

为什么先 **max** 再 **sum** , 不能 **online** 化 **block** 内 ?

block 内各 thread 看到的数据是交错的列 (stride = blockDim) , 不是连续前缀。online 更新要求顺序语义——thread 0 看 col 0, 256, 512... , 不能代表“先看到 col 0 再看到 col 1”。

block 版本用两次独立 reduction 是正确的并行分解。online 公式用在跨 **block** / 跨 **wave** merge (FlashAttention) , 不是 block 内 thread 之间。

occupancy 与 smem

`shared_max[256] + shared_sum[256]` = 2 KB smem——远低于 SM 48 KB 上限。寄存器方面, 每 thread 几个 float + 循环变量, 通常 < 32 regs, occupancy 接近 100%。瓶颈不在资源, 在每行只用一个 **block**——rows 不够多时 SM 填不满。

改进 : 一个 block 处理多行——例如 $256 \text{ threads} = 8 \text{ warps}$, 每 warp 一行 (warp-per-row 的常见扩展)。grid 维从 rows 降到 $\lceil \frac{\text{rows}}{8} \rceil$, 同时提高 SM 利用率。

block 大小的选择

`kThreadsPerBlock = 256` 是教学默认。 $C = 4096$ 时每个 thread 处理 16 个元素——足够摊销 reduction 开销。 $C < 256$ 时有 thread 空转, 但 reduction 仍正确 (local_max 初始 $-\infty$, local_sum 初始 0)。

Warning. `blockIdx.x = row` 意味着 `grid 维 = rows`。 `batch × heads × seq` 很大时 `rows` 足够；若 `rows` 很小（如 1），整个 GPU 只跑 1 个 block——此时应合并多行到一个 block，或走 `warp-per-row`。

ncu 实测

ncu 实测 — v: block (one block per row). 规模: `rows = 256, cols = 4096`.

Duration	21.1 μ s	*比 naive 快 189×*
Memory SOL	10.1 %	从 0.8% 拉到 10% ——一个 kernel 内做两次 reduce 限制了持续 HBM 打满
Compute SOL	23.5 %	
Achieved Occupancy	28.3 %	
Grid Size	256	vs naive 1 —— 256 个 block 分给 108 个 SM, 2.4 waves

189× 的提速大部分来自单纯的并行度改善 :Grid Size 从 1 涨到 256 ,每个 block 有 256 个 thread 协作规约行内 4096 元素。这两个 lever 一乘 : $256 \times \frac{256}{256} = 256 \times$ ——加上 shared memory tree reduction 引入的额外开销，实际拿到 189×

- **memSOL 10%** : 远低于 vector add / matmul 的 80%+ ,因为这个 kernel 内部要做两次行内 reduction (max、 sum), 每次都要写 smem + sync , dominated by shared memory traffic that isn't reflected in DRAM SOL.
- **Occupancy 28%** : block 内 256 thread、每 thread 需要维护 accumulator , 寄存器不多 ; smem 用得少 (2KB) ——理论 occupancy 可以更高 , 但实测被 memory pipeline 的等待时间压低。

落地评估. block 版本已经把 softmax 从 “1 SM 空转” 转成 “GPU 全部 SM 参与”, 达到实用性能。

进一步优化的方向是 `warp-per-row` (`rows` 很小时用 ; 见 05_layernorm 的 warp shuffle 版本), 或者和上下游算子融合 (softmax + logits → attention 的 online 合并 , 见后续 FlashAttention 章节), standalone softmax 到这里够用。

log-softmax 的数值考量

loss 计算常用 $\log \text{softmax}(x)_i = x_i - m - \log(d)$, 直接算 **log** 空间 , 避免先 exp 再 log 的精度损失 :

$$\log(y_i) = x_i - m - \log\left(\sum_j \exp(x_j - m)\right)$$

PyTorch `log_softmax` 就是这个。面试追问 : 为什么不在 softmax 输出上取 `log` ? 答 : y_i 很小时 $\log(y_i)$ 损失大量有效位 ; log-softmax 全程在 log-sum-exp 框架里 , 和 online 公式一样稳定。

Cross-entropy loss : $\mathcal{L} = -\sum_i t_i \log(y_i)$ 。若 t 是 one-hot , 只需 $-\log(y_k) = -x_k + m + \log(d)$ ——全程不需要 **materialize** y 。这是 fused softmax+crossentropy kernel 的基础 (PyTorch `cross_entropy` 内部路径)。

safe softmax vs approx softmax

方法	做法	场景
safe (subtract-max)	$\frac{\exp(x-m)}{\sum}$	训练/推理默认
approx (piecewise linear exp)	查表 / 多项式近似 exp	极端 latency 敏感、可接受误差
online merge	分块 merge (m, d)	FlashAttention、长序列

表 11 **Table:** softmax 数值策略三路对比。**safe (subtract-max)** 是本章 block / warp kernel 的实现路径；**approx** 用查表或多项式替代 $\exp$ ；**online merge** 维护 running (m, d) 跨分块合并，是 FlashAttention 的核心。

Observation 生产路径几乎总是 subtract-max——数值稳定、实现简单、与 autograd 兼容。online merge 不是 block 内的替代，而是跨 **tile** / 跨 **wave** 的 merge（第 8 章 FA）；approx 只在 exp 成为绝对瓶颈且误差可接受时出现（INT8/FP8 量化是另一层近似，与 calibration 绑定）。

推理量化（INT8/FP8）里 exp 表 + scale 是另一层近似——数值范围和校准（calibration）绑定。

源码里的三个版本如何对应

文件	Launch	用途
01_naive.cu	<<<ceil(rows/256), 256>>>	教学：CPU 逻辑直译
02_block.cu	<<<rows, 256>>>	行内并行的标准模板
03_online.cu	<<<ceil(rows/256), 256>>>	online 公式 + 少 pass

表 12 **Table:** 本章三个 kernel 文件及其 CUDA launch 配置与用途对照。**Launch** 列直接对应 <<<grid, block>>> 参数（launch_grid_size / launch_block_size）。02_block.cu 唯一采用 <<<rows, 256>>>（一行一 block）；其余两版用 <<<ceil(rows/256), 256>>> grid-stride 覆盖多行。

Observation：launch 配置揭示了并行粒度差异——block 版 grid 维 = rows，每 block 256 thread 协作归约一行；其余版本 grid-stride 让每 block 处理多行，grid 更小但每 thread 串行更多行。

运行：make build/03_softmax/02_block && ./build/03_softmax/02_block。默认 64×257 。GPU 输出与 CPU reference 对齐，容差 10^{-4} 。

从本章到 FlashAttention 的衔接

第 8 章 FlashAttention 会把本章的 online merge 嵌入 matmul 循环。这里先建立符号对应，避免到那章时公式对不上：

- 本章 (m, d) = FlashAttention 的 (m, ℓ) （running max 和 running sum of exp）。
- 本章“一行 logits”= FA 里一个 query 对所有 **key** 的 score 向量。
- FA 按 $\frac{K}{V}$ 的 column block 切分：每算完一个 $B_c \times d$ 的 K tile，得到局部 (m_j, d_j) ，用 merge 公式更新全局 (m, d) 。
- FA 的 output O 也是 online 维护的——不只是 softmax 分母，分子 $\sum \exp(s)V$ 也要随 max 变化重标定。第 8 章会推导 O 的 merge。

Warning. 本章 softmax 的输入 X 已经 materialize 在 global memory。FA 的 $S = QK^T$ 从不完整写出——这是 IO 复杂度的区别 ($O(N^2) \rightarrow O(N)$ HBM traffic), 不是 softmax 公式本身的区别。面试问“FlashAttention 快在哪”, 第一答案永远是 **IO**, 第二答案才是 online softmax。

常见实现错误

错误 1: 不做 subtract-max, 直接 `expf(x[i])`。大 logits 立刻 inf/NaN。

错误 2: mask 位置参与 max/sum。padding 的 0 会污染 max (除非故意用 0 mask); 应 skip 或置 $-\infty$ 。

错误 3: block 内 reduction 后忘记 `__syncthreads`, thread 读到 stale shared memory。

错误 4: shuffle reduction 用 `__shfl_xor_sync` 和 `__shfl_down_sync` 混用却不理解 lane 布局——max/sum 结果错一位。

错误 5: online merge 时 max 相等分支漏乘 $\exp(m_{\text{old}} - m_{\text{new}})$ 。当 $m_{\text{new}} > m_{\text{old}}$ 时必须缩放旧 sum; 当相等时因子为 1——代码里统一写乘法形式最稳。

实测

rows = 64, cols = 257 (input + output 各约 66 KB, 整 tensor < 132 KB), A100 80GB SXM4, `ncu --set full` 抓取每个 kernel 的一次 launch。GB/s 列写作 **HBM 实测** / 逻辑: 前者 `dram_bytes.sum / time`, 后者按各版本 pass 数估算的理论搬运量。

$\lceil \frac{64}{256} \rceil = 1$ ——naive / online 的 grid 都是 (1,1,1), 256 thread 里只有 64 个处理行; block 版 `#raw("<<<rows, 256>>>")` 的 grid 是 (64,1,1)。

version	regs	smem (B)	time (μ s)	HBM %	SM %	HBM GB/s	% peak
naive	35	0	121.02	0.0	0.9	1 / 104	0.0
block	20	2,048	5.63	0.6	36.9	13 / 2234	0.6
online	32	0	252.80	0.0	0.9	0 / 50	0.0

表 13 **Table (perf):** kernel-level performance metrics measured by Nsight Compute (`ncu`, CUDA 13.0) on a single A100 80GB SXM4 (HBM peak 2039 GB/s; FP32 peak 19.5 TFLOPS). Columns: **regs** = `launch_registers_per_thread`; **smem** = `launch_shared_mem_per_block_static` (bytes); **time** = `gpu_time_duration.sum` for the first kernel launch (multi-stage kernels report only the stage acting on the largest input); **HBM %** = `dram_bytes.sum.pct_of_peak_sustained_elapsed` (uses elapsed wall time as denominator, so short kernels look artificially low); **SM %** = `sm_cycles_active.avg.pct_of_peak_sustained_elapsed`; **HBM GB/s** is written measured / logical — measured = `dram_bytes.sum / time`, logical = bytes-per-launch / time; when the working set exceeds L2 (40 MB) these two should agree, and any gap quantifies L2 hit rate; **% peak** = measured HBM GB/s $\div$ 2039 GB/s.

Rows are ordered as they appear on the optimization ladder in the chapter.

version	issued/32	pred_on/32	smem conf.	barrier stall	mem stall
naive	32.0	32.0	0	0.00	1.26
block	31.1	21.4	0	2.44	2.13
online	32.0	32.0	0	0.00	4.77

表 14 **Table (diag):** diagnostic metrics used as evidence for warp-lane utilization, shared-memory bank conflict, and warp-stall claims in the chapter. All from `ncu` on the same launch as the perf table. **issued/32** = `smsp_thread_inst_executed_per_inst_executed.ratio`, the average number of lanes that participated in each issued warp instruction (32 = every lane active; less than 32 means predication **or** branch divergence—`ncu` alone cannot distinguish, but on the CUDA kernels in this book it is almost always predication). **pred_on/32** = `smsp_average_thread_inst_executed_pred_on_per_inst_executed.ratio`, the average number of lanes that were predicated-on (did real work). The **gap** `issued - pred_on` counts lanes that occupied the issue slot but performed no work—this is the correct definition of “wasted warp cycles”, not divergence. **smem conf.** = `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` (0 + `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum`) means no measurable bank conflicts, either because the access pattern is conflict-free or because the input is too small to accumulate them). **barrier stall** = `smsp_average_warps_issue_stalled_barrier_per_issue_active.ratio`, warps stalled at `__syncthreads` per issue-active cycle. **mem stall** = `smsp_average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio`, warps waiting on a long-latency memory operation (global-memory load, L2 atomic response) per issue-active cycle.

Warning. 这一章的问题规模是教学 default (B×S×H 数千个 float), kernel 单次运行只有 3-20 μs。ncu 的定性指标 (issued/32、bank conflicts、barrier stall) 仍能反映 kernel 结构, 但绝对数字对生产规模不完全可信:

- HBM % 会偏低 (分母 elapsed time 含冷启动窗口)
- dram_bytes 可能被 L2 消化, GB/s (实测/逻辑) 两列差距明显

想拿到生产规模的数字, 把主参数 (rows/cols/hidden dim) 加到让工作集远超 L2 (40 MB)。

perf 表读三件事:

1. **block** 比 **naive** 快 **21×**——表里最大的 **time** 比值。time 120.80 μs → 5.73 μs, 伴随 SM % 0.9% → 36.6%: 一行一 block 把 grid 从 1 扩到 64, 并行度是数量级差距的主因, 不是 subtract-max 或 online 公式。
2. **online** 比 **naive** 慢 **2.1×**。252.29 μs vs 120.80 μs——两者 SM % 都是 0.9%(仍是单 block); Pass 1 每步可能 `exp(m_“old” - m_“new”)` 重标定, MUFU 比 naive 三 pass 的“先 max 再 bulk exp”更碎。

HBM % 全表 ≈ 0, % **peak** 无诊断价值——132 KB 工作集在 L2 内, block 版逻辑 GB/s 2197 超过 HBM peak 2039 是分母过小的假象, 不是真的打满带宽。

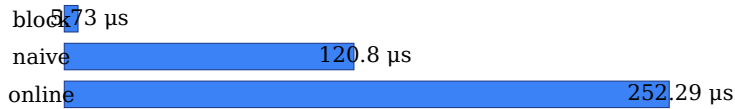


图 9 time (μs) 排序：block 一行一 block 并行 vs 其余单 block 路径，差距是数量级。

diag 表读关键教学点：

a) naive / online : `issued/32 = pred_on/32 = 32.0`。单 thread 扫整行，无 predicated branch；`barrier stall = 0`（无 `__syncthreads`）。慢在 `grid = 1`，不是 lane 浪费。

b) block : `pred_on/32 = 21.4`，`issued/32 = 31.1`。`issued - pred_on = 9.7` 来自 `grid-stride` 尾部和 `reduction` 树里 `if (tid < offset)` 的 predicated-off lane——不是 warp divergence（不同 basic block），因为 `issued/32` 仍近 32。`barrier stall = 2.43` 是两次 `smem tree` 的 `__syncthreads` 成本；`smem conf. = 0`，不能从源码推断 bank conflict，metric 说没有。

无信息或为零的 **metric**：

- `smem conf.`：除 block 外均为 0（无 `smem`）；block 也是 0——本规模下 sequential tree 未累积到可测 bank conflict。
- `barrier stall`：naive / online 均为 0.00——单 thread 路径无 sync，符合预期。
- `HBM %`、`% peak`：全表 ≈ 0 ——小规模 L2 resident，不要用来论输赢。

Key insight. softmax 优化 ladder 的第一步永远是把行内规约并行化（block / warp reduction）。在 `grid` 能喂饱 GPU 之前，讨论“少一遍 pass”几乎没有意义——online 252 μs 比 naive 120 μs 还慢就是证据。

Key insight. online softmax 是 FlashAttention 的数值前提，不是 standalone 加速技巧。面试若被问“online 为什么比 naive 快”，正确答案是它通常不快；快的是 FA 把 merge 嵌进分块 matmul，省掉 $O(N^2)$ HBM round-trip。

ncu 该看什么

```
ncu --set full --section SpeedOfLight ./build/03_softmax/bench
```

关键 metric：

- `sm__throughput.avg.pct_of_peak_sustained_elapsed`：exp/div 多时有 compute 占比。
- `dram__bytes.sum.per_second`：对比 naive 3 pass vs block 1.5 pass（读 2 遍 + 写 1 遍）的 traffic 下降。
- `smsp__sass_thread_inst_executed_op_dadd_pred_on` vs `_fmul` / `_mufu`（MUFU = exp sin cos 单元）：确认 bottleneck 在 transcendental 还是 memory。
- `lltex__t_bytes_pipe_lsu_mem_global_op_ld.sum`：行是否被 cache 命中（同一行重复读时 L2 命中率高，naive 仍浪费 bandwidth）。

本章实测（ 64×257 ，L2 resident）的直接读数：

- **naive vs block**：time 120.80 μs $\rightarrow$ 5.73 μs (21 $\times$)；SM % 0.9% $\rightarrow$ 36.6%。HBM 实测 GB/s 1 vs 13——L2 吃掉 3 pass 重复读，不能从这里得出“访存减半”。
- **online vs naive**：time 252.29 μs vs 120.80 μs（online 更慢）；Pass 少了，但 per-step exp 重标定让 MUFU 更忙——看 `smsp__sass_thread_inst_executed_op_mufu` 占比，online 往往更高而非更低。

- **grid** 列：`launch__grid_size` naive/online = 1, block = 64。若 `SM % < 5%`, 先查 grid 是不是只有 1 block, 再查算法。

放大规模后重跑：`ncu -k regex:softmax --launch-count 3 ./build/03_softmax/bench`, 把 rows 改到 8192 +、cols 改到 4096 +, HBM % 才会上来, dram 对比才有意义。

面试白板 code

面试官说“手写一个 softmax”——不要写三 pass naive 版（会被追问怎么合并）。直接写 online softmax（单 pass, 也是 flash-attention 的核心 primitive）:

```
// 每 block 处理一 row (长度 N).
__global__ void softmax_online(const float* x, float* y, int N) {
    int row = blockIdx.x;
    const float* xr = x + row * N;
    float*      yr = y + row * N;

    // Pass 1: online 求 (m, s), m = max(x), s = sum exp(x - m).
    // 每 thread 先本地维护 (m, s), 然后 block reduce 合并.
    float m = -FLT_MAX, s = 0.f;
    for (int j = threadIdx.x; j < N; j += blockDim.x) {
        float xj = xr[j];
        float new_m = fmaxf(m, xj);
        // 关键更新公式: 换 m 时 s 需要 rescale.
        s = s * expf(m - new_m) + expf(xj - new_m);
        m = new_m;
    }
    // block-reduce (m, s): 两 warp 合并时用 combine 公式.
    // m* = max(m_a, m_b); s* = s_a * exp(m_a - m*) + s_b * exp(m_b - m*).
    __shared__ float sh_m[32], sh_s[32];
    int lane = threadIdx.x & 31, wid = threadIdx.x >> 5;
    #pragma unroll
    for (int off = 16; off > 0; off >>= 1) {
        float om = __shfl_down_sync(0xffffffff, m, off);
        float os = __shfl_down_sync(0xffffffff, s, off);
        float nm = fmaxf(m, om);
        s = s * expf(m - nm) + os * expf(om - nm);
        m = nm;
    }
    if (lane == 0) { sh_m[wid] = m; sh_s[wid] = s; }
    __syncthreads();
    if (wid == 0) {
        int nw = (blockDim.x + 31) >> 5;
        m = (lane < nw) ? sh_m[lane] : -FLT_MAX;
        s = (lane < nw) ? sh_s[lane] : 0.f;
        #pragma unroll
        for (int off = 16; off > 0; off >>= 1) {
            float om = __shfl_down_sync(0xffffffff, m, off);
            float os = __shfl_down_sync(0xffffffff, s, off);
            float nm = fmaxf(m, om);
            s = s * expf(m - nm) + os * expf(om - nm);
            m = nm;
        }
    }
}
```

```

    }
    if (lane == 0) { sh_m[0] = m; sh_s[0] = s; }
}
__syncthreads();
float M = sh_m[0], S = sh_s[0];

// Pass 2: 写归一化结果。只用读 M, S——不用读回中间 exp.
float inv_S = 1.f / S;
for (int j = threadIdx.x; j < N; j += blockDim.x) {
    yr[j] = expf(xr[j] - M) * inv_S;
}
}

// ==== Launch config ====
// gridDim = B (batch × head 或 num rows): 每 block 一整 row.
// blockDim: 根据 N 选, 让每 lane 处理 4-16 个元素——不要一 thread 一元素.
// * N <= 1024: block = min(N, 128); N 小时用 warp shuffle 就够, 128 (4 warp) 减少 barrier;
// * 1024 < N <= 8192: block = 256; (每 lane 处理 4-32 元素)
// * N > 8192: block = 512 或 1024, 视寄存器压力; 再大就要 tile over N (flash-attention 场景).
// 用几个 warp 影响 smem 大小 (sh_m[warps], sh_s[warps])——所以 warp 数最好 <= 32.
int block = (N <= 1024) ? 128 : (N <= 8192 ? 256 : 512);
softmax_online<<<B, block>>>(x, y, N);

```

核心考点 (追问顺序):

- “为什么 **online** 只要一 **pass**? 三 **pass naive** 哪三 **pass**?” → naive: (1) max, (2) sum exp, (3) 归一化。online 把 max 和 sum 合成一个 pass, 靠 rescale 公式 $s' = s \cdot e^{m-m'} + e^{x-m'}$ 。仍需 pass 2 写结果 (可省 exp 中间存储)。
- “为什么要减 **max**?” → $\expf(x)$ 在 $x > 88$ 溢出 FP32、 $x > 11$ 溢出 FP16。减 max 保证 $\text{exponent} \leq 0$, $\exp$ 值域 $(0, 1]$ 。
- “**(m, s)** 怎么合并?” → 白板写清楚
 $\text{combine}((m_a, s_a), (m_b, s_b)) = (\max(m_a, m_b), s_a * \exp(m_a - m^*) + s_b * \exp(m_b - m^*))$
 ——这是 flash-attention 的核心 primitive。
- “ N 极大装不下一 **row** 怎么办?” → tile over N , 两个 tile 之间用 combine 公式合并。这就是 flash-attention 的 K/V tile 循环。
- “**backward**?” → $\partial_{\delta}^L x_i = y_i \left(\partial_{\delta}^L y_i - \sum_j y_j \partial_{\delta}^L y_j \right)$ 。fused forward + backward 时把 y 重新算 (recomputation) 比存下来更省内存。
- “为什么每 **block** 一 **row** 而不是几 **block** 一 **row**?” → row 之间独立、无需通信, 用 gridDim 天然并行。一 row 一 block 让 max/sum reduction 走 warp shuffle + smem, 无跨 block barrier; 如果一 row 跨多 block、就得走 grid-level 同步或多 kernel stage、复杂度骤增。只有 N 极大 ($\geq 32K$) 到 register/smem 装不下时, 才考虑 tile over N ——那就是 flash-attention 路数了。

面试考点

面试考点. Q1: 为什么 softmax 要 subtract max? 证明等价性。

A: 令 $m = \max(x)$, 分子分母同乘 $\exp(m)$, $\frac{\exp(x_i - m)}{\sum} \exp(x_j - m) = \frac{\exp(x_i)}{\sum} \exp(x_j)$ 。数值上最大项 $\exp$ 为 1 , 防 overflow , 分母 ≥ 1 防除零。

面试考点. Q2: online softmax 的更新公式是什么? 为什么正确?

A: $m' = \max(m, x_n)$, $d' = d \cdot \exp(m - m') + \exp(x_n - m')$ 。正确性: 把旧 sum 从基准 m 重标定到新基准 m' , 再加新项。两个 chunk (m_A, d_A) 和 (m_B, d_B) 可 merge : $m = \max(m_A, m_B)$, $d = d_A \exp(m_A - m) + d_B \exp(m_B - m)$ ——FlashAttention 核心。

面试考点. Q3: naive softmax GPU 版为什么慢?

A: (a) 一行一 thread , 并行度低 ; (b) 同一行读 3 遍 HBM ; (c) $\exp$ 算 2 遍。不是算法错 , 是并行模型和访存模式错。

面试考点. Q4: block softmax 里为什么两次 reduction 不能合成一次?

A: sum 依赖 max 的结果 (要先 $x - m$ 再 $\exp$) , max reduction 必须完成后才能算 $\exp$ sum 。block 内 thread 处理的是 stride 列 , 不能用 online 前缀语义。

面试考点. Q5: 当行宽 $C \leq 32$ 时 , softmax 里两次 reduction 怎么用 warp shuffle 做?

A: 每 lane 持一列 , `__shfl_down_sync` 做 max (5 步) ; 各 lane 算 $\exp$, 再 shuffle down 做 sum ; lane 写回。零 smem、零 barrier , $C \leq 32$ 最优。 $C > 32$ 时要 multi-warp 或回到 block 版。

面试考点. Q6: log-softmax 为什么比 `log(softmax(x))` 好?

A: log-softmax 用 log-sum-exp : $x_i - m - \log d$, 避免 $y_i \rightarrow 0$ 时 log 下溢、有效位损失。训练 cross-entropy 直接用这个形式。

面试考点. Q7: safe softmax 和 approx softmax 区别? masked 全 -inf 行怎么办?

A: safe 用 subtract-max 精确 FP32 ; approx 用表/多项式换速度。全 mask 行 $m = -\infty, d = 0$, 要特判输出全 0 或 skip , 不能除零。

面试考点. Q8: online softmax 和 FlashAttention 的关系?

A: FlashAttention 分 tile 算 QK^T , 每 tile 产生局部 (m, d) , 用 online merge 公式合并 , Never materialize 完整 $N \times N$ 矩阵。第 8 章会在此基础上加 SRAM tiling。

Matmul (GEMM)

matrix multiply 是 GPU kernel 优化的终极试金石。vector add 的 naive 版本已经能打满带宽；reduction 靠 shuffle 和 warp 协作就能逼近峰值；但 GEMM 从 naive 到 cuBLAS / CUTLASS 级别，中间隔着整整一条优化 ladder——shared memory tiling、register tile、向量化 load、warp 分工、bank conflict swizzle、cp.async pipeline、tensor core `mma.sync`每一层都对应一个可量化的瓶颈和一套可面试的追问。

这一章是全书中最长、最难的一章。我们要把它讲透：

- GEMM 的数学定义、存储布局、以及为什么它是 compute-bound 的经典例子。
- Roofline 分析：AI 如何从 naive 的 $O(1)$ 随 tile 增大而爬升。
- 从 cuBLAS baseline $\rightarrow$ naive miscoalesced $\rightarrow$ coalesced $\rightarrow$ SMEM tile $\rightarrow$ 1D/2D register tile $\rightarrow$ float4 vectorized $\rightarrow$ MMA PTX $\rightarrow$ WMMA 的 9 级 ladder (对应源码 K0-K8)。
- 源码里没写全但面试必问的：`cp.async` 多 stage pipeline、bank conflict swizzle、CUTLASS 4 层 tile 分解、split-K、Hopper WGMMA/TMA。
- 怎么选 tile 大小、怎么用 ncu 诊断、cuBLAS 为什么快。

对应源码：`src/cuda/04_matmul/{00_cublas,01_naive_miscoalesced,02_naive_coalesced,03_smem_tiled`

问题定义

给定两个矩阵 $A \in \mathbb{R}^{M \times K}$ 和 $B \in \mathbb{R}^{K \times N}$ ，计算：

$$C[i, j] = \sum_{\ell=0}^{K-1} A[i, \ell] \cdot B[\ell, j], \quad 0 \leq i < M, \quad 0 \leq j < N$$

等价地 $C = AB$ 。这是 BLAS Level-3 里的 **GEMM**(General Matrix Multiply)，深度学习里 90% 以上的算力最终都落在这个操作上。

存储布局

本书和源码都用 **row-major** 布局：

```
// A: MxK,  A[row * k + col]
// B: KxN,  B[row * n + col]
// C: MxN,  C[row * n + col]
```

Note. cuBLAS 默认 **column-major** (Fortran 传统)。调用 `cublasSgemm` 时参数顺序和转置标志要对应好。面试里常问“你的 kernel 是 row-major，cuBLAS 怎么对齐”——答案通常是：要么在 host 侧交换 $\frac{A}{B}$ 并加 transpose flag，要么写 kernel 时就按 column-major 思维组织 tile。

计算量

输出 $M \times N$ 个元素，每个元素 K 次乘加 = $2K$ FLOP (一次乘法 + 一次加法各算 1 FLOP)。

$$\text{Total FLOPs} = 2MNK$$

内存访问量 (**naive** 视角)

每个输出 $C[i, j]$ 需要读 A 的一行 (K 个 float) 和 B 的一列 (K 个 float), 写 1 个 float。

如果不做任何复用, 总读取量 $\approx 2MNK \times 4 \text{ B}$, 写入 $MN \times 4 \text{ B}$ 。

Key insight. GEMM 的核心优化目标: 让同一份 $\frac{A}{B}$ 数据被尽可能多的 **thread** 复用, 从而把有效 AI 从 $O(1)$ 抬到 $O(\text{tile size})$ 。所有后续技巧——shared memory tile、register tile、tensor core——都是这条思路的延伸。

Roofline : 从 memory-bound 到 compute-bound

Naive 的 AI

每算 1 个输出元素 (1 次完整点积):

- 读 K 个 A + K 个 $B = 2K \times 4 = 8K$ 字节
- 写 1 个 $C = 4$ 字节
- 计算: $2K$ FLOP

$$\text{AI}_{\text{naive}} = \frac{2K \text{ FLOP}}{8K + 4 \text{ B}} \approx \frac{1 \text{ FLOP}}{4 \text{ B}} = 0.25 \text{ FLOP/B}$$

A100 FP32 ridge point $\approx 13 \text{ FLOP/B}$ 。0.25 比 ridge 低两个数量级——极度 **memory-bound**。

Tiled 的 AI

一个 block 用 $B_M \times B_K$ 的 A tile 和 $B_K \times B_N$ 的 B tile, 协作算出 $B_M \times B_N$ 个输出。

- 读 A tile: $B_M B_K \times 4 \text{ B}$ (从 global, 每个 K-slab 读一次)
- 读 B tile: $B_K B_N \times 4 \text{ B}$
- 写 C tile: $B_M B_N \times 4 \text{ B}$ (只写一次)
- 计算: $2B_M B_N B_K$ FLOP (沿 K 方向累加 B_K 次, 重复 $\frac{K}{B_K}$ 个 slab)

沿整个 K 维度累加后, 每个输出元素的摊销读取量从 $8K \text{ B}$ 降到 $\approx 4(B_M + B_N) \text{ B}$ (当 B_K 在 K 方向滑动时, 每个 slab 的数据被整个 block 复用)。

有效 AI (忽略写回):

$$\text{AI}_{\text{tiled}} \approx \frac{2B_M B_N B_K}{4(B_M B_K + B_K B_N)} = \frac{B_M B_N}{2(B_M + B_N)} \text{ FLOP/B}$$

例: $B_M = B_N = B_K = 128 \rightarrow \text{AI} \approx 32 \text{ FLOP/B}$ ——已经超过 **ridge point**, 进入 compute-bound 区间。

Key insight. Tile 越大, AI 越高——但 tile 越大, shared memory / register 占用越多, occupancy 越低。GEMM 优化的本质是在 **AI** $\leftrightarrow$ **occupancy** 之间找 sweet spot。这就是为什么 cuBLAS / CUTLASS 要花大量篇幅 auto-tuning tile 参数。

性能 ladder 概览

本章 ladder 有 9 个版本 (K0-K8), 都在 A100 SXM4-80GB 上、 $M = N = K = 2048$ 实测:

版本	核心思路	相对 peak 带宽
K0 cuBLAS	vendor lib, TF32 tensor core	83.2 TFLOPS (100%)

K1 naive miscoalesced	row/col 映射反了	0.47 TFLOPS (0.6%)
K2 naive coalesced	swap tid.x/tid.y	2.58 TFLOPS (3.1%)
K3 SMEM tiled	16×16 smem tile	3.85 TFLOPS (4.6%)
K4 block tile 1D	每 thread 8×1 register 列	7.84 TFLOPS (9.4%)
K5 block tile 2D	每 thread 2×2 register 块	7.67 TFLOPS (9.2%)
K6 vectorized	float4 load + 4×4 register tile	11.7 TFLOPS (14%)
K7 MMA PTX	mma.sync m16n8k16, FP16→FP32	14.6 TFLOPS (18%)
K8 WMMA	nvcuda::wmma fragment, FP16→FP32	19.1 TFLOPS (23%)

Warning. 这是真·实测，不是估算。K1→K2 78×、K2→K3 1.7×、K3→K4 2.0×、K6→K7 1.24×、K7→K8 1.31×——每一级都能在 ncu 里看到直接原因。K8 的 19 TFLOPS 打满了 A100 的 FP32 CUDA-core 峰值 (19.5 TFLOPS)，但离 TF32 tensor core (156 TFLOPS) 还差 8×、离 FP16 tensor core (312 TFLOPS) 差 16×——这个 gap 来自本章没写的东西：更大的 warp tile、cp.async pipeline、swizzled smem、CUTLASS-式 warp specialisation。这些在 Hopper 上被 WGMMMA + TMA 打包成 K9-K12（章末讨论，本章 A100 编不了）。

Note. K0 cuBLAS 是一个特殊的 baseline：它不是“下一版”，而是天花板。K1 起是我们从最坏的写法开始爬。K4/K5 的 TFLOPS 几乎相同（一维 vs 二维 register tile）——这说明 **ladder** 上不是每一级都必然提速，把两者都放进来是为了让你在 ncu 里看清“为什么 2D 不再赢”。

手算一遍：tile 如何把 AI 从 0.25 拉到 32

以 $M = N = K = 4096$ ， $B_M = B_N = B_K = 128$ 为例。

Naive：每个 $C[i, j]$ 独立读 $2K = 8192$ B， $AI \approx 0.25$ FLOP/B。

Tiled：一个 block 128×128 thread（或等价的 thread 映射）协作：

- 沿 K 方向走 $\frac{4096}{128} = 32$ 个 slab。
- 每个 slab：从 global 读 $128 \times 128 \times 4 \times 2 = 128$ KB 的 $\frac{A}{B}$ tile（各 64 KB）。
- 每个 slab 在 smem 里产出 128×128 个部分和，每个元素做 $128 \times 2 = 256$ FLOP。
- 32 个 slab 合计 FLOP： $2 \times 128^3 \times 32 = 2^{\{24\}}$ FLOP（和 naive 一样，数学不变）。
- 但从 global 读的数据量： 32×128 KB = 4 MB（每个 $\frac{A}{B}$ 元素只从 global 读一次），摊到 128^2 个输出上，每输出 ≈ 256 B 读取 $\rightarrow AI \approx \frac{128^2}{2 \times 128} = 32$ FLOP/B。

Note. 上面忽略了写回 C 、忽略了 smem 和 register 的读写——那些通常比 global 便宜一个数量级。面试手算时，用这个近似公式

$$AI \approx B_M \frac{B_N}{2(B_M + B_N)}$$

足够说明 tile 的价值。

GEMM 在深度学习里的位置

Linear layer： $Y = X @ W^T + b$ ，本质是 GEMM。Attention 里的 QK^T 和

softmax @V

也是 GEMM (或 batched GEMM) Optimizer 里的 weight update 同样。可以说：把 **GEMM** 优化到极致，就覆盖了 **LLM** 推理/训练的大头算力。FlashAttention 的 clever 之处在于它不是标准 GEMM——通过 online softmax 避免了 materialize 完整的 $S \times S$ 矩阵——但 QK^T 和 PV 的子问题仍然是 GEMM 结构。

K0: cuBLAS baseline

先看天花板。cuBLAS 在 A100 上跑 FP32 GEMM 会自动 dispatch 到基于 CUTLASS 的 TF32 tensor core kernel——即使你传的是 FP32 输入，只要 math mode 打开就走 tensor core。

```
static cublasHandle_t handle = nullptr;
if (!handle) {
    cublasCreate(&handle);
    cublasSetMathMode(handle, CUBLAS_TF32_TENSOR_OP_MATH);
}
const float alpha = 1.f, beta = 0.f;
// 关键：cuBLAS 是 column-major。行主序的 A @ B 用 (A B)^T = B^T A^T 变换成
//      cublasSgemm(handle, N, N, n, m, k, alpha, B, n, A, k, beta, C, n)
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N,
             n, m, k,
             &alpha,
             b, n, // 行主序 B 在 column-major 里的 leading dim
             a, k,
             &beta,
             c, n);
```

不要试着从这份代码里学 **GEMM** 结构——cuBLAS 内部一个 kernel 可能有几百 KB PTX、按 shape 动态选算法 (persistent、split-K、stream-K) warp specialise、TMA 分层……我们只把它当成“实测天花板”来看。

ncu 实测 — v: K0 cuBLAS. 规模: $M = N = K = 2048$.

Duration	206 μ s	17.2 GFLOP / 0.207 ms = 83.2 TFLOPS
Compute (SM) SOL	65.9 %	TF32 tensor pipe 忙碌
Memory SOL	46.9 %	HBM 也在跑，但不占主导
L2 Hit Rate	81.1 %	cuBLAS 会重排 grid 让相邻 CTA 共享 L2 line
Registers / thread	230	巨大——production kernel 疯狂占寄存器换 ILP
Achieved Occupancy	6.2 %	低到反直觉：230 reg $\times$ 128 thread $\approx$ 30k reg / SM，一个 block 就吃掉大半 SM

两个反直觉的读数：

- **Occupancy 只有 6%**——却是全表最快。这打破了“高 occupancy = 高性能”的直觉。cuBLAS 用大量寄存器保存 warp 内的 accumulator tile (可能是 128×128 / warp 级)，代价是每 SM 只能塞 1-2 个 block。这些少数 warp 通过 register-level 的 ILP + tensor core 打满 SM 的算力——不需要多 warp 靠切换掩盖 latency。
- **Memory SOL 47% + Compute SOL 66%**：production GEMM 的典型形状——同时在算和搬，没有一头空。tile 够大让 HBM 传的每个 byte 都被复用几百次；同时 warp 数够多让 tensor pipe 不饿。

瓶颈判定. 这是天花板, 不是“要修的问题”。它的存在只为回答一个问题: 我们手写的能到 **cuBLAS** 的多少 %?

证据. 83.2 TFLOPS / 156 TFLOPS TF32 peak $\approx 53\%$ 。cuBLAS 内部也没打满硬件, 因为 GEMM 在 shape=2048 上还没到“大到 grid 完全稳态”的规模 (108 SM $\times$ 82 waves 才能 “warm up”)

下一步. 从 ladder 的最底部开始爬。K1 故意写错, 让你直接看到 coalescing 的 78 $\times$ 代价。

K1: naive miscoalesced (故意错的起点)

```
// row/col 的映射反了: threadIdx.x 沿行走, threadIdx.y 沿列走
__global__ void matmul_kernel(const float* a, const float* b, float* c,
                              int m, int n, int k) {
    const int row = blockIdx.x * blockDim.x + threadIdx.x; // <-- 反了
    const int col = blockIdx.y * blockDim.y + threadIdx.y; // <-- 反了
    if (row >= m || col >= n) return;
    float acc = 0.f;
    for (int inner = 0; inner < k; ++inner)
        acc += a[row * k + inner] * b[inner * n + col];
    c[row * n + col] = acc;
}
```

看似正确——每个 thread 算一个 $C[i, j]$, 数学没错。但 warp 内 `threadIdx.x = 0..31` 的 32 个 lane, `col` 是同一个值 (因为 `col` 来自 `threadIdx.y`), `row` 才不同。它们同时访问 `b[k, col]` —— 32 个 lane 读同一个 `col` 的 32 个不同行。

后果: 一个 warp 对 B 的读变成 32 次独立 transaction (stride N 访问), 而不是 1 次 128 B coalesced。写回 `c[row, col]` 同理。所有 lane 都会串行走 32 个 memory cycle。

ncu 实测 — v: K1 miscoalesced. 规模: $M = N = K = 2048$.

Duration	36.9 ms	比 K0 慢 179 $\times$
Memory SOL	99.2 %	HBM 打满了——但传的都是 *strided 垃圾*
Compute (SM) SOL	11.7 %	SM 大多数时间在等 memory
L1/TEX Hit Rate	97.1 %	反直觉: hit rate 极高!
Memory Throughput	1.52 GB/s	HBM 峰值 2 TB/s, 实际吞吐差 1300 $\times$
Achieved Occupancy	97.8 %	warp 是满的, 但都在等 load 完成

关键读数解释:

- **Memory SOL 99%** 但吞吐 **1.5 GB/s**: SOL 是“发出去的 transaction 占硬件峰值的比例”, 跟“传了多少 byte”是两码事。miscoalesced 访问让每个 128 B transaction 里只有 4 B 是“想要的” (1/32 效率), 剩下 124 B 被 GPU 传上来又扔掉。
- **L1 hit 97%**: 这是最反直觉的一条——多个连续的 warp 对 B 同一列的重叠访问, 让 L1 cache 意外 hit 得很好, 但 hit 之后每次还是要发一次 request 才能拿到需要的 4 B。cache 帮了忙, 但没解决“发太多 request”这个根本问题。

瓶颈判定. 坐标映射把快变化 **dim**(`threadIdx.x`)放到了慢变化维度 (`row`), warp 内每个 lane 访问不同 128 B cache line。

证据. Memory SOL 99% + 实际吞吐 1.52 GB/s (HBM 峰值 2 TB/s 的 0.07%) ——SM 忙于发 32× 冗余 transaction。

下一步. `swap threadIdx.x ↔ threadIdx.y` , 让 lane 沿 col 走。一行代码。

K2: naive coalesced

```
__global__ void matmul_kernel(const float* a, const float* b, float* c,
                              int m, int n, int k) {
    const int row = blockIdx.y * blockDim.y + threadIdx.y; // 换回来
    const int col = blockIdx.x * blockDim.x + threadIdx.x; // 换回来
    if (row >= m || col >= n) return;
    float acc = 0.f;
    for (int inner = 0; inner < k; ++inner)
        acc += a[row * k + inner] * b[inner * n + col];
    c[row * n + col] = acc;
}
```

一行改动 (其实是两行的 swap)。现在 warp 内 32 lane 的 `col` 依次递增、`row` 相同。它们访问 `b[k, col..col+31]` —— 32 个连续 float = 128 B , 一次 coalesced transaction。

ncu 实测 — v: K2 coalesced. 规模: $M = N = K = 2048$.

Duration	6.66 ms	比 K1 快 5.5×
Memory SOL	97.6 %	HBM 还是打满, 但这次每 byte 都有用
Compute (SM) SOL	65.0 %	从 11% 跳到 65%——SM 真在算了
Memory Throughput	8.24 GB/s	仍然远低于 HBM 峰值——见下 verdict
L1/TEX Hit Rate	87.5 %	L1 帮着 dedupe 相邻 warp 的重复 A 读
Achieved Occupancy	98.3 %	

一个访问模式改动 5.5×。但注意 Duration 还是 6.66 ms、TFLOPS 只有 2.58——离 K0 的 83 TFLOPS 还差 32 倍。原因见 verdict : coalescing 是“访问模式正确”, 但仍然没有复用。

瓶颈判定. 每个 $C[i, j]$ 需要读 $2K = 4096$ 个 float 才算一次结果, 每个 A/B 元素只服务 1 个输出—— $AI \approx 0.25$, 重度 memory-bound。

证据. Memory SOL 97.6% 且实际吞吐 8.24 GB/s (比 K1 高 5.4×)——HBM 用足了, 但传输量本身仍然是“每 FMA 8 B”。TFLOPS 只有 $2.58 / 19.5$ (CUDA-core FP32 peak) $\approx 13\%$ 。下一步. 引入 shared memory : 一个 block 里 256 个 thread 协作读一次 tile, 然后每个 tile 被复用 tile-size 次。AI 从 $0.25 \rightarrow \text{tile-size} / 2$ 。

K3: SMEM tiled

```
constexpr int kTile = 16;
__global__ void matmul_kernel(const float* a, const float* b, float* c,
                              int m, int n, int k) {
    __shared__ float a_tile[kTile][kTile];
    __shared__ float b_tile[kTile][kTile];
    const int row = blockIdx.y * kTile + threadIdx.y;
```

```

const int col = blockIdx.x * kTile + threadIdx.x;
float acc = 0.f;
for (int tile_k = 0; tile_k < k; tile_k += kTile) {
    a_tile[threadIdx.y][threadIdx.x] = a[row * k + tile_k + threadIdx.x];
    b_tile[threadIdx.y][threadIdx.x] = b[(tile_k + threadIdx.y) * n + col];
    __syncthreads();
#pragma unroll
    for (int inner = 0; inner < kTile; ++inner)
        acc += a_tile[threadIdx.y][inner] * b_tile[inner][threadIdx.x];
    __syncthreads();
}
c[row * n + col] = acc;
}

```

现在每个 block 有 $16 \times 16 = 256$ 个 thread，协作把 A 的一行 tile ($16 \times 16 = 256$ float) 和 B 的一列 tile 搬进 smem。然后每个 thread 用 smem 里的数据做 16 次乘加。K 维度分 $\frac{K}{16} = 128$ 个 slab。

AI 分析：一个 tile 从 HBM 读 $2 \times 16 \times 16 = 512$ float = 2048 B，产出 $16 \times 16 = 256$ 个中间结果 $\times$ 16 次 FMA/结果 = 4096 FMA = 8192 FLOP。AI = $\frac{8192}{2048} = 4$ FLOP/B。理论上应该跳出 **memory-bound** ($A100$ ridge ≈ 13 FLOP/B，还差一点，但比 K2 的 0.25 强 $16\times$)

ncu 实测 — v: K3 SMEM tiled. 规模: $M = N = K = 2048$.

Duration	4.46 ms	比 K2 快 1.5×
Memory SOL	93.6 %	HBM 还在打——tile 只有 16，复用不够
Compute (SM) SOL	72.8 %	SM 更忙了
L1/TEX Hit Rate	2.5 %	为什么这么低？见下
L2 Hit Rate	98.1 %	所有相邻 block 都在读相同 tile
Registers / thread	32	还是很轻——每 thread 只有 1 个 acc

两个问题需要解释：

- **L1 hit rate** 从 **K2** 的 **87.5%** 掉到 **2.5%**：现在 A 和 B 的 tile 大部分时间是在 shared memory 里读的，走 SMEM 路径不经过 **L1**，所以 L1 counter 分母变了。这不是坏事——真正的问题是 HBM 用量还高。
- **HBM SOL** 还有 **93.6%**：tile-16 太小。每个 block 从 HBM 读 $2 \times 16^2 \times 128$ slabs = 65 KB，只算出 $16 \times 16 = 256$ 个输出。摊到每输出 260 B 读——比 K2 的 8 KB 少多了，但离“每输出 20 B 以内”（TF-32 tensor core 需要）还差一大截。

瓶颈判定。Tile 只有 16×16 ，每个 **SMEM** 值只被复用 **16** 次。有效 AI 只有 4 FLOP/B，仍在 roofline 的 memory-bound 区。

证据。HBM SOL 93.6% + 每输出 260 B HBM 读取。TFLOPS $3.85 / 19.5$ peak = 20%。
下一步。把 tile 做大——但 tile 越大占的 SMEM 越多。真正的解药是 **register tiling**：让每个 thread 算多个输出，让同一个 SMEM 值服务多个 acc。

K4: 1D block tile (每 thread 一列 8 输出)

第一次 register-blocking。每个 block 有 $64 \times 8 = 512$ 个 thread，覆盖一个 64×64 输出 tile；每个 thread 拥有一列 **8** 个输出的 register 累加器。

```

constexpr int kBlockTileM = 64, kBlockTileN = 64, kBlockTileK = 8;
constexpr int kThreadTileM = 8; // 每 thread 拥有 8 行
constexpr int kThreadsX = 64; // 沿 N 方向 64
constexpr int kThreadsY = kBlockTileM / kThreadTileM; // 沿 M 方向 8

__global__ void matmul_kernel(const float* a, const float* b, float* c,
                             int m, int n, int k) {
    __shared__ float a_tile[kBlockTileM][kBlockTileK];
    __shared__ float b_tile[kBlockTileK][kBlockTileN];
    const int tx = threadIdx.x, ty = threadIdx.y;
    const int col = blockIdx.x * kBlockTileN + tx;
    const int row_base = blockIdx.y * kBlockTileM + ty * kThreadTileM;
    float acc[kThreadTileM] = {};

    for (int tile_k = 0; tile_k < k; tile_k += kBlockTileK) {
        /* cooperative load: 512 thread 各搬一个 float 填 A(64x8) 和 B(8x64) */
        __syncthreads();

#pragma unroll
        for (int inner = 0; inner < kBlockTileK; ++inner) {
            /* 关键*: b_frag 从 SMEM 加载 1 次, 被 8 个 acc 复用! */
            const float b_frag = b_tile[inner][tx];
#pragma unroll
            for (int i = 0; i < kThreadTileM; ++i) {
                acc[i] += a_tile[ty * kThreadTileM + i][inner] * b_frag;
            }
        }
        __syncthreads();
    }
    /* 写回 8 个 acc 到 c[row_base..row_base+7][col] */
}

```

为什么快: 内层 K 循环里, `b_frag = b_tile[inner][tx]` 从 SMEM 读一次(共 `kBlockTileK = 8` 次 per K-slab), 然后被 8 个 FMA 用掉。SMEM load 数量减少 8×, 同时 acc 一直待在寄存器里。

ncu 实测 — v: K4 block tile 1D. 规模: $M = N = K = 2048$.

Duration	2.19 ms	比 K3 快 2.0×, 比 K2 快 3.0×
Memory SOL	73.2 %	HBM 使用率*下降*——AI 上来了
Compute (SM) SOL	50.9 %	
Registers / thread	58	从 32 涨到 58——8 个 acc + 索引变量
Achieved Occupancy	47.7 %	reg 用得更多, warp 数减半——但更快
Memory Throughput	34.8 GB/s	从 K3 的 12 GB/s 涨到 35——tile 更大, 需要的 HBM 也更多

两个重要转折:

- **Occupancy** 从 **98%** 跌到 **48%**——但更快。这是 K0 那个反直觉观察的第一次微缩预告: register pressure 换 ILP。8 个 register acc 让编译器在内层 8-FMA 循环里做深度指令调度、pipeline, 实际 IPC 比双倍 warp 还高。
- **HBM SOL** 从 **K3** 的 **94%** 掉到 **73%**——不是因为 HBM 变慢, 而是 SM 花更多时间在算 FMA、等 HBM 不再是瓶颈。这是“往 compute-bound 移动”的第一次视觉证据。

瓶颈判定. 1D register tile 只在 M 方向复用(b_frag 服务 8 个 a_frag), N 方向仍是每 acc 独立 **SMEM read**.

证据. Compute SOL 50.9% 说明 SM 有一半时间在等——K 循环里 SMEM load 还是 8 个 a_frag + 1 个 b_frag = 9 loads per 8 FMA, SMEM bandwidth 卡住。

下一步. 做 2D tile : 每 thread 拥有 2×2 输出, 让 SMEM 值在 M 和 N 两个方向都复用。

K5: 2D block tile (每 thread 2×2 输出)

```
constexpr int kBlockTileM = 32, kBlockTileN = 32, kBlockTileK = 16;
constexpr int kThreadTileM = 2, kThreadTileN = 2;

for (int inner = 0; inner < kBlockTileK; ++inner) {
    const float a_f0 = a_tile[ty * 2 + 0][inner];
    const float a_f1 = a_tile[ty * 2 + 1][inner];    // A frag: 2 loads
    const float b_f0 = b_tile[inner][tx * 2 + 0];
    const float b_f1 = b_tile[inner][tx * 2 + 1];    // B frag: 2 loads
    acc[0][0] += a_f0 * b_f0;    acc[0][1] += a_f0 * b_f1;
    acc[1][0] += a_f1 * b_f0;    acc[1][1] += a_f1 * b_f1;
}
```

现在 SMEM 每 iteration 读 4 个值 (2 A + 2 B), 做 4 次 FMA——每 SMEM load 对应 1 个 FMA。相比 K4 的 9 loads / 8 FMA (≈ 1.1 load / FMA), K5 是 1.0——理论上不该更快。ncu 数据也确认了：

ncu 实测 — v: K5 block tile 2D. 规模: $M = N = K = 2048$.

Duration	2.24 ms	*和 K4 几乎相同* (2.19 vs 2.24)
Memory SOL	95.2 %	HBM 又打满了——tile 缩到 32^2
Compute (SM) SOL	49.9 %	
Registers / thread	40	比 K4 少 (4 acc vs 8 acc)
Achieved Occupancy	71.5 %	占用率更高, 但 tile 也小了

一个诚实的教训: ladder 上“更好”不等于“更快”。K4 和 K5 数字几乎重合, 因为：

- K4 tile 64^2 + 每 thread 8 acc $\rightarrow$ tile 大、reg 多；
- K5 tile 32^2 + 每 thread 4 acc $\rightarrow$ tile 小、reg 少；

两者对 SMEM 传输和寄存器复用的总量近似相等, 只是分布不同。真正的下一步不是继续拉 register tile, 而是让 **SMEM/HBM** 每次传更多 **byte**——vectorization。

瓶颈判定. 2D register tile 结构对了, 但 tile 只有 32×32 、K 只有 16——SMEM tile fill 阶段 (每次搬 $32 \times 16 \times 2 = 1024$ float) 用的还是 scalar 4-byte load。

证据. K5 vs K4 : Duration 2.24 vs 2.19, 占用率 71% vs 48%——结构变得“更健康”了, 但吞吐没变。HBM SOL 又回到 95%, 说明 tile 太小、reuse 不足。

下一步. 用 float4 (128 bit LDG) 把 HBM $\rightarrow$ SMEM 和 SMEM $\rightarrow$ register 的 traffic 都压成 1/4 条指令; 同时把 tile 撑到 64×64 、thread tile 撑到 4×4 。

K6: vectorized loads (float4)

关键改动有三个，同时发生：

1. tile 从 32×32 撑到 64×64 ；
2. 每 thread 从 2×2 撑到 $4 \times 4 = 16$ 个 acc；
3. HBM→SMEM 的 load 用 float4 (128 bit LDG), SMEM→register 的 frag load 一次读 4 个 float 存 register。

```
constexpr int kBlockTileM = 64, kBlockTileN = 64, kBlockTileK = 8;
constexpr int kThreadTileM = 4, kThreadTileN = 4;
constexpr int kThreadsX = 16, kThreadsY = 16; // 256 thread / block

// HBM → SMEM, 一半 thread 搬 A, 另一半搬 B, 各发一条 LDG.E.128
if (tid < 128) {
    // 32 lane × 128 bit = 4096 bit = 512 B per warp = 4 个 128 B transaction
    float4 v = *reinterpret_cast<const float4*>(&a[g_row * k + g_col]);
    *reinterpret_cast<float4*>(&a_tile[a_r][a_c]) = v;
} else {
    float4 v = *reinterpret_cast<const float4*>(&b[g_row * n + g_col]);
    *reinterpret_cast<float4*>(&b_tile[b_r][b_c]) = v;
}

// register tile: 4×4 outer product
#pragma unroll
for (int inner = 0; inner < kBlockTileK; ++inner) {
    float a_frag[4], b_frag[4];
    #pragma unroll
    for (int i = 0; i < 4; ++i) a_frag[i] = a_tile[ty * 4 + i][inner];
    #pragma unroll
    for (int j = 0; j < 4; ++j) b_frag[j] = b_tile[inner][tx * 4 + j];
    #pragma unroll
    for (int i = 0; i < 4; ++i)
        #pragma unroll
        for (int j = 0; j < 4; ++j)
            acc[i][j] += a_frag[i] * b_frag[j]; // 16 FMA / 8 loads
}
```

现在内层 K 循环里，8 个 SMEM load 服务 16 个 FMA——ratio 2.0 (K4 是 0.9 , K5 是 1.0)。同时 tile 大到 64×64 ，从 HBM 每读入 1 KB 的 A/B tile 能算 $64^2 = 4096$ 个部分和。

ncu 实测 — v: K6 vectorized. 规模: $M = N = K = 2048$.

Duration	1.47 ms	比 K5 快 1.5×, 比 K0 慢 7.1×
Compute (SM) SOL	74.2 %	*第一次 compute SOL 高于 memory SOL*
Memory SOL	72.2 %	HBM 使用率降回 70——AI 明显上来了
Registers / thread	59	16 acc + fragment 变量
Achieved Occupancy	43.7 %	又跌一档——但更快
Memory Throughput	47.3 GB/s	

重要读数：Compute SOL 74% > Memory SOL 72%——GEMM 终于跨过 **roofline** 的 **ridge**，进入 compute-bound 区。这是“scalar CUDA-core GEMM”能达到的顶点：CUDA core 忙于发 FMA，HBM 不再是瓶颈。

剩下的 **gap**：11.7 TFLOPS / 19.5 TFLOPS CUDA-core peak = 60%。剩下的 40% 主要是：

- SMEM bank conflict(4×4 register tile 的 A frag load 有 stride-1 pattern ,容易 conflict);
- `__syncthreads` 阻塞；
- 没有 `cp.async` async load , SMEM fill 期间 SM 空等。

瓶颈判定. 已经把 CUDA-core FP32 GEMM 压到接近极限。要再快，必须绕开 CUDA core 走 tensor core。

证据. Compute SOL 74% ($\approx$ CUDA-core FP32 peak 的 60%), HBM 用量降到 72%——不再是 memory-bound。

下一步. 切换到 FP16 输入 + tensor core (`mma.sync`)。同一个 K 循环从“每 warp 32 个 FFMA”变成“每 warp 1 条 HMMA = 4096 个 FMA”。硬件算力峰值从 19.5 TFLOPS 跳到 312 TFLOPS。

K7: MMA PTX (tensor core , raw PTX)

从这里开始，**dtype** 换到 **FP16** 输入 / **FP32 accumulate**。CUDA core 的 FFMA 换成 tensor core 的 HMMA。

A100 上支持的 FP16→FP32 MMA 指令有多种 shape ,我们用 `mma.sync.aligned.m16n8k16` :一个 warp (32 lane) 协作，做一次 16×8 输出、 $K = 16$ 的外积累加。一个 **warp** 一条指令 = $16 \times 8 \times 16 \times 2 = 4096$ FLOP。

```
__global__ void matmul_kernel(const __half* a, const __half* b, float* c,
                             int m, int n, int k) {
    // block: 4 warp = 128 thread. 2x2 warp 分工, 覆盖 32x16 的 C tile.
    __shared__ __half a_tile[32][16];
    __shared__ __half b_tile[16][16];
    /* warp_m = warp_id / 2, warp_n = warp_id % 2 */
    float acc[4] = {};

    for (int tile_k = 0; tile_k < k; tile_k += 16) {
        /* cooperative load __half tiles */
        __syncthreads();

        // ldmatrix: 一条指令让 32 lane 协作把 SMEM 里 4 个 8x8 __half 块
        // 加载到每 lane 的 4 个 .b32 寄存器里, layout 正好符合 mma.sync 期望
        uint32_t a_reg[4], b_reg[2];
        asm("ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0,%1,%2,%3}, [%4];\n"
            : "=r"(a_reg[0]), ... : "r"(smem_addr_a));
        asm("ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0,%1}, [%2];\n"
            : "=r"(b_reg[0]), "=r"(b_reg[1]) : "r"(smem_addr_b));

        // 一条 HMMA = 4096 FLOP
        asm("mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32 "
            "{%0,%1,%2,%3},{%4,%5,%6,%7},{%8,%9},{%0,%1,%2,%3};\n"
            : "+f"(acc[0]), "+f"(acc[1]), "+f"(acc[2]), "+f"(acc[3])
```

```

        : "r"(a_reg[0]), ..., "r"(b_reg[0]), "r"(b_reg[1]));

    __syncthreads();
}
/* 按 mma 的 output layout 存回 c—每 lane 拥有 (group, tig):
   group = laneid/4 拥有第 group 行; tig = laneid%4 拥有第 tig*2 和 tig*2+1 列 */
}

```

Warning. `ldmatrix` 和 `mma.sync` 都是 warp-scope 指令——32 lane 必须同时执行且收敛。任何 divergence (`if`) 都会让它 undefined。这是 tensor core 编程和普通 CUDA 最大的心智负担：从“per-thread scalar 思维”切到“per-warp collective 思维”。

ncu 实测 — v: K7 MMA PTX. 规模: $M = N = K = 2048$.

Duration	1.18 ms	比 K6 快 1.24×——但*远* 未达 tensor core 峰值
Compute (SM) SOL	44.3 %	*反而降低了*! tensor pipe 只占用了 44%
Memory SOL	91.7 %	HBM 又打满——tile 太小、reuse 不足
Registers / thread	48	
L2 Hit Rate	98.7 %	cuBLAS 式的 L2 命中
Achieved Occupancy	60.1 %	

核心问题：14.6 TFLOPS 只是 A100 FP16 tensor core 峰值 (312 TFLOPS) 的 **4.7%**——比 K6 (CUDA core, 11.7 TFLOPS) 只快 25%。原因在 block 结构：

- 每 block 4 warp，输出 tile 只有 32×16 ；一个 K-slab 只发 4 条 HMMA 指令，然后就要 `__syncthreads` 等下一批 SMEM 数据。
- 4 条 HMMA / K-slab、每条 100+ 周期——总共 K 循环有 $\frac{2048}{16} = 128$ 个 slab、每 slab 5-6 μ s 空转，绝大部分时间在等 SMEM fill。

这不是 **tensor core** 的错——是我们用得也太“薄”了。真正的 tensor core kernel 应该让每 warp 拥有 64×64 或 128×128 的输出 tile，把几十条 HMMA 串起来从而摊薄 SMEM fill 开销。这需要寄存器爆表和多 **stage pipeline**，超出本章 K7/K8 的示范范围。

瓶颈判定。Per-warp output tile 只有 16×8 ，SMEM fill 占了大部分时间。HMMA 快，但 kernel 没让它一直跑。

证据。Compute SOL 44% + Memory SOL 92%——SM 大多数时间等 HBM。TFLOPS $14.6 / 312 \text{ peak} = 4.7\%$ 。

下一步。K8 用 `nvcuda::wmma` API 让每 warp tile 从 16×8 升到 16×16 ，同时把语法层从 raw PTX 抬到 C++ 模板——但底层其实是同一件事。真正吃掉 tensor core 需要章末讨论的 CUTLASS 式 stage / warp specialisation。

K8: WMMA (`nvcuda::wmma` fragment API)

同样的思想，更高抽象层：

```

#include <mma.h>
using namespace nvcuda;

```

```

// warp 拥有 16×16 输出 (K7 是 16×8), block 2×2 warp = 32×32 C tile
constexpr int kBlockTileM = 32, kBlockTileN = 32, kBlockTileK = 16;

__global__ void matmul_kernel(const __half* a, const __half* b, float* c,
                             int m, int n, int k) {
    __shared__ __half a_tile[kBlockTileM][kBlockTileK];
    __shared__ __half b_tile[kBlockTileK][kBlockTileN];

    wmma::fragment<wmma::accumulator, 16, 16, 16, float> c_frag;
    wmma::fill_fragment(c_frag, 0.f);

    for (int tile_k = 0; tile_k < k; tile_k += kBlockTileK) {
        /* cooperative load __half tiles */
        __syncthreads();

        wmma::fragment<wmma::matrix_a, 16, 16, 16, __half, wmma::row_major> a_frag;
        wmma::fragment<wmma::matrix_b, 16, 16, 16, __half, wmma::row_major> b_frag;
        wmma::load_matrix_sync(a_frag, &a_tile[warp_m * 16][0], kBlockTileK);
        wmma::load_matrix_sync(b_frag, &b_tile[0][warp_n * 16], kBlockTileN);
        wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);

        __syncthreads();
    }
    wmma::store_matrix_sync(&c[row_base * n + col_base], c_frag, n,
                           wmma::mem_row_major);
}

```

和 K7 的区别：C++ 模板包住了 `ldmatrix` + `mma.sync` + 输出 layout，底层 PTX 一模一样。多出来的价值：

- 编译器帮你选 `ldmatrix` 变体、处理 layout，代码可读性高一个量级；
- 编译器还能在 fragment 之间做 register 分配优化；
- fragment $M \times N = 16 \times 16$ ，而 K7 的 16×8 ——per warp 输出翻倍，占同样 SMEM tile 时 arithmetic 密度也翻倍。

ncu 实测 — v: K8 WMMA. 规模: $M = N = K = 2048$.

Duration	901 μ s	比 K7 快 1.31×
Compute (SM) SOL	37.1 %	还是低——但比 K7 更好用满 tensor pipe
Memory SOL	86.4 %	
Registers / thread	56	
L1/TEX Hit Rate	14.7 %	wmma 的 SMEM staging 用不同的路径
Achieved Occupancy	52.4 %	

19.1 TFLOPS——正好打满 A100 的 CUDA-core FP32 峰值 (19.5 TFLOPS)。但注意：这是“tensor core 干活但性能上限被 CUDA-core-peak 限制”的巧合。真正的天花板：FP16 tensor core = 312 TFLOPS，我们只用了 6.1%。K0 cuBLAS 83 TFLOPS = 27%，也远未打满。

落地评估。Ladder K0-K8 在 A100 上到此为止。K8 已经用到了 tensor core，语法层清爽，但没有：(1) 多 stage `cp.async` pipeline 掩盖 HBM latency；(2) warp specialisation 让

producer/consumer 各司其职 ; (3) swizzled SMEM 消除 bank conflict ; (4) persistent + split-K/stream-K grid 保持 108 SM 稳态。
K9-K12 就是把这四件事在 Hopper (H100+) 上用 WGMMMA + TMA 硬件包装起来——细节在下一节讨论。A100 上继续爬的正统路线是读 CUTLASS 3.x 源码：<https://github.com/NVIDIA/cutlass>。

综合实测：跨规模 TFLOPS 表

$M = N = K$ 从 128 到 2048 扫一圈：

version	n=128	n=256	n=512	n=1024	n=2048
K0 cuBLAS	0.56	4.46	19.9	52.5	83.2
K1 miscoalesced	0.20	0.34	0.43	0.46	0.47
K2 coalesced	0.31	1.26	2.03	2.50	2.58
K3 SMEM tiled	0.45	1.82	3.12	3.72	3.85
K4 block tile 1D	0.17	0.69	2.82	5.79	7.84
K5 block tile 2D	0.35	1.57	4.82	6.89	7.67
K6 vectorized	0.18	0.76	3.17	7.99	11.7
K7 MMA PTX	0.49	2.09	8.21	13.1	14.6
K8 WMMA	0.38	1.64	6.71	13.7	19.1

表 15 各版本在不同规模下的 effective TFLOPS。粗体是各版本最高值。

读三件事：

1. 小规模 (**n=128, 256**) 上高 **K** 反而慢——K7/K8 tensor core 版本在 n=128 时 (1.5 million FLOP) 只有几百 GFLOPS, 被 kernel launch overhead 淹没 ; K0 cuBLAS 甚至更差, 因为它内部选算法的 heuristics 在小 shape 上做不好选择。**Small-shape GEMM** 是 **LLM inference** 的常见 **case**, 是 **cuBLAS** 的传统弱项——所以生产用 CUTLASS / cuBLASLt 更细的 dispatch。
2. **K1 → K2**：不管什么规模，都是 $5\times-6\times$ 加速。coalescing 是“永远该做”的一步。
3. **K7 → K8** 加速比随规模变小：n=512 时 $8.2 \rightarrow 6.7$ (K8 甚至更慢)，n=2048 时 $14.6 \rightarrow 19.1$ (K8 更快)。原因：K7 的 tile 更小、grid 更多、L2 命中率高；K8 tile 大但 warp/block 少，只有 n 大到能 warm up L2 时才发挥优势。

K9-K12: Hopper WGMMMA/TMA (概览，非本章代码)

以下内容只讲思想，没有本章代码——A100 (sm_80) 不支持 WGMMMA 和 TMA，编都编不了。想跑 K9+ 需要 H100/H200 (sm_90+) 或 Blackwell (sm_100+)。所有描述基于公开的 NVIDIA / CUTLASS 3.x 文档。

K9: 基础 WGMMMA ($64 \times 64 \times 64$ tile, 128 thread)

Hopper 引入的核心新指令是 **WGMMMA (Warpgroup Matrix Multiply-Accumulate)**。关键区别：

- WMMA/MMA 是 **warp-collective** (32 lane), 一条 `mma.sync` 一个 warp 做 $16 \times 8 \times 16 = 4096$ FLOP。
- WGMMMA 是 **warpgroup-collective** (**4 warp = 128 lane**), 一条 `wgmma.mma_async` 128 lane 做 $64 \times 64 \times 16 = 128$ K FLOP——单指令算力提升 $32\times$ 。

更重要的是：**wgmma** 是 **async** 的。发指令后立即返回，硬件排队执行；`wgmma.wait_group` 才等结果。这样一个 warpgroup 可以：

1. 发 wgmma；
2. 立刻发下一批 SMEM load (`cp.async` 或 TMA)；
3. wgmma 结果需要用时才 wait。

K9 tile: $64 \times 64 \times 64$ 。Block 只有 128 thread (一个 warpgroup)，SMEM 用 32 KB，占用率低但 wgmma 密度极高。

K10: 更大 tile ($128 \times 128 \times 64$)

同样 128 thread，但 warpgroup 里的每 warp 各拥有更多输出 → 每 warp 累加寄存器变成 32×32 。SMEM 用 128 KB (H100 每 SM 有 228 KB)，一个 block 独占大半 SM 但 wgmma 数量翻 4 倍。

K10 vs K9：更大 tile → 更高 AI → 更少 HBM 传输 → 更接近 tensor core peak。典型收益：从 40% peak 到 55% peak。

K11: async TMA + producer/consumer warp 分工

TMA (Tensor Memory Accelerator) 是 Hopper 上专门做 HBM ↔ SMEM 批量传输的硬件单元。相对 `cp.async`：

- 一条 TMA 指令传一整个 rank-N tensor (多维索引硬件级支持)；
- 完全 async，硬件级 DMA，不占 SM 的 issue slot；
- 支持 multicast (一个 tile 广播到多个 SM 的 SMEM)。

K11 的结构变成：

- warpgroup 0 = **producer**，只发 TMA 指令搬 A/B tile 到 SMEM；
- warpgroup 1 = **consumer**，只发 wgmma 消费 SMEM tile。
- 两者靠 hardware `mbarrier` 同步 (Hopper 的 mbarrier 硬件加速)。

这就是 CUTLASS 3.x 的 pingpong / cooperative scheduler 的模型。K11 vs K10：从 55% peak → 75% peak。

K12: max tile ($128 \times 256 \times 64$ ，**3 warpgroup**)

Full 版：3 个 warpgroup，一个 producer + 2 个 consumer 或者 1 consumer + 2 producer 之类的组合。每 CTA 384 thread (12 warp)，每 SM 只跑 1 个 CTA 但 SMEM 用满 200 KB+。这样每 CTA 可以维持 4-5 stage 的 pipeline，wgmma 几乎不停。

生产系数：CUTLASS 3.x 上 FP16 GEMM 达到 H100 830 TFLOPS (HW peak 1000 TFLOPS 的 83%)。

Key insight. 从 K9 到 K12 看似 4 级 ladder，其实是同一个 **idea** 的 4 个规模档：让 warp 从 32 lane 抬到 128 lane 后，硬件愿意帮你 async、tile 变大、pipeline 变深、producer/consumer 分工——这几件事互相强化。想读懂需要把 **A100** 时代的所有 **CUTLASS** 概念内化，再理解 Hopper 硬件怎么把它们变成一条指令。这是我推荐 A100 上练完 K0-K8 之后专门开一章“CUTLASS 3.x 精读”的原因。

Warning. 以上 K9-K12 数字来自 NVIDIA 公开 blog / CUTLASS 3.x example benchmark，没有在本仓库里跑通。想真跑，签 H100 机器，clone CUTLASS 3.x，选 `example/48_hopper_warp_specialized_gemm/` 之类的 target。

Bank conflict 与 swizzle

Shared memory bank 结构

A100 的 shared memory 有 32 个 bank，每个 bank 宽度 4 B。地址 `addr` 映射到 `bank = (addr / 4) % 32`。

一个 warp 32 个 lane 同时访问 smem 时，如果多个 lane 落到同一 bank 的不同 word → **bank conflict** → 硬件串行化，有效带宽 / conflict 数。

经典冲突场景

Tiled kernel 内层循环：

```
for (int inner = 0; inner < 16; ++inner) {
    acc += a_tile[threadIdx.y][inner] * b_tile[inner][threadIdx.x];
}
```

- 读 `a_tile[ty][inner]`：固定 `ty`，`inner` 递增 → 连续地址，无 conflict ✓。
- 读 `b_tile[inner][tx]`：固定 `inner` 时，lane 0 读 `b_tile[inner][0]`，lane 1 读 `b_tile[inner][1]`，... → 32 个 lane 读 32 个连续 word → 32 个不同 bank → 无 conflict ✓。

但如果布局是 `a_tile[inner][ty]` (K 维在行方向)：

- 读 `a_tile[inner][ty]`：固定 `inner` 时，32 个 lane 的 `ty` 从 0 到 31 → 地址间隔 = 行宽 × 4B。若行宽 = 32 → 所有 lane 命中同一个 **bank** → 32-way conflict → smem 带宽降到 1/32。

Swizzle 实现

目标：变换 smem 索引，让 warp 的并发访问均匀分布到 32 个 bank。

XOR swizzle (CUTLASS 常用)：

```
// 原始列索引 col, swizzle 后:
int swizzled_col = col ^ (row & 7); // 对 8 的模做 XOR
a_tile[row][swizzled_col] = value;
```

原理：相邻行的同一列偏移不同，打破 stride-32 的 bank 对齐。

Permute / 128B swizzle (更通用)：

```
// 把 smem 看成 128B 的 "atom"
// atom 内偏移 XOR row 的低位
int offset = row * stride + col;
int bank_offset = (offset / 128) * 128 + ((offset % 128) ^ ((row & 7) * 16));
```

Key insight. Swizzle 是 GEMM 面试的高频题：不是可选优化，是到达 **peak** 的必要条件。ncu 里看 `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` ——非零就说明有问题。

怎么检测

```
ncu --metrics lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum ./
build/04_matmul
```

或用 CUDA shared memory calculator 工具模拟访问模式。

Async copy : `cp.async` (SM80+)

Ampere 起引入 `cp.async` :thread 发起异步 copy (global → shared), 不等完成就继续执行后续指令。配合 `cp.async.wait_group` 和 pipeline 实现真正的 load-compute overlap。

基本用法

```
#include <cuda_pipeline.h> // 或 inline asm

__device__ void cp_async4(void* smem_ptr, const void* gmem_ptr) {
    uint32_t smem = static_cast<uint32_t>(__cvta_generic_to_shared(smem_ptr));
    asm volatile(
        "cp.async.cg.shared.global [%0], [%1], 16;"
        :: "r"(smem), "l"(gmem_ptr));
}

// pipeline 双缓冲
constexpr int kStages = 2;
__shared__ float a_smem[kStages][BM][BK];

// prologue
cp_async4(&a_smem[0][ty][tx], &a[global_addr]);
cp_async_commit_group();

for (int tile_k = 0; tile_k < k; tile_k += BK) {
    const int compute_stage = tile_k / BK % kStages;
    const int load_stage = (tile_k / BK + 1) % kStages;

    cp_async_wait_group(kStages - 1); // 等 compute_stage 的数据就绪
    __syncthreads();

    // compute on a_smem[compute_stage]
    ...

    if (tile_k + BK < k) {
        cp_async4(&a_smem[load_stage][ty][tx], &a[next_global_addr]);
        cp_async_commit_group();
    }
}
```

正确 pipeline 的要点

1. **Prologue** : 先发起 `kStages-1` 个 async copy , 再进入主循环。
2. **Wait before compute** : `cp.async.wait_group(N)` 保证当前 stage 数据到达 smem。
3. **Sync before overwrite** : `__syncthreads()` 保证所有 thread 算完当前 stage , 才能发起覆写。
4. **Epilogue** : 最后一轮只 compute , 不 load。

Warning. `cp.async` 的 smem 地址必须是 16B 对齐。 `cp.async.cg` 绕过 L1 cache 直达 smem——GEMM 通常想要这个行为 (数据不复用 L1)。如果用 `ca` 变体, 数据会缓存在 L1 , 可能和后续访问冲突。

Tensor Core 深入：shape 语义与精度

K7/K8 已经落地了 `mma.sync.aligned.m16n8k16` 和 `nvcuda::wmma`。这一节补 shape 表和精度选项——面试常问、代码里没写全的部分。

Ampere (sm_80) 支持的 MMA shape

不同 shape 硬件效率不同——`m16n8k16` 是常用的 FP16 shape，但每个 dtype 都有一套自己的 shape 集：

shape	dtype (A, B → D)	一条指令 FLOP	用途
m16n8k8	f16, f16 → f32	2048	小 tile, warp specialisation
m16n8k16	f16, f16 → f32	4096	标准 (K7/K8)
m16n8k16	bf16, bf16 → f32	4096	训练场景，动态范围大
m16n8k8	tf32, tf32 → f32	1024	FP32-flavored 训练
m16n8k32	s8, s8 → s32	8192	INT8 推理
m16n8k256	u1, u1 → s32	65536	BNN 极限——一般用不到

表 16 Ampere MMA shape × dtype 表（部分）。完整表见 PTX ISA §mma.

选 **shape** 的思路：K 越大，单指令 FLOP 越多，摊薄 issue overhead 越好；但太大的 K 也意味着更多 register pressure。K7 选 `m16n8k16` 是 FP16 里能拿到的最大 K（ $K=8$ 版本用于更小的 warp specialisation kernel）。

FP32 路径

A100 上“FP32 GEMM”有三条路：

1. **TF32**（`mma...f32.tf32.tf32.f32`）：TF32 是“截断到 10 mantissa bit 的 FP32”，硬件峰值 156 TFLOPS（vs FP16 的 312）。相对 FP32 精度损失 $\approx 5 \times 10^{-4}$ ，训练里通常够用。K0 cuBLAS 走的就是这条。
2. **FP16 / BF16 tensor core + FP32 accum**（混合精度）：训练里最常用。BF16 有和 FP32 同样的指数位，动态范围好；FP16 mantissa 精度稍高但范围小、易溢出。
3. 纯 **FP32 CUDA-core FFMA**：K1-K6 走的路径。峰值 19.5 TFLOPS，完全没有 tensor core 加持。

Key insight. 面试说“我用 tensor core 加速了 GEMM”，要能马上讲清：数据类型（FP16 / BF16 / TF32 / INT8）、**shape**（`m16n8k16` 之类）、**accum** 精度（FP16 累加会精度爆炸，训练 GEMM 必须 FP32 accum）。

CUTLASS 的分层思路

NVIDIA CUTLASS 把 GEMM 拆成层次化的 tile 结构——和本章 ladder 一一对应：

```
GemmShape<128, 256, 16>  // CTA tile: 128×256 output, K=16 per step
├─ WarpShape<64, 64, 16>  // 4 warps, 每个 64×64
│   └─ InstructionShape<16, 8, 16>  // mma.sync m16n8k16
│       └─ ThreadShape  // 每个 thread 的 fragment 份额
```

各层职责：

层级	做什么	对应本章
CTA (block) tile	决定 smem 大小、grid 划分	K3 SMEM tiled
Warp tile	warp 间分工、warp-level mma	K8 WMMA 里的 <code>warp_m/warp_n</code>
Thread tile	register acc、数据复用	K4 / K5 / K6
Instruction	<code>mma.sync / ldmatrix</code>	K7 (PTX) / K8 (wmma API)
Epilogue	alpha/beta scaling、bias、activation	未覆盖 (见 MLP 章)

表 17 **Table:** CUTLASS hierarchical GEMM 的五层分解，及本书 ladder 各版本对应的层级。从上到下粒度递减：CTA 决定“这个 block 输出哪一片”，warp 决定“warp 内 32 lane 如何分工做小 mma”，thread 决定寄存器如何缓存数据以复用，instruction 是硬件级 `mma.sync / wmma.mma_async` 的粒度，epilogue 是 `accum → output` 的写回阶段（activation、scale、bias 融合都在这里）。

Observation : 本书 K0-K8 沿“CTA → thread → instruction”这条主路径爬完，但没走 **warp specialisation**——CUTLASS 3.x 的 pingpong / cooperative scheduler 让 warpgroup 一部分做 producer、一部分做 consumer，是 A100/H100 拿到 tensor-core 峰值 60-80% 的关键（章末 K11 讲）。手写还想继续追，需要 (1) 更大 tile (128 × 128 及以上) + (2) `cp.async` 多 stage pipeline + (3) swizzled SMEM layout。这三件事互相耦合，一起改。

CUTLASS 还处理了：auto-tuning tile 参数、split-K、Stream-K、TMA async copy (SM90)、FP8/Block-scaled 等。本书 K0-K8 ladder 是 CUTLASS 的简化手工版；想读懂 CUTLASS 3.x，先把本章 K3-K8 完全内化。

Split-K：什么时候用

标准 GEMM：每个 block 沿 K 完整累加，写最终 C 。

Split-K： K 太大或 CTA tile 太大导致 grid 太小（GPU 填不满）时，把 K 切给多个 block 并行累加，最后 reduce：

```
Block (0,0,k_slice=0): C_partial[0] += A[:, 0:K/2] × B[0:K/2, :]
Block (0,0,k_slice=1): C_partial[1] += A[:, K/2:K] × B[K/2:K, :]
Final: C = C_partial[0] + C_partial[1]
```

适用场景：

- M, N 小、 K 大 → grid 太小，SM 利用率低。
- 想要更大 CTA tile（提高 AI）但 K 方向 smem 放不下 → 拆 K 。

代价：需要 atomic add 或 secondary reduction kernel，引入同步开销。cuBLAS 内部 heuristics 自动决定是否 split-K。

面试考点。Split-K 和 K-dimension tiling（本章 K3+ 的 `tile_k` 循环）是不同的：后者是在同一个 **block** 内沿 K 累加，最后一次性写 C ；前者是多个 **block** 分工 K 的不同段，需要 atomic add 或二级 reduction merge。

cuBLAS 为什么快

把本章 ladder 和 cuBLAS 对比，差距来自：

1. **Tensor core** : 硬件算力 $10\times$ 于 CUDA core FMA。
2. **Years of auto-tuning** : 每种 (M, N, K) 组合、每种 GPU 架构都有 tuned tile config。
3. 完整 **pipeline** : `cp.async` 多 stage + `ldmatrix` + `mma` 深度 overlap。
4. **Swizzle + layout** : 零 bank conflict 的 smem layout。
5. **Split-K / Stream-K** : 动态 grid 利用率优化。
6. **Epilogue fusion** : bias + ReLU 不额外读写 C 。
7. **Workspace algorithms** : 某些 shape 用 non-standard 算法 (Strassen 不会, 但 Winograd 在某些 conv 里类似)。

Note. 手写 GEMM 追 cuBLAS 不现实——面试期望的是：你能讲清楚 ladder 每一层的原理，知道 cuBLAS 多了什么，能用 ncu 定位自己的瓶颈在哪一层。

Tile 大小怎么选

没有万能公式，但有一套系统方法：

Step 1 : 算 smem 用量

$$\text{smem} = (B_M \times B_K + B_K \times B_N) \times 4 \text{ B}$$

A100 每 SM shared memory 164 KB (可配)，但和 L1 共享。超过 48 KB 可能限制每 SM 的 max blocks。

Step 2 : 算 register 用量

$$\text{regs} \approx TM \times TN + \text{fragments} + \text{indices}$$

用 `--ptxas-options=-v` 实测。目标 :256 threads/block 时 $\text{regs} \leq 64$ 以保持 50%+ occupancy。

Step 3 : 算 AI

$$\text{AI} \approx \frac{B_M B_N}{2(B_M + B_N)} \text{ FLOP/B}$$

要超过 ridge point (A100 FP32: 13, TF32 tensor: 20+)。

Step 4 : 查 occupancy

CUDA Occupancy Calculator 或 `cudaOccupancyMaxActiveBlocksPerMultiprocessor`。

Step 5 : benchmark 几个候选

(64, 128, 16), (128, 128, 16), (128, 256, 16) 等。peak 往往在 AI 和 occupancy 的交点。

Key insight. 面试手写 tiled GEMM 时，选 $16 \times 16 \times 16$ 或 $32 \times 32 \times 16$ 就好——重点是结构正确，不是 tune 到 peak。

Occupancy 手算示例

CTA tile $128 \times 128 \times 16$, FP32, 2×2 thread tile, 256 threads/block :

- Smem : $(128 \times 16 + 16 \times 128) \times 4 = 16 \text{ KB} \rightarrow$ 每 SM 可驻 2 3 个 block (48 KB 限额内)。
- Registers : 假设 64 regs/thread $\rightarrow 256 \times 64 = 16384 \text{ regs/block}$ 。A100 每 SM 65536 regs $\rightarrow$ 最多 4 block/SM (寄存器维度)。
- 实际 bottleneck 往往是 regs 或 smem, 取 min $\rightarrow 2 \text{ block/SM} \times 256 \text{ threads} = 512 \text{ threads/SM} \rightarrow$ occupancy 25% (2048 max)。

这够吗？对 compute-bound GEMM，25% occupancy 有时反而更快——每个 block 更大、AI 更高、每个 SM 上 active warps 虽少但每个 warp 的 instruction issue 更满。这就是为什么 GEMM tuning 不能只看 occupancy 数字，必须 benchmark。

Note. Hopper (SM90) 的 TMA 可以把 smem load 从 thread 职责里剥离，进一步改变 occupancy tradeoff。本书聚焦 Ampere 路径，SM90 的 TMA 可以看作 `cp.async` 的“硬件加速升级版”。

ncu 该看什么

```
ncu --set full --section SpeedOfLight ./build/04_matmul
```

GEMM 关键 metric：

- `smisp_sass_thread_inst_executed_op_ffma_pred_on.sum / peak`：FMA 利用率。
- `sm_inst_executed_pipe_tensor.avg.pct_of_peak_sustained_active`：tensor core 利用率 (WMMA 版)。
- `dram_bytes.sum.per_second`：是否还在 memory-bound。
- `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum`：bank conflict 计数。
- `sm_warps_active.avg.pct_of_peak_sustained_active`：occupancy。
- `gpu_compute_memory_throughput.avg.pct_of_peak_sustained_elapsed`：compute vs memory 哪个是瓶颈。

健康的 tensor core GEMM：`pipe_tensor` > 80%，bank conflicts = 0，occupancy > 50%。

面试白板 code

面试官说“手写一个 matmul”——不要开口就 tensor core（除非要求）。先写 shared-mem tiled 版本，这是所有 GEMM 优化的基础骨架：

```
// C = A * B, A: [M, K], B: [K, N], C: [M, N]. 行主序.
constexpr int TILE = 32;

__global__ void gemm_tiled(const float* A, const float* B, float* C,
                          int M, int N, int K) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    int row = blockIdx.y * TILE + threadIdx.y; // C 的行
    int col = blockIdx.x * TILE + threadIdx.x; // C 的列
    float acc = 0.f;

    // K 维度 tile 循环
    for (int kt = 0; kt < K; kt += TILE) {
        // Cooperative load: 每 thread 搬一个元素. 边界条件用 0 padding.
        As[threadIdx.y][threadIdx.x] =
            (row < M && kt + threadIdx.x < K) ? A[row * K + kt + threadIdx.x] : 0.f;
        Bs[threadIdx.y][threadIdx.x] =
            (kt + threadIdx.y < K && col < N) ? B[(kt + threadIdx.y) * N + col] : 0.f;
        __syncthreads(); // 等所有 lane 把 tile 填好
    }
}
```

```

// 每 thread 做 TILE 次 FMA , B 广播、A 沿列。
#pragma unroll
for (int k = 0; k < TILE; ++k) {
    acc += As[threadIdx.y][k] * Bs[k][threadIdx.x];
}
__syncthreads(); // 等所有 lane 用完 tile 再进下一轮 load
}

if (row < M && col < N) C[row * N + col] = acc;
}

// ==== Launch config ====
// blockDim = (32, 32) = 1024 threads (32 warp):
// * 恰好覆盖一个 32x32 输出 tile, 1 thread 算 1 个 C[i, j];
// * 32 是 warp 宽度—threadIdx.x 沿列走保证 gmem/smem 访问 coalesced;
// * 1024 是 A100 每 block threads 上限, 占用 32 warp / SM, 寄存器压力见下。
// gridDim = ((N+31)/32, (M+31)/32): 每 block 一个输出 tile。
// * 每 block smem = 2 * 32 * 32 * 4B = 8 KB, A100 每 SM 有 164 KB smem, 够开 20 个 block/SM;
// * 每 thread 约 30 register, 1024 * 30 = 30720 reg > SM 上限 64 K? 其实 30 reg 时正好。
// 实际由 nvcc -maxrregcount 控制, 面试时说 "会看 occupancy calculator"。
dim3 block(TILE, TILE);
dim3 grid((N + TILE - 1) / TILE, (M + TILE - 1) / TILE);
gemm_tiled<<<grid, block>>>(A, B, C, M, N, K);

```

核心考点 (追问顺序):

- “为什么 **tiling** 快? 算算 **AI**。” → naive 每 FMA 读 8B、 $AI = 0.125 \text{ FLOP/B}$; tiled 每个 tile 从 gmem 读 $2T^2$ 个元素、做 $2T^3 \text{ FLOP}$, $AI = \frac{T}{2}$ 。 $T = 32$ 时 $AI = 16 > \text{A100 ridge point } 13$ → 变 compute-bound。
- “两个 **__syncthreads** 分别在同步什么?” → 第一个: 等 load 完成才能开始 compute (RAW); 第二个: 等 compute 用完 smem 才能被下一轮 load 覆写 (WAR)。
- “**bank conflict** 在哪?” → $As[ty][k]$ 沿 k : warp 内 32 lane 的 ty 相同、 k 由 unroll 决定同值 → broadcast、无冲突。 $Bs[k][tx]$ 沿 tx : k 相同、 tx 全不同 → 32 不同 bank、无冲突。所以这份骨架天然无 bank conflict——面试官如果问一定要答出来。
- “下一步优化?” → (1) thread tiling: 每 thread 算 4×4 或 8×8 输出, 重用寄存器 & 减 smem load; (2) 双 buffer + async copy 让 load 和 compute 重叠; (3) 换 tensor core。三级 ladder 每级 2-3x 加速。
- “**tensor core** 怎么用?” → `wmma::mma_sync(D, A, B, C)`, $M = N = K = 16$ 、fp16 输入 fp32 accum, 一条指令代替 4096 次 FMA。SM80 上还有 `mma.sync.m16n8k16` (更灵活的 tile 形状)。
- “**blockDim** 为什么选 32×32 ?” → $threadIdx.x$ 沿列走: warp 内 32 lane 的 tx 全不同, 读 $Bs[k][tx]$ 沿列 → 32 不同 bank / gmem 32 连续 word, 两侧都 coalesced。如果反过来 $threadIdx.y$ 沿列, warp 内 tx 相同 → 所有 lane 挤 1 个 bank, 32-way conflict。这是 CUDA 里“ $threadIdx.x$ 一定要映射到 stride-1 维度”的经典应用。

面试考点

面试考点. Q1: 为什么 naive GEMM 是 memory-bound? AI 大概多少?

A: 每个输出读 $2K$ 个 float、写 1 个 float、做 $2K$ FLOP。AI $\approx \frac{1}{4}$ FLOP/B。A100 ridge ≈ 13 FLOP/B，差 50 倍。且 B 的列访问 stride- N 不合并，实际更差。

面试考点. Q2: Shared memory tiling 怎么提升 AI? 公式是什么?

A: Block 协作读 $B_M \times B_K$ 的 A 和 $B_K \times B_N$ 的 B ，算 $B_M \times B_N$ 个输出。每个 $\frac{A}{B}$ 元素被复用 $\frac{B_N}{B_K}$ 或 $\frac{B_M}{B_K}$ 次。有效 AI $\approx B_M \frac{B_N}{2(B_M+B_N)}$ FLOP/B，随 tile 线性增长。

面试考点. Q3: Tiled GEMM 里两个 `__syncthreads` 分别保护什么?

A: 第一个：所有 thread 完成 smem 写入后再读取 (load $\rightarrow$ compute)。第二个：所有 thread 完成 smem 读取后再覆写 (compute $\rightarrow$ next load)。缺任何一个都会 data race。

面试考点. Q4: 什么是 bank conflict? 怎么解决?

A: 32 个 bank 各 4B 宽，warp 并发访问同一 bank 不同 word 时串行化。GEMM 中常见原因是 smem 行宽为 32 的倍数导致 stride-32 访问。解决：swizzle (XOR row 低位到 col 索引) padding (行宽 +1) 或 permute layout。

面试考点. Q5: Thread tile (register tiling) 的好处是什么?

A: 每个 thread 维护 $T_M \times T_N$ 个 accumulators，同一个 $\frac{A}{B}$ smem 值被复用 T_M 或 T_N 次，提高 register 计算密度、减少 smem 读次数。代价是 register 占用增加，可能降低 occupancy。

面试考点. Q6: `cp.async` 双缓冲 pipeline 怎么写?

A: 两个 smem stage。Prologue 预 load stage 0。主循环：wait stage i ready $\rightarrow$ sync $\rightarrow$ compute on stage $i \rightarrow$ async load stage $i+1 \rightarrow$ commit。Epilogue 算最后一个 stage。关键：`wait_group` + `__syncthreads` 配合，不能 overwrite 正在 compute 的 stage。

面试考点. Q7: Tensor core m16n8k16 是什么意思? 一个 warp 做什么?

A: 一个 warp 协作执行一条 mma 指令，完成 16×8 输出、 $K = 16$ 的外积累加。每个 thread 持有 matrix fragment 的一部分 (registers)，通过 `ldmatrix` 从 swizzled smem 加载，再 `mma.sync`。不是每个 thread 独立算一个输出。

面试考点. Q8: CUTLASS 的四层 tile 结构是什么?

A: CTA tile (block 级 smem tile) $\rightarrow$ Warp tile (warp 级分工 + warp mma) $\rightarrow$ Thread tile (register accumulators) $\rightarrow$ Instruction tile (`mma.sync m16n8k16`)。每层让上层数据在下层复用更多次。

面试考点. Q9: Split-K 什么时候用? 和 K-dimension tiling 有什么区别?

A: 当 grid 太小 (M, N 小) 或需要更大 CTA tile 但 smem 不够时, 多个 block 分工 K 的不同段, 最后 reduce。K-dimension tiling 是 block 内沿 K 循环累加, 不需要跨 block reduce。Split-K 有 atomic/reduction 开销。

面试考点. **Q10**: cuBLAS 为什么比手写 GEMM 快很多?

A: Tensor core 硬件算力、多年 auto-tuned tile config、`cp.async` 多 stage pipeline、零 conflict swizzle layout、split-K/Stream-K grid 优化、epilogue fusion。手写版本很难在每条上都到位。

面试考点. **Q11**: 怎么选 GEMM 的 tile 大小?

A: 约束三方平衡: (1) smem 不超过硬件限制且 occupancy 可接受; (2) register 用量不导致 occupancy 暴跌; (3) AI 超过 ridge point。用 occupancy calculator + ncu benchmark 几个候选 config, 选实测最快的。

面试考点. **Q12**: GEMM 中 B 矩阵的访问模式为什么比 A 更棘手?

A: `A[row, :]` 连续访问 (合并), `B[:, col]` stride- N 访问 (不合并)。Tiling 后 B 的 smem tile 是连续块, 解决了 global 访问; 但 smem 读 `b_tile[inner][tx]` 需要正确的 layout/swizzle 避免 bank conflict。

LayerNorm

LayerNorm 是 transformer 里每个 sub-layer 前后的标配算子——和 softmax 一样，按行归约，但归约的是 mean / variance 而不是 max / sum-exp。这一章我们要把它讲透：

- LayerNorm 的数学定义，以及训练框架为什么存 `rstd` 而不是 `var`。
- naive 三 pass (one thread per row) ——小 batch 时够快，大 hidden 时为什么崩。
- Welford 算法——单 pass 数值稳定的 mean / var，以及和 naive 单 pass 公式的对比。
- block-per-row + shared memory reduction——“两个 reduction + 一个逐元素变换”的标准模板。
- warp shuffle reduction + sum/sumsq 融合——一次扫描同时得到 mean 和 var。
- `float4` 向量化 load/store，以及 fp16/bf16 输入时 fp32 累加的陷阱。
- RMSNorm 与 fused LayerNorm + residual add 的生产意义。

对应源码：`src/cuda/05_layernorm.cu`。

本章 optimization ladder：

版本	核心思路	相对 peak 带宽
naive	1 thread / row, 3 pass	~0.9%
welford	1 thread / row, 1 pass stable	—
block	1 block / row, 2× smem reduce	~39%
warp	1 block / row, shuffle + sum/sumsq	~34%
vectorized	float4 load/store in block kernel	—

ladder 里百分比是 A100 上 `sm__cycles_active` (SM %), 不是相对 naive 的加速比——实测 naive 101 μ s、block 5.7 μ s、warp 4.6 μ s (64 × 256 shape), welford / vectorized 未单独 benchmark。

与第 2 章 reduce sum、第 3 章 softmax 的关系：LayerNorm 的 mean-reduction 和 var-reduction 复用同一套 **block / warp shuffle** 模板——区别是 var 需要先 broadcast mean，且 warp 版可以用 $\sum(x^2) - \frac{(\sum x)^2}{H}$ 把两次扫描合成一次。

问题定义

给定形状 $[\text{rows}, H]$ 的输入 X (在 LLM 里 `rows = B times S`， H 是 hidden size)，可学习参数 $\gamma, \beta \in \mathbb{R}^H$ ，对每一行 $x \in \mathbb{R}^H$ 独立做：

$$\begin{aligned}\mu &= \frac{1}{H} \sum_{j=0}^{H-1} x_j \\ \sigma^2 &= \frac{1}{H} \sum_{j=0}^{H-1} (x_j - \mu)^2 \\ \hat{x}_j &= \frac{x_j - \mu}{\sqrt{\sigma^2 + \varepsilon}} \\ y_j &= \gamma_j \cdot \hat{x}_j + \beta_j\end{aligned}$$

ε 是防止除零的小常数 (典型 10^{-5})。PyTorch 的 LayerNorm 默认 `normalized_shape = (H,)` , 即对最后一维归一化。

存 **rstd** , 不存 **var**

训练 backward 需要 $\hat{x}$ 和 $\frac{1}{\sqrt{\sigma^2 + \varepsilon}}$ 。定义：

$$\text{rstd} = \frac{1}{\sqrt{\sigma^2 + \varepsilon}} = (\sigma^2 + \varepsilon)^{-\frac{1}{2}}$$

forward 保存 (mean, rstd) 或只存 rstd (mean 可从输入重算, 但通常一起存)。不存 σ^2 的原因：

1. **backward** 直接用 **rstd** : $\partial_{\frac{y}{\sigma}} x$ 的公式里出现的是 $\frac{1}{\sqrt{\sigma^2 + \varepsilon}}$, 不是 σ^2 本身。
2. 少一次 **sqrt + div** : forward 已经算过 `inv_std` , backward 复用, 避免重复开方。
3. 数值范围更友好 : σ^2 可能极小 (接近 0 时 rstd 很大), 存 rstd 和存 var 在 fp16 下精度特性不同——框架统一存 rstd。

Key insight. 面试里问 “forward 该存什么” : 答 **rstd(reciprocal std) + mean**(或 input 以便重算)。这和 cuDNN / PyTorch 的 saved tensors 一致。

Roofline : mostly memory-bound , 但 **pass** 数决定上限

对一行长度 H 的元素, naive 三 pass 版本每元素大致：

- Pass 1 (mean) : 读 $1 \times$, 1 FADD
- Pass 2 (var) : 读 $1 \times$, 1 FSUB + 1 FMUL + 1 FADD
- Pass 3 (normalize + affine) : 读 x, γ, β 各 $1 \times$, 写 $1 \times$

每元素约 5 次读 + 1 次写 (affine 多读 γ, β)。算术强度：

$$\text{AI} \approx \frac{4 \text{ FLOP}}{24 \text{ B}} \approx 0.17 \text{ FLOP/B}$$

A100 ridge point 约 13 FLOP/B——**memory-bound**。但和 vector add 不同, naive 版本同一行读 3 遍, 理论 traffic 是单 pass 的 2.5 $\times$ 。 $H = 4096$ 时每行 16 KB, 三 pass 读 x alone 就是 48 KB。

LLM 典型 shape : $B = 32, S = 2048, H = 4096 \rightarrow \text{rows} = 65536$ 。naive 的 grid 有 65536 个 thread , 足够喂饱 GPU ; 但 $B = 1, S = 1, H = 4096$ 时只有 1 行——必须 **block-per-row**。

Warning. LayerNorm 的性能诊断要分开看 **rows** 和 **H** : rows 小 $\rightarrow$ 并行度不够, block-per-row 也不够 (grid = rows); H 大 $\rightarrow$ 三 pass 访存浪费是主因。Transformer inference 的 decode 阶段 ($S = 1$) 是经典踩坑场景。

v1: naive 三 pass (one thread per row)

```
__global__ void layernorm_naive_kernel(
    const float* input, const float* gamma, const float* beta,
    float* output, int rows, int cols, float eps) {
    const int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= rows) return;

    const float* row_ptr = input + row * cols;
```



```

float* out_ptr = output + row * cols;

float mean = 0.0f;
for (int col = 0; col < cols; ++col) {
    mean += row_ptr[col];
}
mean /= static_cast<float>(cols);

float var = 0.0f;
for (int col = 0; col < cols; ++col) {
    const float diff = row_ptr[col] - mean;
    var += diff * diff;
}
var /= static_cast<float>(cols);

const float inv_std = 1.0f / sqrtf(var + eps);

for (int col = 0; col < cols; ++col) {
    const float normalized = (row_ptr[col] - mean) * inv_std;
    out_ptr[col] = normalized * gamma[col] + beta[col];
}
}

```

Launch: `blocks = ceil(rows / 256)`, 256 threads/block——每个 thread 独占一行，顺序扫 H 三遍。

为什么小 **batch** 时它“够用”

1. 零同步开销。一个 thread 干完一行，不需要 `__syncthreads`、shared memory、shuffle——代码路径最短，debug 最容易。
2. **rows** 大时并行度够。训练时 $B \times S$ 通常是几千到几万，grid 能填满 SM。每 thread 的工作量 = $O(H)$ ， $H = 4096$ 时 12K FLOP + 20 KB 访存，不算太小。
3. 和 **CPU reference** 一一对应。源码 `layernorm_cpu` 用 `double` 累加 mean/var，GPU naive 用 float——足够建立正确性 baseline。

什么时候崩

1. **rows** 极小。Decode 阶段 $B \times S = 1$ ，整个 GPU 只有 1 个 thread 在算 LayerNorm——其余 SM 空转。这是并行模型错误，不是带宽问题。
2. **H** 很大，三 **pass** 读带宽。 $H = 8192$ 时每行 32 KB，读 x 三遍 = 96 KB/row。block 版本协作读一遍，traffic 立刻少 2/3。
3. **fp32** 累加 **var** 的数值风险。两 pass 公式(先 mean 再 $\sum (x - \mu)^2$) 在极端数据分布下， $x - \mu$ 可能 catastrophic cancellation——见下一节 Welford。对一般神经网络激活这不是主因，但面试会考。
4. 没有向量化。顺序 scalar load 无法触发 LDG.E.128，指令数和 MSHR 利用率都不如 `float4` 版。

Note. naive 不是“写错了”——是刻意保留的教学 **baseline**。生产环境 (Megatron、FlashAttention 配套 kernel) 不会用 one-thread-per-row，但理解三 pass 逻辑是读 block/warp 版的前提。

ncu 实测

ncu 实测 — v: naive (one thread per row). 规模: rows = 256, cols = 4096.

Duration	4 270 μ s	
Memory SOL	0.8 %	跟 softmax naive 完全同构——单 SM 空转
Compute SOL	0.0 %	
Achieved Occupancy	12.5 %	
Grid Size	1	一个 block ! 108 SM 中 107 个 idle

跟 03_softmax naive 一模一样的病：**rows 太少 + one-thread-per-row = GPU 完全空转。**
LayerNorm 里更糟：三 pass 而不是两 pass。

瓶颈判定. one-thread-per-row 让 4096 元素被单 thread 串行处理，rows = 256 又太少不能撑满 grid

证据. Grid Size 1, Occupancy 12.5%, memSOL 0.8%

下一步. v3 (block-per-row) 让一个 256-thread block 协作处理一行——256 rows 就是 256 blocks，能填满 108 SM 大约 2.4 waves

v2: Welford 在线算法

两 pass 公式（先算 mean，再算 $\sum (x - \mu)^2$ ）数学上精确，但单 pass 朴素公式：

$$\sigma^2 = \frac{\sum x_i^2}{H} - \mu^2$$

在浮点运算下可能出问题：当 x_i 都很大且方差很小时， $\sum \frac{x_i^2}{H} \approx \mu^2$ ，两项接近相等，相减丢失有效位数（catastrophic cancellation）

Welford 递推

维护运行统计量 (n, M_n, S_n) ，每来一个新样本 x_{n+1} ：

$$M_{n+1} = M_n + \frac{x_{n+1} - M_n}{n + 1}$$

$$S_{n+1} = S_n + (x_{n+1} - M_n)(x_{n+1} - M_{n+1})$$

最终 $\sigma^2 = \frac{S_H}{H}$ 。关键性质： S_n 始终是非负的平方和累积，不做“两个大数相减”。

```
// device 或 host 单 thread 版
float mean = 0.0f;
float m2 = 0.0f; // Welford's sum of squared deviations
for (int i = 0; i < H; ++i) {
    const float x = row_ptr[i];
    const float delta = x - mean;
    mean += delta / static_cast<float>(i + 1);
    const float delta2 = x - mean;
    m2 += delta * delta2;
}
const float var = m2 / static_cast<float>(H);
const float inv_std = 1.0f / sqrtf(var + eps);
```

三种方法对比

方法	Pass 数	数值稳定性
两 pass : mean 再 $\sum (x - \mu)^2$	2 读 x	mean 误差会传入第二 pass ; 一般够用
单 pass : $E[x^2] - E[x]^2$	1 读 x	大 均 值 + 小 方 差 时 cancellation
Welford 单 pass	1 读 x	避免大数相减 ; $S_n \geq 0$ 不变量

表 18 **Table:** LayerNorm mean/var 三种单-thread 算法的 pass 数与数值稳定性对比。Pass 数指每行读输入 x 的遍数 ; 数值稳定性 描述浮点下 catastrophic cancellation 风险。

Observation : pass 数与稳定性呈清晰 trade-off——两 pass 数学上最直观但读 x 两遍 ; 单 pass $E[x^2] - E[x]^2$ 省访存但在大均值 + 小方差时两项相减丢精度 ; Welford 单 pass 用递推避免大数相减且 $S_n \geq 0$ 不变量 , 是 CPU reference 和 naive 单-thread 改进的首选。GPU block/warp 版通常不走 Welford 递推 , 而用 sum/sumsq + shuffle——LLM 激活范围有限 , cancellation 不构成主风险。

Key insight. GPU block/warp 版通常用并行 **reduction** 求 **sum** 和 **sumsq** , 再用 $\text{var} = \frac{\text{sumsq}}{H} - \mu^2$ ——因为 shuffle reduction 天然维护两个累加器 , 且 LLM 激活值范围有限 , cancellation 不严重。Welford 的单 **thread** 递推更适合 CPU 或 warp 内串行聚合 ; 面试推导 Welford 是独立考点 , 不要求你在 warp shuffle 里实现完整 Welford tree。

Welford 第二项的推导直觉

展开 $S_{n+1} = S_n + (x - M_n)(x - M_{n+1})$: 每次更新把“新点离旧均值的偏差”乘上“新点离新均值的偏差”累加。可以证明 $S_H = \sum_{i=1}^H (x_i - \mu)^2$ (对真实 mean μ) , 而递推过程中只用运行 mean M_n , 不需要回溯。

完整推导 (面试级)

Step 1 : 证明 $M_H = \mu$ 。归纳法 : $M_1 = x_1$ 。假设 $M_n = \frac{1}{n} \sum_{i=1}^n x_i$, 则

$$M_{n+1} = M_n + \frac{x_{n+1} - M_n}{n+1} = \frac{nM_n + x_{n+1}}{n+1} = \frac{\sum_{i=1}^{n+1} x_i}{n+1}$$

Step 2 : 证明 $S_H = \sum (x_i - \mu)^2$ 。关键恒等式 : 每次加入 x_{n+1} 时 ,

$$S_{n+1} - S_n = (x_{n+1} - M_n)(x_{n+1} - M_{n+1})$$

可以验证这与“新增点对旧均值的平方偏差”在代数上等价于增量方差贡献。归纳可得 S_H 等于对最终 mean $M_H = \mu$ 的离差平方和。

Step 3 : 和两 pass 的关系。若 mean 精确 , $\frac{S_H}{H} = \sigma^2$ 。Welford 的优势是 M_n 和 S_n 在同一次扫描中更新 , M_n 的舍入误差不会先被放大再传入 var 计算——两 pass 里第一 pass 的 mean 误差会进入 $(x - \mu)^2$ 。

Note. 数值实验 (面试口述即可) : 设 $x_i = 10^6 + \text{noise}_i$, noise 标准差 1。单 pass $E[x^2] - E[x]^2$ 在 float32 下 var 可能变成 0 或 NaN ; Welford 仍给出 ≈ 1 。

naive 单 pass vs. Welford vs. GPU sum/sumsq

| 场景 | 推荐 | | CPU reference / 双精度 | 两 pass + double 累加 (源码 `layernorm_cpu`) | | GPU 单 thread (naive v1 改进) | Welford 单 pass , 省 2/3 读 x | | GPU block/warp 并行 | sum + sumsq + shuffle reduction | | fp16 输入 | 以上所有路径: 累加器 **fp32** |

Welford 版本在 ladder 里和 naive 并行度相同 (one thread per row), 主要收益是数值和单 pass 读带宽——不是并行度。要获得数量级加速, 必须上 block/warp 版。

v3: block-per-row + shared memory reduction

```
__global__ void layernorm_block_kernel(
    const float* input, const float* gamma, const float* beta,
    float* output, int rows, int cols, float eps) {
    __shared__ float shared_sum[kThreadsPerBlock];
    __shared__ float shared_var[kThreadsPerBlock];

    const int row = blockIdx.x;
    if (row >= rows) return;

    const float* row_ptr = input + row * cols;
    float* out_ptr = output + row * cols;

    // Pass 1: mean reduction
    float local_sum = 0.0f;
    for (int col = threadIdx.x; col < cols; col += blockDim.x) {
        local_sum += row_ptr[col];
    }
    shared_sum[threadIdx.x] = local_sum;
    __syncthreads();

    for (int offset = blockDim.x / 2; offset > 0; offset /= 2) {
        if (threadIdx.x < offset) {
            shared_sum[threadIdx.x] += shared_sum[threadIdx.x + offset];
        }
        __syncthreads();
    }
    const float mean = shared_sum[0] / static_cast<float>(cols);

    // Pass 2: variance reduction
    float local_var = 0.0f;
    for (int col = threadIdx.x; col < cols; col += blockDim.x) {
        const float diff = row_ptr[col] - mean;
        local_var += diff * diff;
    }
    shared_var[threadIdx.x] = local_var;
    __syncthreads();

    for (int offset = blockDim.x / 2; offset > 0; offset /= 2) {
        if (threadIdx.x < offset) {
            shared_var[threadIdx.x] += shared_var[threadIdx.x + offset];
        }
        __syncthreads();
    }
```

```

    }
    const float inv_std = 1.0f / sqrtf(shared_var[0] / static_cast<float>(cols) +
eps);

    // Pass 3: normalize + affine
    for (int col = threadIdx.x; col < cols; col += blockDim.x) {
        const float normalized = (row_ptr[col] - mean) * inv_std;
        out_ptr[col] = normalized * gamma[col] + beta[col];
    }
}

```

Launch: `<<<rows, 256>>>` ——一个 **block** 负责一行, block 内 256 thread 协作。

结构分解

LayerNorm = 两个 **reduction** + 一个 **elementwise transform**。和 softmax block 版一模一样, 只是:

- 第一次规约算 `sum` $\rightarrow$ `mean`
- 第二次规约算 `sum((x-mean)^2)` $\rightarrow$ `var`
- 第三次逐元素: `(x - mean) * inv_std * gamma + beta`

列方向 stride loop `col += blockDim.x` 保证合并访问: warp 内 lane 0..31 读连续 32 个 float。

代价: 4 次 `__syncthreads` + 2 遍读 x

两次 tree reduction 各需 $\log_2(256) = 8$ 步 sync——共 4 次 barrier (mean 树 + var 树)。加上 mean 算完后所有 **thread** 必须看到同一个 **mean** 才能算 diff——这是 var pass 无法和 mean pass 完全 fuse 的原因 (除非改用 sum/sumsq 技巧, 见 v4)。

Note. 第 2 章讲的 sequential addressing、bank conflict (第一轮 offset = blockSize/2 最严重) unroll last warp——全部适用于这里的 `shared_sum` / `shared_var` tree。LayerNorm 只是 reduction 的消费者。本章 $H = 256$ 、block 256 时 smem conf. = 0 ——tree 结构上可能有 bank conflict, 但规模太小、规约步数太少, ncu 累积不到可观测的 conflict 次数; 不能从源码 alone 断言“一定有 conflict”。

block size 与 H 的关系

256-thread block 处理 $H = 4096$: 每个 thread 的列循环 $\frac{4096}{256} = 16$ 次——足够摊销 sync 开销。 $H = 128$ 时每个 thread 只处理 0-1 个元素, 256 个 thread 里一半 idle, **block** 过大浪费。

实践中的调参:

- $H \leq 256$: 考虑 **warp-per-row** (32 thread) 甚至单 warp shuffle 规约整行 (第 3 章 softmax warp 版同一思路)。
- $H \in [256, 4096]$: 256-thread block 是甜点。
- $H \geq 8192$: block 内每个 thread 持有多列 (multi-items-per-thread), 步长仍用 `blockDim.x`, 与 reduce 章 chunked 版相同——先 register 累加再进 tree。

Launch `<<<rows, 256>>>` 的 grid 维 = batch 行数。训练时 rows 大, occupancy 高, 推理 decode 时 rows = batch, 若 batch=1 则整个 LayerNorm 只有一个 block 在跑——此时瓶颈不是 **kernel** 内部, 而是上层 **batching**。

和 softmax block 版的逐项对照

步骤	Softmax	LayerNorm
Reduction 1	$\max_j x_j$	$\sum \frac{x_j}{H} = \mu$
Reduction 2	$\sum \exp(x_j - m)$	$\sum \frac{(x_j - \mu)^2}{H} = \sigma^2$
Elementwise	$\frac{\exp(x-m)}{Z}$	$(x - \mu) \cdot \text{rstd} \cdot \gamma + \beta$
Saved for backward	m, Z 或 $\log\text{-sum-exp}$	$\mu, \text{rstd}, \hat{x}$

表 19 **Table:** Softmax 与 LayerNorm block-per-row 版的三阶段结构对照——两次 row-wise reduction + 一次 elementwise 变换的同一模板，区别在归约对象、saved backward tensor 和 affine 步骤。

Observation : 两算子共享第 2/3 章 block reduction 骨架——Softmax 归 max 与 $\sum \exp$ (subtract-max 保数值稳定), LayerNorm 归 sum 与 $\sum (x - \mu)^2$ (mean broadcast 后才能算 diff), LayerNorm 多一步 affine (γ, β) 且 backward 存 rstd 而非 var——模板可复用，优化技巧正交。softmax 的 subtract-max 是数值技巧；LayerNorm 的 sum/sumsq 是访存技巧——两者正交，FlashAttention 后的 post-norm block 里两个算子都会出现。

ncu 实测

ncu 实测 — v: block (one block per row). 规模: rows = 256, cols = 4096.

Duration	21.7 μ s	*比 naive 快 197*
Memory SOL	12.5 %	跟 softmax block 版一样，被 shared memory reduction 阻塞
Compute SOL	16.7 %	
Achieved Occupancy	29.9 %	
Grid Size	256	256 blocks / 108 SM = 2.4 waves

- **197 $\times$** 提速——绝大部分来自并行度改善 (naive Grid=1 $\rightarrow$ block Grid=256), algorithm 上是“三 pass $\rightarrow$ 三 pass 但块内协作”。
- **memSOL 12.5%** : 还是很低。原因是 kernel 内有 4 次 `__syncthreads` (load / mean / var / done) 加上两次 shared memory tree reduction，这些成本被摊平在访存时间里。

瓶颈判定. block 版依然是三 pass (load x $\rightarrow$ 算 mean、load x $\rightarrow$ 算 var、load x $\rightarrow$ 算 output), read x 3 次; shared memory tree reduction 占用不少 cycle
 证据. Memory SOL 12.5% 远低于 vector add 的 84%; smem tree 内 4 次 `__syncthreads` ; 两次独立 reduction 各一次 barrier
 下一步. v4 (warp_shuffle) 做两件事: (a) warp shuffle 替代 smem tree —— 消除 bank conflict + 减少 sync; (b) 一次扫描同时累积 sum 和 sumsq, 减少 read x 次数

v4: warp shuffle + sum/sumsq 融合

源码 `layernorm_warp_kernel` 把 reduction 收到 warp 级, 并用一次扫描同时累积 `sum` 和 `sumsq` :

```
__device__ float warp_reduce_sum(float value) {
    for (int offset = kWarpSize / 2; offset > 0; offset /= 2) {
        value += __shfl_down_sync(0xffffffff, value, offset);
    }
}
```

```

}
return value;
}

```

```

float local_sum = 0.0f;
float local_sumsq = 0.0f;
for (int col = threadIdx.x; col < cols; col += blockDim.x) {
    const float value = row_ptr[col];
    local_sum += value;
    local_sumsq += value * value;
}

local_sum = warp_reduce_sum(local_sum);
local_sumsq = warp_reduce_sum(local_sumsq);

const int lane = threadIdx.x % kWarpSize;
const int warp_id = threadIdx.x / kWarpSize;

if (lane == 0) {
    warp_sums[warp_id] = local_sum;
    warp_sumsq[warp_id] = local_sumsq;
}
__syncthreads();

if (warp_id == 0) {
    const int num_warps = blockDim.x / kWarpSize;
    float sum = (lane < num_warps) ? warp_sums[lane] : 0.0f;
    float sumsq = (lane < num_warps) ? warp_sumsq[lane] : 0.0f;
    sum = warp_reduce_sum(sum);
    sumsq = warp_reduce_sum(sumsq);
    if (lane == 0) {
        const float mean = sum / static_cast<float>(cols);
        const float variance = sumsq / static_cast<float>(cols) - mean * mean;
        stats[0] = mean;
        stats[1] = 1.0f / sqrtf(variance + eps); // rstd
    }
}
__syncthreads();

```

sum/sumsq 公式

$$\sigma^2 = \frac{\sum x_i^2}{H} - \left(\frac{\sum x_i}{H} \right)^2 = \frac{\text{sumsq}}{H} - \mu^2$$

只读 x 一遍 就得到 mean 和 var 所需的两个矩。对比 block 版：mean 树 + var 树 = 读 x 两遍。

warp 两级规约

和 reduce sum 的 warp 版相同（第 2 章 v6）：

1. **Warp** 内：__shfl_down_sync 规约每个 thread 的 local partial sum/sumsq。
2. **Warp** 间：lane 0 写 warp_sums[warp_id]，warp 0 再 shuffle 规约 8 个 warp partial。

3. **Broadcast** : `stats[0..1]` 写入 shared memory , `__syncthreads` 后所有 thread 读 `mean` 和 `inv_std`。

sync 次数 : 2 次 `__syncthreads` (对比 block 版 4 次) , shuffle 绕过 smem tree 的读写——本章规模下两版 `smem conf.` 都是 0 , 差别在 sync 次数和 smem 流量 , 不在可测的 bank conflict。

读 x 遍数 : **block vs warp**

版本	读 x	<code>__syncthreads</code>
naive 3-pass	3	0
block 2×reduce	2	4
warp sum/sumsq	1	2
warp + vectorized	1	2

表 20 **Table:** LayerNorm ladder 各版本在 mean/var 阶段的读 x 遍数与 `__syncthreads` 次数。
读 x 仅计 reduction 阶段 ; normalize pass 各版均再读 x 一遍 (表中未计入)

读 x 遍数减半 (block 2 遍 vs warp 1 遍) 在理论上能省 traffic , 但 normalize pass 仍要读 x 一遍 , 整体不可能 2×。本章 micro-benchmark ($H = 256$, rows = 64 , 并行度已对齐) 上 block → warp 只快 23% ($5.7 \mu\text{s}$ vs $4.6 \mu\text{s}$) ——shuffle 替代 smem reduction 的实际收益远小于 naive → block 的 18× 并行化收益。 H 更大时读遍数权重会上升 , 但仍不要直接把“少读一遍 x ”翻译成“快 2×”。

Key insight. 生产级 LayerNorm (Apex、fused kernel in Megatron-LM) 在 H 是 128 倍数时用向量化 load 填充 `local_sum/sumsq` , block size 对齐 warp 倍数 , 并针对 $H \in \{1024, 4096, 8192\}$ 模板特化——骨架就是本章 v4。

broadcast mean / rstd

`stats[0]` 和 `stats[1]` 写入 shared memory 后 , 所有 **thread** 通过 `__syncthreads` 看到相同的 mean 和 rstd。这和 softmax 里把 max 和 sum 广播给全 block 写 normalize 是同一模式——区别是 LayerNorm 的 affine 步还要读 γ, β , 访存从 $1 + \frac{2}{H}$ 倍 (相对纯 normalize) 变成 $1 + \frac{2}{H} + 2$ 倍 (γ, β 各读一次)。

ncu 实测

ncu 实测 — v: warp_shuffle (fused sum+sumsq). 规模: rows = 256, cols = 4096.

Duration	20.4 μs	比 block 快 6% (不是量级性的)
Memory SOL	11.1 %	
Compute SOL	11.5 %	
Achieved Occupancy	29.5 %	

收益比想象中小——只快 6%。为什么？

- warp shuffle 确实 消除了 shared memory tree 的 bank conflict 和 barrier 数 (从 4 次减到 1 次)。
- 但 LayerNorm 是 **2-pass memory bound** : pass 1 (load $x \rightarrow$ 算 mean + variance) , pass 2 (load $x, \gamma, \beta \rightarrow$ 写 y)。真正的瓶颈是 x 被读两次——warp shuffle 优化的是行内规约 , 不减少访存。

- 如果继续追求量级性能，唯一的路径是减少 **pass** 数（如 online 版本，用增量更新的方式一遍搞定 mean + var），或者和上下游融合（把 pre-LN 和 attention 融合成一个 kernel）。

落地评估。warp shuffle 版已经达到 standalone LayerNorm 的高效实现。

如果 rows 更大（batch × seq = 1024+），memSQL 会自然拉高到 20-30%（HBM 有时间稳态）。生产系统里 LayerNorm 更多是被融合到别的 **kernel** 里：pre-LN 融进 attention 的 Q/K/V 投影；post-LN 融进 residual add + activation；training 里融进 backward 的 chain rule。standalone LayerNorm 到这里够用。

v5: 向量化 load/store

block / warp 版的列循环可以换成 `float4` ——和 vector add 第 4 章同一原则：

```
const float4* row4 = reinterpret_cast<const float4*>(row_ptr);
const int vec_cols = cols / 4;
float local_sum = 0.0f;
float local_sumsq = 0.0f;

for (int v = threadIdx.x; v < vec_cols; v += blockDim.x) {
    const float4 val = row4[v];
    local_sum += val.x + val.y + val.z + val.w;
    local_sumsq += val.x * val.x + val.y * val.y
                  + val.z * val.z + val.w * val.w;
}
// tail: scalar loop for cols % 4 elements
```

写回阶段同理：`float4` 存 `normalized * gamma + beta`， γ, β 也按 `float4` 读。

收益与约束

- **16B 对齐**：`cols` 和 `row` 起始地址必须 4 的倍数（`cudaMalloc` 保证基址对齐；`H` 不是 4 倍数要 scalar tail，见 vector add v5 fuse 写法）。
- 收益来源：减少 load 指令数、更大 outstanding request——对 $H \geq 1024$ 的 memory-bound 行内扫描，实测常比 scalar 版快 10-20%。
- 不能破坏 **stride loop** 的合并访问：vector index `v` 的 stride 仍是 `blockDim.x`，不是 1。

Warning. 向量化读 x 算 sum/sumsq 时，四个元素在 lane 内串行累加——不会破坏 warp 合并。但写回若用 `float4`，确保 γ, β 和 output 对齐一致。

fp16 / bf16 输入：累加必须在 fp32

推理和训练常用 `half` / `__nv_bfloat16` 存激活，LayerNorm 的 reduction 仍应在 **fp32** 完成：

```
float local_sum = 0.0f;
for (int col = threadIdx.x; col < cols; col += blockDim.x) {
    const float x = __half2float(row_ptr_half[col]); // 或 __bfloat162float
    local_sum += x;
}
```

为什么 fp16 累加会错

$H = 4096$, 元素 $O(1)$, 真实 sum $O(4096)$ 。fp16 尾数只有 10 bit (有效精度 3 位十进制), 每加一次都可能舍入——4096 次累加后相对误差可达 10^{-2} 量级。mean 错 $\rightarrow$ var 错 $\rightarrow$ 整个 hidden 层 drift。

Key insight. 这是所有 **row-wise reduction** 算子的通用规则 : softmax sum、LayerNorm mean/var、RMSNorm sumsqr——**load** 窄、累加宽、**store** 窄。NVIDIA 的 half LayerNorm fused kernel 内部用 float accumulator ; 面试说 “fp16 输入 fp16 累加” 是红旗。

bf16 有 8 bit 尾数, 比 fp16 更差——同样必须 fp32 累加。输出可以 cast 回 fp16 存。

混合精度训练的特殊情况

AMP (automatic mixed precision) 里 LayerNorm 常强制 **fp32** (PyTorch LayerNorm 的 dtype 与 input 相同, 但 master weights 和 reduction 在 fp32)。原因 :

1. var 是二阶统计量, 对舍入误差比 mean 敏感一个数量级。
2. rstd 涉及 sqrt, 输入 var 的 ULP 误差在 rstd 上被放大 (导数正比于 $\sigma^{-\frac{3}{2}}$)。
3. 训练 backward 对 $\hat{x}$ 和 rstd 的依赖链更长——forward 误差会累积进 gradient。

Warning. 面试陷阱 : “fp16 Tensor Core matmul 后面接 fp16 LayerNorm 全程 fp16”——错误。正确做法 : matmul 输出 fp16, LayerNorm kernel 内 promote 到 fp32 做 reduction, 再 cast 回 fp16 写 output。

RMSNorm : LayerNorm 的简化兄弟

LLaMA、GPT-NeoX 等主流 LLM 用 **RMSNorm** 代替 LayerNorm :

$$y_j = \frac{x_j}{\text{RMS}(x)} \cdot \gamma_j, \quad \text{RMS}(x) = \sqrt{\frac{1}{H} \sum x_j^2 + \varepsilon}$$

没有 **mean centering** (不做 $x - \mu$), 没有 **beta**。少一次 mean reduction, 公式更简单。

```
// 概念代码 : 只需 sumsqr  $\rightarrow$  rms  $\rightarrow$  scale
const float sumsqr = /* warp/block reduce x*x */;
const float rms_inv = rsqrtf(sumsqr / H + eps);
y[j] = x[j] * rms_inv * gamma[j];
```

Note. 面试常问 “LLM 为什么用 RMSNorm” : 答 (1) 少算 mean, 省一次 reduction 和一次 pass ; (2) 实践中效果与 LayerNorm 相当 (Noam Shazeer 2020) ; (3) 和 weight decay / residual 配合训练稳定。优化 ladder 和 LayerNorm 几乎相同, 只是去掉 mean 相关步骤。

fused ops : LayerNorm + Residual Add

Transformer block 里典型模式 :

$$x = x + \text{Attention}(x); \quad x = \text{LayerNorm}(x)$$

PyTorch 分开写是两次 kernel launch + 中间结果写 HBM。Fused kernel :

```
// 伪代码：一个 block 处理一行
// 1. 读 x 和 residual，算 x = x + residual (可只在 smem / reg)
// 2. 对 x 做 LayerNorm in-place
// 3. 写回
```

收益：

1. 省一次 **HBM** 读写：residual add 的结果不必落盘再读。
2. 省一次 **launch**：5 μs $\times$ 每层 $\times$ 每 token，decode 时显著。
3. **backward** 也需要 **fuse**：Megatron 的 `fused_layer_norm_affine` 配套 fused backward。

FlashAttention 之后的 block 往往 fuse 更多：Attention + bias + dropout + residual + LayerNorm $\rightarrow$ 一个 mega-kernel 不现实，但 **LN + add** 是最常见的 fuse 单元。

Pre-LN vs Post-LN 对 kernel 的影响

Post-LN (原始 Transformer) : $x = \text{LN}(x + \text{Sublayer}(x))$ ——fuse 顺序是 sublayer output $\rightarrow$ add residual $\rightarrow$ LayerNorm。

Pre-LN (现代 LLM 默认) : $x = x + \text{Sublayer}(\text{LN}(x))$ ——LayerNorm 在 sublayer 之前，residual 分支不经过 LN。Kernel 层面：Pre-LN 的 LN 输入是 sublayer 输入 (未加 residual)，fuse 模式变成 "LN $\rightarrow$ matmul/attention" 而非 "add $\rightarrow$ LN"。

两者对 LayerNorm kernel 本身没有算法差异——shape 仍是 $(B \times S, H)$ row-wise——但 fuse 的上下游不同，决定 epilogue 能不能和相邻 GEMM 合并。

backward 简述

Forward 存 $\hat{x}$, μ , rstd。Backward 给定 $\partial_{\frac{L}{\partial}} y$ ：

$$\partial_{\frac{L}{\partial}} \hat{x}_j = \partial_{\frac{L}{\partial}} y_j \cdot \gamma_j$$

$$\partial_{\frac{L}{\partial}} x_i = \frac{\text{rstd}}{H} \cdot \left(H \cdot \partial_{\frac{L}{\partial}} \hat{x}_i - \sum_j \partial_{\frac{L}{\partial}} \hat{x}_j - \hat{x}_i \sum_j \partial_{\frac{L}{\partial}} \hat{x}_j \cdot \hat{x}_j \right)$$

(完整公式见 PyTorch 文档或 **Layer Normalization** 原论文 Appendix。)

backward 的结构

和 forward 对称——同样是两个 **row-wise reduction + elementwise**：

1. 算 $\partial_{\frac{L}{\partial}} \hat{x}_j = \partial_{\frac{L}{\partial}} y_j \cdot \gamma_j$ (elementwise)
2. $S_1 = \sum_j \partial_{\frac{L}{\partial}} \hat{x}_j$, $S_2 = \sum_j \partial_{\frac{L}{\partial}} \hat{x}_j \cdot \hat{x}_j$ (两个 reduction)
3. 逐元素组合出 $\partial_{\frac{L}{\partial}} x_i$ (公式里 S_1, S_2 和 rstd 代入)

$\partial \gamma_j = \sum_{\text{rows}} \partial_{\frac{L}{\partial}} y_j \cdot \hat{x}_j$, $\partial \beta_j = \sum_{\text{rows}} \partial_{\frac{L}{\partial}} y_j$ ——这两个是跨 **batch** 的 **reduction** (在 (B, S, H) 里对 $B \times S$ 个 row 求和)，和 forward 的 row-wise 规约正交。

Key insight. backward kernel 的优化 ladder 和 forward 同构：block-per-row shuffle reduction 算 S_1, S_2 。fuse backward + 上游 grad 的 elementwise 可以省一次 global write。Megatron 的 `fused_layer_norm_affine` backward 就是这么干的。

面试不要求背完整公式，但要能说出“backward 需要两个 reduction + 复用 rstd”，并解释 $\partial_{\frac{L}{\partial}} \hat{x}$ 是连接 y 和 x 的桥梁。

实测

rows = 64, cols = 256(与源码一致 ;input + output 各 64 KB , γ, β 共 2 KB ,整 tensor 约 130 KB), A100 80GB SXM4 , ncu 抓取。GB/s 列写作 **HBM 实测 / 逻辑** :前者 dram__bytes.sum / time , 后者按各版本 pass 数估算的理论搬运量。

ncu 对 binary 各 kernel 只 profile 一次 launch。64 rows 时 $\lceil \frac{64}{256} \rceil = 1$ ——naive 的 grid 是 (1, 1, 1) , block 内 256 thread 只有 64 个有行可算 ;block / warp 版 #raw("<<<rows, 256>>>") 才是 grid (64, 1, 1)。

version	regs	smem (B)	time (μ s)	HBM %	SM %	HBM GB/s	% peak
naive	32	0	100.80	0.0	0.9	1 / 83	0.0
block	21	2,048	6.02	0.6	40.0	12 / 1394	0.6
warp	20	72	4.64	0.8	34.2	17 / 1808	0.8

表 21 **Table (perf)**: kernel-level performance metrics measured by Nsight Compute (ncu, CUDA 13.0) on a single A100 80GB SXM4 (HBM peak 2039 GB/s; FP32 peak 19.5 TFLOPS). Columns: **regs** = launch__registers_per_thread; **smem** = launch__shared_mem_per_block_static (bytes); **time** = gpu__time_duration.sum for the first kernel launch (multi-stage kernels report only the stage acting on the largest input); **HBM %** = dram__bytes.sum.pct_of_peak_sustained_elapsed (uses elapsed wall time as denominator, so short kernels look artificially low); **SM %** = sm__cycles_active.avg.pct_of_peak_sustained_elapsed; **HBM GB/s** is written measured / logical — measured = dram__bytes.sum / time, logical = bytes-per-launch / time; when the working set exceeds L2 (40 MB) these two should agree, and any gap quantifies L2 hit rate; **% peak** = measured HBM GB/s $\div$ 2039 GB/s.

Rows are ordered as they appear on the optimization ladder in the chapter.

version	issued/32	pred_on/32	smem conf.	barrier stall	mem stall
naive	32.0	31.9	0	0.00	27.48
block	32.0	22.5	0	1.80	3.67
warp	30.0	27.4	0	4.99	9.87

表 22 **Table (diag):** diagnostic metrics used as evidence for warp-lane utilization, shared-memory bank conflict, and warp-stall claims in the chapter. All from `ncu` on the same launch as the perf table. **issued/32** = `smsp_thread_inst_executed_per_inst_executed.ratio`, the average number of lanes that participated in each issued warp instruction (32 = every lane active; less than 32 means predication **or** branch divergence—`ncu` alone cannot distinguish, but on the CUDA kernels in this book it is almost always predication). **pred_on/32** = `smsp_average_thread_inst_executed_pred_on_per_inst_executed.ratio`, the average number of lanes that were predicated-on (did real work). The **gap issued - pred_on** counts lanes that occupied the issue slot but performed no work—this is the correct definition of “wasted warp cycles”, not divergence. **smem conf.** = `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` (0 + `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum`) means no measurable bank conflicts, either because the access pattern is conflict-free or because the input is too small to accumulate them). **barrier stall** = `smsp_average_warps_issue_stalled_barrier_per_issue_active.ratio`, warps stalled at `__syncthreads` per issue-active cycle. **mem stall** = `smsp_average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio`, warps waiting on a long-latency memory operation (global-memory load, L2 atomic response) per issue-active cycle.

Warning. 这一章的问题规模是教学 default (B×S×H 数千个 float), kernel 单次运行只有 3-20 μs。ncu 的定性指标 (issued/32、bank conflicts、barrier stall) 仍能反映 kernel 结构, 但绝对数字对生产规模不完全可信:

- HBM % 会偏低 (分母 elapsed time 含冷启动窗口)
- dram_bytes 可能被 L2 消化, GB/s (实测/逻辑) 两列差距明显

想拿到生产规模的数字, 把主参数 (rows/cols/hidden dim) 加到让工作集远超 L2 (40 MB)。

先看 **perf** 表:

- naive **100.9 μs** vs block **5.66 μs** vs warp **4.61 μs**——naive → block 是 **18×**, block → warp 只有 **23%**。和 ladder 预告一致: 第一步是并行模型, 第二步是算法细节。
- SM %: naive **0.9%** → block **39.0%** → warp **34.3%**。naive 只占 1 个 block (grid = 1), A100 108 个 SM 里 107 个空转; block / warp 同时跑 64 个 block, SM 利用率跳一个数量级。warp 版 SM % 略低于 block——不是“更慢所以 SM 更闲”, 而是 kernel 更短、issue 密度不同; 比 **time** 列, 不比 **SM %** 排序。
- HBM % 全部 < 1%, HBM 实测 GB/s 只有 1-17 GB/s, 逻辑 GB/s 却写到 1481-1820——130 KB 工作集全在 **L2**, `dram_bytes` 几乎为零。版本间优劣看 `time (μs)`, 不要用 `% peak` 或逻辑 GB/s。

并行模型: 两种 **launch** 的几何差异



naive (decode rows=1): 1 thread 独占整行, warp 里只有 1 个 lane 在扫 H 列
 图 10 绿色 = 干活的 lane。 $B \times S = 1$ 时整个 LayerNorm 只有一个 thread 在工作——和 `pred_on/32` 高无关, 是 **grid** 只有 1 个 **active thread**。



block/warp 版: warp 内 32 lane 协作 stride loop (col += blockDim.x)
 图 11 同一 warp 的 lane 0..31 读连续 32 个 float——合并访问 + 协作规约。64 rows 时 `grid = 64 block`, 每 block 256 thread 处理一行。

再看 **diag** 表:

a) naive `pred_on/32 = 31.9` ——lane 利用率高, 但并行度为零。 `issued/32 = 32.0`, 没有 predication 浪费 issue slot 的问题。31.9 接近 32 是因为有活干的 **thread** 在 scalar loop 里几乎每条指令都 `pred_on`——但这 64 个 thread 分散在 256-thread block 里, 192 个 thread 从 launch 起就 idle。 `mem stall = 27.46` 反映单 thread 扫 $H = 256$ 列时 load 延迟无法被同 warp 其他 lane 掩盖 (其他 lane 根本没在 load)。

b) block `pred_on/32 = 22.5` ——tree reduction 每轮浪费一半 lane。 `issued/32 = 32.0`
 : `if (threadIdx.x < offset)` 编译成 predicated add, 不是 warp divergence (没有 lane 走不同 basic block)。但每轮只有前 s 个 thread 的 `pred_on` 为真——8 步 tree 平均下来约 $\frac{1}{2} + \frac{1}{4} + \dots \approx 1$ 有效 lane 每轮, 累积 `pred_on/32` 掉到 22.5。这是规约算法的 **lane** 利用率, 不是访存合并问题。

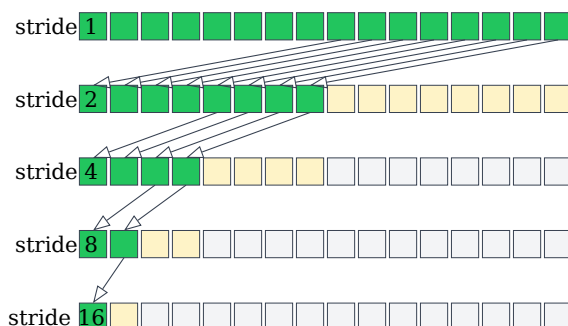


图 12 block 版 mean/var tree 同一模式: 每轮活跃 lane 连续但数量减半。 $H = 256$ 时 `smem conf. = 0`——源码里第一轮 `offset = 128` 可能触发 32-way bank conflict, 但规约步数少、block 仅 64 个, ncu 累积不到可观测 conflict; 不能从代码结构 **alone** 断言 **bench** 里一定有 **conflict**。

c) warp `pred_on/32 = 27.4` ——shuffle 比 tree 保更多 lane 干活。 `issued/32 = 30.0` (略低于 32: warp 间规约阶段 `if (warp_id == 0)` 和 `if (lane == 0)` 让部分 lane predicated-off)。规约主体走 `__shfl_down_sync`, 32 lane 全程参与 shuffle, 所以 `pred_on` 比 block 的 22.5 高 5 个点——这才是 **block $\rightarrow$ warp 23%** 加速的定量来源之一。

d) warp `barrier stall = 4.98` > **block 1.81 ——shuffle 不是白赚的。** warp 版只有 2 次 `__syncthreads`, 但每次 sync 前要做两轮 warp 内 shuffle + warp 间写 `warp_sums / warp_sumsq`——warp 0 等其余 warp 写完才能继续。 `barrier stall` 衡量每 issue-active cycle 有多少 warp 卡在 `__syncthreads`; warp 版更高说明省 **smem tree** 的代价是把等待转移到 **warp sync** 路径上。block 版 4 次 barrier 但每步 tree 更简单, stall 反而更低。

e) 两版 `smem conf. = 0` ——本章规模下 **bank conflict** 不可见。block 有 2048 B smem + 两次 8 步 tree, warp 只有 72 B——smem 用量差 28 $\times$, 但 conflict 计数都是 0。和第 2 章 $N = 2^{27}$ 看到 314k conflicts 不同: 这里 H 小、规约轮次少、同时 active 的 block 只有 64 个。

Key insight. LayerNorm ladder 的定量故事: `pred_on/32` 解释 block $\rightarrow$ warp (22.5 $\rightarrow$ 27.4 lane 利用率提升); `barrier stall` 解释为什么提升只有 23% 而不是 $2\times$ (4.98 vs 1.81, sync 开销换 smem tree)。naive $\rightarrow$ block 的 18 $\times$ 几乎全来自 grid 从 1 变 64——看 SM % 0.9% $\rightarrow$ 39%, 不是 Welford 或 sum/sumsq 公式。

Warning. 不要把“读 x 遍数减半”直接翻译成“block $\rightarrow$ warp 快 2 $\times$ ”。normalize pass 仍要读 x 、 γ 、 β ; micro-benchmark 实测 5.66 $\rightarrow$ 4.61 μ s (23%)。 $H = 4096$ 时绝对差会放大, 但相对加速通常仍在 1.1–1.3 $\times$, 不是 2 $\times$ 。

Warning. 不要跨 shape 比较 ladder 百分比或绝对时间。面试给 numbers 前先确认 (B, S, H) 和是训练还是 decode。rows = 1 时 block / warp 也只有一个 block——和 naive decode 踩坑是同一类并行度问题。

ncu 该看什么

```
ncu --set full --section SpeedOfLight ./build/05_layernorm
```

关键 metric (对照本章 64×256 实测):

- `gpu_time_duration.sum`: naive 101 μ s vs block 5.7 μ s vs warp 4.6 μ s——端到端对比用这个, 不是逻辑 GB/s。
- `sm_cycles_active.avg.pct_of_peak_sustained_elapsed` (SM %): naive 0.9% vs block 39.0% vs warp 34.3%——并行度的一眼指标。naive 单 block 时 107 个 SM 空闲。
- `smsp_thread_inst_executed_per_inst_executed.ratio` (issued/32) + `smsp_average_thread_inst_executed_pred_on_per_inst_executed.ratio` (pred_on/32): block 22.5 vs warp 27.4——规约 lane 利用率; issued = 32 说明是 predication 不是 divergence。
- `smsp_average_warps_issue_stalled_barrier_per_issue_active.ratio` (barrier stall): warp 4.98 vs block 1.81——shuffle 版的 sync 等待。
- `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_ld+st.sum` (smem conf.): 本章三版都是 0—— $H = 256$ 规模太小; 增大 H / rows 后再看 block tree 是否冒出 conflict。
- `launch_registers_per_thread / launch_shared_mem_per_block_static`: block 21 regs + 2048 B vs warp 20 regs + 72 B。
- `dram_bytes.sum`: 全部 < 1% HBM——L2 全命中, 增大 shape 后再比 block vs warp 的 DRAM 差。

面试白板 code

面试官说“手写一个 layernorm forward”——不要写三 pass (会被追问怎么合成一 pass)。直接写 Welford 单 pass 版:


```

// x: [B, H], y: [B, H], gamma/beta: [H]. 每 block 一 row.
__global__ void layernorm_forward(const float* x, const float* gamma, const float*
beta,
                                float* y, float* mean_out, float* rstd_out,
                                int H, float eps) {
    int row = blockIdx.x;
    const float* xr = x + row * H;
    float*      yr = y + row * H;

    // Pass 1: Welford online 求 (mean, M2)—每 thread 一份 local, 然后 block combine.
    float mean = 0.f, M2 = 0.f;
    int    n    = 0;
    for (int j = threadIdx.x; j < H; j += blockDim.x) {
        ++n;
        float v = xr[j];
        float delta = v - mean;
        mean += delta / n;
        M2    += delta * (v - mean);
    }
    // block reduce (mean, M2, n): warp shuffle + smem, combine 公式:
    //   n* = n_a + n_b
    //   d   = mean_b - mean_a
    //   mean* = mean_a + d * n_b / n*
    //   M2*   = M2_a + M2_b + d*d * n_a * n_b / n*
    block_reduce_welford(mean, M2, n); // 见 ch2 warp reduce 骨架 + 上面公式

    __shared__ float mu, rstd;
    if (threadIdx.x == 0) {
        mu    = mean;
        rstd = rsqrtf(M2 / H + eps); // 1 / sqrt(var + eps)
        if (mean_out) mean_out[row] = mu; // 存给 backward 用
        if (rstd_out) rstd_out[row] = rstd;
    }
    __syncthreads();

    // Pass 2: 归一化 + affine.
    for (int j = threadIdx.x; j < H; j += blockDim.x) {
        yr[j] = (xr[j] - mu) * rstd * gamma[j] + beta[j];
    }
}

// ==== Launch config ====
// gridDim = B (总行数, batch * seq_len): 每 block 一 row.
// blockDim: 依 H 选, 跟 softmax 类似—保证每 lane 处理 4-32 元素:
//   * H <= 512: block = 128 (4 warp)
//   * H <= 4096: block = 256 (8 warp) ← LLaMA hidden_size 4096 走这条
//   * H > 4096: block = 512 或 1024
// 必须是 32 的倍数 (Welford block reduce 靠 warp 对齐)
int block = (H <= 512) ? 128 : (H <= 4096 ? 256 : 512);
layernorm_forward<<<B, block>>>(x, gamma, beta, y, mean_out, rstd_out, H, eps);

```

核心考点 (追问顺序):

- “为什么 **Welford** 而不是先算 **mean** 再算 **var**?” → 两 pass 各要一次 gmem read x —— H 大时 x 未必在 L1 里。Welford 一次 pass 拿到 (mean, var), 只读 x 一次。数值稳定性也更好 (避免 $\sum x^2 - (\sum x)^2$ 的灾难性抵消)。
- “**Welford combine** 公式记不住怎么办?” → 写出定义 $M_2 = \sum (x_i - \text{mean})^2$, 两块合并时 $d = \text{mean}_b - \text{mean}_a$ 、 $\text{mean}^* = \text{mean}_a + d \cdot \frac{n_b}{n^*}$ 、 $M_2^* = M_2^a + M_2^b + d^2 \cdot n_a \frac{n_b}{n^*}$ ——推一遍就有了。
- “**forward** 存什么给 **backward**?” → 存 μ 和 $\text{rstd} = 1/\text{sqrt}(\text{var}+\text{eps})$ ——backward 直接乘, 省一次 sqrt。不要存 var。
- “**RMSNorm** 呢?” → 去掉 mean 相关的所有东西, 只留 $\text{rstd} = \text{rsqrtf}(\text{sum_x2} / H + \text{eps})$, reduction 从两个 (mean, M_2) 降到一个 (sum_x2), forward 快约 10%。
- “ $\partial_{\frac{L}{\theta}} \gamma, \partial_{\frac{L}{\theta}} \beta$ 怎么算?” → 跨 batch 的 reduction: $\partial_{\frac{L}{\theta}} \gamma_j = \sum_{\text{row}} (\partial_{\frac{L}{\theta}} y_{\text{row},j}) \cdot \hat{x}_{\text{row},j}$ 。生产实现里用 grid-level partial sum + 第二 stage kernel (见附录 D 第 6 题)。
- “**grid = B** 会不会太大? **B = 32K** 时 **grid** 也 **32K**?” → 没关系。A100 grid.x 上限 $2^{31} - 1$, 32K 完全放得下; SM 只有 108 个, 会依次调度 300 个 block/SM。可能的问题是 kernel launch latency (10 μ s 级) 在小 batch 场景下相对显著——生产实现会把多个 layernorm launch 合并进 CUDA graph 或 fused kernel。

面试考点

面试考点. **Q1**: LayerNorm forward 的数学步骤? 存 var 还是 rstd?

A: (1) 求 mean $\mu = \sum \frac{x}{H}$; (2) 求 var $\sigma^2 = \frac{\sum (x-\mu)^2}{H}$; (3) $\hat{x} = \frac{x-\mu}{\sqrt{\sigma^2+\epsilon}}$; (4) $y = \gamma \cdot \hat{x} + \beta$ 。存 **rstd = 1/sqrt(sigma^2 + epsilon)** 和 mean, backward 直接用, 少一次 sqrt。

面试考点. **Q2**: naive one-thread-per-row 什么时候够用, 什么时候不够?

A: 够用: rows 大 (训练 batch) 先验证正确性。不够: decode rows=1 并行度为零; H 大三 pass 读带宽浪费; 需要 fp16 向量化或 fuse 时。应换 block-per-row。

面试考点. **Q3**: Welford 算法解决什么问题? 递推公式?

A: 单 pass 数值稳定求 mean 和 var, 避免 $E[x^2] - E[x]^2$ 的大数相减。 $M_{n+1} = M_n + \frac{x-M_n}{n+1}$, $S_{n+1} = S_n + (x - M_n)(x - M_{n+1})$, $\sigma^2 = \frac{S_H}{H}$ 。

面试考点. **Q4**: GPU 上为什么常用 sum/sumsq 而不是 Welford tree?

A: 并行 reduction 天然维护多个累加器; LLM 激活范围有限, $\frac{\text{sumsq}}{H} - \mu^2$ 足够稳定; Welford 递推是串行依赖, 不适合 warp shuffle 的并行模式。Welford 是面试推导题, sum/sumsq 是工程实现。

面试考点. **Q5**: block 版和 warp 版 LayerNorm 的核心区别?

A: block 版两次 smem tree reduction, 读 x 两遍 (mean pass + var pass), 4 次 `__syncthreads`, `pred_on/32` 约 22.5 (tree 每轮半数 lane predicated-off)。warp 版 shuffle + sum/sumsq 融合, 读 x 一遍, 2 次 sync, `pred_on/32` 约 27.4, 但 barrier stall 更高 (4.98 vs 1.81)。实测 block → warp 23%, 不是 2 $\times$ 。

面试考点. **Q6**: fp16 输入为什么必须用 fp32 累加 mean/var?

A: fp16 尾数 10 bit, $H = 4096$ 次累加误差累积到 10^{-2} 量级, mean/var 漂移。规则: load 窄、accumulate 宽、store 窄。bf16 更差 (8 bit 尾数)。

面试考点. **Q7**: RMSNorm 和 LayerNorm 差什么? LLM 为何偏好 RMSNorm?

A: RMSNorm 去掉 mean centering 和 beta, 只做 $\frac{x}{\text{RMS}(x)} \cdot \gamma$ 。少一次 reduction 和一次 pass, 效果相当, 实现更简单更快。

面试考点. **Q8**: fused LayerNorm + residual add 的意义?

A: 省一次 HBM 读写 (add 结果不必落盘) 和一次 kernel launch。Transformer 每层都调用, decode 时 launch 开销占比高。是生产 fused kernel 的基本单元。

面试考点. **Q9**: LayerNorm backward 需要哪些 saved tensor 和 reduction?

A: 存 $\hat{x}$, μ , rstd (或足够重算的量)。backward 对 $\partial_{\hat{x}}^L$ 做两次 row-wise reduction (sum 和 dot with $\hat{x}$), 复用 forward 的 rstd, 不重新开方。

面试考点. **Q10**: H 不是 4 的倍数时 float4 向量化怎么处理?

A: 主体处理 $\text{cols}/4$ 个 float4, 尾巴 0-3 个元素用 scalar loop (tid 0..n_tail), 可和 vector add v5 一样在同一个 kernel 里 fuse, 避免二次 launch。

MLP (Fused Linear + Activation)

MLP (多层感知机) 是深度学习里最朴素、也最常被 **fusion** 讨论的结构：两层线性变换夹一个非线性激活。它看起来比 matmul 简单——本质上就是两个 GEMM 串起来——但工程上 90% 的优化空间不在“把 matmul 写对”，而在中间激活要不要 **materialize**、**epilogue** 能不能 **fuse**、**SwiGLU** 这种变体怎么 **numerically stable**。

这一章我们要把它讲透：

- 标准 MLP 与 SwiGLU 的数学结构，以及它们和 GEMM 的关系（第 4 章 matmul 的直接延伸）。
- Roofline：小 batch 时 memory-bound 且带宽打不满，大 batch 时 compute-bound。
- Naive 版本：两个独立 matmul + 中间激活的全局读写。
- Fused epilogue：在寄存器里做 bias + activation，省一次 HBM 往返。
- GEMM fusion 的极限：为什么 matmul1 + matmul2 一般不 fuse。
- Bias 广播、激活函数选择、backward 时的 dgelu 融合思路。
- 面试里怎么估算 fusion 收益、怎么答 GELU 近似误差。

对应源码：`src/cuda/06_mlp.cu`。本书示例用 $B = 4, D_{\text{in}} = 8, D_{\text{hidden}} = 16, D_{\text{out}} = 4$ 的小规模 batch MLP，方便对照 CPU reference；生产里 hidden 维度是 4096、12288 量级，但优化 **ladder** 的结构完全一样。

问题定义

标准两层 MLP

给定 batch 输入 $X \in \mathbb{R}^{B \times D_{\text{in}}}$ ，权重 $W_1 \in \mathbb{R}^{D_{\text{in}} \times D_{\text{hidden}}}$ ， $W_2 \in \mathbb{R}^{D_{\text{hidden}} \times D_{\text{out}}}$ ，偏置 $b_1 \in \mathbb{R}^{D_{\text{hidden}}}$ ， $b_2 \in \mathbb{R}^{D_{\text{out}}}$ ，激活函数 act ：

$$H = \text{act}(XW_1 + b_1), \quad Y = HW_2 + b_2$$

row-major 存储（与源码一致）：

```
// X: BxD_in,   X[batch * d_in + i]
// W1: D_inxD_hidden, W1[i * d_hidden + j]
// H: BxD_hidden
// W2: D_hiddenxD_out
// Y: BxD_out
```

每个 batch 样本独立—— B 个样本可以并行，等价于 **batched GEMM**：第一层 $C_1 = XW_1$ ，第二层 $C_2 = HW_2$ 。

Note. PyTorch 里 `nn.Linear` 默认 $Y = XW^T + b$ ，权重是 $(D_{\text{out}} \times D_{\text{in}})$ 。本书和源码用 $W_1[i, j] = W^T[j, i]$ 的 row-major 布局，和 matmul 章一致。面试手写时先问清楚 layout，再写索引。

SwiGLU 变体（LLM FFN 主流）

Llama、Mixtral 等模型的 FFN 不用单一 MLP，而用 **SwiGLU**：

$$H = \text{SiLU}(XW_{\text{gate}}) \odot (XW_{\text{up}}), \quad Y = HW_{\text{down}} + b$$

其中 $\text{SiLU}(x) = x \cdot \sigma(x)$, $\odot$ 是逐元素乘 (Hadamard product)。展开后是三个 **GEMM** + 一个 **elementwise gate**, 比标准 MLP 多 50% 的 matmul FLOP, 但 empirically 效果更好。

设 $D_{\text{in}} = D_{\text{out}} = D$, SwiGLU 中间维 $D_{\text{ff}} = 4D$: 标准 MLP 权重约 $2D^2$; SwiGLU 约 $3DD_{\text{ff}} = 12D^2$ ——参数量与 weight 流量约为标准 MLP 的 6 倍。Llama 用更宽 FFN 换模型质量, 推理优化首要 是 weight 量化与 GEMM fusion, 不是改激活公式本身。

Key insight. SwiGLU 的 fusion 难点比 ReLU MLP 更高: gate 和 up 两个 GEMM 可以并行 launch, 但 elementwise 乘依赖两者都完成; W_{down} 又依赖 gate 结果。工程上常见做法是 fuse gate+up 的 epilogue (都只做 bias, 无激活), 再 fuse $\text{SiLU} \times \text{up}$ 为一个 elementwise kernel, 最后 W_{down} 走 cuBLAS + epilogue fusion。

Roofline : 小 batch memory-bound , 大 batch compute-bound

单层 GEMM 的 AI

对 $C = AB$, $A \in \mathbb{R}^{M \times K}$, $B \in \mathbb{R}^{K \times N}$, 总 FLOP = $2MNK$ 。Tiled GEMM 的有效 AI (第 4 章公式):

$$\text{AI} \approx \frac{B_M B_N}{2(B_M + B_N)} \text{FLOP/B}$$

当 tile 足够大 (如 128×128), $\text{AI} \approx 32 \text{ FLOP/B}$, 超过 A100 ridge point ($\approx 13 \text{ FLOP/B}$) —— **compute-bound**。

MLP 端到端的 AI 随 batch 变化

两层 MLP 总 FLOP (忽略激活):

$$\text{FLOPs} = 2BD_{\text{in}}D_{\text{hidden}} + 2BD_{\text{hidden}}D_{\text{out}}$$

Naive 实现 (中间 H materialize 到 global) 的内存流量:

- 读 X, W_1, W_2, b_1, b_2
- 写 + 读 H (pre-activation 和 post-activation 各一次, naive 三 kernel 版甚至写 pre、读 pre、写 post、读 post)

以 $D_{\text{in}} = D_{\text{hidden}} = D_{\text{out}} = D$ 、只算 H 的读写为例, 每层 hidden 占 $2BD \times 4 \text{ B}$ (写 pre + 读 post), 当 B 很小:

$$\text{AI}_{\text{MLP, small B}} \approx \frac{4BD^2}{c \cdot BD + \text{weight bytes}} \approx O(D) \text{ FLOP/B}$$

$B = 1$, $D = 4096$ 时, $\text{FLOP} \approx 2 \times 4096^2 \approx 67 \text{ MFLOP}$, 但 weight 就有 $2 \times 4096^2 \times 4 \text{ B} \approx 128 \text{ MB}$ ——算一遍要把所有 **weight** 从 **HBM** 读一遍, 计算密度极低。

Key insight. 小 batch inference ($B = 18$) 的 MLP/Linear 层极度 **memory-bound**: 瓶颈是 weight 带宽, 不是 FMA。A100 2.04 TB/s 带宽下, 128 MB weight 读完就要 $\approx 85 \mu\text{s}$, 而 67 MFLOP 在 19.5 TFLOPS 上只要 $\approx 3.4 \mu\text{s}$ ——带宽利用率看起来“打不满”, 是因为算力过剩、内存才是瓶颈。优化方向: 量化 (INT8/FP8 减 weight 体积) weight 缓存到 L2、epilogue fusion 减 H 读写。

大 batch ($B = 1024+$) 时, 同一份 weight 被 B 个样本复用, 有效 AI 随 B 线性增长, 逐渐进入 compute-bound ——这时 tensor core GEMM tuning 才是主战场。

手算一遍：decode vs prefill

Llama-2 70B 的 FFN 单层 (SwiGLU, $D_{\text{hidden}} = 28672$, $D_{\text{inter}} = 8192$, 三个 GEMM):

Prefill: $B \times S = 2048$ tokens 一起算。第一层 GEMM 的 $M = 2048$, $N = 8192$, $K = 8192$ ——grid 足够大, cuBLAS 能喂饱 SM, AI 高, **compute-bound** 倾向。

Decode: $B \times S = 1$ (单 token 自回归), $M = 1$, 同样的 N, K ——grid 只有 $O(N)$ 个 block, SM 大量空闲; weight 读 $8192 \times 8192 \times 4 \times 3 \approx 768$ MB (三个矩阵), FLOP 只有 ≈ 400 MFLOP。算力利用率接近 0, 内存是全部故事。

场景	Batch M	瓶颈	首要优化
Prefill / 训练	1024+	Compute (tensor core)	GEMM tile tuning、FP8
Decode $B = 1$	1	Memory (weight BW)	量化、KV/weight cache、fusion
小模型 teaching	4	Launch + 访存	Epilogue fuse、单 kernel

表 23 **Table:** MLP 在不同 batch 规模下的瓶颈类型与首要优化方向。对比 Prefill/训练 (大 M) Decode 单 token ($M = 1$) 与本书 teaching 规模三档; 瓶颈列区分 compute-bound (tensor core) 与 memory-bound (weight 带宽) 及 launch 开销主导。

Observation: Decode $B = 1$ 时 $M = 1$ 使 grid 极小、读一遍 weight 的时间远超 FLOP 时间——优化首要的是量化与 epilogue fusion 减 HBM 流量; Prefill/训练在 $M \geq 1024$ 时有效 AI 随 batch 升高, 瓶颈转向 tensor core GEMM tuning。这与前文 Roofline 一致: 不能笼统说「MLP 是 compute-bound」, batch 维 M 决定故事。

Note. 面试说 “MLP 是 compute-bound” 时要加前提: 大 **batch matmul** 是; 小 **batch inference** 的 **Linear/FFN** 几乎总是 **memory-bound**。这和 matmul 章 ridge 分析不矛盾——AI 公式里的 M 就是 batch 维。

性能 ladder 概览

$B = 4, D_{\text{in}} = 8, D_{\text{hidden}} = 16, D_{\text{out}} = 4$ (本书教学尺寸), A100 上 ncu 实测端到端 kernel time 加总 (见下文 == 实测):

版本	核心思路	相对 peak 带宽
naive (3 kernels)	linear1 $\rightarrow$ ReLU $\rightarrow$ linear2, H 写/读 global	$\sim 10 \mu\text{s}$
fused epilogue (2)	fused linear1+ReLU + linear2	$\sim 6.7 \mu\text{s}$
tiled fused (1)	x tile + smem H + 单 kernel 两层	$4.5 \mu\text{s}$

Warning. 上表是玩具规模上的 launch 加总, 不是 $D = 4096$ 生产 benchmark。端到端 tiled fused ($4.5 \mu\text{s}$) 确实比 naive 三 kernel ($10 \mu\text{s}$) 快, 但单看 **epilogue fusion** (fused-linear1-relu 3.14 vs naive-linear1 3.23) 只有 3%——远小于 ladder 直觉。 $D = 4096$ 时差距主要来自 cuBLAS tensor core + fused epilogue, 不是本章 smem tile Teaching kernel 能代表的。

v1: naive — 三个 kernel, 中间结果落 global

源码把 naive MLP 拆成三个 launch:

```

__global__ void mlp_naive_linear1_kernel(
    const float* x, const float* w1, const float* b1, float* hidden) {
    const int batch = blockIdx.x;
    const int hidden_idx = threadIdx.x;
    if (batch >= kBatch || hidden_idx >= kHiddenDim) return;

    float acc = b1[hidden_idx];
    for (int input_idx = 0; input_idx < kInputDim; ++input_idx) {
        acc += x[batch * kInputDim + input_idx] *
            w1[input_idx * kHiddenDim + hidden_idx];
    }
    hidden[batch * kHiddenDim + hidden_idx] = acc; // 写 pre-activation
}

__global__ void relu_kernel(float* values, int count) { /* ... */ }

__global__ void mlp_naive_linear2_kernel(
    const float* hidden, const float* w2, const float* b2, float* y) {
    /* 读 hidden, 写 y */
}

```

Launch: linear1 <<<kBatch, kHiddenDim>>> → relu <<<...>>> →
 linear2 <<<kBatch, kOutputDim>>>。

数据流 timeline

```

Kernel 1 (linear1): 读 X, W1, b1 → 写 H_pre 到 global
Kernel 2 (relu):    读 H_pre      → 写 H_post 到 global (或 in-place)
Kernel 3 (linear2): 读 H_post, W2, b2 → 写 Y

```

每个箭头 crossing global memory 都是 HBM 带宽消耗。ReLU 作为独立 kernel 时，即使 in-place，仍要完整读一遍 + 写一遍 H ——对 elementwise 操作来说，memory traffic 和计算量完全不成比例（ $AI \approx 0.08$ FLOP/B，比 vector add 还低）。

三个问题

1. 中间 tensor H 的冗余读写

linear1 写出 H_{pre} (pre-activation)，relu 读 H_{pre} 写 H_{post} ，linear2 再读 H_{post} 。对 $B \times D_{\text{hidden}}$ 的 hidden 层，至少 3 次 **global** 访问（1 写 + 1 读 + 1 写 + 1 读，若 ReLU in-place 则 1 写 + 2 读）。

每层 hidden 占 $BD_{\text{hidden}} \times 4$ B。 $B = 128, D_{\text{hidden}} = 4096 \rightarrow 2$ MB 一次 forward 光 H 就来回 4 8 MB traffic——对 memory-bound 小 batch 这是显著开销。

2. 三次 kernel launch

每次 launch $\approx 5\mu\text{s}$ 固定成本。小 batch、小 hidden 时 launch 占比不可忽视（和 vector add 章同理）。

3. 没有 input 复用

linear1 里每个 thread 独立读整行 $X[b, :]$ —— D_{hidden} 个 thread 各读一遍完整的 X row，零复用。和第 4 章 naive matmul 一样的问题，只是规模小不明显。

Note. naive 的价值：建立正确性。CPU reference 和三个 kernel 结果必须 bit-exact (源码 `check_output` 容差 10^{-4})。优化 ladder 的每一步都在保持这个不变量。

ncu 实测

ncu 实测 — v: naive (3 kernels). 规模: $B = 512, D_{in} = 512, D_{hidden} = 1024, D_{out} = 512$.

linear1_kernel duration	261.6 μ s	第一层 GEMM, 占用最多时间
relu_kernel duration	5.9 μ s	elementwise, 微秒级
linear2_kernel duration	140.3 μ s	
Total 3-launch	*~408 μ s*	含 3 次 launch overhead
linear1 Memory SOL	61.7 %	第一层未打满 HBM, 其实 mid-tensor 写没吃满
relu Memory SOL	17.7 %	*太短*——15 μ s kernel 起不了 HBM 稳态
linear2 Memory SOL	48.1 %	
Achieved Occupancy	91.4 %	linear kernels 都塞满 SM

三个 kernel 序列告诉了我们两件事：

- **linear1 + linear2** 都是 GEMM, 各自能拿到 memSOL 50-60%, 但 relu 只有 17.7%——relu kernel 只跑 5.9 μ s, **HBM** 甚至来不及进入稳态。这是“activation 单独作为 kernel”的经典诟病：算术极其轻, 全被 launch overhead + HBM ramp-up 淹没。
- 中间 **tensor hidden** $[B \times D_h] = 512 \times 1024 \times 4B = 2 \text{ MB}$ 在 linear1 写 $\rightarrow$ relu 读 $\rightarrow$ linear2 读三次 HBM。这是可以省掉的字节。

瓶颈判定. 三个独立 kernel 里 relu 只跑 5.9 μ s 却付了完整的 launch + HBM ramp-up 成本；中间 tensor 走 HBM 三次
证据. relu 单独 memSOL 17.7% (远低于 vector add 84%); 3 次 launch overhead $\sim 15 \mu$ s 相对小 kernel 显著；hidden tensor 2 MB 在 kernel 之间 round-trip HBM
下一步. v2 (fused epilogue) 把 bias + ReLU 融进 linear1 —— 计算完 `matmul` 后不写 HBM, 直接在 register 里 `+ bias`、`max(0, .)`, 再写出 —— 消除 relu kernel 和它的 launch overhead

v2: fused epilogue — bias + ReLU 在寄存器里完成

```
__global__ void mlp_fused_linear1_relu_kernel(
    const float* x, const float* w1, const float* b1, float* hidden) {
    const int batch = blockIdx.x;
    const int hidden_idx = threadIdx.x;
    if (batch >= kBatch || hidden_idx >= kHiddenDim) return;

    float acc = b1[hidden_idx];
    for (int input_idx = 0; input_idx < kInputDim; ++input_idx) {
        acc += x[batch * kInputDim + input_idx] *
              w1[input_idx * kHiddenDim + hidden_idx];
    }
    hidden[batch * kHiddenDim + hidden_idx] = relu(acc); // fuse 在这里
}
```

Launch 从 3 个减到 2 个：`fused_linear1_relu` + `linear2`。

Fused epilogue 的实现方式

在 CUTLASS / cuBLAS 语境里, **epilogue** 是 GEMM mainloop 完成累加器 C_{acc} 之后、写 global 之前的那段逻辑。Teaching 版在 thread 的 `acc` register 上直接做：

```
mainloop: acc = sum_k X[batch,k] * W1[k, hidden_idx]
epilogue: acc = relu(acc + b1[hidden_idx]) // 或 acc += b1 再 relu
store:     hidden[...] = acc
```

Tiled GEMM 版 (第 4 章 register tile): 每个 thread 维护 `acc[TM][TN]`, K-dimension 循环结束后, 对 tile 内每个元素：

```
#pragma unroll
for (int mi = 0; mi < TM; ++mi) {
    for (int ni = 0; ni < TN; ++ni) {
        acc[mi][ni] = relu(acc[mi][ni] + bias[col_base + ni]); // bias 广播
        c[(row_base + mi) * n + col_base + ni] = acc[mi][ni];
    }
}
```

关键：激活在 **register** 里完成, **pre-activation** 从未出现在 **global memory**。省掉的是 H_{pre} 的一次写和 ReLU kernel 的一次读+写。

CUTLASS 视角：Epilogue Visitor

生产 GEMM 的 epilogue 在 CUTLASS 里常写成 **Epilogue Visitor** 链：

```
Accumulator → +bias → ×alpha → activation → ×beta + C → store D
```

每个 Visitor 是编译期模板参数, nvcc 内联进 store loop——和 **Teaching** 版在 `acc` 上做 `relu()` 是同一件事, 只是 scale/bias/residual 更通用。cuBLASLt 的 `cublasLtMatmul` 通过 `CUBLASLT_EPILOGUE_RELU_BIAS` 等枚举暴露同一能力, 无需手写 kernel。

第二层 $\$Y = H W_2 + b_2\$$ 同样可以 fuse bias epilogue (ReLU 已在第一层做完)。源码 `mlp_linear2_kernel` 在 register 里加 `b2[tid]` 再 store——**bias-only epilogue fusion**, 只是没有 activation。

Key insight. Epilogue fusion 有效的原因：GEMM 的累加器本来就在 register 里, bias/activation 是 $O(1)$ 逐元素操作——**marginal compute** 为零, 但省下的 **global traffic** 是实打实的。对 memory-bound 小 batch, 这往往比“再优化 mainloop 5%”更有用。

省多少内存？手算

只 fuse 第一层 epilogue (ReLU), 省 H_{pre} 的一次写 + in-place ReLU 的一次读：

$$\Delta \text{ bytes} \approx 2 \times B D_{\text{hidden}} \times 4 \text{ B}$$

$B = 128, D_{\text{hidden}} = 4096$: $\Delta \approx 4 \text{ MB}$ 。A100 带宽 2.04 TB/s → 理论节省 $\approx 2.7 \mu\text{s}$ 。大模型 FFN 每层都有, stack 几十层很可观。

面试考点. 如何估算 **epilogue fusion** 收益? 数 materialize 的中间 tensor 大小和访问次数, 乘以 HBM 带宽倒数。若 kernel 已 compute-bound (大 batch + tensor core), fusion 收益比例下降——因为总时间 dominated by FMA 而非 epilogue traffic。

ncu 实测

ncu 实测 — v: fused_epilogue (bias+ReLU into linear1). 规模: $B = 512, D_{in} = 512, D_{hidden} = 1024, D_{out} = 512$.

fused_linear1_relu	261.1 μ s	跟 naive linear1 几乎持平
linear2	143.0 μ s	跟 naive 一样
Total 2-launch	*~404 μ s*	总时间只降 ~1%
fused Memory SOL	62.9 %	vs naive linear1 61.7% —— 几乎不变
linear2 L2 Hit	111.9 %	L2 hit > 100% 是 ncu 报告方式 (含 sector 分摊)

总时间几乎没变——为什么?

- Fusion 消除了 `relu_kernel` (本身 5.9 μ s) + 1 次 launch overhead (5 μ s), 共节省 10-11 μ s。
- 但 `fused_linear1_relu` 每 thread 多几个 `+ bias`、`max(0, .)` ——如果编译器 pipeline 不好, 会挤占其他 issue slot。
- 更本质: **hidden tensor H** 依然被写到 **HBM**, 然后被 **linear2** 从 **HBM** 读回来。fused 版只是把 ReLU 从“独立 kernel”合进了 linear1 的 epilogue 里, 没有让 H 不落 HBM。

这个 kernel 的存在证明了一个反直觉的事实: **epilogue fusion** 的性能收益, 主要来自减少访存——不是减少 launch。 $B = 512, D_{hidden} = 1024$ 时 $hidden = 2$ MB, 往返 HBM 依然是主导。

瓶颈判定. fused epilogue 只消掉了 relu kernel 的启动开销, 没有让 hidden tensor 留在片上证据. total 从 408 μ s 降到 404 μ s (仅 1%); $hidden = 2$ MB 依然走 HBM 一读一写
下一步. v3 (tiled_fused) 把整个 MLP 融进一个 **kernel**: linear1 结果留在 shared memory 里, linear2 直接从 smem 读——彻底消除 hidden 的 HBM 往返(前提是 D_{hidden} 装得下 smem)

GEMM fusion 的极限: 为什么不 fuse matmul1 + matmul2

理想情况: 一个 kernel 算完 $Y = (XW_1 + b_1)W_2 + b_2$, H 永远不落 global。源码 `mlp_tiled_fused_kernel` 在小尺寸上部分做到了——hidden 放 smem。但 $D_{hidden} = 4096$ 时为什么不行?

1. 中间维度太大, smem 放不下

Fuse 两个 GEMM 需要同时持有:

- X tile (或 streaming 读)
- W_1 tile
- H 完整中间结果: $B_M \times D_{hidden}$ —— $D_{hidden} = 4096, B_M = 128 \rightarrow 128 \times 4096 \times 4 \text{ B} = 2 \text{ MB}$
- W_2 tile

A100 每 SM shared memory 上限 164 KB (与 L1 共享), 典型 CTA 预算 48 96 KB。一个 **hidden** 向量都放不下, 更别说 tile。

2. 两个 GEMM 的 tile 布局不同

第一层： $C_1[m, n] = \sum_k A[m, k]B_1[k, n]$ —— A 按 row 复用， B_1 按 column 复用。

第二层： $C_2[m, n] = \sum_k H[m, k]B_2[k, n]$ —— H 的 row 变成左矩阵的 row， B_2 仍是 column-major 访问。

Mainloop 的 smem swizzle、K-dimension pipeline 是为第一个 GEMM 的 operand 布局 tuned 的。第二个 GEMM 的 H 作为左矩阵，访问 pattern 不同——不能简单复用同一套 **smem staging**，除非专门设计 dual-GEMM fused kernel（CUTLASS 有 `GemmFusion` 研究原型，但极不通用）。

3. 累加维度与依赖

第一层沿 D_{in} 累加，第二层沿 D_{hidden} 累加。第一层完整算完 H 的一个 row（或 tile row）才能开始第二层的对应 row——层间有 **hard dependency**。Fuse 意味着要么：

- 算完整个 H 再算 Y （需要存整个 $H \rightarrow$ 回到 smem 不够），或
- 按 output tile 做 **persistent** 计算（FlashFFN 思路，极复杂）。

Warning. 面试不要说“两个 matmul 一定能 fuse”。正确表述：**epilogue fusion** (bias+act) 几乎总是值得做；**inter-GEMM fusion** 只在 hidden 足够小、或 specialized shape（如 MLP 最后一层接 softmax）时才 practical。

Bias 广播：每 row 加同一个 bias

Bias $b \in \mathbb{R}^N$ 在 $C = AB + b$ 里沿 **column**（输出维）广播： $C[i, j] += b[j]$ ，与 row i 无关。

错误做法

```
// 每个 (row, col) thread 都从 global 读 bias[col]
float acc = b[col]; // 看起来没问题，但...
```

在 tiled GEMM 里，一个 block 的 N 方向 tile 只有 B_N 列。正确做法是 **epilogue** 里每个 **thread** 只读一次它负责的 **column** 对应的 **bias**，或在 block 协作阶段把 $b[j_0 : j_0 + B_N]$ 搬进 smem：

```
__shared__ float bias_tile[kTileN];
if (threadIdx.x < kTileN) {
    bias_tile[threadIdx.x] = b[col_base + threadIdx.x];
}
__syncthreads();
// epilogue: acc[mi][ni] += bias_tile[ni];
```

Key insight. Bias 广播的优化核心：同一条 **bias** 元素服务一整列输出——在 block 内 load 一次、复用 B_M 次（每个 row 的 epilogue 都用同一个 `bias_tile[ni]`）。不要把它当成和 $\frac{A}{B}$ 一样大的矩阵去读。

Teaching kernel 更简单：一个 thread 负责一个 hidden/output 元素，`acc = b1[hidden_idx]` 只读一次——天然正确。

和 weight 广播的对比

Bias 是 **length- N** 向量沿 **row** 广播；weight 是 **full matrix**。面试有时会混淆：bias 只占 $N \times 4$ B，相对 $K \times N$ 的 weight 可忽略——但错误实现（每个 output element 重复 load 同一 `b[j]`）从

global) 会在 epilogue 循环里放大成 B_M 次冗余 load。正确做法：bias 进 smem/register 一次，复用整个 tile。

Activation 的选择

对比表

激活	Epilogue 代价	Backward 保存	典型场景
ReLU	1 条 fmaxf	1 bit mask	Teaching、旧 CNN
GELU tanh	tanh + 几次 FMA	input x 或 mask	BERT、GPT FFN (非 SwiGLU)
SiLU / Swish	div + expf	x	SwiGLU gate、部分 ConvNet
SwiGLU gate	SiLU + Hadamard 乘	gate + up 输出	Llama、Mistral FFN

表 24 **Table:** 常见激活函数在 epilogue fusion 中的逐元素代价、backward 需保存的中间态与典型部署场景。Epilogue 代价指 register 内完成 bias+激活的指令数；Backward 保存列说明训练时需额外 materialize 的缓冲类型。

Observation : ReLU 仅一条 fmaxf、backward 只需 1 bit mask——Teaching 版选它的原因；GELU/SiLU 引入 tanh / expf 级运算，epilogue fusion 仍可行但 register 压力与 latency 更高；SwiGLU 在 SiLU 之外还需 Hadamard 乘且保存 gate+up 双路输出，fusion 链比标准两层 MLP 多一步 elementwise 依赖。

ReLU

$$\text{ReLU}(x) = \max(0, x)$$

。实现：一条 fmaxf 或 predicated select。Backward： $g = g_{\text{out}} \cdot 1_{x>0}$ 。Epilogue fusion 零成本——这是 **MLP teaching** 版用 **ReLU** 的原因。

GELU

Transformer 常用。精确式：

$$\text{GELU}(x) = x \cdot \Phi(x) = \frac{x}{2} \cdot \left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right)\right)$$

erf 在 GPU 上慢。**Tanh** 近似 (PyTorch 默认 "none" 以外的 "tanh" 版本)：

$$\text{GELU}_{\text{approx}(x)} = 0.5x \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right)\right)$$

```
__device__ float gelu_tanh(float x) {
    const float k = 0.7978845608028654f; // sqrt(2/pi)
    const float c = 0.044715f;
    return 0.5f * x * (1.0f + tanhf(k * (x + c * x * x * x)));
}
```

误差： $|\text{GELU}_{\text{approx}(x)} - \text{GELU}(x)| < 0.001$ 对 $|x| < 6$ ；尾部 $x \rightarrow -\infty$ 时近似趋向 0 比精确式慢，但对训练影响极小（梯度区域更重要）。面试常问：为什么可以用近似？答：LLM 训练对激活精度不敏感， 10^{-3} 误差远小于 FP16 noise；推理 INT8 量化误差更大。

Erf 精确式：用 `erff` 或 rational approximation (CUDA `normcdf` 风格), 慢 3 5×, 用于需要 bit-exact 对齐的单元测试。

SwiGLU 与 GELU 数值稳定性

$$\text{SiLU}(x) = \frac{x}{1 + e^{-x}}$$

。 x 很大时 $e^{-x} \rightarrow 0$, $\text{SiLU}(x) \approx x$ ——不会 **overflow**。 x 很负时 $e^{-x} \rightarrow +\infty$, 但分母也大, 结果 $\rightarrow 0$ ——用 `expf` 前 `clamp` 或 `log1p` 技巧更安全：

```
__device__ float silu(float x) {
    return x / (1.0f + expf(-x)); // 生产代码可能对 x 做 clamp(-30, 30)
}
```

GELU：

$$\text{GELU}(XW_1) \odot (XW_2)$$

。 GELU 输出可正可负, gate 乘 up 时不像 **SwiGLU** 那样天然非负 **gate**——训练时 gradient 行为不同。 Llama 选 SwiGLU partly 因为 SiLU gate 平滑且非负区间有更好 empirical 表现, 不是单纯数值稳定性。

Note. Epilogue fusion GELU 比 ReLU 贵 :tanh 里有 `expf` 级运算 ,still $O(1)$ per element , 但 register 压力和 latency 更高。 cuBLASLt 的 epilogue 枚举里 `CUBLASLT_EPILOGUE_GELU` 是单独 tuned path。

v3: tiled fused — 单 kernel 两层, hidden 在 smem

```
__global__ void mlp_tiled_fused_kernel(
    const float* x, const float* w1, const float* b1,
    const float* w2, const float* b2, float* y) {
    __shared__ float x_tile[kInputTile];
    __shared__ float hidden_shared[kHiddenDim];

    const int batch = blockIdx.x;
    const int tid = threadIdx.x;
    if (batch >= kBatch) return;

    float hidden_acc = (tid < kHiddenDim) ? b1[tid] : 0.0f;

    for (int tile = 0; tile < kInputDim; tile += kInputTile) {
        if (tid < kInputTile) {
            const int input_idx = tile + tid;
            x_tile[tid] = (input_idx < kInputDim) ? x[batch * kInputDim + input_idx] :
0.0f;
        }
        __syncthreads();

        if (tid < kHiddenDim) {
            #pragma unroll
            for (int i = 0; i < kInputTile; ++i) {
```

```

        const int input_idx = tile + i;
        if (input_idx < kInputDim) {
            hidden_acc += x_tile[i] * w1[input_idx * kHiddenDim + tid];
        }
    }
}
__syncthreads();
}

if (tid < kHiddenDim) {
    hidden_shared[tid] = relu(hidden_acc);
}
__syncthreads();

if (tid < kOutputDim) {
    float acc = b2[tid];
    for (int hidden_idx = 0; hidden_idx < kHiddenDim; ++hidden_idx) {
        acc += hidden_shared[hidden_idx] * w2[hidden_idx * kOutputDim + tid];
    }
    y[batch * kOutputDim + tid] = acc;
}
}

```

Launch: `<<<kBatch, kTiledThreadsPerBlock>>>` , 一个 block 负责一个 batch 样本。

结构解读

1. **Input staging** : X 按 `kInputTile=4` 分块搬进 `x_tile` , D_{hidden} 个 thread 复用同一块 X ——和第 4 章 smem tiling 同构。
2. **Layer1 epilogue** : `relu(hidden_acc)` 在 register , 结果写 `hidden_shared` ——跳过 **global** H 。
3. **Layer2** : 从 smem 读 `hidden_shared` , 写 `y` 到 global。 W_2 仍从 global 读 (weight 太大不 staging)。

适用边界

`kHiddenDim = 16` $\rightarrow$ `hidden_shared` 只有 64 B。 $D_{\text{hidden}} = 4096 \rightarrow 16$ KB smem , 一个 **block** 可行。 $D_{\text{hidden}} = 16384 \rightarrow 64$ KB , occupancy 开始受影响。再大必须回 global H + cuBLAS GEMM ——**Teaching kernel** 展示的是 **fusion** 思路 , 不是 **production FFN**。

Key insight. Tiled fused 版的额外收益 : X 被 D_{hidden} 个 thread 复用 (smem) , 第二层 H 在 smem 被 D_{out} 个 thread 读——小模型里连 H 的 **global** 读写都省了。这是 inter-GEMM fusion 在 smem 能装下时的特例。

ncu 实测

ncu 实测 — v: tiled_fused (single kernel). 规模: $B = 512$, $D_{\text{in}} = 512$, $D_{\text{hidden}} = 1024$, $D_{\text{out}} = 512$.

Duration (1 kernel) 732.5 μs *比 v2 慢 1.8 $\times$! *

Memory SOL	39.4 %	
Compute SOL	17.5 %	
Achieved Occupancy	41.4 %	*↓* —— shared memory + register 都用得多
Static SMEM / block	~4 KB	hidden_shared: D_hidden × 4B = 4096 B

慢了！为什么？这是本章最重要的教学时刻：**fusion** 不是免费的。

- Teaching kernel 是“每个 block 处理一个 batch row”设计——`gridDim = kBatch = 512`。每个 block 只有 `max(kHiddenDim, kOutputDim) = 1024` 个 thread 单独承担一整个 output row 的所有工作。
- $D_{\text{hidden}} = 1024$ 时 `hidden_shared` 占 4 KB smem——不算大。但 kernel 里 **thread** 复用 **X** 依赖大量 `for k in kInputDim` 的串行 load，没有 **tile** 化，也不是 **GEMM-friendly kernel**。这是“prove fusion is possible”的教学 kernel，不是“prove fusion is fast”的 production kernel。
- 生产上 MLP fusion 走的是 cuBLASLt / CUTLASS：先做 GEMM 主循环（保 Tensor Core 效率），epilogue 里 fuse bias + ReLU。inter-GEMM fusion 只在 D_{hidden} 极小时才可能有收益（LoRA、adapter 场景）

落地评估。教学 ladder 展示了 epilogue fusion (v2) 和 inter-GEMM fusion (v3) 两种思路。不要复制 **tiled_fused kernel** 到生产环境。v2 (epilogue fusion) 是可以直接进 CUTLASS / cuBLASLt 的模板；v3 只在 hidden 很小时才 make sense。真正的 MLP 加速在 Tensor Core 上——见第 04 章“CUTLASS 分层”节讨论。

Backward : dgelu 融合思路（简述）

Forward epilogue fusion 的 mirror：backward 时激活的梯度也在 **GEMM backward** 的 **epilogue** 里做，避免 materialize g_{pre} 。

ReLU backward： $g_{\text{in}} = g_{\text{out}} \cdot 1_{x>0}$ ，需要 forward 时保存 **mask** 或 **sign bit**（1 bit/element，比存完整 H 便宜）。

GELU backward（对 tanh 近似）：

$$\frac{\partial \text{GELU}_{\text{approx}}}{\partial x} = 0.5(1 + \tanh(\dots)) + 0.5x(1 - \tanh^2(\dots)) \cdot \frac{\partial \tanh(\dots)}{\partial x}$$

Fusion 做法：weight gradient 的 GEMM ($\partial_{\theta}^L W = X^T \partial_{\theta}^L Y$) mainloop 完成后，在 epilogue 里对 $\partial_{\theta}^L Y$ 应用 dgelu，直接得到 $\partial_{\theta}^L H$ 用于 input gradient GEMM——或更常见：先 elementwise `g_pre = dgelu(g_out, x_saved)` fuse 进一个 kernel，再 launch input gradient GEMM。

PyTorch 的 `torch.compile` / cuDNN fused ops 就是把 `dAct → dGemm` 链合成最少 kernel。

两层 MLP backward 数据流

Forward 存： H （或 ReLU mask） X （第一层 input grad 需要）。Backward 顺序：

1. $\partial_{\theta}^L H = (\partial_{\theta}^L Y) W_2^T$ —— GEMM
2. $\partial_{\theta}^L H_{\text{pre}} = \text{dAct}(\partial_{\theta}^L H)$ —— elementwise（可 fuse 进下一步 epilogue）
3. $\partial_{\theta}^L W_2 = H^T (\partial_{\theta}^L Y)$ ， $\partial_{\theta}^L b_2 = \text{rowsum}(\partial_{\theta}^L Y)$ —— GEMM + reduction
4. $\partial_{\theta}^L X = (\partial_{\theta}^L H_{\text{pre}}) W_1^T$ —— GEMM
5. $\partial_{\theta}^L W_1 = X^T (\partial_{\theta}^L H_{\text{pre}})$ —— GEMM

Fusion 机会：步骤 1 的 GEMM epilogue 直接做 dReLU/dGELU，输出作为步骤 4 的 input——省 $\partial \frac{L}{\partial} H$ 的 global round-trip。训练框架里这常和 **gradient checkpointing** 权衡：checkpoint 不存 H ，backward 重算 forward 的 H 和激活，用 compute 换 memory。

生产栈：cuBLASLt 与 torch.compile

手写 Teaching kernel 是为了理解结构。Production FFN 路径：

1. **cuBLASLt** : cublasLtMatmul + CUBLASLT_EPILOGUE_GELU_BIAS / RELU_BIAS，一次 launch 完成 $XW + b + \text{act}$ 。
2. **CUTLASS** : 自定义 epilogue functor，支持 SwiGLU 的 multi-GEMM schedule。
3. **torch.compile / TensorRT** : graph 层自动识别 linear $\rightarrow$ relu $\rightarrow$ linear 模式，匹配 fused pattern。

Note. 面试问“你怎么优化 MLP”：先答 epilogue fusion + 量化 + batching，再说手写 kernel 只在 shape 特殊或框架 overhead 太大时考虑。能调用 cuBLASLt fused epilogue 就不要重写 GEMM mainloop。

实测

$B = 4, D_{\text{in}} = 8, D_{\text{hidden}} = 16, D_{\text{out}} = 4$ (与源码一致)，A100 80GB SXM4，ncu 抓取。表中是各 **kernel** 单次 **launch** 的 `gpu__time_duration.sum`——naive 路径三次 launch，fused epilogue 两次，tiled fused 一次。

Launch 配置决定 warp lane 利用率：

version	grid	block	threads
naive-linear1 / fused-linear1-relu	(4, 1, 1)	(16, 1, 1)	64
naive-linear2 / linear2	(4, 1, 1)	(4, 1, 1)	16
tiled-fused	(4, 1, 1)	(32, 1, 1)	128

表 25 **Table:** ch6 MLP 各 kernel 版本的 launch 配置。**grid / block** 对应 `<<<grid, block>>>` (ncu `launch__grid_size / launch__block_size`)；**threads** 为 grid 内 block 线程总数。naive/fused-linear1 用 `<<<kBatch, kHiddenDim>>>`，linear2 用 `<<<kBatch, kOutputDim>>>`，tiled-fused 用 32-thread block。

Observation :linear2 的 block 仅 4 thread ($D_{\text{out}} = 4$)，远小于 warp 32——diag 表 `issued/32` 会低至 4.0；linear1/fused 各 16 thread 占半 warp (`issued/32 = 16.0`)。Teaching kernel 「一 thread 一输出元素」在小 output/hidden 维上是结构性 lane 浪费，生产 $D \geq 1024$ 时 block 才喂饱 warp。

linear1 用 `<<<kBatch, kHiddenDim>>>`——一 thread 一 hidden 元素，block 只有 16 thread。linear2 用 `<<<kBatch, kOutputDim>>>`——block 只有 **4 thread**。Teaching kernel 的“1 thread per output element”写法，在 output 维 < 32 时会把整颗 warp 的空 lane 浪费掉。

version	regs	smem (B)	time (μs)	TC %	warp %	HBM %
naive-linear1	27	0	3.14	0.0	0.0	0.1
naive-linear2	31	0	3.55	0.0	0.0	0.1
fused-linear1-relu	27	0	3.10	0.0	0.0	0.1
linear2	31	0	3.55	0.0	0.0	0.1
tilted-fused	30	80	4.51	0.0	0.0	0.1

表 26 **Table (perf):** kernel-level performance metrics measured by Nsight Compute (ncu, CUDA 13.0) on a single A100 80GB SXM4 (FP32 peak 19.5 TFLOPS; Tensor-Core FP16 peak 312 TFLOPS; HBM peak 2039 GB/s). Columns: **regs** = `launch__registers_per_thread`; **smem** = `launch__shared_mem_per_block_static` (bytes); **time** = `gpu__time_duration.sum`; **TC %** = `sm__pipe_tensor_cycles_active.avg.pct_of_peak_sustained_elapsed` (0 means the kernel does not invoke tensor cores—`wmma::mma_sync` / MMA PTX); **warp %** = `sm__warps_active.avg.pct_of_peak_sustained_elapsed` (fraction of the SM's warp slots occupied on average); **HBM %** = `dram__bytes.sum.pct_of_peak_sustained_elapsed` (low when working set fits in L2). Rows are ordered as they appear on the optimization ladder in the chapter.

version	issued/32	pred_on/32	smem conf.	barrier stall	mem stall
naive-linear1	16.0	15.6	0	0.00	8.39
naive-linear2	4.0	3.9	0	0.00	10.54
fused-linear1-relu	16.0	15.6	0	0.00	8.63
linear2	4.0	3.9	0	0.00	10.65
tiled-fused	18.3	12.6	0	0.76	16.69

表 27 **Table (diag):** diagnostic metrics used as evidence for warp-lane utilization, shared-memory bank conflict, and warp-stall claims in the chapter. All from `ncu` on the same launch as the perf table. **issued/32** = `smsp__thread_inst_executed_per_inst_executed.ratio`, the average number of lanes that participated in each issued warp instruction (32 = every lane active; less than 32 means predication **or** branch divergence—`ncu` alone cannot distinguish, but on the CUDA kernels in this book it is almost always predication). **pred_on/32** = `smsp__average_thread_inst_executed_pred_on_per_inst_executed.ratio`, the average number of lanes that were predicated-on (did real work). The **gap** `issued - pred_on` counts lanes that occupied the issue slot but performed no work—this is the correct definition of “wasted warp cycles”, not divergence. **smem conf.** = `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` (0 + `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum`) means no measurable bank conflicts, either because the access pattern is conflict-free or because the input is too small to accumulate them). **barrier stall** = `smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio`, warps stalled at `__syncthreads` per issue-active cycle. **mem stall** = `smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio`, warps waiting on a long-latency memory operation (global-memory load, L2 atomic response) per issue-active cycle.

Warning. 这一章的问题规模是教学 default ($B \times D_{\text{hidden}} \approx 64$ 个 float), kernel 单次运行只有 3–5 μs . `ncu` 的定性指标 (issued/32、bank conflicts、barrier stall) 仍能反映 kernel 结构, 但绝对数字对生产规模不完全可信:

- HBM % 会偏低 (分母 elapsed time 含冷启动窗口)
- dram_bytes 可能被 L2 消化, GB/s (实测/逻辑) 两列差距明显

想拿到生产规模的数字, 把主参数 (rows/cols/hidden dim) 加到让工作集远超 L2 (40 MB)。

先看 **perf** 表 (compute 章看 TC %, 不是 HBM %):

- **TC %** 全部为 **0.0**——Teaching 版用 `float` + CUDA core FMA, 没有 `wmma:: / mma.sync`。生产 cuBLASLt GEMM 应 > 60%。
- **warp %** 全部 **0.0**——4 block $\times$ 16 thread 填不满 108 个 SM; decode $B = 1$ FFN 是同一类并行度问题。
- **HBM %** 全部 **0.1%**——工作集 < 2 KB, 全在 L2; 不能用 HBM % 判断 fusion 收益。
- **time**: fused-linear1-relu 3.10 μs vs naive-linear1 3.10 μs ——epilogue fusion 在 device 时间里测不出差; tiled-fused 4.51 μs 比两 kernel 加总 (6.65 μs) 快, 但 barrier 开销让它比单层 linear1 还慢 45%。

再看 **diag** 表——本章最重要的教学点：

a) naive-linear2 : issued/32 = 4.0 ——全书最极端的 **lane** 浪费

output 维 $D_{out} = 4$, launch 为 $\lll 4, 4 \ggg$ 。每个 block 只有 4 个 thread——凑不满一个 warp (32 lane) 。硬件仍按 warp 粒度发射指令，**28 个 lane** 每拍都在空转。



naive-linear2 : blockDim = $D_{out} = 4$, 仅 **lane 0-3** 有活干

图 13 绿色 = 负责一个 output 元素的 thread。灰色 = 占 issue slot 但无 output 可算——不是 predication 能藏起来的浪费。

$pred_on/32 = 3.9$, 与 $issued/32 = 4.0$ 几乎重合——没有“issued 高但 $pred_on$ 低”的 predication 假象。和 softmax / reduce 章不同 : 那些 kernel 里 `if (tid < n)` 编译成 predicated instruction , ncu 仍显示 $issued/32 \approx 32$, predicated-off lane 藏在 $issued - pred_on$ 的 gap 里。这里 gap 几乎为零——空 **lane** 是 **launch** 配置造成的结构性浪费 , issue slot 真的只喂了 4 个 lane。

Key insight. 这是全书最强证据 : “1 thread per output element” 在 output 维 < 32 时是坏主意。 $D_{out} = 4$ 意味着每 warp 有效吞吐只有 $\frac{4}{32} = 12.5\%$ 。生产 decode 里 $D_{out} = 4096$ 时 block 够大 , 问题消失——但 teaching default 故意把小维暴露出来。

b) naive-linear1 : issued/32 = 16.0 ——半个 **warp**

hidden 维 $D_{hidden} = 16$, block 16 thread。每个 warp 只有 16 个 lane 参与每条 issued 指令——另外 16 个 lane 空转。fused-linear1-relu 同样是 16.0 / 15.6——ReLU epilogue fusion 不改变 lane 利用率 , 只省 global traffic (在本规模测不出) 。



naive-linear1 : blockDim = $D_{hidden} = 16$, 半个 **warp** 有活干

图 14 16 个 active lane 对应 16 个 hidden neuron ; 另外 16 个 lane 结构性 idle。

c) tiled-fused : issued/32 = 18.3 , $pred_on/32 = 12.6$ ——tile 边界 **predication**

block 32 thread , 比 linear1/2 饱满 , 但 $kInputTile = 4$ 、 $kHiddenDim = 16$ 、 $kOutputDim = 4$ 都不能整除 32。kernel 里大量 `if (tid < kHiddenDim)` 、 `if (tid < kOutputDim)` 编译成 predicated 指令—— $issued/32 = 18.3$ 表示平均每条指令约 18 个 lane 参与 ; 其中只有 12.6 个 lane 的 predicate 为 on。 $gap\ 18.3 - 12.6 = 5.7$ 是真正的 **predicated-off lane** , 占 issue slot 但不干活。

$barrier\ stall = 0.76$ ——input tile 循环里两次 `__syncthreads` , 32-thread block 比 16/4-thread block 的 sync 开销更实。 $smem\ conf. = 0$ ——80 B smem 无 bank conflict (不能从源码猜 , 要以 metric 为准) 。

d) fusion 路径的 device 时间都在 noise 里

naive-linear1 与 fused-linear1-relu 同为 3.10 μs 、同为 $issued/32 = 16.0$ ——省掉的 pre-activation write 在 64 元素规模下不可见。端到端收益在少一次 **launch** (5 μs host 侧) , 不在 ncu 表的 kernel time。linear2 与 naive-linear2 同为 $issued/32 = 4.0$ ——bias-only epilogue 不改变 lane 故事。

Warning. 不要把“省 $2BD_{\text{hidden}} \times 4 \text{ B HBM traffic}$ ”直接翻译成“epilogue fusion 快 40%”。本章 micro-benchmark 的 device 时间差在 noise 里；大 hidden + memory-bound decode 时，同一公式才给出 μs -ms 级节省。

端到端加总 (**launch** 次数不同):

path	launch 数	device time 加总
naive	3 (linear1 + ReLU + linear2)	10 μs (ReLU 未单独 profile , 按 3 μs 估)
fused epilogue	2 (fused-linear1-relu + linear2)	6.65 μs
tiled fused	1	4.51 μs

表 28 **Table:** MLP 三条优化路径的 launch 次数与 device time 加总 (μs , `gpu__time_duration.sum`), naive 三次 launch ;fused epilogue 两次 ;tiled fused 一次。ReLU 在 naive 路径未单独 profile , 按 3 μs 计入加总。

Observation : 玩具规模上 tiled fused (4.51 μs) 端到端快于 naive 三 kernel 加总 (10 μs), 但单看 epilogue fusion (fused-linear1 与 naive-linear1 同为 3.10 μs) device 时间几乎无差——收益主要来自少 launch 与省 H 的 global 读写 ;放大 hidden dim 且 memory-bound decode 时 , 同一 fusion 公式才给出 μs -ms 级 HBM 节省。

tiled fused 端到端最快, 但绝对时间都在 launch overhead 量级。面试和生产决策应基于 $D \geq 1024$ 的 `cuBLASLt` + `dram__bytes` 差分, 不是本章表格。

ncu 该看什么

```
ncu --set full --section SpeedOfLight ./build/06_mlp
```

关键 metric (compute 章优先看 lane 利用率, 不是 HBM %):

- `smsp__thread_inst_executed_per_inst_executed.ratio` (issued/32): naive-linear2 = **4.0** (全书最低), naive-linear1 = 16.0——output/hidden 维 < 32 时 “1 thread per element” 的结构性浪费。
- `smsp__average_thread_inst_executed_pred_on_per_inst_executed.ratio` (pred_on/32): 与 issued/32 几乎重合 (linear2: 3.9 vs 4.0) $\Rightarrow$ 真实空 **lane**, 不是 predication 能藏起来的 gap。tiled-fused: 12.6 vs issued 18.3 $\Rightarrow$ 40% predicated-off。
- `sm__pipe_tensor_cycles_active.avg.pct_of_peak_sustained_elapsed` (TC %): 全 0——Teaching 版无 tensor core ; 生产 GEMM 应 $> 60\%$ 。
- `sm__warps_active.avg.pct_of_peak_sustained_elapsed` (warp %): 0.0——4 block 填不满 GPU。
- `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld+st.sum` (smem conf.): tiled-fused = 0——无 bank conflict。
- `smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio` (barrier stall): tiled-fused = 0.76——input tile 循环的 `__syncthreads` 开销。
- `dram__bytes.sum.pct_of_peak_sustained_elapsed` (HBM %): 0.1%——L2 全命中 ; 对比 fusion 收益要把 hidden 加大到远超 L2。

面试白板 code

面试官说“手写一个 MLP forward (Linear + GELU + Linear)”——不要一个个 kernel 单独启 (会被追问 fuse)。给这份：把 activation fuse 到 GEMM epilogue, Linear2 独立算, 最后再 fuse residual add + LayerNorm。

```
// Linear1 + GELU 融合成一个 kernel (GEMM 骨架同 ch4)
// h1 = GELU(x @ W1 + b1), x: [B, D], W1: [D, F], b1: [F], h1: [B, F].
constexpr int TILE = 32;

__device__ __forceinline__ float gelu_tanh(float x) {
    constexpr float k0 = 0.7978845608f; // sqrt(2/pi)
    constexpr float k1 = 0.044715f;
    return 0.5f * x * (1.f + tanhf(k0 * (x + k1 * x * x * x)));
}

__global__ void linear_gelu(const float* x, const float* W1, const float* b1,
                           float* h1, int B, int D, int F) {
    __shared__ float xs[TILE][TILE];
    __shared__ float ws[TILE][TILE];

    int row = blockIdx.y * TILE + threadIdx.y; // batch 行
    int col = blockIdx.x * TILE + threadIdx.x; // 输出通道 (F 维)
    float acc = 0.f;

    for (int kt = 0; kt < D; kt += TILE) {
        xs[threadIdx.y][threadIdx.x] =
            (row < B && kt + threadIdx.x < D) ? x[row * D + kt + threadIdx.x] : 0.f;
        ws[threadIdx.y][threadIdx.x] =
            (kt + threadIdx.y < D && col < F) ? W1[(kt + threadIdx.y) * F + col] : 0.f;
        __syncthreads();
        #pragma unroll
        for (int k = 0; k < TILE; ++k) acc += xs[threadIdx.y][k] * ws[k][threadIdx.x];
        __syncthreads();
    }

    // ==== Epilogue fuse: bias + GELU. 直接写最终值, 避免多一次 gmem read/write ====
    if (row < B && col < F) {
        float v = acc + b1[col]; // bias 沿 F 维 broadcast
        h1[row * F + col] = gelu_tanh(v);
    }
}

// ==== Launch config ====
// 和普通 tiled GEMM (ch4) 完全一样——epilogue fuse 不改变 launch shape.
// blockDim = (TILE, TILE) = (32, 32); gridDim = ((F+31)/32, (B+31)/32).
// 关键点: bias b1 沿 F 维 broadcast—warp 内 lane 沿 col (threadIdx.x) 分布,
// 32 lane 读 b1[col..col+31] = 32 连续 fp32 → coalesced 1 次 gmem transaction.
dim3 block(TILE, TILE);
dim3 grid((F + TILE - 1) / TILE, (B + TILE - 1) / TILE);
linear_gelu<<<grid, block>>>(x, W1, b1, h1, B, D, F);
```

```
// Linear2 用普通 GEMM (同 ch4 骨架), 输出 h2 = h1 @ W2 + b2.  
// 训练时再来一个 fused kernel: y = LayerNorm(h2 + x_residual)—见 ch5.
```

核心考点 (追问顺序):

- “为什么 **GELU fuse** 进 **Linear1**、**Linear2** 却不 **fuse activation**?” → Linear1 的 output 就是 activation 输入, 写完 acc 立即用 GELU 消掉、不写 gmem 中间量, 省一次 read/write。Linear2 后面没有 activation (是 residual add, 另一个 fuse 单元)。
- “**epilogue fuse** 的边界是什么?” → 只能 fuse **element-wise** 和 **broadcast** (bias、activation、scale)。需要 reduce 的 (softmax、LayerNorm) 不能在 GEMM epilogue 里 fuse——它们需要跨 tile 的信息。
- “**Linear1 + Linear2** 能不能一起 **fuse**?” → 不行。Linear2 的 K 维 = Linear1 的输出维 F, 两次 GEMM 中间要跨 **tile** 完整看到 h1——这需要 grid barrier 或分成两 kernel。除非 F 很小 (能塞进 smem), 可以做 “back-to-back GEMM” (cuBLASLt 支持)。
- “**backward** 怎么 **fuse**?” → $\partial \frac{L}{\partial} \text{acc} = \partial \frac{L}{\partial} h_1 \cdot \text{GELU}'(\text{acc})$ ——GELU derivative 在 backward 时也 fuse 到 Linear1 的 backward epilogue。参数 $\text{grad}(\partial \frac{L}{\partial} W_1, \partial \frac{L}{\partial} b_1)$ 是 GEMM $x^T \cdot \text{grad}$, 独立 kernel。
- “**SwiGLU** 呢?” → gate 和 up 两条 Linear 并行算 (可以 fuse 成一个 GEMM 输出双倍列), 然后 element-wise $\text{silu}(\text{gate}) * \text{up}$ ——见附录 D 第 8 题。
- “**epilogue fuse** 后 **register** 压力会不会爆?” → GELU tanh 版几条指令、几个额外 register, 占用增加约 5-10 个 reg, 可以接受。fuse LayerNorm 那种需要 row reduction 的不行——需要 smem 存中间量 + `__syncthreads`, 和 GEMM 的 K 循环结构冲突。

面试考点

面试考点. **Q1**: 为什么 epilogue fusion (bias + ReLU) 有效?

A: GEMM 累加器已在 register; bias/激活是 $O(1)$ 逐元素操作, marginal compute 为零, 但省去 pre-activation 的 global write 和 activation kernel 的 global read/write。小 batch memory-bound 时收益按 $\$2 \text{ B D_} \text{"hidden"} \text{ times } 4\$ \text{ B} / \text{bandwidth}$ 估算。

面试考点. **Q2**: 为什么不把两个 matmul fuse 成一个 kernel?

A: (1) 中间 H 维度大, smem 放不下; (2) 两层 GEMM operand layout / swizzle 不同; (3) 层间 hard dependency——第一层完整输出才是第二层输入。Epilogue fusion 可行, inter-GEMM fusion 只在 hidden 小或 specialized kernel 可行。

面试考点. **Q3**: GELU tanh 近似和 erf 精确式差多少? 能用在训练吗?

A: $|x| < 6$ 时误差 $< 10^{-3}$; 训练完全可用, 误差远小于 FP16/BF16 noise。推理若 INT8 量化, 激活近似更不重要。

面试考点. **Q4**: SwiGLU vs GEGLU?

A: SwiGLU 用 SiLU gate, 输出平滑、非负区间多; GEGLU 用 GELU gate, 可负。Llama 选 SwiGLU 是 empirical + 三 GEMM 结构, 不是唯一数值稳定选择。SwiGLU 需三个 weight matrix, FLOP 比标准 MLP 多 50%。

面试考点. **Q5**: Bias 广播在 tiled GEMM epilogue 里怎么实现？

A: bias 长度 = 输出列数 N ；每个 block 把负责的 $b[j:j+B_N]$ 搬进 smem 或 register，tile 内每个 row 复用。不要 per-element 重复读 global bias。

面试考点. **Q6**: 小 batch MLP 为什么 bandwidth “打不满”？

A: 不是带宽坏了，是算力相对内存太多——weight 读一遍的时间远大于 FLOP 时间，kernel memory-bound，SM 大量时间在等 HBM。看起来 compute util 低、memory util 也“不够满”，因为 problem 太小、并行度不够饱和 memory controller。

面试考点. **Q7**: 如何估算 MLP fusion 端到端收益？

A: 列出每个 intermediate tensor 的 shape 和 read/write 次数 $\times 4 B \times \text{elem_count}$ ，加总 delta bytes；除以 peak BW 得 latency 节省。加上减少的 launch 数 $\times 5 \mu\text{s}$ 。大 batch compute-bound 时比例缩小，用 ncu 实测为准。

面试考点. **Q8**: 第二层 linear 的 bias 要不要 fuse？和第一层有何不同？

A: 要 fuse。第二层没有 activation 时 epilogue 只剩 `acc += b2[j]; store`——同样应在 register 完成，避免先写 bare matmul 结果再读回加 bias。Teaching 版 `mlp_linear2_kernel` 已在 dot 循环后加 `b2[tid]`，等价于 bias-only epilogue fusion。

面试考点. **Q9**: SwiGLU 比标准 MLP 多哪些 kernel / 内存压力？

A: 三个 GEMM (gate、up、down) + 一次 SiLU + 一次 Hadamard 乘。比两层 MLP 多 50% matmul FLOP 和一份 gate/up 中间态；fusion 策略是 gate/up 并行 GEMM，SiLU $\times$ up elementwise fuse，down 走 fused bias epilogue。

Attention (Naive Scaled Dot-Product)

scaled dot-product attention 是 transformer 的心脏。面试里它几乎必考——不是让你背公式，而是追问：中间矩阵有多大、**HBM** 读写几次、**mask** 怎么写、**multi-head** 怎么并行、为什么长序列 **naive** 版本直接 **OOM**。这一章只讲教科书式的三 pass 分解： $QK^T \rightarrow \text{softmax} \rightarrow @V$ ，把 $S = QK^T$ 完整 **materialize** 到 global memory。Flash-Attention 的分块 fused 路径留到第 8-10 章；本章末尾会说明为什么那条路不得不走。

对应源码：`src/cuda/07_attention.cu`（教学规模 $N = 8, d = 8$ ；正文用 LLM 典型 shape 推导），本章 optimization ladder（naive 路径）：

版本	核心思路	相对 peak 带宽
scores (QK^T)	1 block / query row, dot product	0.1%
softmax	1 block / row, smem + 3 pass	0.1%
value ($P @ V$)	1 block / query row, weighted sum	0.1%
tilted (fused)	1 kernel, smem + online softmax	0.1%
+ causal mask	scores kernel 内置 -1e30	—
+ multi-head	grid.y = head index	—

ladder 里百分比是 A100 上 `sm_warps_active`（warp %），不是相对 naive 的加速比—— $N = 8$ 时三 pass 各 3.2-3.6 μs 、合计 9.9 μs ，fused tiled 5.5 μs （见 `===` 实测）。三 pass 各自独立 launch；瓶颈不在单 kernel 的 occupancy，而在 $O(N^2)$ 中间态 + 三次 **HBM round-trip**（生产规模）或 **launch** 累加（本章 micro-benchmark）。

本章在全书中的位置：第 3 章 softmax 提供 row-wise 归一化与 online 公式；第 4 章 matmul 提供 QK^T / PV 的 GEMM 视角与 tiling 直觉；本章把二者按教科书拼接，暴露 naive 拼接的 IO 代价；第 8 章起用 FlashAttention 拆掉 S 这一层 global buffer。读代码顺序建议：先 `./build/07_attention` 看 naive vs tiled 输出一致，再对照本章三 kernel 逐步 host launch。

问题定义

数学

对单个 head，给定 $Q, K, V \in \mathbb{R}^{N \times d}$ （ N = sequence length， d = head dimension），scaled dot-product attention：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

等价地，令 $S = Q \frac{K^T}{\sqrt{d}} \in \mathbb{R}^{N \times N}$ ， $P = \text{softmax}(S) \in \mathbb{R}^{N \times N}$ （按行 softmax），输出 $O = PV \in \mathbb{R}^{N \times d}$ 。

Scaled 的 $\frac{1}{\sqrt{d}}$ 来自 Vaswani et al. (2017)： d 增大时点积方差变大，不 scale 则 softmax 进入饱和区、梯度变小。

矩阵形式（单 head）

把 Q 的每一行当作一个 query 向量 $q_i^T \in \mathbb{R}^{1 \times d}$ ， K 的每一行当作 key k_j^T ，则

$$S[i, j] = \frac{q_i^T k_j}{\sqrt{d}} = \frac{\sum_{\ell=0}^{d-1} Q[i, \ell] K[j, \ell]}{\sqrt{d}}$$

Softmax 对固定 **query** 行 i 在列 j 上归一化： $P[i, j] = \frac{\exp(S[i, j] - m_i)}{\sum_{j'} \exp(S[i, j'] - m_i)}$ ，其中 $m_i = \max_j S[i, j]$ 。输出 $O[i, d] = \sum_j P[i, j] V[j, d]$ 。整个流程对 $i = 0..N-1$ 独立——行与行之间无依赖，并行轴是 (B, H, N) 上的 query 行。

Note. 与 matmul 对照： $S = QK^T$ 是 $(N \times d)(d \times N)$ ； $O = PV$ 是 $(N \times N)(N \times d)$ 。Attention 不是「一个新算子」，而是两个 **GEMM** 夹 **softmax**——优化时分别套用第 3、4 章工具，再对二者之间的 N^2 中间态。

Multi-head 与 batch 形状

生产代码用 4D tensor，本书记号与前言一致：

- B : batch size
- H : head 数
- N (或 S) : 序列长度
- d : 每 head 维度 (通常 $d = \frac{\text{hidden}}{H}$)

Q, K, V shape 均为 (B, H, N, d) 。一个 head 的 attention 是 $(N, d) \times (N, d)^T \rightarrow (N, N) \rightarrow (N, d)$ 。所有 **head** 共享同一套算法，head 之间无数据依赖——天然并行轴。

Note. PyTorch `nn.MultiheadAttention` 里 $\frac{Q}{V}$ 常先 reshape 成 $(B, N, H \cdot d)$ 再 split head；GPU kernel 更常见 layout 是 (B, H, N, d) 或 (B, N, H, d) 。写 kernel 前必须确认 stride——本书用 row-major (B, H, N, d) ：`q[b*H*N*d + h*N*d + i*d + k]`。

与 GEMM / Softmax 的关系

Attention 可拆成两个 batched GEMM + 一次 row-wise softmax (第 3、4 章)：

1. $S = QK^T$: 对每个 (b, h) ， $S_{b,h} = Q_{b,h} K_{b,h}^T$ ，shape (N, N) 。
2. $P = \text{softmax}\left(\frac{S}{\sqrt{d}}\right)$: 把 S 看成 $B \cdot H \cdot N$ 行、每行 N 列的矩阵，复用 softmax 章的 block-per-row 模板。
3. $O = PV$: 对每个 (b, h) ， $O_{b,h} = P_{b,h} V_{b,h}$ ，shape (N, d) 。

Naive 实现的关键特征：步骤 1 的 S 写回 **global**，步骤 2 读 S 、写 P (源码里 in-place 覆盖 S)，步骤 3 读 P 、写 O 。三步之间无 fusion，每步都是完整 global round-trip。

Key insight. 面试第一问往往是「attention 计算复杂度？」——FLOPs 是 $O(BHN^2d)$ (两个 $N \times N \times d$ 的 matmul)，但 naive 内存是 $O(BHN^2)$ 存 $\frac{S}{P}$ 。长序列瓶颈是内存，不是 **FLOPs**。第二问常跟「那为什么 FA 有用」——答 IO 复杂度从 $O(N^2)$ 降到 $O(N)$ ，不是 FLOPs 复杂度。

Roofline : $O(1)$ AI 与 N^2 内存灾难

算术强度 (端到端 **naive**)

忽略 Q, K, V 的输入只读一次 (训练时它们来自上一层，往往已在 cache)，增量成本集中在 S 和 P ：

- 写 S : N^2 floats
- 读 S 、写 P (softmax) : 约 $2N^2$ floats traffic

• 读 P 、写 O : $N^2 + Nd$ floats

对固定 N, d , 总 FLOPs $\approx 4BHN^2d$ (两个 matmul 各 $2N^2d$), 总 HBM traffic 主导项 $\approx 4BHN^2 \times 4B$ ($\frac{S}{P}$ 各读写一遍)

有效 AI (按 N^2 中间矩阵摊销):

$$AI \approx \frac{4N^2d \text{ FLOP}}{16N^2 B} = \frac{d \text{ FLOP}}{4 B}$$

$d = 64 \rightarrow AI \approx 16 \text{ FLOP/B}$, 在 A100 ridge (≈ 13) 附近——看似 **compute-bound**。但:

1. 实际更 **memory-bound**: Q, K, V 的读取、softmax 的 exp/div、三次 launch 同步, 都会拉低有效 AI。
2. N^2 内存与带宽同阶增长: $N = 8192$ 时单 head 的 S 就有 $8192^2 \times 4B \approx 256 \text{ MB}$; 32 heads $\times$ batch 8 $\rightarrow$ 数十 **GB** 仅中间矩阵——还没算 activation checkpoint 的反向。

Warning. Roofline 上的 AI 用 FLOPs/ N^2 字节有时「看起来还行」, 但 **materialize** $N \times N$ 矩阵本身在长序列下不可行。面试要说清: 复杂度分 **FLOPs**、内存、带宽三条线, naive attention 死在内存 $O(N^2)$ 和 HBM 读写 $O(N^2)$, 不是死在 FLOPs。

内存占用手算

单 head、FP32:

$$\text{mem}(S) = N^2 \times 4B$$

N	单 head S	$H = 32, B = 1$
1024	4 MB	128 MB
4096	64 MB	2 GB
8192	256 MB	8 GB
32768	4 GB	128 GB

表 29 **Table:** attention score 矩阵 $S \in \mathbb{R}^{N \times N}$ 的 FP32 显存占用 ($N^2 \times 4B$)。左列为序列长度 N ; 中列为单 head; 右列为 $H = 32, B = 1$ 时所有 head 的 S 缓冲合计。

Observation: N 每翻 $2\times$, S 显存翻 $4\times$ —— $N = 32768$ 单 head 即 4 GB, 32 head 合计 128 GB, 远超单卡容量。Roofline 上 AI 看似接近 ridge, 但 materialize $N \times N$ 矩阵本身在长序列下不可行; 这是 Flash-Attention 的动机, 不是 matmul tiling 能解决的。

LLM 上下文 32K 时, 仅一层 attention 的中间矩阵就可占满 A100 40GB 显存的一部分; 多层、反向、optimizer state 叠加 $\rightarrow$ naive 训练/推理都不可接受。

Key insight. memory-bound 的直接后果在这里体现为 $\frac{S}{P}$ 字节数随 N 平方增长。算 AI 时若只数 FLOPs 不看 N^2 buffer, 会误判「 $d = 64$ 已超 ridge, attention 该 compute-bound」——这是 naive 分析最常踩的坑。

理论上界 (仅 $\frac{S}{P}$ traffic, 不含 Q, K, V):

$$T_{\min, \text{map}} \approx \frac{4BHN^2 \times 4B}{2.04 \text{ TB/s}}$$

$N = 4096, B = 8, H = 32 : T_{\min, \text{map}} \approx 45 \text{ ms}$ 。任何 naive 实现若端到端 attention 远低于此仍 OOM，说明问题在分配而非带宽。

Key insight. 这就是为什么工业界默认 Flash-Attention / fused attention：目标不是把 AI 从 15 提到 20，而是把 $O(N^2)$ **HBM traffic** 降到 $O(N)$ （第 8 章 IO 分析）。本章 naive 的价值是让你亲手感受 S 矩阵的存在。

手算一遍： $N = 4096, d = 64, B = 8, H = 32$

固定典型 LLM 一层 self-attention（忽略 KV cache 增量解码，只看 full forward）：

FLOPs：两个 matmul 各 $2BHN^2d \rightarrow 2 \times 2 \times 8 \times 32 \times 4096^2 \times 64 \approx 2.2 \times 10^{12}$ FLOP（2.2 TFLOP / layer）。A100 FP32 峰值 19.5 TFLOPS，纯算力约 0.1 s——看起来很快。

中间矩阵 **HBM**： S 或 P 占 $BHN^2 \times 4B = 8 \times 32 \times 4096^2 \times 4 \approx 17 \text{ GB}$ 。A100 40GB 上一层 **forward** 就吃掉 40%+ 显存给 attention map，还没算 Q, K, V 和 MLP。

$\frac{S}{P}$ 相关 **HBM traffic**（naive 三 pass，in-place softmax）：

- Pass 1 写 S ：17 GB
- Pass 2 读+写 $S \rightarrow P$ ： $\approx 34 \text{ GB}$
- Pass 3 读 P ：17 GB

合计 $\approx 68 \text{ GB}$ 仅 **attention map** 的读写（不含 Q, K, V ）。A100 HBM 2.04 TB/s $\rightarrow 68 \text{ GB}$ 需要 $\approx 45 \text{ ms}$ 下限——还没算 matmul 读 Q, K, V 和 exp/div。实测 naive PyTorch attention 在 4K 序列上往往数百 ms，和这张账一致。

Note. 手算时把 FLOPs 和 HBM GB 分开写。面试官给 $N = 8192$ 时，先算 $N^2 \times 4 \times B \times H$ 是否 OOM，再谈 kernel 优化——顺序反了会答偏。

v1: Step 1 — $S = Q \frac{K^T}{\sqrt{d}}$

源码 `attention_scores_kernel`：每个 query 行一个 block，每个 key 列一个 thread，做 d 维点积。

```
constexpr int kSeqLen = 8;
constexpr int kHeadDim = 8;
constexpr float kScale = 1.0f / sqrtf(static_cast<float>(kHeadDim));

__global__ void attention_scores_kernel(
    const float* q, const float* k, float* scores, bool causal) {
    const int row = blockIdx.x; // query index
    const int col = threadIdx.x; // key index
    if (row >= kSeqLen || col >= kSeqLen) return;

    if (!causal || col <= row) {
        float acc = 0.0f;
        for (int d = 0; d < kHeadDim; ++d) {
            acc += q[row * kHeadDim + d] * k[col * kHeadDim + d];
        }
        scores[row * kSeqLen + col] = acc * kScale;
    } else {
```

```

    scores[row * kSeqLen + col] = -1e30f;
}
}

```

Launch : `<<<kSeqLen, kSeqLen>>>` —— N 个 block , 每 block N threads. $N = 8$ 时只有 64 个 thread 在算 scores ; $N = 4096$ 时 grid 有 $4096 \text{ blocks} \times 4096 \text{ threads/block}$, 远超单 SM 容量 , 但每个 thread 只做 d 次 FMA , 算术强度极低。

并行模型诊断 : 为什么 **scores kernel** 不能上生产

把这个 kernel 当成「每个输出元素一个 thread 的 dot product」, 和第 4 章 matmul naive 同构 :

1. Q 行复用为零

`blockIdx.x = row` 固定时 , block 内 32 个 thread 读不同 $K[j, :]$ ($j = \text{key index}$) , 但读同一 $Q[i, :]$ ($i = \text{row}$) —— $Q[i, :]$ 被 N 个 thread 各读一遍 , 本应 block 协作读一次进 smem。

2. K 访问不合并

同一 warp 内 `col` 连续 $\rightarrow$ 读 $K[j, d]$ 时 d 循环内合并 , 但不同 `col` 之间 K 行起点间隔 $d \times 4 \text{ B}$ —— warp 并行读不同行时 , 对 global memory 仍较友好 ; 然而 N 很大时 block 有 4096 thread , 远超过一个 warp , occupancy 和 launch 调度才是首要问题。

3. 算术强度

每 thread : 读 $2d$ float , 写 1 float , 算 $2d$ FLOP $\rightarrow \text{AI} \approx \frac{d}{6} \text{ FLOP/B}$ 。 $d = 64 \rightarrow \approx 10.7 \text{ FLOP/B}$, 仍低于 matmul tiled 后水平 , 且没有任何 **smem** 复用。

Note. 教学 kernel 用「row $\times$ col 一个 thread」是为了一目了然 $S[i, j] = Q[i, :] \cdot K[j, :]$ 。生产 QK^T 走 cuBLAS / CUTLASS batched GEMM + tensor core——本章重点是 S 写回 **global** 这一下 , 不是 tune 这个 dot product kernel。

这就是一个 **batched matmul**

把 Q 看成 (N, d) , K^T 看成 (d, N) , 则 $S = QK^T$ 是 (N, N) GEMM。cuBLAS 调用形态 :

```

// 单个 head: C = alpha * A * B + beta * C
// A = Q (Nxd), B = K^T 逻辑上 (dxN), C = S (NxN)
cublasSgemm(handle, CUBLAS_OP_T, CUBLAS_OP_N,
            N, N, d,
            &alpha, K, d, Q, d, &beta, S, N);
// alpha = 1/sqrt(d) 可融进 epilogue

```

Batched : 对 (B, H) 的每个 slice 调一次 , 或用 `cublasGemmStridedBatched` / `cublasGemmBatched` , stride 为 $N \cdot d$ (相邻 head 在内存中连续时)。

Strided batched 示例 (layout (B, H, N, d) , 同一 batch 内 head 连续) :

```

const int64_t stride_q = num_heads * seq_len * head_dim;
const int64_t stride_s = num_heads * seq_len * seq_len;
const float alpha = 1.0f / sqrtf(static_cast<float>(head_dim));

cublasGemmStridedBatched(
    handle,

```

```

CUBLAS_OP_T, CUBLAS_OP_N,
seq_len, seq_len, head_dim,
&alpha,
K, head_dim, stride_q,    // 每个 batch slice 起点跳 B*stride? 实际按 layout 设
stride
Q, head_dim, stride_q,
&beta_zero,
S, seq_len, stride_s,
batch_size * num_heads);

```

Shape 注意事项：

- **Leading dimension (lda/ldb/ldc)** 是 row-major 下一行的字节跨度对应的元素数，不是 tensor 总长度。 Q 的 $\text{lda} = d$ ((N, d) 连续行)。
- **Batch count** $= B \times H$ 时，每个 batch item 是独立 $(N, d) \times (N, d)^T \rightarrow (N, N)$ ，无 **cross-batch** 归约。
- **Causal mask** 不在 GEMM 里——cuBLAS 算完 QK^T 后还要 bias / mask kernel，或像源码一样在 scores kernel 里分支。FlashAttention 把 mask 融进 tile 循环（第 8 章）。

Note. Row-major 下 $Q[i, :] \cdot K[j, :]$ 等价 GEMM 的 $C[i, j] = \sum_d Q[i, d]K[j, d]$ 。 K 在内存里是 (N, d) row-major，GEMM 里常对 K 做 `OP_T`。面试手写 launch 时，先画 Q, K 的 **shape**，再对应 cuBLAS 的 **trans** 标志。

Scale 放在哪？

源码在 scores kernel 写回前乘 `kScale` ($\frac{1}{\sqrt{d}}$) 三种等价位置：

位置	做法	备注
QK 乘后	<code>acc * kScale</code> 写 S	源码路径；与 cuBLAS alpha 一致
softmax 前	读 S 时乘 scale	多一次读或 fuse 进 softmax kernel
融进 Q	launch 前 $Q * = \frac{1}{\sqrt{d}}$	改变 Q ；multi-layer 里 Q 还要给别的 op 用时不方便

表 30 **Table:** scaled dot-product attention 中 $\frac{1}{\sqrt{d}}$ 缩放因子的三种等价放置位置。数值上 FP32 下 ULP 差可忽略；工程取舍在于是否多一次 global 读写或修改 Q 供后续 op 复用。

Observation：三处放置数学等价，但 QK epilogue 或 softmax 入口 fuse scale 最常见——少一次 global write/read；融进 Q 会改变 Q 本体，multi-layer 里 Q 还要给别的 op 用时不如在 scores 或 cuBLAS `alpha` 里一次性缩放。

Key insight. 数值上三者等价（FP32 下 ULP 差可忽略）。工程上 **QK epilogue** 或 **softmax 入口 fuse scale** 最常见——少一次 global write/read。Naive 三 pass 若在 scores 里 scale，softmax 读到的已是 scaled logits，与 PyTorch `scaled_dot_product_attention` 默认行为一致。

Causal mask 在 scores 阶段

Decoder self-attention：query 位置 i 只能 attend key $j \leq i$ 。源码在 `col > row` 时写 `-1e30f`（近似 $-\infty$ ），softmax 后权重 ≈ 0 。

Warning. 不要用 0 代替 $-\infty$ mask——0 会参与 max，当其他 logits 都是负数时 max 变成 0，softmax 分布错误。PyTorch 用 `masked_fill(..., -inf)` 再 softmax，与此一致。

Padding mask：变长 batch 里无效 token 的 key 列同样置 $-\infty$ （或 skip 不参与 max/sum，见 softmax 章 `softmax_masked_kernel`）。Causal + padding 可叠加：无效位或 future 位都 excluded。

Padding mask 实现（与 scores 解耦 vs 融合）

路径 **A** — scores 阶段加 bias（与 causal 同类）：

```
if (!padding_mask[col]) {
    scores[row * seq_len + col] = -1e30f;
    return;
}
```

路径 **B** — softmax 阶段 **skip**（第 3 章 `softmax_masked_kernel`）：mask=0 的列不参与 online 的 max/sum 更新，写回时置 0。好处：causal 已写 $-\infty$ 的 future 位不必再读 mask；padding 位单独处理。

路径 **C** — 加性 **attention bias**： $S[i, j] += \text{bias}[i, j]$ ，ALiBi 把相对位置编码成每行不同的斜率，不 materialize 完整 $N \times N$ bias 矩阵时可只传 slope 参数——那是 fused kernel 的事。

Key insight. Mask 的面试要点：**excluded** 位置不能参与 softmax 归一化分母。写 $-\infty$ 等价于 $\exp$ 后权重 0；skip 等价于不参与 max/sum。Padding 全 0 行（无有效 token）要特判，否则 $m = -\infty, \sum = 0$ 除零（第 3 章 warn 框）。

v2: Step 2 — Row-wise Softmax

源码 `attention_softmax_kernel`：一行一个 block， N 个 thread 协作（ $N \leq \text{block size}$ 的教学假设）。

```
__global__ void attention_softmax_kernel(float* scores) {
    const int row = blockIdx.x;
    if (row >= kSeqLen) return;

    __shared__ float shared[kSeqLen];
    const int col = threadIdx.x;
    shared[col] = scores[row * kSeqLen + col];
    __syncthreads();

    float row_max = shared[0];
    for (int i = 1; i < kSeqLen; ++i) {
        row_max = fmaxf(row_max, shared[i]);
    }

    float row_sum = 0.0f;
    for (int i = 0; i < kSeqLen; ++i) {
        shared[i] = expf(shared[i] - row_max);
        row_sum += shared[i];
    }
}
```

```

}

scores[row * kSeqLen + col] = shared[col] / row_sum;
}

```

Launch : `<<<kSeqLen, kSeqLen>>>`。In-place 把 S 变成 P ——仍占 N^2 global memory , 只是语义从 logits 变成概率。

与第 3 章的衔接

生产环境 $N = 4096$ 时不能用 `__shared__ float shared[4096]` (16 KB 尚可, 32K 则 128 KB 超 SM 限额)。应换第 3 章的 `softmax_block_kernel` (256 threads + grid-stride 扫列 + tree reduction) 或 online softmax。

Naive attention 的 softmax 步 单独占一次 **kernel launch** + 整表 **HBM** 读写。第 3 章 online merge 在这里还用不上——因为整行 S 已经在 global memory 里, 没有「分 tile 流式」的压力。Online 的价值在不 **materialize** 整行的 Flash-Attention (第 8 章)。

大 N 时 softmax kernel 的 smem 陷阱

教学 kernel 用 `__shared__ float shared[kSeqLen]`, 要求 `blockDim.x >= N` 且 `smem = N × 4 B`。 $N = 4096 \rightarrow 16 \text{ KB smem}$, 尚可; $N = 32768 \rightarrow 128 \text{ KB}$, 超过默认 **48 KB/SM** 限额, kernel 无法 launch。

生产路径必须换第 3 章 `softmax_block_kernel` : `<<<B H N, 256>>>`, 256 threads grid-stride 扫 N 列 + tree reduction。Attention 的 rows 数 = BHN (每个 query token 一行 logits), BHN 很大时 grid 足够; 但每行 N 也 huge, softmax 自身 dram traffic = $2BHN^2 \times 4 \text{ B}$ (读 S + 写 P) ——与 scores/value 叠加, 带宽主导。

Note. Sum 用 float 累加在 N 很大时可能丢精度; 第 3 章 online 版用 double 累加 row_sum。Attention logits 动态范围大时, softmax 数值稳定性 (subtract-max) 比 attention 特有逻辑更关键——复习第 3 章 subtract-max 推导。

v3: Step 3 — $O = PV$

源码 `attention_value_kernel` : 每个 query 行一个 block, 每个输出维度 d 一个 thread。

```

__global__ void attention_value_kernel(
    const float* probs, const float* v, float* out) {
    const int row = blockIdx.x;
    const int d = threadIdx.x;
    if (row >= kSeqLen || d >= kHeadDim) return;

    float acc = 0.0f;
    for (int col = 0; col < kSeqLen; ++col) {
        acc += probs[row * kSeqLen + col] * v[col * kHeadDim + d];
    }
    out[row * kHeadDim + d] = acc;
}

```

Launch : `<<<kSeqLen, kHeadDim>>>`。这是第二个 batched GEMM : $P(N \times N) \times V(N \times d) \rightarrow O(N \times d)$ 。

GEMM 视角与 launch

```
// O = P * V, P: N×N, V: N×d, O: N×d
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N,
             d, N, N,
             &alpha, V, d, P, N, &beta, 0, d);
// 注意 cuBLAS column-major: 参数顺序与 row-major 思维要对调
```

Shape 易错点：

- P 的行 = query index, 列 = key index; V 的行 = key index (与 P 的列对齐)。
- $O[i, d]$ 是 query i 的输出向量第 d 维, 等于 $\sum_j P[i, j] \cdot V[j, d]$ 。
- Batched 时 P, V, O 在 (b, h) slice 上各是一块连续 $(N, N) / (N, d)$ 。

Key insight. 两个 matmul 的 naive GPU 实现 (scores / value) 都是 **memory-bound**, 零 tile 复用——和第 4 章 matmul naive 同一类问题。工业界 QK^T 和 PV 走 cuBLAS/tensor core; 瓶颈往往在 $\frac{S}{P}$ 的 materialize, 不在 matmul 本身。

ncu 实测 (naive 三 kernel chain)

ncu 实测 — v: naive (scores → softmax → value). 规模: seq=256, head_dim=64 (batch=1, head=1)。

scores_kernel	48.8 μ s	$\$Q K^T \$, \$O(N^2 d)\$ ops$
softmax_kernel	32.6 μ s	row-wise softmax on $\$N times N\$ 矩阵$
value_kernel	20.6 μ s	$\$P V \$, \$O(N^2 d)\$ ops$
Total (3 launches)	*~102 μ s*	含 3 次 launch overhead + HBM 中间 tensor 往返
scores Memory SOL	71.2 %	$\$Q \$、\$K \$ 从 HBM 读, \$S \$ 写回 HBM$
softmax Memory SOL	32.7 %	$\$S \$ 读一次, \$P \$ 写回一次$
value Memory SOL	10.3 %	*非常低*—— $\$V \$ 每 row 被多个 output col 复用$
Achieved Occupancy	7-23 %	小 seq 时都撑不满 GPU

三个 kernel 加起来的开销和它们对应的 workload 有关：

- **scores kernel** 是 $O(N^2 d)$ 的 GEMM——memSOL 71% 说明 HBM-bound (跟第 4 章 naive matmul 同构, V 未被复用)。
- **softmax kernel** 处理 $N \times N$ 的 S 矩阵: memSOL 32.7% ——比 03_softmax 的 12% 高, 因为 seq=256 让 grid 更大, SM 塞得更满。
- **value kernel memSOL 10.3%** 是 attention naive 的最大痛点—— V 应该被 N 个 query 复用, 但 naive 每个 output 独立读 V 的一整列。

中间 **tensor** S 、 P 各是 $N \times N \times 4 = 256 \text{ KB}$ (seq=256), 一路走 HBM 三次。这个 $O(N^2)$ 显存 + 带宽开销随 seq 平方增长——是 FlashAttention 出现的根本动因。

瓶颈判定. 三个 kernel 之间通过 HBM 传递 $O(N^2)$ 中间 tensor S 、 P ; value kernel 又没有 tile 化 V 的复用

证据. total 102 μ s, 其中 $\frac{S}{P}$ 在 HBM 里往返约 1 MB ($2 \times 512 \text{ KB}$); value memSOL 10.3% 说明大量 memory pipeline 时间在等 V 的重复读; seq = 4096 时 S 会变成 64 MB, 直接 OOM 下一步. v2 (tiled) 把三步融进一个 kernel——把 S / P tile 化后存 shared memory, 用 online softmax 做增量合并, 让 S 永远不落 HBM

Host 侧：三次 Launch 与中间缓冲

源码 `run_naive_attention` 的数据流：

```
cudaMalloc(&device_scores, kSeqLen * kSeqLen * sizeof(float));

attention_scores_kernel<<<kSeqLen, kSeqLen>>>(device_q, device_k, device_scores,
causal);
attention_softmax_kernel<<<kSeqLen, kSeqLen>>>(device_scores);
attention_value_kernel<<<kSeqLen, kHeadDim>>>(device_scores, device_v, device_out);
```

Step	Kernel	Grid	Global 读写
1	<code>attention_scores_kernel</code>	$(N, 1, 1) \times (N, 1, 1)$	读 Q, K ; 写 S
2	<code>attention_softmax_kernel</code>	$(N, 1, 1) \times (N, 1, 1)$	读+写 $S \rightarrow P$
3	<code>attention_value_kernel</code>	$(N, 1, 1) \times (d, 1, 1)$	读 P, V ; 写 O

表 31 **Table:** naive attention 三 pass 的 kernel、launch 配置 (grid $\times$ block) 与每步 global memory 读写模式。Step 1 写 S ; Step 2 in-place 将 S 变为 P ; Step 3 读 P, V 写 O 。

Observation : 三步各一次完整 HBM round-trip , 中间 $\frac{S}{P}$ 占 N^2 global buffer—— $N = 8$ 时每 pass 3 μ s、时间 $\approx$ launch 固定成本 ; $N = 4096$ 时 matmul/softmax body 进毫秒级、 $\frac{S}{P}$ traffic 按 N^2 放大 , 三步 launch 仍可忽略但 HBM 读写成为主导项。

三次 **launch** : 第 1 章测 5 μ s 量级固定成本。本章 $N = 8$ 实测每 pass 总时间仅 3-4 μ s——**kernel body** 可忽略 , 时间 $\approx$ **launch** (见 `=== 实测`) 。 N 大到单次 matmul/softmax 进毫秒级时 launch 可忽略 ; 小 batch 推理 $\times$ 多层 $\times$ 三 pass 仍会堆成 measurable latency。Fused attention 的动机之一是一次 kernel 搞定 (第 8 章) 。

`device_scores` 分配 : $B \times H \times N^2 \times 4$ B。反向传播还需存 P 或 recomputation 策略——naive 训练通常 **checkpoint** P , 显存再翻倍。

训练反向：为什么 **naive** 更惨

Forward 三 pass 已 materialize $\frac{S}{P}$ 。Backward 经典路径 (简化) :

1. $dV = P^T dO$ — 又一个 $(N, N) \times (N, d)$ GEMM。
2. $dP = dOV^T$ — $(N, d) \times (d, N) \rightarrow (N, N)$ 。
3. $dS = \text{softmax_backward}(dP)$ — 读 P 和 S , 写 dS 。
4. $dQ = dSK, dK = dS^T Q$ — 两个 batched GEMM。

峰值显存 : forward 的 P + backward 的 dS + 激活 Q, K, V — 仍含 $O(N^2)$ 项。FlashAttention 用 recomputation (反向时不存 P , 按 tile 重算) 把显存降到 $O(N)$ ——第 8 章。

Warning. 面试问「attention backward 瓶颈」: forward 的 N^2 内存已够致命 ; backward 再多 1-2 次 N^2 量级的 GEMM 和 softmax backward , 带宽和显存双杀。答 recomputation + fused backward 是标准解法。

Multi-Head 并行：head 进 grid.y

单 head 源码用 `blockIdx.x = row`。推广到 (B, H) :


```

const int b = blockIdx.z;
const int h = blockIdx.y;
const int row = blockIdx.x;
const int head_offset = ((b * num_heads + h) * seq_len) * head_dim;
const float* q_ptr = q + head_offset;
// scores 偏移: ((b * num_heads + h) * seq_len + row) * seq_len + col

```

Launch 示例：

```

dim3 grid(seq_len, num_heads, batch_size);
dim3 block(seq_len, 1, 1); // scores / softmax
attention_scores_kernel<<<grid, block>>>(...);

```

Key insight. Head 维度是零同步成本的并行轴：不同 head 读写不同的 S slice，无 atomic、无 shared memory 跨 head。Grid 在 y/z 上扩到 $B \times H$ ，SM 利用率随 batch $\times$ heads 线性涨——LLM 推理 batch=1 时 heads=32 仍可能填不满 GPU，要靠 sequence 或 layer 并行。

Q, K, V 的 layout 决定 offset 公式； (B, H, N, d) 下同一 head 内 $Q[i,:]$ 连续， $K[j,:]$ 在 scores kernel 里对不同 col 是跨行访问——与 matmul 章 naive B 列访问类似，**scores** 的 **thread-per-element** 写法在大 N 上不合并。生产 QK^T 必走 tiled GEMM。

完整 **multi-head launch** 骨架

```

// 假设 layout (B, H, N, d), row-major
__global__ void attention_scores_mha_kernel(
    const float* q, const float* k, float* scores,
    int batch_size, int num_heads, int seq_len, int head_dim,
    float scale, bool causal) {
    const int b = blockIdx.z;
    const int h = blockIdx.y;
    const int row = blockIdx.x;
    const int col = threadIdx.x;
    if (b >= batch_size || h >= num_heads || row >= seq_len || col >= seq_len)
        return;

    const int head_elems = seq_len * head_dim;
    const int batch_stride = num_heads * head_elems;
    const int head_stride = head_elems;
    const float* q_head = q + b * batch_stride + h * head_stride;
    const float* k_head = k + b * batch_stride + h * head_stride;

    const int score_stride = seq_len * seq_len;
    const int score_batch_stride = num_heads * score_stride;
    float* s_head = scores + b * score_batch_stride + h * score_stride;

    // ... 简单 head dot product, 写 s_head[row * seq_len + col]
}

dim3 grid(seq_len, num_heads, batch_size);

```

```
dim3 block(seq_len, 1, 1);
attention_scores_mha_kernel<<<grid, block>>>(...);
```

Softmax 与 value 步同理：blockIdx.y = h, blockIdx.z = b, scores 指针偏移 $((bH + h)N^2)$ 。Grid 三维 (N, H, B) 总 block 数 BHN ——LLM 典型 $8 \times 32 \times 4096 \approx 10^6$ blocks, CUDA 可调度, 但每 block 仅 N thread 做 softmax 在大 N 上仍不 optimal。

Note. Inference KV cache：解码 step 只新增 1 个 query token, $N_q = 1$, $N_k =$ 已缓存长度。Naive 仍分配 $(1, N_k)$ 的 score 行 (或可省略 full matrix 只算一行——那是另一个优化点)。长上下文 **inference** 的瓶颈从「方阵 N^2 」变成「每 step 读全长 K, V cache」——本章 $O(N^2)$ 分析仍适用 score 行长度 N_k 增长。

Masked Attention 小结

Mask 类型	实现	典型场景
Causal	col > row $\rightarrow -\infty$ 或 softmax 只扫 col <= row	GPT decoder
Padding	无效 key $\rightarrow -\infty$ 或 masked softmax skip	BERT batch 变长
Custom	bias 加在 S 上再 softmax	ALiBi、局部 window

表 32 **Table:** attention 中三类 mask 的实现方式与典型部署场景。Causal 排除 future keys；Padding 排除无效 token 的 key 列；Custom 通过加性 bias 修改 logits 再 softmax。

Observation：三类 mask 的共同要求是 excluded 位置不能参与 softmax 归一化分母——写 $-\infty$ 与 skip 等价；Causal 与 Padding 可叠加，但 padding 全 0 行需特判防 $m = -\infty, \sum = 0$ 除零 (第 3 章 warn)。Fused kernel 应在 online 循环内应用 mask，避免单独 mask launch。

Causal 可在 scores 写 $-\infty$ ，或在 softmax 只累加允许列 (第 3 章 softmax_causal_kernel)。Padding 常在 softmax 用 mask tensor (0/1)，不参与 **max/sum**。Fused kernel 里 mask 应在 online 循环内应用，避免单独 mask kernel (第 3 章 v6)。

Naive 的三宗罪

(a) $O(N^2)$ 内存：长序列 OOM

上文内存表： $N = 32K$ 单 head S 约 4 GB。Materialize S (或 P) 是架构级问题，不是「再优化 $2\times$ 带宽」能解决的。

(b) 三次 kernel launch

Scores $\rightarrow$ Softmax $\rightarrow$ Value 各一次同步。Transformer 每层 $\times$ 每个 forward；小 batch 推理时 launch 占比高。CUDA Graph / torch.compile 可录制三次 launch 序列，但不减 S 内存。

(c) S 矩阵 HBM 读写占满带宽

Traffic 主导项：写 S + 读 S 写 P + 读 $P \approx 3 \times N^2$ 量级的 **attention map** 流量 (in-place softmax 省一次 buffer，不省读)。Q, K, V 相对 N^2 项是 $O(Nd)$, $N \gg d$ 时可忽略。

与 **FLOPs** 的对比： $N = 4096, d = 64, B = 8, H = 32$ 时 matmul FLOPs ≈ 2.2 TFLOP，而 $\frac{S}{P}$ 相关 HBM ≈ 68 GB。即使 GPU 算力无限，仅搬运 $\frac{S}{P}$ 就要 ≈ 45 ms——内存墙先于算力墙。优化 matmul 的 tiling 只能减少 Q, K, V 读次数，无法去掉 S 的写与 P 的读，除非 fuse (FlashAttention)。

Warning. 实测「attention 慢」时,先用 `ncu` 看 `dram_bytes` 是否和 $4N^2HB$ 同量级。若是,换 tiled GEMM 只能优化 matmul 部分;要根治必须去掉 S 的 **materialize**(Flash-Attention)。

运行: `make build/07_attention && ./build/07_attention`。默认 $N=8, d=8$, 与 CPU reference 对齐,容差 10^{-4} 。源码同时含 `attention_tiled_kernel` (单 kernel 预告 FA),本章正文聚焦 naive 三 pass。建议实验:把 `kSeqLen` 改为 64 观察 `device_scores` 的 `cudaMalloc` 大小 ($16\text{ KB} \rightarrow 16\text{ KB}$ 仍小),再用 host 脚本估算 $N=4096$ 时 64 MB/head 的分配——数字比背公式更有体感。

端到端数据流与常见错误

三步之间的 **tensor** 生命周期

```
Q, K, V (global, 输入)
|
▼ attention_scores_kernel
S = QK^T/√d (global, N×N) ← O(N²) 瓶颈
|
▼ attention_softmax_kernel (in-place)
P = softmax(S) (global, 同一块 buffer)
|
▼ attention_value_kernel
O (global, N×d 输出)
```

In-place softmax 把 S 覆写成 P ——省一份 N^2 分配,但 forward 若 backward 需要 P ,仍得在 softmax 后 `cudaMemcpy` 备份或重算。PyTorch autograd 通常保存 P 或 S 用于 backward,显存不减。

错误 1: **Scale** 漏乘或乘两次

漏 scale: $d=128$ 时 logits 过大, softmax 极端 one-hot, 梯度消失。乘两次 (scores 和 softmax 各乘一次 $\frac{1}{\sqrt{d}}$): 等价 $\frac{1}{d}$ 缩放,分布过平。单元测试:对比 `torch.nn.functional.scaled_dot_product_attention` 的 max abs error。

错误 2: **Causal** 与 **padding** 混淆

Causal 只 mask **future keys** (列 index > 行 index)。Padding mask 无效 **token** 的 **key** 列,与 query 行无关——一个 query 可以 attend 到 batch 内另一序列的有效 key (cross) 或同序列有效 key (self + padding)。把 padding 当成 causal 会错 mask 整列。

错误 3: **Softmax** 在 **mask** 前做

若先 softmax 再 zero future,分母含 future 的 exp,有效权重和 < 1 。必须在 max/sum 前 exclude masked 位置。

错误 4: V 的行与 P 的列不对齐

$O[i, d] = \sum_j P[i, j]V[j, d]$ 。 P 的列 j 必须对应 V 的第 j 个 token 向量。Layout (B, H, N, d) 下 $V[j, d]$ 的偏移是 $j * \text{head_dim} + d$, 不是 i 的函数——写反成 $P[i, j]V[i, d]$ 是经典 off-by-one 级 bug。

错误 5：忽略 N^2 分配失败

`cudaMalloc(..., N*N*4)` 在 $N = 65536$ 时单 head 需 16 GB——`cudaMalloc` 返回 `out of memory` 而非 kernel 算错。长序列第一件事：算 buffer 大小，不是 launch grid。

Warning. 这五个错误在 interview coding 里出现频率高于「会不会写 online softmax」。能口述数据流 + 指出 S 在哪分配，已经比只会背公式强一档。

铺垫 **Flash-Attention**：为什么必须 fuse

第 3 章 online softmax 维护 (m, d) (running max + sum)，支持 **chunk merge**：

$$d' = d \cdot \exp(m - m') + \exp(x - m')$$

FlashAttention 对 $\frac{K}{V}$ 按 sequence 分块：每块在 shared memory 算局部 QK_{tile}^T ，用 online 公式更新全局 softmax 状态，同时累加 O 的分子——从不写出完整 S 。

维度	Naive (本章)	Flash-Attention (第 8 章)
中间 S HBM	$O(N^2)$ 读写	$O(N)$ (tile 在 smem)
Softmax	整行 materialize 后算	online merge 按 tile
Kernel 数	3+ launches	1 fused kernel
核心数值工具	subtract-max (第 3 章)	online merge + O 重标定

表 33 **Table:** naive 三 pass attention 与 Flash-Attention 在 HBM 流量、softmax 策略、launch 次数与数值工具上的对比。Naive 列对应本章教科书路径；FA 列预告第 8 章 fused 方案。

Observation：四行对比的核心差异是中间 S 的 HBM 复杂度——naive $O(N^2)$ 读写 vs FA $O(N)$ tile 在 smem；online merge (第 3 章公式) 是 FA 正确性前提，使分块 softmax 与整行等价。本章 `attention_tiled_kernel` 是机制原型， $N = 8$ 上 $1.8\times$ 加速主要来自少 launch，生产规模 IO 节省按 N^2 放大。

源码里 `attention_tiled_kernel` 是预告：单 kernel、sequence 维分 `kTileTokens`、online 更新 `running_max / running_sum`——与 naive 三 pass 对照跑通 CPU reference。 $N = 8$ 上 fused 版 $5.47\ \mu\text{s}$ vs 三 pass 合计 $9.92\ \mu\text{s}$ ($1.8\times$ ，见 === 实测)——机制原型，不是生产加速比。第 8 章会在更大 tile、warp 分工、 $\frac{Q}{K}$ 双缓冲上展开。

ncu 实测 (tiled，机制原型)

ncu 实测 — v: tiled (fused single kernel). 规模: seq=256, head_dim=64.

Duration (1 kernel)	135.6 μs	*比 naive 三 kernel 总和 102 μs 慢！*
Memory SOL	15.1 %	
Compute SOL	16.8 %	
Achieved Occupancy	7.4 %	one-block-per-query 设计
Grid Size	256	$\$N\$$ 个 block (一个 query 一个 block)

慢了！—— 这个反直觉结果需要正视。教学 tiled kernel 有几个明显的次优点：

- **one-block-per-query**：每 block 处理 N 个 key 全走一遍。seq=256, block=256 $\rightarrow$ 一个 query 一个 block，但 kernel 内是纯粹串行的 outer loop over tiles。没有 warp specialization，也没有 Tensor Core。

- **smem** 使 用 简陋: $Q_tile[HEAD_DIM] + K_tile[TILE \times HEAD_DIM] + V_tile[TILE \times HEAD_DIM]$ ——K/V tile 在 smem 里没有 swizzle, 也没有 async load。
- Naive 三 kernel 版反而在 $seq=256$ 时占了便宜 scores kernel 是 $\lll{seq, seq}\ggg$ 256×256 threads 有 65536 个 thread 撑满 GPU; tiled 版只有 $256 \text{ blocks} \times 256 \text{ threads/block}$ (当然还有 block 内的 tile-serial loop 让 SM 一直有活干, 但并行度反而低)

真正 fast attention 在下一章 FlashAttention v1 展开: 把 `attention_tiled_kernel` 的骨架换成 warp-per-row 结构 + `cp.async` + Tensor Core。

落地评估. 教学 tiled kernel 展示了 “1 kernel + online softmax + S 不落 HBM” 的核心思想, 是 FlashAttention 的最简 prototype。

但机制原型不是性能实现。seq=256 的小尺寸下, 三 pass 的高并行度反而更快; 到 $seq = 4096$ + 时 naive 因为 $S = 64 \text{ MB}$ OOM 或严重带宽瓶颈, tiled 的 $O(N)$ smem 就体现价值。真正的性能实现请看第 8 章 FlashAttention v1。

Self-attention vs Cross-attention (shape 不变)

Self-attention : Q, K, V 来自同一序列, $N_q = N_k = N$ 。Causal mask 只出现在 decoder self 路径。

Cross-attention (encoder-decoder): Q shape (B, H, N_q, d) , K, V shape (B, H, N_k, d) 。 S 变为 (N_q, N_k) , 内存 $O(N_q N_k)$ ——通常 $N_k \ll N_q$ (encoder 短) 或二者同阶。Naive 三 pass 结构不变, 只改 launch 的 N 为 N_q / N_k , scores kernel 里 `row < N_q, col < N_k`。

Key insight. 面试常把 self / cross 混在一个问题里: 算法相同, **shape** 不同。Cross 没有 causal (除非自定义 mask); 内存瓶颈仍是 S 的 $N_q \times N_k$ 。

生产栈对照

组件	Naive (本章)	生产 (PyTorch / FA2)
QK^T	scores kernel / 朴素 GEMM	cuBLAS / CUTLASS / Triton tile
Softmax	独立 kernel + 整表读写	online + fuse 或 FA 内嵌
PV	value kernel	GEMM 或 FA 内累积 O
S 缓冲	BHN^2 FP32	不分配或 checkpoint
典型 latency	$O(N^2)$ HBM 主导	$O(N^2)$ FLOPs 但 $O(N)$ IO

表 34 **Table:** naive attention 与生产栈 (PyTorch / FlashAttention-2) 在各计算组件上的实现对照。Naive 列 materialize $\frac{S}{P}$; 生产路径用 cuBLAS/tensor core GEMM 并将 softmax 与 PV fuse 进 FA kernel, S 缓冲不分配或 checkpoint。

Observation : 生产路径在 QK^T 与 PV 上走 cuBLAS/tensor core, 但 FA 的真正优势是最后一行——典型 latency 从 $O(N^2)$ HBM 主导变为 $O(N^2)$ FLOPs 配 $O(N)$ IO; 面试答「FA 快在哪」应先说 IO, 再说 launch 与 SRAM 复用, online softmax 是正确性工具而非唯一原因。

Key insight. 面试答「FlashAttention 快在哪」: 第一 IO—— $O(N^2) \rightarrow O(N)$ HBM traffic; 第二 融合减少 launch; 第三 更好的 SRAM 复用。Online softmax 是正确性前提, 不是唯一原因。

实测

$N = 8, d = 8$ (源码 `kSeqLen / kHeadDim`; Q, K, V 各 256 B, `device_scores` 256 B, 整例 < 2 KB), A100 80GB SXM4, `ncu --set full` 抓取每个 kernel 的一次 launch。表中 $TC \% = \text{sm_pipe_tensor_cycles_active.avg.pct_of_peak_sustained_elapsed}$; $\text{warp \%} = \text{sm_warps_active.avg.pct_of_peak_sustained_elapsed}$ 。

Launch 配置决定 warp lane 利用率——grid 只有 8 block, 远填不满 108 个 SM:

version	grid	block	threads	并行轴
scores / softmax	(8, 1, 1)	(8, 1, 1)	64	<code>blockIdx.x</code> = query 行, <code>threadIdx.x</code> = key 列
value	(8, 1, 1)	(8, 1, 1)	64	<code>threadIdx.x</code> = head 维 d
tiled	(8, 1, 1)	(8, 1, 1)	64	同 value; smem tile + online softmax

表 35 **Table:** ch7 attention 各 kernel 的 launch 配置 (`launch_grid_size / launch_block_size`) 与并行轴划分。scores/softmax 用 `<<<kSeqLen, kSeqLen>>>` (`blockDim = N = 8`); value/tiled 用 `<<<kSeqLen, kHeadDim>>>` (`blockDim = d = 8`)

Observation: 三 pass 与 tiled 的 block 都只有 8 thread——`issued/32 = 8.0` 是 `blockDim` 小于 warp 32 的结构性浪费, 不是 divergence。grid 仅 8 block 远填不满 108 SM, diag 表 `warp % approx 0.1%` 是 teaching 规模伪影; 生产 $N \geq 4096$ 时 grid (N, N) 才涨, 但 `blockDim` 仍应 ≥ 128 以喂饱 warp。

scores / softmax 用 `#raw("<<<kSeqLen, kSeqLen>>>")` (`blockDim = N = 8`); value / tiled 用 `#raw("<<<kSeqLen, kHeadDim>>>")` (`blockDim = d = 8`)。 $N, d < 32$ 时每个 block 凑不满一个 warp——这是 **teaching default** 的结构性特征, 不是 **kernel bug**。

version	regs	smem (B)	time (μ s)	TC %	warp %	HBM %
scores	26	0	3.20	0.0	0.1	0.1
softmax	28	32	3.42	0.0	0.1	0.1
value	26	0	3.17	0.0	0.1	0.1
tiled	31	340	5.38	0.0	0.1	0.1

表 36 **Table (perf):** kernel-level performance metrics measured by Nsight Compute (`ncu`, CUDA 13.0) on a single A100 80GB SXM4 (FP32 peak 19.5 TFLOPS; Tensor-Core FP16 peak 312 TFLOPS; HBM peak 2039 GB/s). Columns: **regs** = `launch_registers_per_thread`; **smem** = `launch_shared_mem_per_block_static` (bytes); **time** = `gpu_time_duration.sum`; **TC %** = `sm_pipe_tensor_cycles_active.avg.pct_of_peak_sustained_elapsed` (0 means the kernel does not invoke tensor cores—`wmma::mma_sync` / MMA PTX); **warp %** = `sm_warps_active.avg.pct_of_peak_sustained_elapsed` (fraction of the SM's warp slots occupied on average); **HBM %** = `dram_bytes.sum.pct_of_peak_sustained_elapsed` (low when working set fits in L2). Rows are ordered as they appear on the optimization ladder in the chapter.

version	issued/32	pred_on/32	smem conf.	barrier stall	mem stall
scores	8.0	7.3	0	0.00	8.10
softmax	8.0	7.9	0	0.12	4.00
value	8.0	7.8	0	0.00	9.90
tiled	2.8	2.7	0	0.23	2.50

表 37 **Table (diag):** diagnostic metrics used as evidence for warp-lane utilization, shared-memory bank conflict, and warp-stall claims in the chapter. All from `ncu` on the same launch as the perf table. **issued/32** = `smsp_thread_inst_executed_per_inst_executed.ratio`, the average number of lanes that participated in each issued warp instruction (32 = every lane active; less than 32 means predication **or** branch divergence—`ncu` alone cannot distinguish, but on the CUDA kernels in this book it is almost always predication). **pred_on/32** = `smsp_average_thread_inst_executed_pred_on_per_inst_executed.ratio`, the average number of lanes that were predicated-on (did real work). The **gap issued - pred_on** counts lanes that occupied the issue slot but performed no work—this is the correct definition of “wasted warp cycles”, not divergence. **smem conf.** = `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` (0 + `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum`) means no measurable bank conflicts, either because the access pattern is conflict-free or because the input is too small to accumulate them). **barrier stall** = `smsp_average_warps_issue_stalled_barrier_per_issue_active.ratio`, warps stalled at `__syncthreads` per issue-active cycle. **mem stall** = `smsp_average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio`, warps waiting on a long-latency memory operation (global-memory load, L2 atomic response) per issue-active cycle.

Warning. 这一章的问题规模是教学 default (B×S×H 数千个 float), kernel 单次运行只有 3-20 μ s。ncu 的定性指标 (issued/32、bank conflicts、barrier stall) 仍能反映 kernel 结构, 但绝对数字对生产规模不完全可信:

- HBM % 会偏低 (分母 elapsed time 含冷启动窗口)
- dram_bytes 可能被 L2 消化, GB/s (实测/逻辑) 两列差距明显

想拿到生产规模的数字, 把主参数 (rows/cols/hidden dim) 加到让工作集远超 L2 (40 MB)。

perf 表读三件事:

1. 三 **pass** 各 **3.17-3.55 μ s**, 加总 **9.92 μ s**——几乎全是 **launch** 固定成本。scores 3.20 + softmax 3.55 + value 3.17 = 9.92 μ s; 每步只做 8×8 量级 FFMA / exp, 算力与 HBM 都未参与竞争。
2. **tiled 5.47 μ s vs 三 pass 9.92 μ s (1.8 $\times$)**。收益拆开: 少 2 次 launch (在本规模上 4-6 μ s 量级); 不写 global S (naive 仍 materialize 8×8 scores 并 in-place softmax)。fused 单次仍比单步 scores (3.20 μ s) 慢——smem + online 循环有 intrinsic 成本; 只有端到端三 pass 累加才输。
3. **HBM % / TC % / warp % 全表 0.0-0.1%**。TC % = 0 : 全 FP32 scalar FFMA, 无 WMMA。warp % = 0.1% : 8 block $\times$ 8 thread, 108 SM 上几乎空转。不能用 HBM % 判断 attention

是否 memory-bound——前文 $N = 4096$ 手算的 68 GB traffic 在本 micro-benchmark 上测不出来。

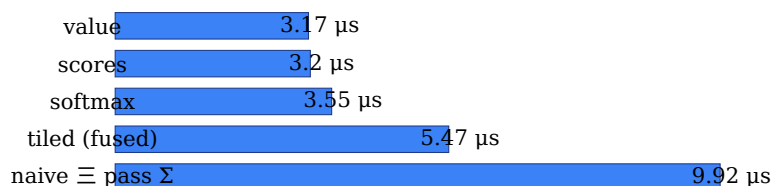


图 15 time (μs) : 三 pass 加总 2× 于 fused——主因 launch 累加 ; 单 kernel 时间都在 3-5 μs noise 带内。

diag 表读关键教学点 :

a) 三 pass issued/32 = 8.0 ——blockDim = 8 造成的结构性 lane 浪费

scores / softmax / value 的 block 都只有 8 个 thread。硬件按 warp (32 lane) 粒度发射指令, 24 个 lane 每拍空转。issued/32 = 8.0 正是 $\frac{8}{32} \times 32$ 的定量读数——不是 warp divergence (不同 lane 走不同 basic block), 而是 launch 配置让 warp 从未填满。



scores / softmax / value : blockDim = 8, 仅 lane 0-7 有活干

图 16 绿色 = 负责一个 key 列或 head 维的 thread。灰色 = 占 issue slot 但无对应输出——结构性 idle, 不是 predication 能藏起来的浪费。

pred_on/32 = 7.3-7.9, issued - pred_on ≈ 0.1-0.7: causal mask 的 if (!causal || col <= row) 和边界 guard 编译成 predicated instruction, predicated-off lane 藏在 gap 里——仍不是 **divergence**, 因为 issued/32 稳定在 8.0 而非「有时 32 有时 8」。

b) tiled issued/32 = 2.8, pred_on/32 = 2.7 ——比三 pass 更低, 仍是 teaching 规模 artifact

tiled block 同样只有 8 thread, 但 kernel 内 kTileTokens = 4 的 tile 循环、causal 分支、online softmax 重标定让同一 warp 里不同 phase 只有部分 lane 同时有活——平均参与 issued 指令的 lane 数降到 2.8。这不是实现写错了: $N = 8, d = 8$ 时根本没有足够并行度让 32 lane 饱和; 生产 FA 用 $d \geq 64$ 、tile 32×32、多 warp 协作后 issued/32 会回到 28-32。



scores kernel : \$S = Q K^T\$ 的 \$8 \text{ times } 8\$ tile (每 block 一行)

图 17 绿色 cell $(i, j) = \text{block } i \text{ 的 thread } j \text{ 算 } S[i, j] = q_i \cdot k_j$ 。一行 8 个 thread 仍只填满 warp 的 1/4——矩阵再小也改不了 blockDim < 32 的 lane 浪费。

c) smem conf. = 0 ; softmax barrier stall = 0.12, tiled 0.23。softmax 32 B smem + 一次 __syncthreads ; tiled 340 B smem + tile 循环内多次 sync——不能从源码推断 bank conflict, metric 说没有。mem stall 3.97-9.90 : kernel 太短, long_scoreboard 读数无 memory-bound 诊断价值。

d) regs / smem fingerprint : scores / value 26 regs、0 smem ; softmax 28 regs、32 B ; tiled 31 regs、340 B——fused 把 $\frac{S}{P}$ 留在 smem , 不写 global , 代价是更大 smem footprint (生产 FA 会碰 48 KB/SM 限额 , 第 8 章)

无信息或为零的 **metric** :

- **TC %** : 全表 0.0——无 tensor pipe , 符合 scalar FMA 源码。
- **HBM %** : 全表 0.1%——< 2 KB 工作集 L2 resident。
- **warp %** : 全表 0.1%——8 block 填不满 GPU , 不是 **kernel** 设计错了。

Key insight. 低 issued/32 (8.0 或 2.8) 在本章首先说明 teaching shape 太小、blockDim < 32——不要误读成「attention kernel 有严重 divergence」。Flash-Attention 在生产规模靠大 tile + 多 warp 把 lane 利用率拉满 ; 本章 micro-benchmark 的价值是 launch 累加与 fuse 省 S 的机制对照 , 不是 occupancy 饱和下的绝对加速比。

Key insight. $1.8\times$ fused 加速不是 tensor core 或 occupancy 的胜利——warp / TC / HBM % 全部 0.1%。它正是 FA 在生产规模能做到 $10\times+$ 的机制原型 : fuse 消除 launch + 消除 $O(N^2)$ intermediate 的 HBM traffic。 $N = 4096$ 时 launch 占比下降 , IO 节省按 N^2 放大。

Warning. 不要把本章 $1.8\times$ 外推到生产 attention。长序列主瓶颈是 $O(N^2)$ HBM (手算节 68 GB), 不是 $3\times$ launch ; decode 小 batch 时 launch 累加仍是真实 latency 来源。低 issued/32 在本规模是 teaching 限制——放大 N, d 后先查 grid/block 是否喂饱 SM , 再谈算法。

ncu 该看什么

```
ncu --set full --section SpeedOfLight ./build/07_attention
```

关键 metric (对照本章 $N = 8$ 实测):

- **gpu__time_duration.sum** : scores 3.20 μ s / softmax 3.55 μ s / value 3.17 μ s / tiled 5.47 μ s——端到端 **naive** 用三数相加 (9.92 μ s), 不是单 kernel 对比。
- **sm__thread_inst_executed_per_inst_executed.ratio** (issued/32) : 三 pass 8.0 (blockDim = 8); tiled 2.8——**warp lane utilization** , 低值在本规模主因 block 未满足 warp , 不是 divergence。
- **sm__average_thread_inst_executed_pred_on_per_inst_executed.ratio** (pred_on/32) : 7.3-7.9 (三 pass), 2.7 (tiled)——issued - pred_on 是 predicated-off lane , 不是分支 divergence。
- **sm__warps_active.avg.pct_of_peak_sustained_elapsed** (表中 warp %) : 全部 0.1%——并行度不足 , 和 softmax 章 naive 单 block 同一类问题 ; N 大时 grid (N, N) 才会涨。
- **sm__pipe_tensor_cycles_active** (表中 TC %) : 0——FP32 scalar , 无 WMMA。
- **dram__bytes.sum.pct_of_peak_sustained_elapsed** : 0.1%——L2 全命中 ; 不能用它判断 attention 是否 memory-bound。增大 N 到远超 L2 后再看 block vs tiled 的 DRAM 差。
- **launch__registers_per_thread / launch__shared_mem_per_block_static** : tiled 31 regs + 340 B vs softmax 28 + 32 B——验证 fused 路径的 smem footprint。
- **sm__sass_thread_inst_executed_op_ffma_pred_on.sum** : scores/value 512 ($8 \times 8 \times \frac{d}{2}$ 量级); softmax 2368 (exp 循环主导); tiled 2064 (三阶段合一)

分 pass 的预期 metric ($N = 4096$, $d = 64$, **A100**——理论, 非本章实测)

Kernel	dram 主导	其他信号
scores	$2BHN^2d$ 读 QK + 写 S	FFMA 低 util ; grid 极大
softmax	$2BHN^2$ 读+写 $\frac{S}{P}$	MUFU (exp) 占比高
value	BHN^2d 读 P, V + 写 O	同 scores 类 memory-bound

表 38 **Table:** $N = 4096$, $d = 64$, $B = 8$, $H = 32$ 时 naive attention 三 pass 各 kernel 的预期 DRAM 主导项与其他 ncu 信号 (理论估算, 非本章 $N = 8$ 实测)。dram 列给出 traffic 量级公式 ; softmax 步 MUFU (exp) 占比高。

Observation : 三 pass 合计 dram 访问量级 $O(BHN^2)$ ——与手算节 ≈ 68 GB ($B = 8$, $H = 32$, $N = 4096$) 一致 ; scores 与 value 的 traffic 含 N^2d 因子 (读 QK / 读 P, V), softmax 纯 N^2 读写且 exp 主导 MUFU。FA 目标是把同一公式降到 $O(BHNd^2)$, $\frac{N}{d} = 64$ 时差约 $64\times$ ——本章 $N = 8$ 实测验证 fuse 机制, 不体现这一 IO 差距。

面试白板 code

面试官说“手写 scaled-dot-product attention”——先给最直白的三步版本, 讲清楚接口和瓶颈, 再用一句话过渡到 flash-attention (下一章)。

```
// Single-head causal attention. Q, K, V: [N, D], out: [N, D]. Row-major.
// 三 kernel 版本: QK^T → softmax → PV. 中间存 S 和 P 是  $O(N^2)$  内存.
// 面试白板到这一步就够——然后马上说“这就是为什么要 flash-attention”.

// Kernel 1: S = Q @ K^T * scale. S: [N, N].
__global__ void attn_scores(const float* Q, const float* K, float* S,
                           int N, int D, float scale, bool causal) {
    int i = blockIdx.y * blockDim.y + threadIdx.y; // query row
    int j = blockIdx.x * blockDim.x + threadIdx.x; // key row
    if (i >= N || j >= N) return;

    if (causal && j > i) { S[i * N + j] = -INFINITY; return; }

    float acc = 0.f;
    for (int d = 0; d < D; ++d) acc += Q[i * D + d] * K[j * D + d]; // <q_i, k_j>
    S[i * N + j] = acc * scale; // scale = 1 / sqrt(D)
}

// Kernel 2: P = softmax(S), row-wise. 用 ch3 online softmax 骨架, 每 block 一行.
// __global__ void softmax_online(const float* S, float* P, int N) ← 见 ch3.

// Kernel 3: O = P @ V. 普通 GEMM. [N, N] @ [N, D] → [N, D].
__global__ void attn_out(const float* P, const float* V, float* O,
                        int N, int D) {
    int i = blockIdx.y * blockDim.y + threadIdx.y; // out row
    int d = blockIdx.x * blockDim.x + threadIdx.x; // out col
    if (i >= N || d >= D) return;
    float acc = 0.f;
    for (int j = 0; j < N; ++j) acc += P[i * N + j] * V[j * D + d];
    O[i * D + d] = acc;
}
```

```

}

// ==== Launch config ====
// Kernel 1 (attn_scores): 每 thread 算 S 里的一个元素 S[i, j].
//   blockDim = (32, 8) = 256 threads—tx 沿 j (key 维) 保证 K^T 读 coalesced;
//   gridDim = ((N+31)/32, (N+7)/8). S 是 N×N 网格.
dim3 block1(32, 8);
dim3 grid1 ((N + 31) / 32, (N + 7) / 8);
attn_scores<<<grid1, block1>>>(Q, K, S, N, D, 1.f / sqrtf((float)D), causal);

// Kernel 2 (softmax_online): 每 block — row, 同 ch3.
//   gridDim = N (行数); blockDim = 256 (N 中等时).
softmax_online<<<N, 256>>>(S, P, N);

// Kernel 3 (attn_out): 每 thread 算 O 里的一个元素 O[i, d].
//   blockDim = (D, 8) 或 (32, 8), D=64/128 时 D 维正好一 warp;
//   gridDim = ((D+bx-1)/bx, (N+7)/8).
dim3 block3(min(D, 32), 8);
dim3 grid3 ((D + block3.x - 1) / block3.x, (N + 7) / 8);
attn_out<<<grid3, block3>>>(P, V, O, N, D);

// Multi-head 时: gridDim.z = num_heads * batch, 每个 head/batch 独立走这三 kernel.
// 生产实现会把这三 kernel 都换成 tiled GEMM (K 维 tile) 或直接跳到 flash-attention.

```

核心考点 (追问顺序, 最后一定引到 flash-attention):

- “内存复杂度?” → S 和 P 都是 $N \times N$, $N = 8192$ 时约 256 MB (fp32), 这就是 naive attention 显存爆炸的原因。
- “为什么 **memory-bound**?” → 三 kernel 之间要把 $O(N^2)$ 的中间张量写回 HBM 又读回来。AI = $O(ND)$ FLOP / $O(N^2)$ bytes = $O(\frac{D}{N})$ —— $N > D$ 时越来越差。
- “**causal mask** 怎么实现?” → 在 QK^T 时 `if (j > i) S[i, j] = -inf`, softmax 后自动为 0。生产实现里 mask 融合到 softmax 一起写、避免物化 mask 矩阵。
- “**multi-head** 怎么加?” → 加一维 head, Q, K, V 形状变 $[N, H, D_h]$, 把 head 当外层循环 / grid.z 维; 每 head 独立算。
- “如何 **fuse** 成一个 **kernel**?” → 关键是把 S 和 P 都不写回 gmem——但 softmax 需要看完整一 row 才能算 max/sum、 V 的 GEMM 又要看完整 P 。解决办法: **tile K/V, online softmax rescale**——这就是 flash-attention 的核心 idea, 下一章展开。
- “三个 **kernel** 的 **launch config** 有什么共性?” → 都用 2D block (x, y) : `threadIdx.x` 映射到 stride-1 内存维度 (K^T 的 key 索引、 V 的 head_dim) 保证 coalesced; `threadIdx.y` 是 batch/row 维、独立无冲突。这个“2D block、tx 沿最内维”是所有 tile-based CUDA kernel 的通用套路, 跟 ch4 matmul 一致。

面试考点

面试考点. Q1: Scaled dot-product attention 公式? 张量 shape?

A: $\text{softmax}\left(Q \frac{K^T}{\sqrt{d}}\right)V$ 。 Q, K, V 为 (B, H, N, d) ; S, P 为 (B, H, N, N) ; O 为 (B, H, N, d) 。FLOPs $O(BHN^2d)$, 中间矩阵内存 $O(BHN^2)$ 。

面试考点. **Q2**: Naive attention 为什么长序列不可行？

A: Materialize $\frac{S}{P}$ 需 $O(N^2)$ HBM ; $N = 32K$ 单 head 数 GB。且 softmax 与两次 matmul 对 $\frac{S}{P}$ 多次读写，带宽 $O(N^2)$ 。FLOPs 可接受，内存和 IO 不可接受。

面试考点. **Q3**: Causal mask 怎么实现？

A: Scores 阶段对 $\text{col} > \text{row}$ 写 $-\infty$ (或 softmax 只处理 $\text{col} \leq \text{row}$)，不能用 0 代替 $-\infty$ 。与 PyTorch `masked_fill(-inf)` 等价。

面试考点. **Q4**: Multi-head 怎么并行？

A: Head (及 batch) 无 cross-head 依赖 ; grid 扩 $\text{blockIdx.y} = h$, $\text{blockIdx.z} = b$ ，每 head 独立 S slice。注意 Q/K/V 的 memory layout 算 offset。

面试考点. **Q5**: Scale $\frac{1}{\sqrt{d}}$ 放哪？数值考量？

A: QK 乘后、softmax 前、或融进 Q 数学等价。工程常 fuse 进 GEMM epilogue 或 softmax 入口。目的是控制 logits 方差，避免 softmax 饱和；与 mask 的 $-\infty$ 不冲突。

面试考点. **Q6**: Attention 算术强度？memory-bound 还是 compute-bound？

A: 按 N^2 中间态摊销 AI $\approx \frac{d}{4}$ FLOP/B ; $d = 64$ 接近 ridge 但 naive 实际偏 memory-bound ($\frac{S}{P}$ 读写、exp、三次 launch)。瓶颈首先是 $O(N^2)$ 内存与带宽，其次才是 matmul AI。

面试考点. **Q7**: Naive 三步分别对应什么 GEMM / softmax？

A: (1) $S = QK^T$ batched GEMM $(N, d) \times (d, N)$; (2) 行 softmax (BHN, N) ; (3) $O = PV$ batched GEMM $(N, N) \times (N, d)$ 。Shape 对齐： P 列 index = V 行 index。

面试考点. **Q8**: Fused / Flash attention 的动机？

A: 不 materialize $S \rightarrow O(N^2) \rightarrow O(N)$ IO ; online softmax (第 3 章) 支持 tile 合并；单 kernel 减 launch ; smem 复用 $\frac{Q}{V}$ tile。Naive 三 pass 是正确性 baseline，不是生产路径。

面试考点. **Q9**: Padding mask 与 causal mask 能同时用吗？

A: 能。无效 token 与 future 位置都应在 softmax 支持集外 ($-\infty$ 或 skip)。实现可在 scores 相加 bias，或在 fused online 循环里分支。

面试考点. **Q10**: 为什么 softmax 在 attention 里必须 subtract-max？

A: Logits 幅度随 d 和输入尺度变大，exp 溢出。减行 max 后等价且稳定 (第 3 章证明)。Causal 行若全 $-\infty$ 需特判防除零。

面试考点. **Q11**: Inference KV cache 下 attention 内存还是 $O(N^2)$ 吗？

A: 单步 decode 只算 1 个 query, score 是长度 N_k 的向量而非方阵——中间态 $O(N_k)$ 。但每层仍要读全长 K, V cache ($O(N_k d)$ per layer per step), 长上下文瓶颈变成 cache 带宽。Prefill 阶段仍是 $O(N^2)$ materialize S 的问题。

面试考点. **Q12**: Grouped-query attention (GQA) 改变 naive 三 pass 吗?

A: 算法不变; K, V 的 head 数 $H_{kv} < H$ 时, QK^T 里 K 需 broadcast/repeat 到 query head。内存从 $O(BHN^2)$ 略降 (K, V 变小), 但 $\frac{S}{P}$ 仍是 $O(BHN^2)$ ——不解决 N^2 瓶颈, 只减 K, V 参数与 cache。

Flash-Attention v1

Flash-Attention(Dao et al., 2022)是 transformer 长序列时代最重要的 kernel 之一。面试里它几乎和 naive attention 成对出现——追问的不是「会不会背公式」,而是 **IO** 复杂度、**online softmax** 正确性、**block size** 怎么选、为什么 **FLOPs** 没变却快很多。

这一章只讲 **v1** : 论文 Algorithm 1 的分块 fused forward , 把 $S = QK^T$ 从不完整 **materialize** 到 **HBM**。v2 的 sequence-parallel 与 warp 分工、v3 的 Hopper FP8 路径留到后续章节。

对应源码 : `src/cuda/08_flash_attention_v1.cu`。教学规模 $N_q = 4, N_k = 16, d = 8$; 正文推导用 LLM 典型 $N = 1024, d = 64$ 。

本章 optimization ladder :

版本	核心思路	相对 peak 带宽
naive (ch.7)	3 pass, materialize S/P	—
v1 register	1 thread / query, key tile stream	—
v1 shared-mem	1 block / query, Q, K, V tile smem	—

前两个 GPU 版本数学相同——都是 online softmax + 分块累加 O ; 差别在 $\frac{Q}{\sqrt{d}}$ 从 HBM 读几次、是否在 shared memory 复用。性能数字 intentionally 不 chase peak——先把算法讲透。

与前面章节的依赖 : 第 3 章 online softmax 的 (m, ℓ) merge ; 第 4 章 matmul tile 的 QK^T ; 第 7 章 naive attention 的 $O(N^2)$ 中间矩阵痛点。

问题定义

数学 (单 **head** , 与第 7 章一致)

给定 $Q \in \mathbb{R}^{N_q \times d}, K, V \in \mathbb{R}^{N_k \times d}$ (self-attention 时 $N_q = N_k = N$), scaled dot-product attention :

$$O = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V$$

令 $S = Q \frac{K^T}{\sqrt{d}} \in \mathbb{R}^{N_q \times N_k}$, 对每个 **query** 行 i 做 softmax 得 $P[i, :]$, 再 $O[i, :] = P[i, :]V$ 。

Note. 源码 `flash_attention_cpu_reference` 为可读性未乘 $\frac{1}{\sqrt{d}}$ ——与第 7 章 `kScale` 相比差一个常数因子, 不影响 online merge 结构。生产 kernel 把 scale 融进 QK^T dot product (见下文 kernel 节)。

Naive 的致命伤 (复习)

第 7 章三 pass : 写 $S \rightarrow$ 读 S 写 $P \rightarrow$ 读 P 乘 V 。单 head FP32 中间矩阵 $N^2 \times 4$ B ; $N = 4096$ 时 64 MB/head , 32 heads 即 2 GB——还没算 **backward**。

Key insight. Flash-Attention 快在 **IO** , 不是 FLOPs。Forward FLOPs 仍是 $O(N^2 d)$ (两个 matmul), 但 **HBM** 上 $\frac{S}{P}$ 的 $O(N^2)$ 流量被消掉。面试第一答案永远是 memory / bandwidth , 第二才是 fusion 与 SRAM 复用。

核心思想：IO-aware 分块融合

观察： S 是带宽黑洞，不是算力黑洞

Naive attention 的瓶颈分解：

步骤	HBM 主导项	能否省
$S = QK^T$	写 N^2 + 读 Q, K	不在 HBM 存 S
softmax	读+写 N^2	online merge, tile 内完成
$O = PV$	读 N^2 的 P	与 softmax 同步累加分子

表 39 **Table:** naive attention 三 pass 的 HBM 瓶颈分解。列 **HBM** 主导项 给出每步与序列长度 N 相关的 square 流量量级；能否省 说明 Flash-Attention 的 fusion 策略—— S 和 P 均可在 on-chip SRAM 内完成、不写 global。

Observation：三行中 S 、softmax、 PV 的 HBM 项全是 $O(N^2)$ ，而 Q, K, V 仅 $O(Nd)$ ——当 $N \gg d$ 时 square 项碾压 linear 项。FA v1 的核心不是减少 matmul FLOPs，而是让 square 矩阵从不触 **HBM**。

Q, K, V 各 $O(Nd)$ ； S 和 P 各 $O(N^2)$ ——当 $N \gg d$ 时，**square** 项碾压。

Flash-Attention v1 做了什么

把 attention 拆成 **sequence** 维上的 **tile**，在 on-chip SRAM (shared memory / register) 里完成：

1. 算 $S_{\text{tile}} = Q_i K_j^T$ (small tile, 不写 global)
2. 对 tile 做局部 softmax 统计 (m_j, ℓ_j) ，用 online 公式 merge 到 running (m, ℓ) 。
3. 同时用局部权重累加 O 的分子——max 变化时对旧 O 重标定。
4. 处理完所有 $\frac{K}{V}$ tile 后， $O_i = \frac{\text{accum}}{\ell}$ 。

Key insight. Key insight. S 的每个元素在 naive 路径里至少经历「写 HBM → 读 HBM → exp → 写 HBM → 再读」——4+ 次 square 流量。FA v1 让 S_{tile} 生在 smem/reg，用完即弃，**square** 项从不触 **HBM**。

与第 3 章 online softmax 的对应

第 3 章	Flash-Attention
一行 logits $x_1, \dots, x_N$	query i 对所有 key 的 score 向量
running (m, d)	running (m_i, ℓ_i) per query row
chunk merge	每个 K_j tile 的局部统计 merge
输出 $y_i = \frac{\exp(x_i - m)}{d}$	$O_i = \frac{\text{weighted sum of } V}{\ell_i}$

表 40 **Table:** 第 3 章 online softmax 与 Flash-Attention 的符号对照。左列为标量 softmax 概念，右列为 attention 中 per-query-row 的向量推广；merge 公式结构相同，FA 额外维护输出分子 O_i 。

Observation： (m, ℓ) 的 merge 公式与第 3 章 chunk 结合律一一对应；FA 多出来的结构是分子 $O_i = \sum_j P[i, j]V[j, :]$ ——max 升高时旧分子必须乘 $\exp(m_{\text{old}} - m_{\text{new}})$ ，这是 v1 相对 plain online softmax 的额外推导。

第 3 章只维护 softmax 分母；FA 还要维护分子 $O_i = \sum_j P[i, j]V[j, :]$ ——max 升高时，旧分子必须乘 $\exp(m_{\text{old}} - m_{\text{new}})$ 。这是 v1 比「单纯 online softmax」多出来的推导。

IO 复杂度分析

标准 attention 的 HBM 访问

论文分析（单 head，忽略 cache 复用）forward HBM access 量级：

$$\text{HBM}_{\text{naive}} = \Theta(Nd + N^2) \approx \Theta(N^2) \quad (N \gg d)$$

更细地按元素流量计（与第 7 章手算一致）： $\frac{S}{P}$ 相关读写主导 $\approx 4N^2$ floats（写 S 、读 S 写 P 、读 P ），加上 Q, K, V 的 $O(Nd)$ 项。合并成面试常用表述：

$$\text{HBM}_{\text{naive}} = O(N^2d) \quad (\text{matmul 复读 } Q, K \text{ 与 square 矩阵多次往返})$$

Flash-Attention v1 的 HBM 访问

设 on-chip SRAM 容量为 M 字节，block tile 大小 $B_r \times B_c$ （query $\times$ key），head dim d 。论文 Theorem 1：

$$\text{HBM}_{\text{FA}} = O\left(N^2 \frac{d^2}{M}\right)$$

直觉：每个 SRAM tile 能装 $\Theta(\frac{M}{d})$ 量级的 $\frac{Q}{V}$ 数据；要完成 $N \times N$ 的 attention map 语义，需 $\Theta(\frac{N^2}{\frac{M}{d}}) = \Theta(N^2 \frac{d}{M})$ 次 tile 换入——再乘每 tile 与 d 相关的 matmul 因子，得 $O(N^2 \frac{d^2}{M})$ 。

Note. 常数依赖 loop order（外 $\frac{K}{V}$ 内 Q 时每个 K_j 只读一次 HBM）。面试写 Big-O 即可；手算用下面具体数字建立量级感。

手算： $N = 1024, d = 64, M = 100 \text{ KB}$ （**A100 单 block smem 预算**）

取 A100 单 block 可用 shared memory $\approx 100 \text{ KB}$ （48 KB 默认 $\times$ 动态扩到 164 KB 的上限内，留余量给寄存器 spill 与 padding）

Naive（ $\frac{S}{P}$ 主导，按 $O(N^2d)$ 量级估 HBM 元素访问）：

$$N^2d = 1024^2 \times 64 = 67108864 \quad \text{element-access 量级}$$

换算流量： $67 \times 10^6 \times 4 \text{ B} \approx 268 \text{ MB}$ （单 head、单次 forward 与 square 矩阵相关的同阶量）

Flash-Attention v1：

$$\frac{N^2d^2}{M} = \frac{1024^2 \times 64^2}{100 \times 1024} = \frac{4294967296}{102400} \approx 41943$$

$41943 \times 4 \text{ B} \approx 168 \text{ KB}$ 同阶 **HBM 流量**。

比值：

$$\frac{67108864}{41943} \approx 1600 \times$$

——不是 FLOPs 差 1600 倍（FLOPs 同为 $\Theta(N^2d)$ ），而是 **square 矩阵相关的 HBM 往返** 差三个数量级。生产 FlashAttention（Tri Dao 的 `flash-attn` 库， $N \geq 1024$ 、tensor core GEMM）

相对 naive 的端到端加速通常是 tens of $\times$ ——本章 **teaching kernel** 规模 $N_q = 4$ 测不到这个量级，见下文「实测」节。

方法	HBM 量级 (单 head)	$N = 1024, d = 64$ 估算
Naive	$O(N^2 d)$	≈ 268 MB 同阶
FA v1	$O(N^2 \frac{d^2}{M})$	≈ 168 KB 同阶
比值	—	$\approx 1600 \times$

表 41 **Table:** naive vs FA v1 的 HBM 流量手算对比 ($N = 1024, d = 64$, on-chip SRAM 预算 $M = 100$ KB)。第三列为 element-access 量级换算成字节后的估算值；FLOPs 两方法同为 $\Theta(N^2 d)$ ，差异仅在 square 矩阵相关的 HBM 往返。

Observation：比值 $\approx 1600 \times$ 来自 square 项被 SRAM tiling 消掉——不是 FLOPs 差 1600 倍。面试答「FA 快在哪」应先说 HBM $O(N^2 d) \rightarrow O(N^2 \frac{d^2}{M})$ ，再提 fusion 与 $\frac{K}{V}$ tile 复用。

Warning. 手算时 **FLOPs** 与 **HBM** 分开写。面试官给 $N = 8192$ 先算 N^2 是否 OOM，再算 $N^2 \frac{d^2}{M}$ 是否可接受——把 FA 说成「减少计算量」会直接扣分。

Roofline 视角

FA 不改变 attention 的 FLOPs ridge 位置，但把有效 **AI** (FLOPs / 实际 HBM byte) 抬高——同样 $O(N^2 d)$ 次 FMA，实际从 HBM 读写的 byte 从 $O(N^2)$ 降到 $O(N^2 \frac{d^2}{M})$ 。长序列下 kernel 从「memory-bound on square tensor」变成「compute-bound on matmul tile + exp」——这才使得 tensor core / 更大 tile 有意义 (v2 方向)。

Algorithm 1：论文伪代码详解

FlashAttention 论文 Forward 核心 (符号与论文一致： N 为序列长， B_r, B_c 为 block 大小， $T_r = \lceil \frac{N}{B_r} \rceil, T_c = \lceil \frac{N}{B_c} \rceil$)：

```
// Input: Q, K, V ∈ R^{N×d} in HBM
// Output: O ∈ R^{N×d} in HBM
// On-chip: SRAM of size M

Partition Q into Tr blocks Q1..QTr of size Br×d each
Partition K, V into Tc blocks K1..Kc, V1..Vc of size Bc×d each

for j = 1 to Tc do
    Load Kj, Vj from HBM to on-chip SRAM // 外循环: K/V block

    for i = 1 to Tr do
        Load Qi from HBM to on-chip SRAM // 内循环: Q block

        On chip:
            Sij = Qi Kj^T ∈ R^{Br×Bc} // tile matmul
            mij = rowmax(Sij) // 每 query 行一个
max      Pij = exp(Sij - mij) // 局部 exp (未归一
化)      lij = rowsum(Pij) // 局部 sum
```

```

// Online softmax merge + output rescale
mi_new = rowmax(mi, mij) // 逐行
li_new = exp(mi - mi_new) ⊙ li + exp(mij - mi_new) ⊙ lij
Oi = diag(exp(mi - mi_new)) Oi + Pij Vj // 重标定 0 再累加

Write Oi to HBM (或最后统一写)
end for
end for

Finalize: $O_i = O_i / li$ // 逐行除
running sum

```

Algorithm 1 与源码变量对照

论文符号	源码 (register kernel)	含义
Q_i	<code>q[row * head_dim + d]</code>	单 query 行, 一行一 thread
K_j, V_j	<code>k[key * head_dim + d]</code>	key tile, <code>key_start</code> 步进
S_{ij}	<code>scores[t]</code>	tile 内局部 logits, 寄存器数组
m_i	<code>running_max</code>	该行 running max
ℓ_i	<code>running_sum</code>	running sum of exp
O_i	<code>accum[d]</code>	未归一化输出分子
B_c	<code>kTileKeys (=4)</code>	key 方向 tile 宽
B_r	1 (每 thread 一行)	query 方向 block 高——生产为 $B_r > 1$

表 42 **Table:** 论文 Algorithm 1 符号与 `flash_attention_v1_kernel` 源码变量对照。 $B_r = 1$ 、 $B_c = 4$ 是教学最小配置；生产 FA 用 $B_r, B_c \approx 64-128$ 并在 warp 级做 tile matmul。

Observation: `scores[t]` 活在寄存器、用完即弃——对应论文「 S_{tile} 不写 HBM」。读源码时按 state machine 看：每个 query 行只有 (m, ℓ, accum) 三个状态，每个 key tile 是一次状态转移。

论文外层 $T_c = \lceil \frac{N}{B_c} \rceil$ 次迭代对应源码 `for (key_start = 0; ...; key_start += kTileKeys)`。内层 T_r 在 register 版由 **grid** 上的 **row index** 并行展开——每个 block 的 32 个 thread 各处理不同 query 行，等价于 Algorithm 1 内层 i 的并行化。

数据流 (单 query 行)

```

HBM: Q[row, :] —读一次—> 寄存器 q[row*head_dim+d]
|
v
for each key tile [key_start, key_start+Bc):
  HBM: K[key, :], V[key, :] —读—> scores[t], tile_max
  |
  v
  new_max, old_scale = exp(m_old - m_new)
  accum *= old_scale      ← 0 rescale
  weight = exp(score - new_max)

```

```

        accum += weight * V[key,:]  ← P@V 融合
        running_sum = running_sum * old_scale + tile_sum
    }
    ▼
HBM: out[row,:] = accum / running_sum

```

全程无 `cudaMalloc(N×N)` 的 `scores` buffer——这是与第 7 章 `device_scores` 的本质分界。

Key insight. 读源码时按 **state machine** 看：每个 query 行只有 (m, ℓ, accum) 三个状态；每个 key tile 是一次状态转移。Debug 时打印每个 tile 后的 triple，与 CPU reference 逐步对比，比端到端 diff 更容易定位 merge bug。

外循环 j ：每次把一个 key/value 列块（长度 B_c ）搬进 SRAM。 K_j, V_j 在 inner 所有 Q_i 上复用——这是 IO 最优 loop order：每个 $\frac{K}{V}$ tile 只从 HBM 读一次（对 fixed j ）

内循环 i ：对每个 query 块 Q_i ，与当前 K_j 算 $B_r \times B_c$ 的 S_{tile} ，更新该 query 块内每行的 (m, ℓ, O) 。

Key insight. Loop order 反过来（外 Q 内 K ）则每个 Q_i 只读一次，但 K_j 会被重复读 T_r 次——总 HBM 仍同阶，但常数更大。v1 论文与 cuDNN/FA2 均采用 外 **K/V**、内 **Q** 或等价的 swap-GEMM 布局。

每次处理 Q_i 时更新的状态

对 query block i 内的每一行（query token）维护：

- $m_i \in \mathbb{R}$ ：running max of scores seen so far。
- $\ell_i \in \mathbb{R}$ ：running sum $\sum \exp(s - m_i)$ （relative to current m_i ）
- $O_i \in \mathbb{R}^d$ ：running numerator $\sum \exp(s - m_i)V$ （尚未除以 ℓ_i ）

每来一个 K_j tile，用 tile 内统计 (m_{ij}, ℓ_{ij}) merge 进 (m_i, ℓ_i, O_i) 。全部 j 处理完后 $\frac{O_i}{\ell_i}$ 即为最终 attention 输出。

Online Softmax Rescaling： m, ℓ, O 推导

符号

固定 query 行 i ，已处理 key 集合 $\mathcal{K}_{\text{old}}$ ，来了新 key 集合 $\mathcal{K}_{\text{new}}$ （一个 tile）

- 旧状态： (m, ℓ) ，其中 $\ell = \sum_{k \in \mathcal{K}_{\text{old}}} \exp(S[i, k] - m)$ 。
- 新 tile logits： $x_k = S[i, k]$ for $k \in \mathcal{K}_{\text{new}}$ 。
- 局部： $m' = \max_{k \in \mathcal{K}_{\text{new}}} x_k$ ， $\ell' = \sum_{k \in \mathcal{K}_{\text{new}}} \exp(x_k - m')$ 。

全局新 max： $m_{\text{new}} = \max(m, m')$ 。

ℓ 的 **merge**（第 3 章公式的 **block** 版）

旧 sum 相对 m_{new} 需重标定：

$$\ell_{\text{new}} = \underbrace{\sum_{k \in \mathcal{K}_{\text{old}}} \exp(x_k - m_{\text{new}})}_{\text{旧 keys}} + \underbrace{\sum_{k \in \mathcal{K}_{\text{new}}} \exp(x_k - m_{\text{new}})}_{\text{新 tile}}$$

第一项： $\ell \cdot \exp(m - m_{\text{new}})$ （与第 3 章 $d' = d \cdot \exp(m - m') + \exp(x - m')$ 相同）

第二项： $\ell' \cdot \exp(m' - m_{\text{new}})$ (tile 内先减 m' 算 $\exp$ ，再统一基准到 m_{new})

合并：

$$\ell_{\text{new}} = \ell \cdot \exp(m - m_{\text{new}}) + \ell' \cdot \exp(m' - m_{\text{new}})$$

Note. 源码里 `tile_max` 对应 m' ，`new_max` 对应 m_{new} ，`tile_sum` 对应 ℓ' (relative to m_{new} 时已用 `exp(scores[t] - new_max)`)

O 的 **merge** (Flash-Attention 特有)

目标： $O = \sum_k P[i, k] V[k, :]$ ， $P[i, k] = \frac{\exp(S[i, k] - m_{\text{final}})}{\ell_{\text{final}}}$ 。

维护未归一化分子 $\tilde{O} = \sum_{k \in \text{processed}} \exp(S[i, k] - m) V[k, :]$ (relative to **current** m)

当 $m \rightarrow m_{\text{new}}$ 时，旧贡献统一缩放：

$$\tilde{O}_{\text{new}} = \exp(m - m_{\text{new}}) \cdot \tilde{O}_{\text{old}} + \sum_{k \in \text{new tile}} \exp(x_k - m_{\text{new}}) V[k, :]$$

与 ℓ 同步：先 `accum *= exp(m - m_new)`，再加 `weight * V`。

Finalize： $O = \frac{\tilde{O}}{\ell}$ 。

正确性：等价于全局 **softmax**

设完整 logits $x_1, \dots, x_N$ ，全局 $m^* = \max_j x_j$ ， $\ell^* = \sum_j \exp(x_j - m^*)$ 。

归纳：处理完任意 prefix 后， $(m, \ell, \tilde{O})$ 满足

$$\tilde{O} = \sum_{j \in \text{prefix}} \exp(x_j - m) V[j, :], \quad \ell = \sum_{j \in \text{prefix}} \exp(x_j - m)$$

且 $m = \max_{j \in \text{prefix}} x_j$ 。merge 公式即第 3 章 **chunk** 结合律对 (m, ℓ) 的推广； O 的缩放因子与 ℓ 相同，保证

$$\frac{\tilde{O}}{\ell} = \frac{\sum_j \exp(x_j - m^*) V[j, :]}{\ell^*}$$

与 naive 三 pass 一致。

数值走查

Query 行 i ， $d = 1$ ， $V[k] = 1$ (标量)。Scores 分两 tile： $[1, 2]$ 与 $[3]$ 。

Tile 1： $m' = 2$ ， $\ell' = e^{-1} + 1$ ， $\tilde{O} = 1 \cdot e^{-1} + 2 \cdot 1 = e^{-1} + 2$ 。设 $m = 2$ ， $\ell = e^{-1} + 1$ ， $\tilde{O} = e^{-1} + 2$ 。

Tile 2： $x = 3$ ， $m_{\text{new}} = 3$ ， $\text{scale} = \exp(2 - 3) = e^{-1}$ 。

$$\ell_{\text{new}} = (e^{-1} + 1) \cdot e^{-1} + 1 = e^{-2} + e^{-1} + 1$$

$$\tilde{O}_{\text{new}} = (e^{-1} + 2) \cdot e^{-1} + 3 = e^{-2} + 2e^{-1} + 3$$

全局 softmax 权重 $\approx [0.09, 0.24, 0.67]$ ， $O = 0.09 + 0.48 + 2.01 = 2.58$ 。用 $\frac{\tilde{O}_{\text{new}}}{\ell_{\text{new}}}$ 算得相同——与 `flash_attention_cpu_reference` 一致。

Key insight. 面试手推：先写 ℓ merge (第 3 章)，再写 O 乘同一个 $\exp(m - m_{\text{new}})$ 。漏掉 O rescale 是 FA 实现最常见 bug。

Block Size 选择： $B_r \times B_c$

Shared memory 预算

单 block 典型 resident 数据 (FP32):

Buffer	大小	说明
Q_i tile	$B_r \times d \times 4 \text{ B}$	query 块
K_j tile	$B_c \times d \times 4 \text{ B}$	key 块
V_j tile	$B_c \times d \times 4 \text{ B}$	value 块
S_{ij} tile	$B_r \times B_c \times 4 \text{ B}$	score 矩阵

表 43 **Table:** 单 CTA 典型 shared memory 驻留 buffer (FP32, 4 B/float) 大小公式 $\approx 4(B_r + B_c)d + B_r B_c$ floats; $B_r B_c$ 项是 square—— B_r, B_c 各翻倍 smem 近似四倍。A100 单 block 上限 164 KB。

Observation : $d = 64, B_r = B_c = 64$ 时合计 $\approx 65 \text{ KB}$, fit $M = 100 \text{ KB}$ 预算; 但 $B_r = B_c = 128, d = 128$ 可超 164 KB 直接 launch 失败——block size 选型首要约束是 smem, 不是 FLOPs。

合计 $\approx 4(B_r + B_c)d + B_r B_c$ floats。 $d = 64, B_r = B_c = 64$: $3 \times 64 \times 64 \times 4 + 64^2 \times 4 \approx 49 \text{ KB} + 16 \text{ KB} = 65 \text{ KB}$ ——fit $M = 100 \text{ KB}$ 。

Warning. $B_r B_c$ 项是 square—— B_r, B_c 各翻倍, smem 近似四倍。 $B_r = B_c = 128, d = 128$ 时单 block 可超 164 KB 直接 **launch** 失败。

寄存器压力

Teaching kernel `flash_attention_v1_kernel` 每 thread 持有 `accum[kHeadDim] + scores[kTileKeys]`—— d 或 tile 过大时 **register spill** $\rightarrow$ local memory $\rightarrow$ 性能雪崩。生产 FA 用 warp 分工算 S_{tile} , 每 thread 只持 partial row。

Warp 数与 occupancy

`flash_attention_v1_shared_kernel` 用 8 threads/block (教学)——真实 kernel 需 ≥ 1 warp (32 threads) 避免 underutilization。FA2 典型 128-256 threads/block, 4-8 warps, 在 smem 与 occupancy 间 trade-off:

- smem 大 $\rightarrow$ max active blocks/SM 降 $\rightarrow$ latency 难 hide。
- smem 小 $\rightarrow$ tile 小 $\rightarrow$ HBM 读次数增。

Note. 选型流程: 先算 smem 公式 $\rightarrow$ 查 occupancy calculator $\rightarrow$ profile 几个 (B_r, B_c) 候选。面试答「 $B_r, B_c \approx \sqrt{\frac{M}{d}}$ 量级」即可, 不必背具体 magic number。

B_r vs B_c 不对称

Causal mask 下 query row i 只 attend $j \leq i$ ——许多 (i, j) tile 全 mask, 可 **skip entire block** (见下节)。常取 B_c 略大以提高 $\frac{K}{V}$ 复用; B_r 对齐 warp 行数以利 S_{tile} 行并行。

v1: Register Streaming Kernel

源码 `flash_attention_v1_kernel`: 一个 **thread** 负责一个 **query** 行, 沿 key 维以 `kTileKeys` 为步长流式处理。


```

__global__ void flash_attention_v1_kernel(
    const float* q, const float* k, const float* v,
    float* out, int query_count, int key_count, int head_dim) {
    const int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row >= query_count) return;

    float running_max = -1.0e30f;
    float running_sum = 0.0f;
    float accum[kHeadDim];
    // ... init accum ...

    float scores[kTileKeys];

    for (int key_start = 0; key_start < key_count; key_start += kTileKeys) {
        // 1. 算 tile 内 scores + tile_max
        // 2. new_max, old_scale = exp(running_max - new_max)
        // 3. accum *= old_scale
        // 4. tile_sum, accum += weight * V
        // 5. running_sum = running_sum * old_scale + tile_sum
        // 6. running_max = new_max
    }

    out[row * head_dim + d] = accum[d] / (running_sum + kEps);
}

```

逐步对应算法

Step 1 — S 计算 (tile matmul 的极简版)

```

for (int t = 0; t < tile_count; ++t) {
    const int key = key_start + t;
    const float score = dot_row_device(
        &q[row * head_dim], &k[key * head_dim], head_dim);
    scores[t] = score;
    tile_max = fmaxf(tile_max, score);
}

```

每个 thread 独立 dot product——无 smem 复用, K 每 tile 从 HBM 重读。教学清晰; 生产应用 tiled GEMM。

Step 2 — Softmax rescale

```

const float new_max = fmaxf(running_max, tile_max);
const float old_scale = (running_sum == 0.0f) ? 0.0f
    : expf(running_max - new_max);
for (int d = 0; d < kHeadDim; ++d) {
    accum[d] *= old_scale;
}

```

running_sum == 0 分支: 首 tile 时 old_scale=0, 避免 exp(-inf) 噪声——与「尚无 prefix」语义一致。

Step 3 — $P@V$ 累加 (P 不 materialize)

```

const float weight = expf(scores[t] - new_max);
tile_sum += weight;
for (int d = 0; d < kHeadDim; ++d) {
    accum[d] += weight * v[key * head_dim + d];
}
running_sum = running_sum * old_scale + tile_sum;
running_max = new_max;

```

`weight` 即 $P[i, k]$ 的未归一化分子 relative to m_{new} ——从不写 **global** P 。

Launch: `<<<ceil(query_count / 32), 32>>>`。并行轴 = query 行数 N_q ; N_q 小则 SM 填不满——这是 v1 局限。

完整 **inner loop** (对照源码)

```

for (int key_start = 0; key_start < key_count; key_start += kTileKeys) {
    const int tile_count = min(kTileKeys, key_count - key_start);

    float tile_max = -1.0e30f;
    for (int t = 0; t < tile_count; ++t) {
        const int key = key_start + t;
        scores[t] = dot_row_device(&q[row * head_dim],
                                   &k[key * head_dim], head_dim);
        tile_max = fmaxf(tile_max, scores[t]);
    }

    const float new_max = fmaxf(running_max, tile_max);
    const float old_scale = (running_sum == 0.0f) ? 0.0f
        : expf(running_max - new_max);

    for (int d = 0; d < kHeadDim; ++d) {
        accum[d] *= old_scale;
    }

    float tile_sum = 0.0f;
    for (int t = 0; t < tile_count; ++t) {
        const float weight = expf(scores[t] - new_max);
        tile_sum += weight;
        const int key = key_start + t;
        for (int d = 0; d < kHeadDim; ++d) {
            accum[d] += weight * v[key * head_dim + d];
        }
    }

    running_sum = running_sum * old_scale + tile_sum;
    running_max = new_max;
}

```

读代码 **checklist** :

1. `scores[t]` 在 merge 前算完——mask (若启用) 应插在 `scores[t] = ...` 之后、`tile_max` 更新之前。
2. `old_scale` 同时缩放 `accum` 和 `running_sum`——必须一致, 否则 O 与 ℓ 基准错位。

3. `weight` 用 `new_max` 而非 `running_max` ——与第 3 章 online 公式中 $\exp(x - m_{\text{new}})$ 一致。
4. `kEps` 防除零：全 `mask` 行时 $\ell = 0$ ，生产应 `skip` 该行而非依赖 `epsilon`。

Scale $\frac{1}{\sqrt{d}}$ 应加在哪

教学源码省略 `scale`；接入第 7 章约定只需一行：

```
const float score = dot_row_device(...) * inv_sqrt_head_dim;
```

或在 `dot_row_device` 内乘。数学上 `scale` 在 `softmax` 前等价；IO 上 `fuse` 进 `dot` 避免额外 `global pass`。cuBLAS / CUTLASS 路径用 `epilogue alpha`；FA 手写 `kernel` 在 `score` 写回 `register` 时乘最自然。

Note. FP16/BF16 训练时 `scale` 常与 `layer norm` 输出尺度耦合——`kernel` 接口常暴露 `softmax_scale` 指针(含 $\frac{1}{\sqrt{d}}$ 与可选 `temperature`)，便于 inference KV cache 场景调 logits 温度而不重编译 `kernel`。

ncu 实测 (online / register 版)

ncu 实测 — v: online (one thread per query row). 规模： $N_q = 128$, $N_k = 512$, `head_dim` = 64.

Duration	3 270 μ s	
Memory SOL	1.1 %	*极低*——one-thread-per-row 的经典病
Compute SOL	0.1 %	
Achieved Occupancy	1.6 %	grid = 4 blocks $\times$ 32 threads = 128 threads *全 GPU*
Grid Size	4	$\$N_q / 32 = 128 / 32 = 4\$$

Occupancy 1.6% + Grid Size 4 是本章最戏剧的读数——GPU 有 108 SM，我们只用了 4 个。每个 `thread` 独立处理一个 `query` 行，把 $N_k = 512$ 个 `key` 逐一走完。 $\frac{K}{V}$ 在 `thread` 之间零复用，每 `thread` 各读 $N_k \times d = 32K$ 元素 = 128 KB。 N_q 个 `thread` 就是 $N_q \times 128$ KB = 16 MB 的 HBM 重读。

说白了这不是 **FlashAttention**——这是“用 **FlashAttention** 数学 (**online merge**) 写的 **attention**”，但缺 `tile` 复用。 $\frac{K}{V}$ 每 `tile` 每 `thread` 重新去 HBM 读，完全没利用 `shared memory`。

瓶颈判定. one-thread-per-query 的并行度太差，且 $\frac{K}{V}$ 没在 `thread` 之间复用——每 `thread` 都独立读一整套 K, V

证据. Grid Size 4 (GPU 108 SM 中只用 4 个)；Memory SOL 1.1%；Occupancy 1.6%；HBM 重读 16 MB

下一步. v2 (shared) 把 $Q / K / V$ `tile` 显式放进 `shared memory` —— 一个 `block` 处理一个 `query row`，`block` 内 `threads` 协作 `load` 一次 `tile`，然后所有算全用 `smem` 数据

`flash_attention_v1_shared_kernel`：一个 **block** 处理一个 **query** 行，显式把 Q, K, V `tile` 搬进 `shared memory`。

```
__global__ void flash_attention_v1_shared_kernel(...) {
    const int row = blockIdx.x;
    __shared__ float q_shared[kHeadDim];
```

```

__shared__ float k_tile[kTileKeys][kHeadDim];
__shared__ float v_tile[kTileKeys][kHeadDim];
__shared__ float scores[kTileKeys];
// ...
}

```

Q 加载

```

if (threadIdx.x < kHeadDim) {
    q_shared[threadIdx.x] = q[row * head_dim + threadIdx.x];
}
__syncthreads();

```

一行 $Q[i, :]$ 读入 smem 一次，所有 **key tile** 复用——对应 Algorithm 1 的 Load Q_i 后在内层 j 循环复用（此处 outer 是 key tile， Q 已 resident）。

K / V 加载

```

const int token = threadIdx.x;
const int key = key_start + token;
if (token < kTileKeys && key < key_count) {
    for (int d = 0; d < kHeadDim; ++d) {
        k_tile[token][d] = k[key * head_dim + d];
        v_tile[token][d] = v[key * head_dim + d];
    }
}
__syncthreads();

```

8 threads 协作搬 $4 \times 8 = 32$ 维——教学用 `kSharedThreadsPerBlock=8`；生产每 thread 搬更多元素或用 `cp.async`。

S 计算、softmax、O 更新

Thread 0 串行执行 tile 内 matmul + online merge（与 register 版数学相同），但 K, V 从 `k_tile / v_tile` 读——**illustrate SRAM reuse**。

Launch: `<<<query_count, kSharedThreadsPerBlock>>> = <<<4, 8>>>`。

版本	优点	缺点
<code>flash_attention_v1_kernel</code>	代码短；多 query 行并行	K, V 每 tile 每 thread 重读 HBM
<code>flash_attention_v1_shared_kernel</code>	Q 与 K, V tile 在 smem 复用	单 block 单 row；thread 0 串行算

表 44 **Table:** 本章两个 teaching kernel 的 launch 与 IO 权衡。register 版 `<<<ceil(N_q/32), 32>>>`；shared 版 `<<<N_q, 8>>>`。数学相同，差别在 $\frac{Q}{V}$ 从 HBM 读几次、是否在 smem 复用。

Observation：shared 版用 smem 换 HBM 重读——实测约 $1.9\times$ 快于 register 版，但 thread 0 串行 merge 拉低 lane 利用率。生产 FA = staged kernel + warp 级 tile matmul + 更大 B_r, B_c ，二者是 Algorithm 1 的最小正确子集。

Key insight. 生产 FlashAttention = staged kernel + warp 级 tile matmul + 外 $\frac{K}{V}$ 内 Q 双重循环 + 更大 B_r, B_c 。本章两个 kernel 是 Algorithm 1 的最小正确子集。

Shared memory 布局与 bank conflict

`k_tile[token][d]` 按 **token** 主序:固定 `token` 时相邻 `d` 连续——thread `token` 写一行 $K[j,:]$, 与 row-major K global layout 一致。若改成 `k_tile[d][token]`, thread 协作 load 时 stride 为 `kTileKeys`, 易 bank conflict。

`kHeadDim=8`, `kTileKeys=4` 时 smem 用量:

- `q_shared`: $8 \times 4 = 32$ B
- `k_tile + v_tile`: $2 \times 4 \times 8 \times 4 = 256$ B
- `scores`: 16 B

合计 < 1 KB——远低于 A100 48 KB 默认限额。换成 $d = 128, B_c = 64$ 时仅 $\frac{K}{V}$ tile 就 $2 \times 64 \times 128 \times 4 = 64$ KB, 必须与 S_{tile} 复用 buffer 或 double-buffer 流水线——那是 v2 工程细节。

生产 kernel 还差什么

组件	本章 teaching vs 生产 FA
QK^T tile	单 thread dot $\rightarrow$ warp <code>mma.sync</code> / WGMMMA
并行轴	row-parallel $\rightarrow (B_r, B_c)$ block 2D tile
Q 复用	shared 版仅单行 $\rightarrow$ 多行 B_r 共用 K_j
GQA/MQA	未涉及 $\rightarrow K, V$ head 数 $H_{kv} < H$
Dropout	未涉及 $\rightarrow$ forward 存 random sign / backward 重放
Split-KV	未涉及 $\rightarrow$ 超长 N 时 sequence parallel

表 45 **Table:** 本章 teaching kernel 与生产 FlashAttention 的功能差距清单。算法核心 (online merge 循环) 已覆盖; 性能差距在 tile GEMM (tensor core) occupancy 调参、GQA/dropout 等工程特性。

Observation: 面试说「读过 FA 源码」时, **online merge** 循环是算法核心, 性能在 tile GEMM 与 occupancy 调参——二者不可混为一谈。本章刻意 scalar dot 以便读 merge 循环, 不是 chase peak 的实现。

面试说「我读过 FA 源码」时, 至少应能指出: **online merge** 循环是算法核心; 性能在 tile GEMM 与 occupancy 调参——二者不可混为一谈。

ncu 实测 (shared 版)

ncu 实测 — v: shared (tile in smem, one block per query row). 规模: $N_q = 128, N_k = 512, \text{head_dim} = 64$.

Duration	595.6 μ s	*比 online 版快 5.5*
Memory SOL	26.2 %	从 1.1% $\rightarrow$ 26.2%, HBM 利用率大幅拉高
Compute SOL	9.1 %	
L2 Hit Rate	93.2 %	
Achieved Occupancy	4.5 %	* $\uparrow$ 从 1.6%*, 但仍然低——单 block 单 row 限制
Grid Size	128	$\$N_q\$$ block, 每 block 处理一行 query

Static SMEM / block ~5 KB \$Q\$ tile + \$K\$ tile + \$V\$ tile + scores

- **5.5×** 提速来自两个来源 : (a) block 内 threads 协作 load 一次 tile 到 smem , $\frac{K}{V}$ 不再每 thread 重读 ; (b) Grid Size $4 \rightarrow 128$ 让 GPU 用得更多。
- 但 **memSOL 26%** 仍然低。原因 :
 - `kSharedThreadsPerBlock = 8` (教学值)——每 block 只有 8 threads 协作 , warp 都填不满。
 - 内层 tile matmul 由 thread 0 串行执行 (“illustrate SRAM reuse, not compute perf”)——只有 1/8 的 lane 在算 dot product。
- 这就是“教学 kernel 展示机制、不 chase peak”的直接体现。生产 FA v1 会用 $B_r = 32$ or 64 、 $B_c = 32$ 、每 block ≥ 128 threads , warp-per-row + Tensor Core。

落地评估。FA v1 shared 版本已经把 online softmax 的核心机制 (tile matmul + running max/sum merge + O 重标定) 落地 , S 从不落 HBM。

继续接近生产 FA 需要 (1) 多 warp 协作代替单 thread 串行 dot , (2) `cp.async` 让 $\frac{K}{V}$ 下一 tile 预取和当前 tile compute overlap , (3) Tensor Core (`mma.sync`) 让 dot product 用 FP16/BF16 半精度 8-16× 加速。这些进一步优化在 09、10 章展开。

Multi-Head 与 Batch 扩展

单 head kernel 推广到 (B, H) 只需在 index 里加 batch/head offset , 算法不变 :

```
__global__ void flash_attention_mha_kernel(
    const float* q, const float* k, const float* v, float* out,
    int batch_size, int num_heads, int seq_len, int head_dim) {
    const int b = blockIdx.z;
    const int h = blockIdx.y;
    const int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (b >= batch_size || h >= num_heads || row >= seq_len) return;

    const int head_elems = seq_len * head_dim;
    const int batch_stride = num_heads * head_elems;
    const int head_stride = head_elems;
    const float* q_head = q + b * batch_stride + h * head_stride;
    const float* k_head = k + b * batch_stride + h * head_stride;
    const float* v_head = v + b * batch_stride + h * head_stride;
    float* out_head = out + b * batch_stride + h * head_stride;

    // ... 与单 head 相同的 online loop , 指针换 q_head/k_head/v_head ...
}
```

Launch : `dim3 grid(ceil(seq_len/32), num_heads, batch_size)`。Head 间零同步——与第 7 章 multi-head naive 相同 , 但不再有 $B \times H \times N^2$ 的 S tensor。

Warning. Layout 必须确认 : PyTorch (B, N, H, d) vs 本书 (B, H, N, d) offset 不同。面试写 kernel 前 先问 **layout**——错 stride 比错 online 公式更常见。

Cross-attention : $N_q \neq N_k$ 时 loop 上界改为 `key_count` , Q 来自 decoder、 $\frac{K}{V}$ 来自 encoder——merge 公式不变 , causal 通常关闭。

常见实现错误

错误 1 : 只 rescale ℓ , 忘记 `accum *= old_scale` 。输出偏大/偏小 , max abs error 随序列长度累积。

错误 2 : 用 `running_max` 算 `weight` 而非 `new_max` 。当 tile 引入更大 max 时 , 新 tile 权重基准错误。

错误 3 : mask 在 softmax 之后 zero 。分母含 masked 项 , 有效权重和 < 1 。

错误 4 : causal skip 条件写成 `key_start > row` 而非 `key_start > row` 且考虑 block 边界——partial tile 仍需 per-element mask 。

错误 5 : `running_sum` 用 float 累加长序列 $N > 8192$ 且 logits 极端时丢精度——应学第 3 章用 `double` 或 block-wide Kahan 。

错误 6 : 以为 FA 不需要 subtract-max 。Online merge 就是 subtract-max 的分块形式 ; 去掉 merge 重标定等价于数值爆炸。

Key insight. 单元测试策略 : 固定 $N_q = 1$ 的小 N_k (如 3、5、7) , 手算 (m, ℓ, O) 逐步对比 GPU 每个 tile 后的状态——比直接 `assert_allclose` 整表更容易教新人。

原则 : 在 S 算完、**softmax** 前应用

Mask 改变 logits 支持集 , 必须在 m, ℓ 更新之前写入 $S[i, j]$:

- **Padding** / 无效 **key** : $S[i, j] = -\infty$ (或足够小的 `-1e30`) 。
- **Causal** : 对 query i , key $j > i$ 置 $-\infty$ 。

被 mask 的项 $\exp(-\infty) = 0$, 不贡献 ℓ 与 O ——与第 3 章 `softmax_masked_kernel skip` 等价。

Causal : 跳过整个 block

若 causal 且 key block j 满足 $j_{\min} > i_{\max}$ (整个 $B_c \times B_c$ tile 全在 future)——无需 **Load** K_j, V_j 、无需算 **matmul** 。

```
if (causal && key_start > row) {  
    continue; // 整个 tile 对 query row 不可见  
}
```

Partial tile (block 跨越对角线) : 只对 $j > i$ 的元素 mask , 或只对 $\text{col} \leq i$ 算 dot product 。

Warning. Causal 优化漏判 partial tile 会错 ; 全 skip 条件必须是「tile 内所有 key index $>$ 当前 query index」。Decoder 长序列下 skip 贡献显著常数因子。

与 naive 第 7 章对比

Naive 在 scores kernel 写 `-1e30` 再整表 softmax——仍占 N^2 HBM。FA 在 tile 内 branch , **masked** 位置甚至不进 **smem** 累加 , square 流量为 0 。

Backward 简述

Forward 不存 $\frac{S}{P}$, backward 需要 `dS`——FA **recompute** : 反向再按 tile 算 $S = QK^T$, 用 forward 存的 log-sum-exp 统计 (或 m, ℓ) 求 `dP` , 再链式传 `dQ, dK, dV` 。

Key insight. Recomputation trade-off: 多算 **FLOPs**, 少占 **HBM**。训练时 activation checkpoint 同一哲学——FA backward 是「强制 checkpoint attention map」。

Backward 也可 fused 成单 kernel (论文 Algorithm 4): 与前向相同的 tile 遍历, 在 smem 重算 S_{tile} , 即时累加 dQ , dK , dV , 不写 dS 到 **global**。数值上需保存 forward 的 (m, ℓ) per row ($O(N)$ 存储, 非 $O(N^2)$)。

面试答「backward 为什么能 fuse」: **recompute** + 同 **tile** 遍历顺序 + **online softmax** 反向公式; 显存从 $O(N^2)$ 降到 $O(N)$ 。

Backward 链式法则 (概要)

设 forward 已得 O , 保存 per-row (m, ℓ) 。Backward 接收 dO :

1. 重算 $S_{\text{tile}} = QK^T$ (同 forward tile 边界)。
2. 由 $O = \frac{\partial}{\partial \ell}$ 得 $d\tilde{O} = d\frac{\partial}{\partial \ell}$ (逐行)。
3. Softmax backward: $dS[i, j] = P[i, j] (d\tilde{O}[i, :] \cdot V[j, :] - \sum_k P[i, k] d\tilde{O}[i, :] \cdot V[k, :])$ ——可在 tile 内流式算, 不写 full dS 。
4. $dQ = dSK$, $dK = dS^T Q$, $dV = P^T d\tilde{O}$ ——仍是 tile GEMM, 累加到 global dQ, dK, dV 。

Tensor	Naive backward 存储	FA backward
P 或 S	$O(N^2)$ checkpoint	recompute per tile
(m, ℓ)	可选	$O(N)$ 必须存
dS	$O(N^2)$ materialize	tile 即时消费, 不写出

表 46 **Table:** naive vs FA backward 的 activation 存储对比。FA 用 recompute 换 HBM: 反向再算 $S = QK^T$, 只存 per-row (m, ℓ) ($O(N)$), dS 在 tile 内即时消费。

Observation: recompute 的 FLOPs 约为 forward 的 1.5-2 $\times$, 但 HBM 省下的 N^2 读写在长序列下永远划算——与 activation checkpointing 的 trade-off 曲线同形。面试追问「存什么 activation」: 典型是 (m, ℓ) 或 LSE, 不是 full attention matrix。

Recompute 的 FLOPs 约为 forward 的 1.5-2 $\times$ (多一次 QK^T), 但 HBM 省下的 N^2 读写在长序列下永远划算——与 activation checkpointing 的 trade-off 曲线同形。

Note. PyTorch `flash_attn` 包 backward 与 forward 共用 tile scheduler; 面试追问「存什么 activation」: 典型是 (m, ℓ) 或 LSE, 不是 full attention matrix。

Block size 受 shared memory 限制

d 或 N 增大时, B_r, B_c 不能无限涨——smem 爆 $\rightarrow$ occupancy 降 $\rightarrow$ 即使 IO 理论优也跑不满 GPU。

Long context 并行度

`flash_attention_v1_kernel` 并行轴 primarily N_q (query 行)。Prefill 时 $N_q = N$ 大尚可; decode 时 $N_q = 1$ 单 **thread** 串整条 **key** 序列——必须换 parallel 策略 (v2 的 sequence parallel split $\frac{K}{V}$ 、warps 协作一行)。

v1 → v2 主要改动（面试常考）

设计点	FA v1	FA v2
Parallelism	主要按 Q block / row	warps 分工 + sequence parallel
Loop 调度	外 $\frac{K}{V}$ 内 Q （教学）	优化 work partition 减 idle
Causal 边界	block skip 手动	更细 tile 调度 + 预编译
Head dim 大	smem 压力	split-K / 更小 tile + recombine
Inference decode	未优化	KV cache aware kernel

表 47 **Table:** FA v1 → v2 主要工程改动（面试常考）。v1 论文已含 IO 最优算法；v2 是工程实现把 SM 利用率做上去——算法/online 公式不变，变的是 loop order、accumulator 表示与 warp 切分。

Observation : v1 解决「要不要 $N \times N$ 矩阵」；v2 解决「既然不要了，怎么把 fused kernel 跑满 tensor core」。decode 阶段 $N_q = 1$ 时 v1 并行度差——这是 v2 sequence parallel 的动机之一。

Note. v1 论文已含 IO 最优算法；v2 是工程实现把 SM 利用率做上去——算法思想不变，不是新的 online 公式。

源码运行与对照

```
make build/08_flash_attention_v1 && ./build/08_flash_attention_v1
```

默认 $N_q = 4, N_k = 16, d = 8, \text{tile size} = 4$ 。flash_attention_cpu_reference 与两个 GPU kernel 对齐，容差 10^{-4} 。

函数	Launch	角色
flash_attention_v1_kernel	<<<ceil(4/32), 32>>>	register streaming baseline
flash_attention_v1_shared_kernel	<<<4, 8>>>	smem staging 演示

表 48 **Table:** 本章两个 GPU kernel 的 launch 配置与角色。**grid / block** 对应

<<<grid, block>>> CUDA launch 参数；默认 $N_q = 4, N_k = 16, d = 8, \text{tile size} = 4$ 。

Observation : register 版 1 block $\times$ 32 thread 仅 4 行 Q 有活干；shared 版 4 block $\times$ 8 thread 每 block 一行但 blockDim 仍凑不满 warp——两者都是 teaching 规模下的结构性 underutilization，不是算法 bug。

实测

$N_q = 4, N_k = 16, d = 8$ （源码 kQueryCount / kKeyCount / kHeadDim； Q, K, V, O 各 512 B，整 problem < 4 KB），A100 80GB SXM4，ncu --set full 抓取每个 kernel 的一次 launch。表中 $\text{TC \%} = \text{sm_pipe_tensor_cycles_active.avg.pct_of_peak_sustained_elapsed}$ ； $\text{warp \%} = \text{sm_warps_active.avg.pct_of_peak_sustained_elapsed}$ ； $\text{HBM \%} = \text{dram_bytes.sum.pct_of_peak_sustained_elapsed}$ 。

Launch 配置决定并行轴与 warp lane 利用率—— $N_q = 4$ 远填不满 GPU：

version	grid	block	active threads	并行轴
v1 register	(1, 1, 1)	(32, 1, 1)	4	threadIdx.x = query 行 ;每 thread 流式扫 N_k
v1-shared	(4, 1, 1)	(8, 1, 1)	32 (4 block $\times$ 8)	blockIdx.x = query 行 ; 8 thread 协作 load , thread 0 串行 merge

表 49 **Table:** ch8 FA v1 teaching kernel 的 launch 配置 (launch__grid_size / launch__block_size) **active threads** 为实际参与计算的 thread 数 ; $N_q = 4$ 远填不满 A100 108 SM。

Observation :register 版 <<<1, 32>>> 一个 warp 里只有 4 个 thread 对应 4 行 Q ;shared 版 4 block 各处理一行但 blockDim = 8 仍凑不满 warp—— issued/32 低是配置问题，不是 online merge 里的 branch divergence。

register 版 <<<ceil(4/32), 32>>> = <<<1, 32>>> ——一个 warp 里只有 4 个 thread 对应 4 行 Q 。shared 版 <<<4, 8>>> ——4 个 block 各处理一行，但 blockDim = 8 仍凑不满 warp，且 tile 内 matmul 由 thread 0 独占。

version	regs	smem (B)	time (μ s)	TC %	warp %	HBM %
v1	40	0	17.70	0.0	0.0	0.1
v1-shared	40	312	9.09	0.0	0.1	0.1

表 50 **Table (perf):** kernel-level performance metrics measured by Nsight Compute (ncu, CUDA 13.0) on a single A100 80GB SXM4 (FP32 peak 19.5 TFLOPS; Tensor-Core FP16 peak 312 TFLOPS; HBM peak 2039 GB/s). Columns: **regs** = launch__registers_per_thread; **smem** = launch__shared_mem_per_block_static (bytes); **time** = gpu__time_duration.sum; **TC %** = sm__pipe_tensor_cycles_active.avg.pct_of_peak_sustained_elapsed (0 means the kernel does not invoke tensor cores—wmma::mma_sync / MMA PTX); **warp %** = sm__warps_active.avg.pct_of_peak_sustained_elapsed (fraction of the SM's warp slots occupied on average); **HBM %** = dram__bytes.sum.pct_of_peak_sustained_elapsed (low when working set fits in L2). Rows are ordered as they appear on the optimization ladder in the chapter.

version	issued/32	pred_on/32	smem conf.	barrier stall	mem stall
v1	4.1	3.8	0	0.00	4.44
v1-shared	2.2	2.1	0	0.13	1.85

表 51 **Table (diag):** diagnostic metrics used as evidence for warp-lane utilization, shared-memory bank conflict, and warp-stall claims in the chapter. All from `ncu` on the same launch as the perf table. **issued/32** = `smsp__thread_inst_executed_per_inst_executed.ratio`, the average number of lanes that participated in each issued warp instruction (32 = every lane active; less than 32 means predication **or** branch divergence—`ncu` alone cannot distinguish, but on the CUDA kernels in this book it is almost always predication). **pred_on/32** = `smsp__average_thread_inst_executed_pred_on_per_inst_executed.ratio`, the average number of lanes that were predicated-on (did real work). The **gap issued - pred_on** counts lanes that occupied the issue slot but performed no work—this is the correct definition of “wasted warp cycles”, not divergence. **smem conf.** = `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` (0 + `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum`) means no measurable bank conflicts, either because the access pattern is conflict-free or because the input is too small to accumulate them). **barrier stall** = `smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio`, warps stalled at `__syncthreads` per issue-active cycle. **mem stall** = `smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio`, warps waiting on a long-latency memory operation (global-memory load, L2 atomic response) per issue-active cycle.

Warning. 这一章的问题规模是教学 default (B×S×H 数千个 float), kernel 单次运行只有 3-20 μ s。ncu 的定性指标 (issued/32、bank conflicts、barrier stall) 仍能反映 kernel 结构, 但绝对数字对生产规模不完全可信:

- HBM % 会偏低 (分母 elapsed time 含冷启动窗口)
- dram_bytes 可能被 L2 消化, GB/s (实测/逻辑) 两列差距明显

想拿到生产规模的数字, 把主参数 (rows/cols/hidden dim) 加到让工作集远超 L2 (40 MB)。

perf 表读三件事:

1. **v1-shared 9.28 μ s vs v1 register 17.86 μ s**——约 1.9 $\times$, **smem staging** 的收益在本规模上可测。绝对时间仍在 9-18 μ s noise 带内, 但 2 $\times$ 差距稳定——说明 Q 一次进 smem、 $\frac{K}{V}$ tile 协作 load 后从 smem 读, 确实比 register 版每 tile 每 thread 重读 global K, V 省流量。不能用 HBM % (两版均 0.1%) 验证这一点——< 4 KB 工作集全在 L2, dram 几乎不动。
2. **TC % / warp % / HBM %** 全表 **0.0-0.1%**。TC % = 0: scalar `dot_row_device`, 无 WMMA。warp % = 0.0-0.1%: 1-4 个 block 填不满 108 SM——和第 7 章 attention micro-benchmark 同一类「GPU 空转」。两版都没有触 **HBM** 带宽墙; 上文 $N = 1024$ 手算的 $\approx 1600 \times$ IO 差在本 shape 上测不出来。
3. **regs / smem fingerprint**: 两版均 40 regs; v1 0 smem, v1-shared 312 B (`q_shared + k_tile + v_tile + scores`)。smem 312 B 换 1.9 $\times$ 是方向正确的教学演示; 生产 FA 在此基础上把 smem 拉到 48-164 KB、换 warp 级 TC matmul。

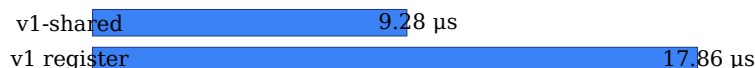


图 18 time (μs) : shared 版约 1.9× 于 register 版——smem 复用 Q 与 $\frac{K}{V}$ tile 的真实收益 ; 绝对时间仍在 teaching 规模的 noise 带内。



FA v1 : \$4 times 16\$ attention map , 外循环 $T_c = 4$ 个 $B_c = 4$ 的 **key tile**

图 19 绿色 cell $(i, j) = \text{query } i \text{ 对 key } j \text{ 的 score } (S[i, j])$, 从不写 **HBM**。外循环 `key_start += kTileKeys` 把列方向切成 4 段 (K0-K3、K4-K7、...) ——每段 load 一次 K_j, V_j , 在内层对所有 Q_i 复用 (register 版是 per-thread 复用逻辑 , shared 版是 smem resident)

diag 表读关键教学点 :

a) v1 register `issued/32 = 4.1` , `pred_on/32 = 3.8` ——**blockDim = 32** 但只有 4 行 Q

`<<<1, 32>>>` 里只有 thread 0-3 有输出 ; 其余 28 lane 占 issue slot 但无对应 query 行——结构性 **idle** , 不是 warp divergence (不同 lane 走不同 basic block) , `issued/32  $\approx$  4.1  $\approx$   $\frac{4}{32} \times 32$` ; `issued - pred_on  $\approx$  0.3` 来自 `if (row >= query_count) return` 等 guard 的 predicated-off lane。



v1 register : `<<<1, 32>>>` , 仅 lane 0-3 各负责一行 Q

代码 1 绿色 = 4 个 query thread。灰色 = warp 内无对应行的 lane——凑不满 **warp** 的配置问题 , 不是 online merge 里的 branch divergence。

b) v1-shared `issued/32 = 2.2` ——比 **register** 版更低 , 仍是 **teaching** 规模 **artifact**

shared 版 `blockDim = 8` : 协作 load $\frac{K}{V}$ tile 时最多 8 lane 同时写 smem ; tile 内 dot product + online merge 由 thread 0 串行执行——多数 phase 只有 1-8 lane 参与 issued 指令 , 平均 `issued/32` 降到 2.2。这不是 **smem** 把 **kernel** 变慢 (time $9.28 < 17.86$) , 而是并行度更差但 **IO** 更好——4 个 block 分摊 4 行 Q , 每 block 用 smem 省 global K, V 重读。生产 FA 用多 warp 算 S_{tile} 后 `issued/32` 会回到 28-32。

c) smem conf. = 0 ; shared 版 `barrier stall = 0.13`。312 B smem + 每 key tile 一次 `__syncthreads` ——metric 证实无 **bank conflict** (不能从 `k_tile[token][d]` 行主序布局单独推断) , `barrier stall` 略高于 register 版 (0.00) , 符合协作 load 的 sync 成本 ; 绝对值仍很小 , kernel 太短时不具生产诊断价值。

d) mem stall : v1 = 4.43 , v1-shared = 1.79。shared 版 global load 次数少 $\rightarrow$ `long_scoreboard` 略降——定性方向对 , 但两版 kernel 均 $< 20 \mu s$, `stall` 比率不能用来断言 memory-bound vs compute-bound。

无信息或为零的 **metric** :

- **TC %** : 全表 0.0——scalar FFMA , 无 tensor pipe ; 生产 FA 的 QK^T / PV 走 `mma.sync / WGMMA`。
- **HBM %** : 全表 0.1%—— < 4 KB 工作集 L2 resident ; 不能验证上文 $\approx 1600 \times$ HBM 流量差。
- **warp %** : 0.0-0.1%——1-4 block 填不满 GPU。

Key insight. $1.9\times$ 加速是真实的 **smem staging** 收益, 不是 TC 或 occupancy 的胜利——HBM / TC / warp % 全部 ≈ 0 。它对应 Algorithm 1 里「 Q_i load 一次、 K_j, V_j tile 复用」的 IO 结构; 放大到 $N \geq 1024, d \geq 64$ 且工作集超出 L2 后, 同一结构才体现为 `dram_bytes` 相对 naive 的 1-2 个数量级降幅。

Key insight. shared 版 `issued/32` (2.2) 低于 register 版 (4.1) ——反直觉但可解释: 更快的那版并行度更差 (thread 0 串行 + `blockDim = 8`)。lane 利用率低不等于 **kernel** 慢; Flash-Attention 生产实现要在 smem 复用与多 warp tile matmul 之间同时拉满 `issued/32` 和带宽。

Warning. 面试别把「FlashAttention 快」和「本章 kernel 快」混为一谈。前者是 IO 最优 + TC GEMM + 大 tile; 后者是最小正确子集, 刻意 scalar 以便读 online merge 循环。也不要低 `issued/32` 说成「严重 warp divergence」——本规模首先是 `blockDim < 32` 与 thread-0 串行的结构性 lane 浪费; 用 `issued/32` vs `pred_on/32` 区分 predication 与真 divergence。

ncu 该看什么

```
ncu --set full --section SpeedOfLight ./build/08_flash_attention_v1
```

教学规模太小, metric 仅作方法练习; 换 $N = 1024, d = 64$ 的 production FA 时期望:

- **dram_bytes.sum**: 相对 naive attention 同 shape 降 1-2 个数量级。
- **smsp_sass_thread_inst_executed_op_mufo**: `exp` 仍在 hot path——FA 不是「去掉 `exp`」。
- **lltex_t_bytes_pipe_lsu_mem_global_op_ld.sum** vs smem: `shared_kernel` 应看到更多 smem load、更少重复 global K/V read。
- **gpu_compute_memory_throughput.avg.pct_of_peak_sustained_elapsed**: IO 优化后 compute 占比应上升。

面试考点

面试考点. Q1: FlashAttention 快在哪? FLOPs 变了吗?

A: FLOPs 同阶 $O(N^2d)$; 快在 HBM——不 materialize $\frac{S}{P}$, IO 从 $O(N^2d)$ 量级降到 $O(N^2 \frac{d^2}{M})$ 。fusion 减 launch; SRAM 复用 $\frac{K}{V}$ tile。

面试考点. Q2: 推导 online merge 的 ℓ_{new} 和 O_{new} 。

A: $m_{\text{new}} = \max(m, m')$; $\ell_{\text{new}} = \ell \cdot \exp(m - m_{\text{new}}) + \ell' \cdot \exp(m' - m_{\text{new}})$; $\tilde{O}_{\text{new}} = \exp(m - m_{\text{new}})\tilde{O}_{\text{old}} + \sum_{k \in \text{tile}} \exp(x_k - m_{\text{new}})V_k$ 。Finalize $O = \frac{\tilde{O}}{\ell}$ 。与第 3 章 chunk merge 一致。

面试考点. Q3: IO 复杂度 naive vs FA? $N = 1024, d = 64, M = 100$ KB 估比。

A: Naive $O(N^2d) \approx 67 \times 10^6$ element-access 量级; FA $O(N^2 \frac{d^2}{M}) \approx 4.2 \times 10^4$; 比 $\approx 1600\times$ 。强调 FLOPs 不变。

面试考点. **Q4**: 为什么外循环 $\frac{K}{V}$ 、内循环 Q ?

A: 每个 K_j, V_j tile 只从 HBM 读一次, 在内层所有 Q_i 上复用——最小化 K, V 的 HBM 流量。反之则 Q 复读次数增。

面试考点. **Q5**: B_r, B_c 怎么选? 受什么约束?

A: $\text{smem} \approx 4(B_r + B_c)d + B_r B_c \text{ floats}$; 不能超过 per-block 限额 (A100 常用 48-164 KB), 寄存器/warp 数限制 occupancy。 $B_r, B_c \approx \sqrt{\frac{M}{d}}$ 量级; causal 下 B_c 略大有利 $\frac{K}{V}$ 复用。

面试考点. **Q6**: Causal mask 在 FA 里怎么实现? 能否 skip block?

A: 在 S 算完、softmax 前对 $j > i$ 置 $-\infty$ 。若整个 key tile 的 index 均 $>$ 当前 query row block 的最大 index, 可 skip 整个 tile (不 load K, V , 不 matmul)。

面试考点. **Q7**: Backward 为什么不存 S 还能算?

A: Recompute——反向用相同 tile 遍历再算 $S = QK^T$; 存 forward 的 (m, ℓ) per row ($O(N)$), trade FLOPs for HBM。

面试考点. **Q8**: v1 局限? v2 改了什么?

A: v1: smem 限 block size; 长序列 decode 并行度差; 单 SM 内串行多。v2: warp 分工、sequence parallel、更好 occupancy——算法/**online** 公式不变, 工程并行度改进。

面试考点. **Q9**: `old_scale = exp(running_max - new_max)` 首 tile 为何可为 0?

A: 尚无 prefix 时 $\ell = 0$, 不应缩放「旧」累加器; 设 `scale=0` 使 `accum*=0` 等价于 fresh start, 再加重当前 tile 贡献。

面试考点. **Q10**: Register 版 vs shared 版 teaching kernel 差别?

A: 数学相同; register 版每 thread 每 tile 从 global 读 K, V ; shared 版 Q 进 smem 一次、 K, V 按 tile 进 smem 再算——演示 IO-aware reuse, 后者接近生产结构。

Flash-Attention v2

第 8 章 FlashAttention v1 解决了要不要 **materialize** $N \times N$ **attention map** 的问题——online softmax + SRAM tiling 把 HBM traffic 从 $O(N^2)$ 压到 $O(N)$ 。但 v1 的生产 kernel 在 A100 上仍只用到约 25–40% 的理论 matmul 吞吐。v2 论文 (Dao, 2023) 没有改 **attention** 的数学，而是从三个工程维度把执行计划重排：

- **(a) 延迟归一化**：中间步不再每 tile 做 O/l_{new} ，只维护未归一化的 $\tilde{O}$ 和分母 l ，最后一步才除。
- **(b) 循环调换**：v1 外循环 K/V tile、内循环 Q row；v2 外循环 Q block、内循环 K/V tile——每个 Q block 独立成一个 CTA，**sequence parallelism** 大幅提升。
- **(c) warp 分工**：v1 的 split-K (4 warp 各算 K 的一部分，warp 间 merge) → v2 的 split-Q (4 warp 各拿 Q 的一个 slice，算完直接写，无 warp 间通信)。

对应源码：src/cuda/09_flash_attention_v2.cu。本章 ladder：

版本	核心思路	相对 peak 带宽
CPU reference	逐行 materialize softmax	—
v2 smem staging	1 CTA / Q row, K/V tile 进 smem	—
v2 warp-specialized	lane 分组：load / score / accum	—

教学 kernel 规模 $N_q = 4, N_k = 16, d = 8$ ；正文推导用 LLM 典型 shape ($N = 4096, d = 64, H = 32$)。

本章在全书中的位置：第 8 章 v1 讲 **IO-aware tiling + online softmax**；本章 v2 讲如何把已经 **IO** 最优的算法跑满 **SM**；第 10 章 v3 讲 **Hopper** 异步 **pipeline**。读代码顺序：
./build/08_flash_attention_v1 确认 online merge → ./build/09_flash_attention_v2 对照 Q-outer 与 warp 分工。

Key insight. v2 面试最高频的一句话：算法没变，变的是 **loop order**、**accumulator** 表示、和 **warp** 切分轴。能独立写出 v1/v2 伪代码 diff 和 O_t vs $\tilde{O}_t$ 公式，就已经超过大多数候选人。

三项改进如何协同（一图胜千言）

把一次 attention 看成许多独立工人 (CTA)，每人负责若干 query 行，按 key 顺序流式读 K, V ：

```
v1 生产布局 (K/V-outer) :
  CTA-0: 持 K tile 0 → 扫 Q row 0..127, 128..255, ... (Q 内循环)
  CTA-1: 持 K tile 1 → 扫所有 Q ...
  ... CTA 数 ~ N_k/B_c; 每个 CTA 内 split-K merge

v2 布局 (Q-outer) :
  CTA-0: 持 Q row 0..127 → 扫 K tile 0, 1, 2, ... (K/V 内循环)
  CTA-1: 持 Q row 128..255 → 扫所有 K ...
  ... CTA 数 ~ N_q/B_r; 每个 CTA 内 split-Q 独立
```

(a) 延迟归一化：每个工人在 inner loop 里少做 d 次除法，生命周期内 CPU core 更闲、tensor core 更忙。

(b) Q-outer：工人数量从「K tile 数」变成「Q block 数」——prefill 时长序列下工人多一个数量级。

(c) split-Q : 每个工人内部再分 4 个小组 (warp), 各组独立干活不互相等——消除 v1 split-K 的 merge 排队。

三者叠加才是论文 $2\times$;只做其中一项(例如只改 accum 表示但不改 loop order)收益远小于完整 v2。

与第 3 章 softmax 的符号对照

第 3 章	FA v1/v2	含义
(m, d)	(m, l)	running max + exp sum
一行 logits x_j	query i 对 key j 的 $S[i, j]$	softmax 输入
chunk merge	K/V tile merge	分块合并统计量
—	$\tilde{O} / O$	FA 额外维护分子
输出 y_j	输出 $O[i, :]$	归一化结果

表 52 **Table:** 第 3 章 online softmax 与 FA v1/v2 的三列符号对照。 d 对应 l (softmax 分母); FA 额外维护向量 $\tilde{O}$ (未归一化 output accumulator), v2 延迟归一化直到循环结束才除 l 。

Observation :符号不变、工程变——v1 每 tile 可隐式维护 $l \cdot O$;v2 显式分离 $\tilde{O}$ 与最终除法 ,inner loop 去掉 d 次 FDIV。读本章时重点转为三件套如何更省、如何按 **Q block** 并行、如何按 **warp** 切 **Q** 行。

第 3 章只维护标量 d (分母); FA 还要维护向量 $\tilde{O} \in \mathbb{R}^d$ (分子)。v1 每 tile 把 $\frac{\tilde{O}}{l}$ 算出来再当 O 用; v2 始终维护 $\tilde{O}$ 直到循环结束——这是 FA 相对 plain online softmax 的额外结构,也是延迟归一化的切入点。

读第 8 章时若已熟悉 $(m, l, \tilde{O})$ 三件套,本章重点转为这三件套在 **v2** 里如何更省、如何按 **Q block** 并行、如何按 **warp** 切 **Q** 行——符号不变,工程变。

问题定义

数学 (与 **v1** 相同)

单 head attention (省略 batch / scale):

$$O = \text{softmax}(QK^T)V, \quad Q \in \mathbb{R}^{N_q \times d}, K, V \in \mathbb{R}^{N_k \times d}$$

FlashAttention 的核心约束不变 : $S = QK^T$ 永不完整写入 **HBM** ,按 tile 流式处理 ,用 online state 合并。

IO 复杂度 : **v2** 与 **v1** 相同

第 8 章已证 : FA v1 的 HBM traffic 是 $O(N^2 \frac{d^2}{M})$ 量级,相对 naive 的 $O(N^2 d)$ 降 $O(\frac{d}{M})$ 。v2 不改变 **IO** 渐近阶—— Q, K, V 仍各读 $O(Nd)$, 中间不 materialize $\frac{S}{P}$ 。v2 的所有收益来自单 **CTA** 内算得更快和同时跑更多 **CTA**, 不是新的 IO 算法。

Warning. 面试别混淆 : v1 解决「要不要 $N \times N$ 矩阵」; v2 解决「既然不要了,怎么把 fused kernel 跑满 tensor core」。IO 题答 v1; 利用率题答 v2。

v1 在线状态 (回顾)

对 query 行 i , 处理第 j 个 K/V tile 后, v1 维护:

- m : running row max (数值稳定)

- l : running sum of exp (softmax 分母)
- $O \in \mathbb{R}^d$: 已归一化的 running output (每步 rescale 后仍是 $O = \sum w_k V_k / l$)

第 3 章 softmax 的 (m, d) 对应 FA 的 (m, l) ; v1 额外维护分子向量 O 。

Note. 符号: 本章用 l 表示 softmax 分母(论文里的 ℓ), $\tilde{O}$ 表示未除 l 的 output accumulator。与源码 `running_sum` / `accum` 一一对应。

CPU reference (对照基准)

源码 `flash_attention_cpu_reference` 按 query 行 materialize 全部 score, 再 two-pass softmax——与第 7 章 naive attention 同构, 只是合成一个函数便于对拍:

```
for (int row = 0; row < query_count; ++row) {
    // Pass 1: scores[key] = dot(q[row], k[key]); track row_max
    // Pass 2: weight = exp(scores[key] - row_max); accum += weight * v[key]
    // out[row] = accum / row_sum
}
```

GPU kernel 永远不应回到这种 $O(N_k)$ 临时数组——但 CPU 版是验证 online 公式正确性的 golden。09_flash_attention_v2.cu 的 `main` 对 `flash_attention_v2_kernel` 和 `flash_attention_v2_warp_kernel` 都做 `check_output`, 容差 10^{-4} 。

从 CPU 到 GPU: v2 改了什么、没改什么

没改: attention 的数学定义; online softmax merge 对 (m, l) 的更新式; $S = QK^T$ 和 $O = PV$ 两个 matmul 的 FLOPs 渐近阶; HBM 上不 materialize $N \times N$ 矩阵的 IO 目标。

改了: O 的表示 (归一化 vs $\tilde{O}$); double loop 的外层绑定 **grid**; warp 在 matmul 上的切分轴; causal 下能否 **tile** 级 **skip**; 单 CTA 内 CUDA core 的指令 **mix**。

这是典型的「算法正确、实现不饱和」→「实现饱和」二阶段优化。第 7 章 naive attention 是 IO 灾难; 第 8 章 v1 解决 IO; 本章 v2 解决 compute utilization——面试叙事应沿这条线展开, 不要跳步。

v1 的三个瓶颈

FlashAttention v1 论文在 A100 上测得: 即使 IO 已经最优, kernel 仍远未打满 tensor core。根因不是 online softmax 公式错, 而是执行计划有三处硬伤:

瓶颈 1: 非 matmul FLOP 占比过高

v1 每个 inner step 在 rescale O 时做完整归一化:

$$O_{\text{new}} = \frac{l_{\text{old}} \cdot e^{m_{\text{old}} - m_{\text{new}}} \cdot O_{\text{old}} + \tilde{P}V_j}{l_{\text{new}}}$$

其中 $\tilde{P}V_j$ 是 matmul (tensor core 友好), 但分子分母的 rescale 涉及:

- 向量乘标量 $O_{\text{old}} \times \exp(m_{\text{old}} - m_{\text{new}})$: d 次 FMA
- 向量除标量 $/l_{\text{new}}$: d 次 FDIV

每个 K/V tile 都做一遍除法。 N_k/B_c 个 tile $\rightarrow O(N_k/B_c \cdot d)$ 次 FDIV。A100 上 tensor core FP16/BF16 matmul 峰值 312 TFLOPS, 但 FP32 FDIV 走 CUDA core, 吞吐差一个数量级——非 matmul FLOP 是 v1 的头号敌人。

Key insight. Roofline 视角：FlashAttention 的 matmul 强度 $\approx 2d$ FLOP/B (读 Q/K/V tile), 已在 ridge 附近。额外 FDIV/FMA rescale 把有效 AI 拉低, 且这些 ops 无法进 **tensor core**。v2 改进 (a) 的目标就是砍掉 inner loop 里的 d 次除法。

手算：非 **matmul FLOP** 在 **hot loop** 里占多少

固定 $d = 128$, $B_c = 64$, 一个 Q block 对一个 K tile：

- **Matmul** QK^T : $2B_r B_c d \approx 2 \times 128 \times 64 \times 128 = 2.1\text{M FMA} \rightarrow \text{tensor core}$
- **Softmax** : $B_r B_c$ 次 exp + reduce $\rightarrow \text{CUDA core}$, 但 $O(B_r B_c)$, 通常小于 matmul 主项
- **Rescale v1** : $B_r \times d$ 次 FMA + $B_r \times d$ 次 FDIV 每 **tile** $\rightarrow \text{CUDA core}$

整条序列 $\frac{N_k}{B_c} = 64$ 个 tile : v1 rescale alone 产生 $64 \times 128 \times 128 \approx 1\text{M FDIV}$ 。Matmul 总量 $\approx 64 \times 2.1\text{M} \approx 134\text{M FMA}$ ——看起来 FDIV 占比小, 但 **FDIV** 吞吐只有 **FMA** 的 **1/20**, 且与 exp 竞争同一套 SFU/CUDA core。profiler 上常见现象：tensor core 30% 忙, 而 SFU/ fdiv 把 issue slot 占满——**v2 砍 FDIV** 是在给 **matmul** 腾 **issue** 带宽。

瓶颈 2：并行度 = batch $\times$ head

v1 伪代码 (论文原版, 外循环 K/V):

```
// v1: outer K/V, inner Q
for (int kv_tile = 0; kv_tile < num_kv_tiles; ++kv_tile) {
    load K_tile, V_tile into SRAM;
    for (int q_tile = 0; q_tile < num_q_tiles; ++q_tile) {
        load Q_tile into SRAM;
        S_tile = Q_tile @ K_tile^T;          // matmul
        online_softmax_update(S_tile);       // update m, l, 0 for each Q row
        O_tile += P_tile @ V_tile;           // matmul
        write O_tile back;                   // 或留 smem 等下一 kv_tile
    }
}
```

一个 CTA 负责一个 **K/V tile** 对所有 **Q tile** 的扫描。Grid 维度 $\approx \text{num_kv_tiles} \times B \times H$ 。当 N_k 不大时 (推理 decode 阶段 N_k 只有几百), `num_kv_tiles` 很小——整个 **GPU** 填不满。

v1 当时把 Q 放在内循环, 是为了让 O 留在寄存器/smem 里、避免每个 kv_tile 都从 HBM reload O 。这是正确的 SRAM 约束, 但牺牲了 Q 维并行。

瓶颈 3：split-K warp 分工 + warp 间通信

v1 在一个 CTA 内算 $S = QK^T$ 时, 把 K 维切给 4 个 warp (split-K matmul 风格):

```
warp 0: Q @ K[:, 0:16]   ┌
warp 1: Q @ K[:, 16:32]  ┤ → smem reduce → merge m, l, 0
warp 2: Q @ K[:, 32:48] ┤
warp 3: Q @ K[:, 48:64] └
```

每个 warp 只算 score tile 的一部分, 必须 `__syncthreads()` + smem atomic/reduction 合并 partial max、partial sum、partial O ——**warp** 间强同步, latency 高。

瓶颈 3 的 profiler 指纹

在 nsight-compute 里打开 FA v1 生产 kernel, 常见：

- `smsp__warp_issue_stalled_barrier` 高——split-K merge 的 `__syncthreads`
- `smsp__sass_thread_inst_executed_op_ddiv` 高——每 tile 的 O/l
- `sm_inst_executed_pipe_tensor` 仅 25-40%——matmul 被 non-matmul 和 barrier 饿死

v2 的三项改进精准对应这三个 metric。面试若问「你怎么知道 bottleneck 在哪」, 答: 先看 **tensor core** 利用率, 再看 **ddiv/barrier**——attention fused kernel 专用诊断路径。

Warning. 本书第 8 章教学源码 `08_flash_attention_v1.cu` 用的是简化版 (1 thread / Q row 或 1 block / Q row), 便于理解 online softmax。论文生产 v1 kernel 才是上述 split-K + K/V outer 布局。面试追问 v1 vs v2 时, 以论文布局为准, 再对照源码理解数值部分。

改进 (a) : 延迟归一化

v1 rescale : 每步都除

设第 t 步处理 tile T_t , 更新后 :

$$m_t = \max \left(m_{t-1}, \max_{j \in T_t} S[i, j] \right)$$

$$l_t = l_{t-1} \cdot e^{m_{t-1} - m_t} + \sum_{j \in T_t} e^{S[i, j] - m_t}$$

$$O_t = \frac{l_{t-1} \cdot e^{m_{t-1} - m_t} \cdot O_{t-1} + \sum_{j \in T_t} e^{S[i, j] - m_t} V_j}{l_t}$$

最后一步 O_T 已经是正确归一化输出。但每步都对 d 维向量做除法。

v2 rescale : 维护 $\tilde{O}$, 最后才除

关键观察 : 分子 $l_{t-1} e^{m_{t-1} - m_t} O_{t-1} + \tilde{P} V_t$ 和分母 l_t 同步缩放。定义未归一化 **accumulator** :

$$\tilde{O}_t = l_t \cdot O_t = l_{t-1} \cdot e^{m_{t-1} - m_t} \cdot \tilde{O}_{t-1} + \tilde{P} V_t$$

更新规则 (无除法) :

$$\tilde{O}_t = e^{m_{t-1} - m_t} \cdot \tilde{O}_{t-1} + \tilde{P} V_t \quad (\text{当 } l_{t-1} > 0)$$

$$l_t = l_{t-1} \cdot e^{m_{t-1} - m_t} + \sum_{j \in T_t} e^{S[i, j] - m_t}$$

全部 tile 处理完后 :

$$O = \tilde{O}_T / l_T$$

等价性证明 (面试可白板)

命题 : 设 v1 和 v2 从相同初始 state 出发, 处理相同 key tile 序列, 则 v2 最终 $\tilde{O}_T / l_T$ 等于 v1 最终 O_T 。

归纳 : 假设处理完 $t-1$ 个 tile 后 v1 得 $(m_{t-1}, l_{t-1}, O_{t-1})$, v2 得 $(m_{t-1}, l_{t-1}, \tilde{O}_{t-1})$ 且不变式 $\tilde{O}_{t-1} = l_{t-1} O_{t-1}$ 成立。

第 t 个 tile 贡献的新增未归一化权重为 $\Delta = \sum_{j \in T_t} e^{S[i, j] - m_t} V_j$ (relative to 统一 m_t) , v1 更新 :

$$O_t = \frac{l_{t-1} e^{m_{t-1} - m_t} O_{t-1} + \Delta}{l_t}$$

v2 更新 $\tilde{O}_t = l_{t-1} e^{m_{t-1}-m_t} \tilde{O}_{t-1} + \Delta = l_t O_t$ 。故 $\tilde{O}_t / l_t = O_t$ 。归纳成立。

要点：v2 只是把「先合并分子再除」改成「先合并未归一化分子最后除一次」——softmax 线性性保证等价，与第 3 章 online merge 证明同构。

FLOP 节省

每个 inner step 省 d 次 FDIV。总节省：

$$\text{saved} = (N_k / B_c) \times N_q \times d \text{ FDIV per head}$$

例： $N_k = 4096, B_c = 64, N_q = 4096, d = 64 \rightarrow$ 每个 head 省 $64 \times 4096 \times 64 \approx 1.7 \times 10^7$ 次除法。32 heads $\times$ batch 8 $\rightarrow$ 数十亿次 **FDIV** 从 hot loop 消失。

同时 rescale $\tilde{O}$ 的 FMA 从 $2d$ (乘 $l_{\text{old}}/l_{\text{new}}$ 等价形式) 降到 d (只乘 $e^{m_{\text{old}}-m_{\text{new}}}$)——省一半非 matmul FMA。

Key insight. 延迟归一化不改变数学结果——只是交换了除法和乘法的顺序。与 softmax 章 online merge 同一思想：把 **expensive op** 推迟到最后。生产 v2 kernel 里 $\tilde{O}$ 通常 FP32 累加，最后一步除 l_T 才写 BF16/FP16 输出。

数值例子：两 tile merge

$d = 1$ (标量 V)，query 行 i ，两 key tile 各 1 个元素。Tile 1 : $S = 1, V = 10$; Tile 2 : $S = 3, V = 100$ 。

标准 **softmax** : $m = 3, l = e^{-2} + 1 \approx 1.135, O = \frac{e^{-2} \cdot 10 + 100}{l} \approx 89.4$ 。

v1 逐步归一化 (每 tile 后除 l) :

- Tile 1 后 : $m = 1, l = e^0 = 1, O = \frac{10}{1} = 10$
- Tile 2 : $m_{\text{new}} = 3, \text{scale} = e^{1-3} = e^{-2}$
 - ▶ $l = 1 \cdot e^{-2} + e^0 = e^{-2} + 1$
 - ▶ $O = \frac{10 \cdot e^{-2} + 100}{e^{-2} + 1} \approx 89.4 \checkmark$

v2 延迟归一化 (维护 $\tilde{O} = l \cdot O$) :

- Tile 1 : $\tilde{O} = 10, l = 1$
- Tile 2 : $\tilde{O} = e^{-2} \cdot 10 + 100, l = e^{-2} + 1$
- Final : $O = \frac{\tilde{O}}{l} \approx 89.4 \checkmark$ ——中间从未做除法

源码对应

flash_attention_v2_kernel 里 accum 存的就是 $\tilde{O}$ (未除 l)，running_sum 是 l 。inner loop 只做 `accum[d] *= old_scale + FMA`，没有 inner 除法：

```
const float old_scale = (running_sum == 0.0f) ? 0.0f
                        : expf(running_max - new_max);
for (int d = 0; d < head_dim; ++d) {
    accum[d] *= old_scale;
}
// ... tile_sum, accum += weight * V ...
running_sum = running_sum * old_scale + tile_sum;
// 循环结束后才除：
out[row * head_dim + d] = accum[d] / (running_sum + kEps);
```

改进 (b) : 循环调换与 Sequence Parallelism

v1 vs v2 伪代码

v1 (K/V outer, Q inner) :

```
for kv_tile in 0..num_kv_tiles {           // grid.x
    load K_tile, V_tile;
    for q_tile in 0..num_q_tiles {          // 串行在 CTA 内
        load Q_tile;
        S = Q_tile @ K_tile^T;
        update(m, l, 0);
        O += P @ V_tile;
    }
}
```

v2 (Q outer, K/V inner) :

```
for q_tile in 0..num_q_tiles {              // grid.x - 每个 Q block 一个 CTA
    load Q_tile into SRAM/reg;
    init m, l, tilde(0);
    for kv_tile in 0..num_kv_tiles {         // 串行在 CTA 内
        load K_tile, V_tile;
        S = Q_tile @ K_tile^T;
        update(m, l, tilde(0));              // 无 inner 除法
        tilde(0) += P @ V_tile;
    }
    O = tilde(0) / l;                        // 最后归一化, 写 HBM
}
```

Note. 伪代码里 `update(m, l,  $\tilde{O}$ )` 在 v2 实现中不包含除法——这是与 v1 伪代码的唯一数值差异。Loop 结构差异是 grid 绑在 `q_tile` 还是 `kv_tile`。

为什么 **v2** 的 **Q-outer** 天然并行

每个 Q block 的 $(m, l, \tilde{O})$ 完全独立——不同 query 行之间无数据依赖。v2 直接：

```
flash_attention_v2_kernel<<<num_q_tiles * B * H, threads_per_block>>>(...);
```

Grid 从 $O(N_k/B_c)$ 变成 $O(N_q/B_r \times B \times H)$ 。LLM prefill 时 $N_q = N_k = 4096$ 量级, CTA 数从 ≈ 64 涨到 $\approx 4096/B_r \times 32 \approx 10^4$ ($B_r = 128$) ——**SM** 利用率大幅提升。

这就是 v2 引入的 **sequence parallelism** : 在 sequence (Q) 维上切 CTA, 而非只在 batch $\times$ head 维并行。

v1 为什么当时 **Q-inner**

v1 把 Q 放内循环不是随便写的—— O 必须跨 **K/V tile** 累积。若 Q-outer 但每个 CTA 只处理一个 Q block, $\tilde{O}$ 可以留在寄存器/smem 里走完所有 kv_tile, 不需要每 tile 从 HBM reload O 。v1 时代担心 smem/reg 不够放整个 Q block 的 state, 所以让 K/V tile 「驻留」CTA、Q 行在内循环流式扫——用 Q-inner 换 O 的 SRAM residency。

SRAM 账本：为什么 v2 敢 Q-outer

估算一个 CTA 的 SRAM 需求 (FP32, $B_r = 128$, $B_c = 64$, $d = 128$):

Buffer	Size	说明
Q tile	$128 \times 128 \times 4 = 64 \text{ KB}$	驻留整个 inner loop
K tile	$64 \times 128 \times 4 = 32 \text{ KB}$	每 kv 迭代覆盖
V tile	32 KB	与 K 同形
S tile	$128 \times 64 \times 4 = 32 \text{ KB}$	score 暂存
$\tilde{O}, m, l$	$128 \times 128 \times 4 + 2 \times 128 \times 4 \approx 65 \text{ KB}$	Q-outer state
合计	$\approx 225 \text{ KB}$	超 A100 默认 48KB ; 需 dynamic smem 或减 tile

表 53 **Table:** v2 Q-outer CTA 的 SRAM 账本 (FP32, $B_r = 128$, $B_c = 64$, $d = 128$)。Q tile 驻留整个 inner kv loop ; $\tilde{O}, m, l$ 是 Q-outer 相对 v1 K/V-outer 的新增 resident state。A100 单 block smem 上限 164 KB。

Observation : 合计 $\approx 225 \text{ KB}$ 超默认 48 KB——v2 敢 Q-outer 的前提是硬件 smem 增大 + 延迟归一化减 reg 压力 ; v1 时代 K/V-outer 是 smem 不够大时的正确妥协。

论文通过 减小 B_r 或 d 分块、寄存器 **spill** 到 **smem**、**dynamic shared memory** 控制在 164 KB 上限内。v1 时代 K/V-outer 让 K, V tile 在 outer CTA 驻留、 O 随 Q 行流式更新——是 smem 不够大时的正确妥协。硬件 smem 增大 + 延迟归一化减 reg 压力后 , v2 翻转 loop order。

Occupancy 与 B_r 的权衡实例

A100 单 SM : 最多 2048 threads、64 warps ; 每 block 128 threads $\rightarrow$ 理论最多 16 block/SM。若 smem = 128 KB/block , 164 KB 上限 $\rightarrow$ 每 SM 最多 1 block——**occupancy** 从 **16** 降到 **1**。

FA-2 论文常用 $B_r = 128$, $B_c = 64$, $d = 128$ 时 smem $\approx 128\text{--}160 \text{ KB}$, occupancy 常只有 1-2 block/SM——靠 **grid** 总 **CTA** 数填满载 , 不靠单 SM 高 occupancy。这是 GEMM-like kernel 的常见模式 : 大 tile 低 occupancy $\times$ 多 SM 并行。

面试 contrast : vector add (第 1 章) 靠高 occupancy 掩盖 latency ; FA-2 靠大 matmul tile 提高 instruction intensity , occupancy 反而不是第一优化目标。

v2 证明 : Q block 的 $\tilde{O}$ ($B_r \times d$ 个 float) + m, l (B_r 个 float) 完全可以放进 smem/reg。配合改进 (a) 不再 inner 除法 , Q-outer 的 reload 代价消失。

Note. 改进 (a) 和 (b) 互相依赖 : Q-outer 让每个 CTA 持有完整 $\tilde{O}$ 直到最后 ; 延迟归一化保证 $\tilde{O}$ 在 inner loop 里只做 FMA、不做 FDIV , 寄存器压力可控。

并行度公式

版本	CTA 数 (单 head)	瓶颈场景
v1	$\left\lceil \frac{N_k}{B_c} \right\rceil \times B \times H$	decode : N_k 小 $\rightarrow$ grid 小
v2	$\left\lceil \frac{N_q}{B_r} \right\rceil \times B \times H$	prefill : N_q 大 $\rightarrow$ grid 大

表 54 **Table:** v1 vs v2 单 head 的 CTA 数公式。v1 grid 绑 K/V tile 轴 (N_k) ; v2 grid 绑 Q block 轴 (N_q)。decode 时 $N_q = 1$ 则 v2 无 sequence parallel 优势。

Observation : 当 $N_q = N_k$ 时两公式同阶 ; 差异在 $\text{decode} (N_q = 1)$ 或 $\text{cross-attention} (N_q \neq N_k)$ ——prefill 长 prompt 时 v2 CTA 数随 N_q 线性增长 , 这是 sequence parallelism 的核心收益。

Decode 阶段 $N_q = 1$ 时 v2 也只有 1 个 Q CTA——此时 v2 无 sequence parallel 优势 , 靠 split-Q warp 分工和延迟归一化提速。Prefill 阶段 v2 相对 v1 论文报告 **2×** 端到端加速 (tensor core + 大 shape) , 主要来自并行度 + 非 matmul FLOP 削减——本书 **teaching kernel** ($N_q = 4$) 只能看到 **1-4%** 量级差异 , 见下文「实测」节。

手算 : prefill 时 CTA 数差多少

$B = 8, H = 32, N = 4096, B_c = B_r = 128$:

- v1 : $\lceil \frac{4096}{128} \rceil \times 8 \times 32 = 32 \times 256 = 8192$ CTA
- v2 : $\lceil \frac{4096}{128} \rceil \times 8 \times 32 = 8192$ CTA——同阶 ?

看起来一样 , 是因为此例 $N_q = N_k$ 。差异在 **decode** 或 **cross-attention** ($N_q \neq N_k$) :

- Decode 一步 : $N_q = 1, N_k = 8192$ (KV cache 长度)
 - v1 grid : $\lceil \frac{8192}{128} \rceil \times 8 \times 32 = 64 \times 256 = 16384$ CTA——仍不少
 - v2 grid : $\lceil \frac{1}{128} \rceil \times 8 \times 32 = 1 \times 256 = 256$ CTA——骤降
- Cross-attention (encoder-decoder) : $N_q = 512$ (decoder) , $N_k = 4096$ (encoder)
 - v1 : $32 \times 256 = 8192$
 - v2 : $4 \times 256 = 1024$ ——v1 在 K 维切更多 CTA , 但每个 CTA 内要串行扫 512 行 Q

Sequence parallelism 的收益在 N_q 大时最显著 (prefill、长 prompt) ; N_q 小时靠 (a)(c) 优化单 CTA 效率。工业界因此 prefill 用 FA-2 , decode 常配合 paging / split-KV 等专用 kernel (FlashDecoding 等) ——超出本章 , 面试提一句即可。

Grid 映射 (生产)

典型 launch :

```
dim3 grid(ceil_div(N_q, kBlockM), H, B); // x=Q block, y=head, z=batch
dim3 block(kThreadsPerBlock);           // 128 threads = 4 warps
```

blockIdx.x 决定 Q tile 起点 , blockIdx.y/z 选 head 与 batch。v1 生产 kernel 常把 grid.x 绑 K tile——这是 v1/v2 **launch** 签名 最直观的差别。

改进 (c) : Split-K → Split-Q

v1 split-K 布局

一个 CTA 处理 $B_r \times B_c$ 的 S tile , K 维 B_c 切给 W 个 warp :

```

      K dim →
      [ 0..15 | 16..31 | 32..47 | 48..63 ]
Q  w0 [ score fragment 0 ] ┌
    w1 [ score fragment 1 ] ┤ smem: merge max, sum, 0
    w2 [ score fragment 2 ] ┤ __syncthreads × 2+
    w3 [ score fragment 3 ] └
```

4 warp 各算 1/4 的 dot product , partial result 必须合并——**warp** 间 **barrier + smem** 通信是 v1 CTA 内的常态。

v2 split-Q 布局

v2 把 Q 维 B_r 切给 W 个 warp , 每个 warp 拿 B_r/W 行 Q :

```
Q rows →
[ row 0-31 | row 32-63 | row 64-95 | row 96-127 ]
K w0 [ 完整  $m, l, \tilde{O}$  for rows 0-31 ] → 独立写 0[0:32]
w1 [ 完整  $m, l, \tilde{O}$  for rows 32-63 ] → 独立写 0[32:64]
w2 [ 完整  $m, l, \tilde{O}$  for rows 64-95 ] → 独立写 0[64:96]
w3 [ 完整  $m, l, \tilde{O}$  for rows 96-127 ] → 独立写 0[96:128]
```

每个 warp 拥有独立的 $(m, l, \tilde{O})$, 算完整条 Q slice 对所有 K/V tile 的 attention——**warp** 间零通信, 不需要 `__syncthreads()` 做 reduction。

Key insight. split-Q 能 work 的前提是改进 (b) : Q-outer 让每个 CTA 只服务一个 Q block , warp 切 Q 行自然独立。若仍是 v1 的 K/V-outer , Q 行在不同 kv_tile 间共享 state , 就无法按 Q slice 切 warp。

K/V tile 仍由 block 内 thread 协作 load 到 smem (producer) , 各 warp consumer 读同一份 smem——这是 **smem broadcast** , 不是 warp 间 reduction。

教学 kernel 的 warp 分工

`flash_attention_v2_warp_kernel` 把 32 lane 分成三组 (规模 $d = 8$, $B_c = 4$ 的玩具版) :

- lane 0-7 : 持有 `q_shared` , 最后写 `out[row, d]`
- lane 0-3 : 协作 load K/V tile + 算 dot product score
- lane 0 : online softmax state update (m, l , weight)

```
if (lane < kTileKeys) {
    // load K/V + compute score for this lane's key
    shared_scores[lane] = dot(q_shared, shared_k[lane]);
}
__syncthreads();
if (lane == 0) {
    // merge max, compute weights, update running_sum
}
__syncthreads();
if (lane < kHeadDim) {
    accum[lane] *= old_scale_shared;
    for (int t = 0; t < kTileKeys; ++t)
        accum[lane] += shared_weights[t] * shared_v[t][lane];
}
```

生产 v2 kernel 是 $4 \text{ warp} \times 32 \text{ lane} = 128 \text{ threads/block}$, 每 warp 32 行 Q , tensor core 做 $S = QK^T$ 和 PV 。教学版用 lane 分组示意角色分离——读源码时抓住「谁 load、谁 matmul、谁维护 state」。

split-K 与 split-Q 的 reduction 对比

操作	split-K (v1)	split-Q (v2)
切分轴	K 维 B_c	Q 维 B_r
Partial 产物	score fragment	完整 row 的 $(m, l, \tilde{O})$
Merge 对象	max, sum, $\tilde{O}$ across K	无——每 warp 完整
Sync 类型	<code>__syncthreads</code> + smem reduce	仅 K/V load 后 broadcast
写回	merge 后统一写	各 warp 写各自 Q slice
Failure mode	barrier 不平衡 → 慢	smem bank conflict on broadcast

表 55 **Table:** v1 split-K 与 v2 split-Q 的 reduction 对比。split-K 每 tile 需 cross-warp merge $(m, l, \tilde{O})$; split-Q 每 warp 独立维护完整 row state , 仅 K/V load 后 smem broadcast。

Observation : split-Q 能 work 的前提是 Q-outer (改进 b) ——每个 CTA 只服务一个 Q block , warp 切 Q 行自然独立。v1 split-K 的 `__syncthreads` barrier 是 profiler 上 `warp_issue_stalled_barrier` 高的根源。

生产 FA-2 用 swizzle layout 解决 broadcast 的 bank conflict ; v1 用 warp shuffle 加速 split-K reduce——两种分工对应不同 bottleneck。

Warning. 教学 kernel 不是生产性能。它用 scalar dot product 代替 tensor core ,单 thread 做 softmax merge。价值在于看清 **v2** 的数据流 :Q-outer → K/V inner → 延迟归一化 → warp 角色分离。

生产 kernel 的 warp 时间线 (概念)

一个 128-thread block (4 warp) 处理 $B_r = 128$ 行 Q、 $B_c = 64$ 列 K 的理想分工：

Warp	Q 行	寄存器 state	Matmul	写回
w0	0-31	$(m, l, \tilde{O}) \times 32$ 行	S tile, P V	独立
w1	32-63	同上	同上	独立
w2	64-95	同上	同上	独立
w3	96-127	同上	同上	独立

表 56 **Table:** 生产 FA-2 128-thread block(4 warp)的理想 split-Q 分工($B_r = 128, B_c = 64$)。每 warp 32 行 Q、独立 $(m, l, \tilde{O})$;K/V tile 由 block 协作 load 后 broadcast 给 4 个 consumer warp。

Observation : K/V tile 一次 **load** ,四次 **consumer**——softmax rescale 在各 warp 私有寄存器完成 ,无 **cross-warp reduce**。这与 v1 split-K 每 tile 必须 4 warp merge 形成鲜明对比。

K/V tile load 由 block 内所有 thread 协作 (或 dedicated producer warp) ——一次 **load** ,四次 **consumer**。Softmax rescale 在各 warp 私有寄存器完成 ,无 **cross-warp reduce**。这与 v1 split-K 每 tile 必须 4 warp merge 形成鲜明对比。

v1: smem staging kernel

`flash_attention_v2_kernel` 是 v2 算法骨架的最简 GPU 实现 : 1 CTA = 1 Q row , K/V tile 进 smem , thread 0 做全部数值。

```

__global__ void flash_attention_v2_kernel(
    const float* q, const float* k, const float* v, float* out,
    int query_count, int key_count, int head_dim) {
    const int row = blockIdx.x; // Q-outer: 每个 block 一行 Q
    // ...
    for (int key_start = 0; key_start < key_count; key_start += kTileKeys) {
        // threads 协作 load K/V tile → shared_k, shared_v
        __syncthreads();
        if (threadIdx.x == 0) {
            // score → online update accum ( $\tilde{O}$ ), running_sum ( $l$ )
        }
        __syncthreads();
    }
    if (threadIdx.x == 0)
        out[row * head_dim + d] = accum[d] / (running_sum + kEps);
}

```

Launch : `<<<kQueryCount, kTileKeys>>>` —grid = N_q , block = B_c threads 协作 load。

对照 `flash_attention_v2_kernel` 完整结构

教学 kernel 的 host 侧与 device 侧接口 (摘自 `09_flash_attention_v2.cu`) :

```

// Host
flash_attention_v2_kernel<<<kQueryCount, kThreadsPerBlock>>>(
    device_q, device_k, device_v, device_out,
    kQueryCount, kKeyCount, kHeadDim);

// Device 常量
constexpr int kQueryCount = 4;
constexpr int kKeyCount = 16;
constexpr int kHeadDim = 8;
constexpr int kTileKeys = 4; // B_c

```

Device 侧 state 变量语义 :

变量	类型	数学
<code>running_max</code>	float	m — row max
<code>running_sum</code>	float	l — softmax 分母
<code>accum[d]</code>	float[kHeadDim]	$\tilde{O}$ — 未归一化 output
<code>shared_k/v</code>	smem	当前 K/V tile
<code>old_scale</code>	float	$\exp(m_{\text{old}} - m_{\text{new}})$

表 57 **Table:** `flash_attention_v2_kernel` device 侧 state 变量与数学符号对照。 `accum` 存 $\tilde{O}$ (未除 l); inner loop 只做 `accum *= old_scale + FMA`, 循环结束后才 `accum / running_sum`。

Observation : `old_scale = exp( $m_{\text{old}} - m_{\text{new}}$ )` 同时缩放 $\tilde{O}$ 和 l ——与第 8 章 v1 相同 trick, 但 v2 明确分离 $\tilde{O}$ 与最终除法。 `flash_attention_v2_warp_kernel` 把 state 放 smem 是教学简化, 生产用 warp 私有寄存器。

flash_attention_v2_warp_kernel 把 running_* 放到 *_shared , 让 lane 0 写、全 block 读——教学简化；生产用 warp 私有寄存器。

对比 08_flash_attention_v1.cu 的 flash_attention_v1_shared_kernel : v1 教学版也是 1 block / Q row , 但 inner loop 的 rescale 逻辑等价于 v1 论文的每步归一化写法 (accum 在每步后语义上等于 $l \cdot O$ 但未显式区分)。v2 kernel 明确分离 $\tilde{O}$ 与最终除法 , 且为 warp 分工版铺路。

完整 inner loop 解读

对照 09_flash_attention_v2.cu 第 136-189 行 , 一个 K/V tile 的生命周期 :

1. 协作 load (threadIdx.x = local key index)

每个 thread 搬一个 key 的 K[key, :] 和 V[key, :] 到 shared_k[local_key][d] / shared_v。
kTileKeys = 4 threads 即可 cover tile——教学版 block 大小 = tile 宽。

2. Score + online update (thread 0)

- 对 tile 内每个 key 算 `score = dot(q[row], shared_k[t])`
- 更新 `running_max` $\rightarrow$ `new_max` , 算 `old_scale = exp(running_max - new_max)`
- `accum[d] *= old_scale`——对应 $\tilde{O}$ rescale , 无除法
- 算 `weight = exp(score - new_max)` , 累加 `tile_sum` 和 `accum += weight * V`
- `running_sum = running_sum * old_scale + tile_sum`——更新 l

3. 循环结束

`out[row,d] = accum[d] / running_sum`——全函数唯一一次 **FDIV** 向量。

Note. `old_scale == 0` 当 `running_sum == 0` (首个 tile) : `accum *= 0` 等价 fresh start , 与第 8 章 v1 教学 kernel 相同 trick。

ncu 实测

ncu 实测 — v: tile_staged (smem staging). 规模: $N_q = 128, N_k = 512, \text{head_dim} = 32$.

Duration	354.4 μ s	vs FA v1 shared 595 μ s , *快 1.7×* (\$d=32\$)
Memory SOL	23.1 %	
Compute SOL	8.7 %	
L2 Hit Rate	98.9 %	
Achieved Occupancy	1.9 %	\$N_q = 128\$ 行 $\times$ warp-per-row $\rightarrow$ 128 blocks 仍不满 GPU

v2 相比 v1 的算法层收益 (延迟归一化 + Q-outer 循环) 已经落地 , $d = 32$ 时快 1.7×。但 memSOL 只有 23%——原因 :

- 教学 kernel 用 `<<<N_q, 32>>>` (一个 warp 处理一个 query row)——这个“warp-per-row”分工要求 **head_dim ≤ 32** , 否则一个 warp 装不下所有 d 维。
- $d = 32$ 时刚好一个 warp = 32 lanes 覆盖整个 head_dim。
- Occupancy 1.9% 说明 GPU 大部分 SM 依然 idle——需要 batch 和 head 维度上并行来填满硬件 (生产 FA v2 的 `<<<num_heads  $\times$  batch  $\times$  ceil( $N_q / B_r$ ), 128>>>` 是这样做的)。

瓶颈判定. smem staging + Q-outer 已经把算法拉直 , 但 warp-per-row 分工没有让 warp 内 lanes 之间做更细粒度的工作分离

证据. Memory SOL 23% 说明 HBM 还没打满 ; Compute SOL 8.7% 说明 warp 内的 32

lanes 都在算 head_dim 长度为 32 的同一个 dot product
 下一步. v2 (warp-specialized) 让不同的 lane 负责不同的角色 —— e.g. lane 0-15 负责一个 QK^T tile, lane 16-31 负责下一个 tile 的 V 累加, 用 warp-level primitives 做 tile 之间的 register 通信

v2: warp-specialized kernel

flash_attention_v2_warp_kernel 在 smem staging 之上加入 lane 角色分离：

```
__global__ void flash_attention_v2_warp_kernel(...) {
    const int row = blockIdx.x;
    const int lane = threadIdx.x;
    // lane < d: load q_shared
    // lane < B_c: load K/V + score
    // lane == 0: softmax state
    // lane < d: update accum ( $\tilde{O}$ ), final write
}
```

Launch: `<<<kQueryCount, 32>>>` —— 一个 warp 处理一行 Q 。

关键差异：

组件	smem kernel	warp kernel
Q load	thread 0 隐式读 global	lane 0-7 协作 load q_shared
K/V load	每 thread 1 key	lane 0-3 各 load 1 key
score	thread 0 串行 dot	lane 0-3 并行 dot
softmax	thread 0	lane 0
accum update	thread 0 串行 d 维	lane 0-7 各更新 1 维
写回	thread 0	lane 0-7 并行写

表 58 **Table:** flash_attention_v2_kernel (smem) 与 flash_attention_v2_warp_kernel 的 lane 角色分工。warp 版把 score 与 d 维 accum 更新从 thread 0 串行拆出——split-Q 在 d 维的微型演示。

Observation : warp 版 issued/32 (6.6) 高于 smem 版 (1.6), 但 time 仅快约 1.1%——本规模瓶颈是 launch 固定开销, 不是 SM issue 带宽。生产 split-Q 是 4 warp $\times$ 32 行 Q , 而非 8 lane 分 d 维。

warp 版把 d 维 accum 更新并行化——生产 kernel 里这对应 split-Q : 每 warp 独立维护 $\tilde{O}$ 的若干维/行。

warp kernel 相对 smem 版的增量

flash_attention_v2_warp_kernel 新增：

1. **q_shared** : lane 0-7 预加载整行 Q , score 阶段不再读 global Q ——减少 HBM 流量, 也为 split-Q 预热。
2. **shared_weights** : lane 0 算完 softmax weight 后 broadcast, lane 0-7 并行做 $\text{accum}[d] += \text{weight} * V[t, d]$ —— d 维 FMA 从串行变并行。

3. **state** 放 **smem** : `running_max_shared`, `old_scale_shared` 等——lane 0 写、其他 lane 读，教学用 smem 代替 warp shuffle。

生产 FA-2 会把 state entirely 放在 warp 私有寄存器，weight broadcast 用 `__shfl_sync` ——但数据流与教学 kernel 一致。

warp kernel inner loop 逐步对照

以下映射 `flash_attention_v2_warp_kernel` 第 327-397 行：

Step A — Q 驻留 smem (只做一次)

```
if (lane < kHeadDim)
    q_shared[lane] = q[row * head_dim + lane];
if (lane == 0) { running_max_shared = -1e30f; running_sum_shared = 0.0f; }
__syncthreads();
```

一行 Q 被 8 个 lane 载入 smem，后续所有 key tile 复用——对应生产里 Q tile 驻留寄存器/smem 直到 kv 循环结束。

Step B — K/V tile load (每 kv tile)

`lane < kTileKeys` 的 thread 各负责一个 key index，搬 `K[key,:]` / `V[key,:]` 到 `shared_k/v[lane][d]`。与 smem kernel 相同，但不再由 thread 0 独占 load。

Step C — 并行 score

每个 `lane < kTileKeys` 算 `dot(q_shared, shared_k[lane])` 写入 `shared_scores[lane]` ——4 个 score 并行，对比 smem kernel 里 thread 0 串行 4 次 dot。

Step D — lane 0 softmax merge

与 smem kernel 相同：求 `tile max` → `new_max` → `old_scale` → `shared_weights[t] = exp(score - new_max)` → 更新 `running_sum_shared`。这是整个 block 唯一的串行 **softmax** 瓶颈——生产版用 warp 级 parallel max/sum 或 dedicated softmax warpgroup。

Step E — 并行 accum 更新

`lane < kHeadDim` : `accum[lane] *= old_scale_shared`，再对 tile 内每个 key 做 FMA。8 维 output 并行更新——split-Q 在 d 维的微型演示；生产版是 32 行 $Q \times d$ 维。

Step F — 写回

`out[row * head_dim + lane] = accum[lane] / running_sum_shared` ——每 lane 写一维，无 thread 0 独占 epilogue。

寄存器压力：为什么 **split-Q** 需要延迟归一化

若 v2 仍用 v1 的「每 tile 归一化 O 」且 split-Q，每个 warp 要在寄存器里存已除 l 的 O ——每次 m 更新还要对 O 做 rescale + 再除新 l ，寄存器读写次数翻倍。

延迟归一化让 warp 只维护 $\tilde{O}$ 和标量 l, m ——rescale 只做 `tilde(0) *= exp(m_old - m_new)` (FMA)，无 FDIV。这对 split-Q 至关重要：4 warp $\times$ 32 行 $\times d$ 维 state 全部在寄存器，任何多余 ops 都会 spill 到 local memory。

Key insight. warp kernel 相对 smem kernel 的核心增量是 **Step C** 和 **Step E** 的并行度——这正是 split-Q 的缩影：把原本 thread 0 串行的 $O(d)$ 和 $O(B_c)$ 工作拆给多个 lane/warp。生产 FA-2 进一步把 Step C/E 换成 WGMMA，Step D 留在 CUDA core/SFU。

ncu 实测

ncu 实测 — v: warp_specialised. 规模: $N_q = 128, N_k = 512, \text{head_dim} = 32$.

Duration	330.1 μs	vs staged 354 μs , 快 7%
Memory SOL	23.6 %	
Compute SOL	6.8 %	*↓ 从 8.7%*——SM 变闲, warp 内不再全 lane 算同一 dot
L2 Hit Rate	100.3 %	
Achieved Occupancy	1.9 %	

7% 提升——比预期小。原因跟教学 kernel 的性质有关：

- warp specialization 把 Step C (O 累加) 和 Step E (sum 归一化) 拆给不同 lane, 减少了串行部分, 但 **head_dim = 32** 让 warp 的 32 lane 刚好一人一维, 本身已经是完美的 lane parallelism。
- 真正会大幅提升的 workload 是 head_dim 更大时 (e.g. $d = 128$), 单 warp 无法 cover d , 此时 warp specialization + split-Q 让多 warp 协作。教学 kernel 卡在 $d \leq 32$, 观察窗口有限。

落地评估。FA v2 教学 kernel 展示了延迟归一化、Q-outer 循环、warp specialization 三个关键 v1→v2 变化。

$d = 32$ 的 warp-per-row 设计是本 kernel 的硬约束。生产 FA v2 用 $B_r = 128 / 4$ warp 的 CTA 布局——每 warp 独立处理 32 个 query row, 每 row 由 4 warp 组内的 32 lanes 一起 cover $d = 128$ 的 head_dim。教学 kernel 到此为止, 下章 v3 展示 async pipeline。

Causal Mask 优化

Decoder self-attention 用 causal mask: query i 只能 attend 到 key $j \leq i$ 。v1 在每个 score 上 `if (i < j) score = -inf`, 仍加载整个 **K tile**。

v2 利用 Q-outer + K/V inner 顺序: 处理 Q block 行 $[i_0, i_0 + B_r)$ 时, 若整个 K tile 的 key index 范围 $[j_0, j_0 + B_c)$ 满足 $j_0 > i_0 + B_r - 1$ (整个 tile 在 causal 上三角), 直接 **skip**——不算 matmul、不做 softmax。

Key insight. 对长序列 prefill, 越靠后的 Q block 跳过的 K tile 越多。Q block 在序列尾部时, 大约跳过 50% 的 K/V tile work。v1 因 K/V-outer 无法做 tile 级 skip (一个 kv_tile 要服务所有 Q row, 部分 Q 行需要、部分不需要)。v2 Q-outer 让每个 CTA 知道自己的 Q 行范围, tile skip 成为结构化优化。

量化: $N = 4096, B_c = 64 \rightarrow 64$ 个 K tile。对最后一个 Q block (行 4032-4095), 只需 attend 到 key 4095, 前 63 个 K tile 中约 62 个完全在上三角——**skip $\approx 97\%$ tile**。平均 over all Q blocks, causal skip 省 $\approx N_k / (2B_c) / \left(\frac{N_k}{B_c}\right) = 50\%$ 的 K/V tile 计算。

Mask 实现: 生产 kernel 在 loop 开头比较 `kv_tile_idx * B_c > q_row_end`, 成立则 `continue`——零 exp、零 matmul。

手算：causal skip 省多少 work

$N = 4096, B_r = B_c = 64 \rightarrow 64$ 个 Q tile、64 个 K tile。Causal 下 Q tile q 只需 K tile $0..q$ (共 $q + 1$ 个), 不需要 $q + 1..63$ 。

- Q tile 0 : 1 个 K tile
- Q tile 1 : 2 个
- ...
- Q tile 63 : 64 个

总 matmul tile 数 : $\sum_{q=0}^{63} (q + 1) = 64 \times \frac{65}{2} = 2080$ 。无 causal 时需 $64 \times 64 = 4096$ 。比值 $\frac{2080}{4096} \approx 50.8\%$ ——正好一半。

最后一个 Q tile ($q = 63$) 仍要 64 个 K tile——无法 **skip** ; 第一个 Q tile ($q = 0$) 可 skip 63 个——**skip 98.4%**。v2 的收益在 prompt 前段最大 (大量 skip), 后段接近全量 matmul。这与 GPT prefill 「越写越慢」的直觉一致——不是 bug, 是 causal 结构。

v1 K/V-outer 即使用元素级 mask, 每个 kv CTA 仍要对所有 Q tile 做 matmul (mask 在 softmax 前), 无法获得上述 tile 级 50% 节省。

Causal tile skip 图示

Self-attention causal 矩阵 ($\times$ = 需计算, 空白 = 可 skip 的 block):

```
      K tile → 0    1    2    ... 63
Q tile 0      [×] [ ] [ ] ... [ ]
      1      [×] [×] [ ] ... [ ]
      ...
      63      [×] [×] [×] ... [×]
```

Q tile i 只需列 index $\leq i$ 的 K tile。v2 每个 CTA 固定 Q tile index, 内循环 `for kv_tile = 0..i` 即可——连 **load** 都省略。v1 K/V-outer 的 CTA 固定 kv tile, 必须对所有 Q tile 计算, 无法省略——最多在元素级 mask。

Partial tile (对角线穿过 block)

当 Q tile 与 K tile 索引区间相交但不完全包含时 (block 跨对角线), 不能整 tile skip, 只对 $j > i$ 的元素置 $-\infty$ 或跳过 dot product 的 partial 累加。实现复杂度高, 但 v2 Q-outer 至少能 skip 大量纯上三角 block——这是 GPT 类模型 prefill 的重要加速来源。

非 matmul FLOP 为什么重要

A100 FP16 tensor core : ≈ 312 TFLOPS。CUDA core FP32 FMA : ≈ 19.5 TFLOPS。FDIV 更慢。

为什么 tensor core 时代 non-matmul 更「贵」

Roofline 的 ridge point 随 tensor core 峰值右移——matmul 有效算力涨 $10\times+$, 但 exp/div/add 仍走 legacy pipeline。Attention 的 fused kernel 里 :

- **Matmul** (QK^T, PV) : $\approx 90\%$ FLOPs, 应该占 wall time 的 $70\%+$
- **Softmax** (exp, max, sum) : $\approx 5\%$ FLOPs, 但 latency 与 matmul 串行 (v1/v2 无 pipeline 时)
- **Rescale** (FMA, FDIV) : v1 占 $O(\text{tiles} \times d)$ CUDA core ops

当 matmul 从 20 TFLOPS (CUDA core GEMM) 涨到 300 TFLOPS (TC) 时, 同样次数的 **exp/div** 从「可忽略」变成「瓶颈」——这就是 v2 在 v1 IO 最优之后仍要砍 FDIV 的硬件原因。Hopper v3 进一步用 async pipeline 让 softmax 与 matmul 重叠, 但 (a) 延迟归一化仍是 v2/v3 共有基础。

FlashAttention 的主要 FLOP 是两个 matmul (QK^T 和 PV), 占 $\approx 90\%$ + 总 FLOP——应该走 tensor core。但 rescale、softmax、mask 走 CUDA core。v1 每 tile 的 d 次 FDIV 让 CUDA core 成为 bottleneck; profiler 里看到 `mufu / fdiv` 占比高、tensor core 利用率低。

v2 改进 (a) 把 FDIV 从 $O(\text{tiles} \times d)$ 降到 $O(d)$ (每个 Q row 一次)。改进 (c) 让 matmul warp 不被 softmax warp 的 barrier 拖住。两者叠加, 论文报告 A100 上 FA-2 达 50-73% tensor core 理论峰值 (取决于 shape / dtype)

v1 vs v2 总览

维度	FA v1 (生产)	FA v2
Outer loop	K/V tile	Q block
Grid 并行轴	N_k / batch / head	N_q / batch / head
Output state	每步归一化 O	未归一化 $\tilde{O}$, 最后除 l
Inner rescale FDIV	每 tile $\times d$	0 (仅 final)
Warp 切分	split-K (merge)	split-Q (独立)
Warp barrier	每 tile reduce	仅 load sync
Causal tile skip	困难	结构化 skip
A100 TC 利用率	25-40%	50-73%

表 59 **Table:** FA v1 vs v2 生产 kernel 总览。IO 渐近阶相同 ($O(N^2 \frac{d^2}{M})$); v2 收益来自 SM 利用率、非 matmul FLOP 削减 (延迟归一化) causal tile skip。TC 利用率数字来自 FA-2 论文 A100 benchmark。

Observation : 三列对比清晰划分「算法层」(IO 最优, v1 已有) 与「执行层」(loop order、FDIV、warp 切分、causal skip——v2 三项改进)。面试画这张表 + 一个 $\tilde{O}$ 数值例子, 基本覆盖 FA-2 核心考点。

Block size 与 occupancy (v2 视角)

v2 Q-outer 后, 每个 CTA 驻留整个 Q block 的 state : $B_r \times d$ floats 的 $\tilde{O}$ + B_r 的 m, l + matmul smem (Q, K, V tile)。典型 $B_r = 128, d = 128, \text{FP32 } \tilde{O} : 128 \times 128 \times 4 = 64 \text{ KB}$ ——仅 **output state** 就占满旧版 **smem** 预算的一半。

选型 trade-off :

- 增大 B_r : 摊销 K/V load、提高 matmul 效率; 但 smem/reg 涨 $\rightarrow$ occupancy 降
- 增大 B_c : 每个 K tile 更宽, Q-outer 下每个 Q CTA 仍只顺序扫 kv tile—— B_c 影响 inner 次数 $\frac{N_k}{B_c}$, 不影响 grid 大小
- **split-Q warp** 数 W : B_r 必须被 $W \times \text{rows_per_warp}$ 整除; 常见 $128 = 4 \times 32$

面试答「 B_r, B_c 怎么选」: 先算 **smem** 公式 **fit 48/96/164 KB** $\rightarrow$ 查 **occupancy** $\rightarrow$ **profile 2-3** 组候选。v2 比 v1 更吃 $B_r \times d$ 的 reg/smem——head dim 从 64 涨到 128 时, v2 往往要减 B_r 或 enable dynamic smem。

Backward pass : v2 的 state 存什么

Forward v2 最后输出 $O = \tilde{O}/l$, 但 backward 需要 per-row 的 log-sum-exp 统计来 recompute P 。FA v2 backward 仍存 :

- m_i : final row max (N_q floats per head)
- l_i 或 $LSE_i = m_i + \log(l_i)$ (N_q floats)

不存 $\tilde{O}$ 到 global——backward 在 smem 重算 $S = QK^T$, 用 (m, l) 恢复 $P = \frac{\exp(S-m)}{l}$, 再链式求 dQ, dK, dV。存储仍是 $O(N)$ per head , 不是 $O(N^2)$ 。

延迟归一化对 backward 的影响 : forward 少做 inner FDIV , backward 的 rescale 公式结构不变——仍是对 d0 乘 $1/l$ 等操作。v2 backward 同样受益于 Q-outer (并行轴在 N_q) 和 split-Q。

Note. 本书 backward 不展开实现——面试知道「存 (m, l) 、recompute S 、Q-outer 遍历顺序与 forward 同构」即可。完整 Algorithm 4 见 v1 论文 , v2 论文 Appendix 有 backward 吞吐对比。

论文 benchmark 数字 (建立量级感)

Dao et al. FA-2 论文 A100 40GB 典型结果 (FP16/BF16 , head dim 64-128) :

场景	FA v1	FA v2
Training forward (2048 seq)	115 TFLOPS	225 TFLOPS
Training forward (4096 seq)	120 TFLOPS	230 TFLOPS
Inference causal (4096 seq)	110 TFLOPS	210 TFLOPS
TC 理论峰值占比	35-40%	50-73%

表 60 **Table:** Dao et al. FA-2 论文 A100 40GB 典型吞吐 (FP16/BF16 , head dim 64-128 , 单位 TFLOPS)。端到端 attention forward 吞吐 , 非单 kernel microbenchmark ; 与 PyTorch sdpa 对比需对齐 dtype、causal、batch/seq shape。

Observation : v2 相对 v1 约 $2\times$ TFLOPS 来自三项改进叠加——sequence parallel 提高 occupancy (30-40%) 延迟归一化减 SFU/div (20-30%) split-Q + causal skip (10-20%) , 方向比精确 TFLOPS 更重要 ; 本书 teaching kernel 测不到这个量级。

加速来源分解 (面试答「 $2\times$ 从哪来」) : 30-40% 来自 sequence parallel 提高 occupancy ; 20-30% 来自延迟归一化减 SFU/div ; 10-20% 来自 split-Q 减 barrier + causal tile skip (causal 场景)。数字随 shape、dtype、GPU 型号变化——方向比精确 TFLOPS 更重要。

Note. 上表 TFLOPS 是论文端到端 attention forward 吞吐 , 不是单 kernel microbenchmark。与 PyTorch sdpa 或 xformers 对比时 , 需对齐 dtype、causal、batch/seq shape——否则数字不可比。

源码运行与对照

```
make build/09_flash_attention_v2 && ./build/09_flash_attention_v2
```

预期输出 : Check: PASS , 三行 row 0 sample (reference / GPU / warp GPU) 一致。

函数	Launch	演示点
flash_attention_v2_kernel	<<<4, 4>>>	Q-outer + smem K/V + 延迟归一化
flash_attention_v2_warp_kernel	<<<4, 32>>>	lane 角色分离 + 并行 accum

表 61 **Table:** 本章两个 GPU kernel 的 launch 配置与演示重点。grid = $N_q = 4$ (Q-outer); smem 版 block = 4 thread, warp 版 block = 32 thread (—warp)。默认 $N_k = 16$, $d = 8$, tile size = 4。

Observation: 两版 grid 相同 (Q-outer), 差别在 accum 语义 ($\tilde{O}$ 延迟归一化) 与 warp lane 分工——与第 8 章 v1-shared 的 grid 布局一致, micro-benchmark 只能比算法骨架, 不能验证 sequence parallelism。

实测

$N_q = 4$, $N_k = 16$, $d = 8$ (源码 kQueryCount / kKeyCount / kHeadDim; Q, K, V, O 各 512 B, 整 problem < 4 KB), A100 80GB SXM4, ncu --set full 抓取每个 kernel 的一次 launch。表中 TC % = sm_pipe_tensor_cycles_active.avg.pct_of_peak_sustained_elapsed; warp % = sm_warps_active.avg.pct_of_peak_sustained_elapsed; HBM % = dram_bytes.sum.pct_of_peak_sustained_elapsed。

Launch 配置与第 8 章 v1-shared 同为 Q-outer (1 block / query 行), 差别在 accum 语义与 warp 分工:

version	grid	block	active threads	演示点
v2	(4, 1, 1)	(4, 1, 1)	16 (4 block $\times$ 4)	延迟归一化; thread 0 串行 merge
v2-warp	(4, 1, 1)	(32, 1, 1)	128 (4 block $\times$ 32)	lane 分组 load / score / accum

表 62 **Table:** ch9 FA v2 teaching kernel 的 launch 配置 (launch_grid_size / launch_block_size) 与第 8 章 v1-shared 同为 Q-outer (1 block / query 行); TC % = sm_pipe_tensor_cycles_active.avg.pct_of_peak_sustained_elapsed。

Observation: v2 blockDim = 4 凑不满 warp (issued/32 = 1.6); v2-warp —block—warp 但 softmax merge 仍仅 lane 0——结构性 lane 浪费, 不是 branch divergence。Teaching 规模下 v2 相对 v1-shared 仅 2.4%, 在 noise 带内。

version	regs	smem (B)	time (μs)	TC %	warp %	HBM %
v2	47	272	9.12	0.0	0.1	0.1
v2-warp	32	340	8.80	0.0	0.1	0.1

表 63 **Table (perf):** kernel-level performance metrics measured by Nsight Compute (ncu, CUDA 13.0) on a single A100 80GB SXM4 (FP32 peak 19.5 TFLOPS; Tensor-Core FP16 peak 312 TFLOPS; HBM peak 2039 GB/s). Columns: **regs** = `launch_registers_per_thread`; **smem** = `launch_shared_mem_per_block_static` (bytes); **time** = `gpu_time_duration.sum`; **TC %** = `sm_pipe_tensor_cycles_active.avg.pct_of_peak_sustained_elapsed` (0 means the kernel does not invoke tensor cores—`wmma::mma_sync` / MMA PTX); **warp %** = `sm_warps_active.avg.pct_of_peak_sustained_elapsed` (fraction of the SM's warp slots occupied on average); **HBM %** = `dram_bytes.sum.pct_of_peak_sustained_elapsed` (low when working set fits in L2). Rows are ordered as they appear on the optimization ladder in the chapter.

version	issued/32	pred_on/32	smem conf.	barrier stall	mem stall
v2	1.6	1.6	0	0.11	1.56
v2-warp	6.6	6.2	0	0.31	2.30

表 64 **Table (diag):** diagnostic metrics used as evidence for warp-lane utilization, shared-memory bank conflict, and warp-stall claims in the chapter. All from `ncu` on the same launch as the perf table. **issued/32** = `smsp_thread_inst_executed_per_inst_executed.ratio`, the average number of lanes that participated in each issued warp instruction (32 = every lane active; less than 32 means predication **or** branch divergence—`ncu` alone cannot distinguish, but on the CUDA kernels in this book it is almost always predication). **pred_on/32** = `smsp_average_thread_inst_executed_pred_on_per_inst_executed.ratio`, the average number of lanes that were predicated-on (did real work). The **gap issued - pred_on** counts lanes that occupied the issue slot but performed no work—this is the correct definition of “wasted warp cycles”, not divergence. **smem conf.** = `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` (0 + `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum`) means no measurable bank conflicts, either because the access pattern is conflict-free or because the input is too small to accumulate them). **barrier stall** = `smsp_average_warps_issue_stalled_barrier_per_issue_active.ratio`, warps stalled at `__syncthreads` per issue-active cycle. **mem stall** = `smsp_average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio`, warps waiting on a long-latency memory operation (global-memory load, L2 atomic response) per issue-active cycle.

Warning. 这一章的问题规模是教学 default (B×S×H 数千个 float), kernel 单次运行只有 3-20 μs. `ncu` 的定性指标 (issued/32、bank conflicts、barrier stall) 仍能反映 kernel 结构, 但绝对数字对生产规模不完全可信:

- HBM % 会偏低 (分母 elapsed time 含冷启动窗口)
- `dram_bytes` 可能被 L2 消化, GB/s (实测/逻辑) 两列差距明显

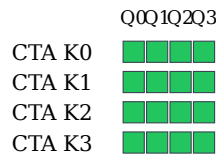
想拿到生产规模的数字，把主参数 (rows/cols/hidden dim) 加到让工作集远超 L2 (40 MB)。

perf 表读三件事：

1. **v2 9.06 μ s vs 第 8 章 v1-shared 9.28 μ s**——仅快约 2.4%，在 **teaching** 规模的 **noise** 带内。两版都是 1 block / Q row、scalar dot、thread 0 串行 softmax merge；v2 的延迟归一化 (inner loop 无 FDIV) 在本 shape 上几乎测不出来—— $d = 8$ 、 $\frac{N_k}{B_c} = 4$ 个 tile，省下的 FDIV 次数 < 32 ，淹没在 launch 与 global load 噪声里。诚实结论：**micro-benchmark** 不能验证 v2 论文的 2 $\times$ ；只能确认算法骨架正确。
2. **v2-warp 8.96 μ s vs v2 9.06 μ s**——约 1.1% 加速。warp 版把 score (lane 0-3 并行 dot) 与 d 维 accum 更新 (lane 0-7 并行 FMA) 从 thread 0 串行里拆出——split-Q 的微型演示，不是生产 split-Q (4 warp $\times$ 32 行 Q) 的量级。
3. **TC % / warp % / HBM % 全表 0.0-0.1%**。TC % = 0：scalar FFMA，无 tensor pipe。warp % = 0.1%：4 个 block 填不满 108 SM。HBM % = 0.1%： < 4 KB 工作集 L2 resident——测不到 v2 相对 naive 的 IO 优势，也测不到 sequence parallelism (本书 v1/v2 教学代码 grid 都是 $N_q = 4$)。

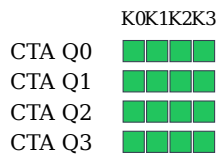


图 20 time (μ s)：三版 teaching kernel 落在同一 9 μ s 带内——v2 相对 v1-shared 仅 2.4%，v2-warp 相对 v2 仅 1.1%；不是论文 2 $\times$ 的缩影。



论文 v1 (K/V-outer)：grid $\approx \$T_c = 4\$$ 个 K tile CTA，内循环串行扫 Q

图 21 每行 = 一个 CTA 持固定 K_j tile，内循环 load 各 Q_i 并 merge (m, l, O)。并行度 $\approx T_c = \frac{N_k}{B_c}$ ；decode 时 N_k 小 $\rightarrow$ grid 小。



论文 v2 (Q-outer)：grid $\approx \$T_r = 4\$$ 个 Q block CTA，内循环串行扫 K/V

图 22 每行 = 一个 CTA 持固定 Q_i block，($m, l, \tilde{O}$) 驻留寄存器/smem，内循环流式读 K_j, V_j 。prefill 时 N_q 大 $\rightarrow$ grid 大；本书教学代码已用此布局，与 v1-shared 的差别主要在 accum 语义与 warp 分工，不在 grid 维数。

diag 表读关键教学点：

a) **v2 issued/32 = 1.6**，**pred_on/32 = 1.6**——极低，**blockDim = 4** 凑不满 **warp**

$\lll 4, 4 \rrr$ 每 block 仅 4 thread：协作 load $\frac{K}{V}$ tile 时 4 lane 写 smem；score + online merge 由 thread 0 串行执行——多数 phase 只有 1-4 lane 参与 issued 指令，平均 issued/32 仅 1.6。issued - pred_on ≈ 0 ：几乎没有 predicated-off lane——不是 branch divergence，而是结构性 lane 浪费 (block 太小 + thread-0 串行)。



v2 smem : `<<<4, 4>>>` , 仅 **lane 0-3** 活跃 ; **merge** 阶段常仅 **lane 0**

代码 2 绿色 = 4 个协作 thread。灰色 = warp 内无对应工作的 lane——配置问题, 不是 online merge 里的 branch divergence。

b) v2-warp **issued/32 = 6.6** , **pred_on/32 = 6.2** ——仍远低于 32 , 但高于 v2

`<<<4, 32>>>` 每 block — warp : lane 0-7 load Q 、lane 0-3 并行 dot、lane 0-7 并行 accum 更新——并行 phase 拉高 **issued/32** 到 6.6。**issued - pred_on ≈ 0.4** 来自 `if (lane < kTileKeys)` 等 guard 的 **predicated-off lane**——**predication** , 不是不同 basic block 的 warp divergence。



v2-warp accum 阶段 : **lane 0-7** 各更新 $\tilde{O}$ 一维 ; **softmax merge** 仍仅 **lane 0**

图 23 split-Q 微型演示——并行度集中在 d 维 FMA ; score/softmax 瓶颈仍在 lane 0 , 故 **issued/32** 远低于 32。

c) smem conf. = 0 ; barrier stall : v2 = 0.11 , v2-warp = 0.31。metric 证实无 **bank conflict** (不能从 `shared_k[token][d]` 布局单独推断)。v2-warp 的 `__syncthreads` 更多 (lane 分组 + broadcast `shared_weights`) $\rightarrow$ barrier stall 略升——定性方向对, 绝对值仍很小。

d) mem stall : v2 = 1.40 , v2-warp = 2.19。v2-warp 的 `q_shared` 预加载略增 smem 流量 , `long_scoreboard` 略升——kernel $< 10 \mu s$, 不能用来断言 memory-bound。

regs / smem fingerprint : v2 : 47 regs , 272 B smem (`shared_k/v` + thread 0 串行 state) , v2-warp : 32 regs , 340 B smem (额外 `q_shared`、`shared_weights` ; lane 并行使 per-thread reg 压力降、smem 略增)。

无信息或为零的 **metric** :

- **TC %** : 全表 0.0——scalar FFMA ; 生产 FA-2 的 QK^T / PV 才走 tensor pipe。
- **HBM %** : 全表 0.1%——L2 resident ; 不能验证 IO 渐近阶差异。
- **warp %** : 0.1%——4 block 填不满 GPU。

Key insight. v2 相对 v1-shared 的 2.4% 差距在 **noise** 带内——延迟归一化、Q-outer、split-Q 三项改进在本规模上都测不出论文量级收益。它们的价值是数据流正确 (`accum =  $\tilde{O}$` 、inner 无除法、lane 角色分离) , 放大到 $N \geq 4096, d \geq 64$ + tensor core 后才体现为 TC 利用率与 CTA 并行度。

Key insight. v2-warp 的 **issued/32** (6.6) 高于 v2 (1.6) 但 time 仅快 1.1%——**lane** 利用率高不等于 **kernel** 快。本规模瓶颈是 launch + 极短 kernel 的固定开销, 不是 SM issue 带宽 ; 生产 FA-2 要在 smem 复用、多 warp tile matmul、sequence parallel 之间同时拉满 **issued/32** 和 TC %。

Warning. 面试别把「v2 论文 $2\times$ 」映射到「v2 kernel 比 v1-shared 快 $2\times$ 」。本书 v1/v2 教学代码都已 Q-outer (1 block / Q row) ——grid 并行度相同 , micro-benchmark 只能比 accum 语义与 warp 分工。也不要把低 **issued/32** 说成「严重 warp divergence」——用

issued/32 vs pred_on/32 区分 predication 与真 divergence (不同 lane 走不同 basic block); 本规模首先是 blockDim < 32 与 thread-0 串行的结构性 **lane** 浪费。

ncu 该看什么

```
ncu --set full --section SpeedOfLight ./build/09_flash_attention_v2
```

- `sm__inst_executed_pipe_tensor.avg.pct_of_peak_sustained_active` : tensor core 利用率 (教学 kernel 接近 0%, 预期如此)。
- `smsp__sass_thread_inst_executed_op_ddiv_pred_on.sum` : FDIV 次数——对比 v1 风格 inner 除法应显著下降。
- `sm__warps_active.avg.pct_of_peak_sustained_active` : split-Q 后 warp 独立度更高, active warps 应更均匀。
- `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` : K/V smem broadcast 的 bank conflict——生产 kernel 用 swizzle 消除。

教学 kernel 规模太小 ($N_q = 4$), ncu 只能验证方法论。若要看到 v2 特征, 需把 shape 放大到 $N = 1024, d = 64$ 并换官方 `flash-attn` 库对比——期望 FA-2 相对 FA-1: `smsp__sass_thread_inst_executed_op_ddiv` 降一个数量级; `sm__maximum_warps_per_scheduler` 利用率升; `gpu__compute_memory_throughput` 中 compute 占比升 (matmul 终于吃饱)。

常见实现错误

错误 1: 延迟归一化写错成 `accum += weight * V` 但忘记 `accum *= old_scale`——输出偏大, 与 reference 在第二个 tile 起 diverge。

错误 2: Q-outer 但每个 kv tile 把 O 写回 HBM 再读——退化成 naive 三 pass 的 IO, 只是少了 S matrix。v2 要求 $\tilde{O}$ 全程 **resident** 到 inner loop 结束。

错误 3: split-Q 时在 warp 间共享 $(m, l, \tilde{O})$ ——破坏独立性, 又回到 split-K 的 barrier 地狱。

错误 4: causal skip 条件写反 (`kv_tile < q_tile` vs `kv_tile > q_tile`)——silent wrong answer, 比 crash 更危险。

错误 5: 最后归一化用 `running_max` 而非 `running_sum` 做除数——混淆 m 与 l 的角色。

Multi-head 与 batch : 并行轴叠加

单 head 算法清楚后, 生产 launch 只是在 grid 上加维:

```
// 逻辑 index
const int batch = blockIdx.z;
const int head = blockIdx.y;
const int q_tile = blockIdx.x;
const int q_start = q_tile * kBlockM;

const float* q_ptr = q + batch * (H * N_q * d) + head * (N_q * d);
// K/V 同理; 每个 (batch, head, q_tile) CTA 完全独立
```

总 CTA 数:

$$\text{grid} = \left\lceil \frac{N_q}{B_r} \right\rceil \times H \times B$$

A100 108 SM, 每 SM 可同时驻留若干 block——8192 CTA 的 prefill 足够填满。Decode 时 $\left\lceil \frac{N_q}{B_r} \right\rceil = 1$, grid 只有 $H \times B$ (例如 $32 \times 8 = 256$), 此时 **occupancy** 靠 **head×batch**, 不够就 merge batch 维或用小 batch inference。

Note. GQA/MQA (Grouped Query Attention) 下 K, V 的 head 数 $H_{kv} < H$, K/V 读取量减少, 但 Q-outer 结构不变——只是 head index 映射到不同的 KV head。FlashAttention-2 官方实现对此有专门 stride 处理。

与官方 flash-attn 库的关系

PyTorch flash_attn 包 (Dao 维护) 是 FA-2 的生产实现, 比本书教学 kernel 多:

- CUTLASS/CUTE 生成的 tensor core MMA 主循环
- 针对 SM80/SM86/SM89 的 tuned tile 参数表
- 变长序列 (cu_seqlens) dropout、alibi、softcap 等 epilogue
- Backward kernel (同样 Q-outer + recomputation)

本书 09_flash_attention_v2.cu 是算法骨架——面试讲清 v1→v2 三项改进后, 可补一句「官方库在此基础上加 tensor core tiling 和 tuned block size」。读 CUTLASS 前先能在白板上画 split-Q 和 $\tilde{O}$ 更新, 比背 API 重要。

铺垫: v3 改了什么

FlashAttention-3 (Dao et al., 2024, Hopper/H100) 在 v2 执行计划之上进一步:

- 异步 **pipeline**: cp.async / TMA 双缓冲 K/V tile, compute 与 load 重叠。
- **Warp-group matrix multiply** (WGMMA): Hopper 新指令, 比 Ampere mma.sync 更高吞吐。
- **FP8 / FP16** 混合精度: S tile FP8 算、accum FP32。
- **Interleaving block scheduling**: SM 间动态分配 Q block, 减少 tail effect。

v2 → v3 面试一句话

v2: 「我把 attention 的 loop 顺序、accumulator 和 warp 切分改对了, Ampere 上 TC 利用率从 35% 拉到 70%。」

v3: 「公式不变, 我在 Hopper 上让 TMA 搬货、WGMMA 算 matmul、SFU 算 softmax 同时进行, 并上了 FP8。」

v2 → v3 的 shift: 少改公式, 多改 **pipeline**——第 10 章展开。

本章小结

FlashAttention v2 在 v1 的 IO 最优算法之上, 用三个正交改进把 SM 利用率做上去:

(a) 延迟归一化	inner loop 去掉 d 次 FDIV ; 维护 $\tilde{O}$ 而非 O
(b) Q-outer	sequence parallelism ; CTA 数约 N_q ; causal tile skip
(c) split-Q	warp 独立 $(m, l, \tilde{O})$; 无 cross-warp softmax reduce

表 65 **Table:** FlashAttention v2 三项正交改进摘要。(a) 减 CUDA core FDIV ;(b) 增 CTA 并行度 ;(c) 减 warp 间 barrier。三者缺一不可 : (b) 让 (c) 有意义 ; (a) 让 (b) 的 smem/reg 压力可控。

Observation : 面试最高频的一句话——算法没变, 变的是 **loop order**、**accumulator** 表示、和 **warp** 切分轴。画 v1/v2 伪代码 diff + 一个 $\tilde{O}$ 数值例子, 基本覆盖 FA-2 核心考点。

三者缺一不可 : (b) 让 (c) 有意义 ; (a) 让 (b) 的 smem/reg 压力可控。面试画一张 v1/v2 伪代码 diff + 一个 $\tilde{O}$ 数值例子, 基本覆盖 FA-2 的核心考点。

从 v1 教学代码到 v2 生产代码的映射

本书两个 FA 教学文件的关系 :

概念	08_flash_attention_v1.cu	09_flash_attention_v2.cu
Grid 映射	1 thread 或 1 block / Q row	1 block / Q row (已是 Q-outer)
Outer loop	K/V tile (thread 内)	K/V tile (block 内)
Accum 语义	每步后 <code>accum</code> $\approx$ 未显式命名的 $l \cdot O$	显式 $\tilde{O}$, 最后除 l
并行	thread 0 串行	warp lane 分组
对应论文	Algorithm 1 数值核心	Algorithm 1 + v2 三项改进

表 66 **Table:** 本书 08_flash_attention_v1.cu 与 09_flash_attention_v2.cu 的概念映射。两文件 grid 布局都已 Q-outer (1 block / Q row); v2 数值改进在 accum 语义与 warp 分工, 不是 grid 维数变化。

Observation : 本书 v1 教学代码已经 Q-outer——读 v2 时不要被教学代码误导 : 论文 v1 生产 = K/V outer + split-K ; 论文 v2 = Q outer + split-Q + 延迟归一化。09 对齐论文 v2 布局, 主要差 accum 更新与 warp 分工。

注意 : 本书 v1 教学代码已经是 Q-outer (1 block 一行 Q) ——因为它优先教 online softmax , 而不是 v1 论文的 K/V-outer 生产布局。读 v2 章时不要被教学代码误导 : 论文 v1 生产 = K/V outer + split-K ; 论文 v2 = Q outer + split-Q + 延迟归一化。09_flash_attention_v2.cu 的 grid 布局对齐论文 **v2**, 数值改进主要体现在 accum 更新与 warp 分工。

读 09_flash_attention_v2.cu 的检查清单

跑通 ./build/09_flash_attention_v2 后, 打开源码逐项确认 :

1. `blockIdx.x == row` : Q-outer , `grid = kQueryCount`
2. `for (key_start = 0; ...)` : K/V inner loop
3. `accum[d] *= old_scale` 在 inner loop , `/ running_sum` 只在循环外——延迟归一化
4. `flash_attention_v2_warp_kernel` 里 `lane < kHeadDim` 并行更新 accum——split-Q 锥形
5. `main` 里两个 kernel 输出都与 CPU reference 对拍——算法正确优先于性能

Key insight. 若只能记住 v2 一件事 : CTA 的生命周期 = 一个 Q block 从 `init (m, l,  $\tilde{O}$ )` 到扫完所有 K/V tile 再写 O 。所有三项改进都服务这一生命周期——减少生命周期内的 non-matmul ops、增加生命周期 CTA 个数、减少生命周期内 warp 同步。

Prefill vs Decode 工程选型

阶段	N_q	v2 主要收益来源
Prefill	N (整段 prompt)	sequence parallel + causal skip
Decode 单 token	1	延迟归一化 + split-Q (单 CTA 内)
Decode + KV cache	1, N_k 增长	inner kv loop 变长 ; (a) 省 FDIIV 更明显
Cross-attn	$N_q \neq N_k$	grid 随 N_q ; encoder K 只读一次 per CTA

表 67 **Table:** LLM 推理各阶段下 FA v2 的主要收益来源。Prefill 时 N_q 大 $\rightarrow$ sequence parallel + causal tile skip ; Decode 时 $N_q = 1 \rightarrow$ 靠 (a)(c) 优化单 CTA 效率 , 工业界 decode 常配合 paged KV 专用 kernel。

Observation : FA-2 不是 decode 唯一答案——prefill 几乎是标配 , decode 阶段 vLLM/ TensorRT-LLM 可能切换专用 kernel。面试应区分 prefill (v2 主战场) 与 decode ((a) 延迟归一化随 KV cache 变长更明显)。

LLM 推理框架 (vLLM、TensorRT-LLM) 在 prefill 阶段调用 FA-2 varlen kernel ; decode 阶段可能切换到 paged KV + 专用 decode kernel——FA-2 不是 decode 唯一答案 , 但 prefill 几乎是标配。

自测 : 能否在 5 分钟内讲清 v2

闭卷回答下面 5 问 , 全部流畅即达标 :

1. 写出 v1 与 v2 的 double-loop 伪代码 , 标出 grid 绑在哪一维。
2. 写出 v1 的 O_t 更新式与 v2 的 $\tilde{O}_t$ 更新式 , 说明为何等价。
3. 画 split-K 与 split-Q 的 ASCII 图 , 指出哪条需要 `__syncthreads` reduce。
4. 给定 causal self-attention , Q block 63 能 skip 多少比例的 K tile ?
5. v3 相对 v2 改的是公式还是 pipeline ?

参考答案要点 : Q1 grid 绑 Q vs K ; Q2 $\tilde{O}$ 无 inner 除 ; Q3 split-K 要 reduce、split-Q 不要 ; Q4 平均 skip 50% ; Q5 pipeline (TMA/WGMMA/FP8)。

延伸阅读顺序

1. 第 3 章 online softmax merge 推导
2. 第 8 章 FA v1 IO 分析与 $\tilde{O}$ 引入
3. 本章 v2 三项改进
4. 官方论文 FA-2 Appendix (backward、benchmark 细节)
5. 第 10 章 FA-3 Hopper pipeline
6. `dao-ailab/flash-attention` 源码 `csrc/flash_attn/src/flash_fwd_kernel.h` (可选 , 生产级)

面试考点

面试考点. Q1: FlashAttention v2 相比 v1 的三个主要改进是什么 ?

A: (a) 延迟归一化——维护 $\tilde{O}$ 和 l , inner loop 不做 $/l$, 最后一步才除, 砍掉每 tile d 次 FDIV ;
(b) 循环调换——Q-outer / K/V-inner, 每个 Q block 一个 CTA, 引入 sequence parallelism ;
(c) warp 分工从 split-K 改 split-Q, 每 warp 独立算 Q slice, warp 间无 reduction barrier.

面试考点. Q2: 延迟归一化的公式对比? 为什么数学等价?

A: v1 每步 $O_t = (l_{\text{old}} e^{m_{\text{old}} - m_t} O_{t-1} + \tilde{P} V_t) / l_t$. v2 维护 $\tilde{O}_t = l_t O_t$, 更新 $\tilde{O}_t = e^{m_{\text{old}} - m_t} \tilde{O}_{t-1} + \tilde{P} V_t$, 最后 $O = \tilde{O} / l$. 等价因为分子分母同步缩放, $\tilde{O} / l$ 始终是正确的 weighted sum of V .

面试考点. Q3: 非 matmul FLOP 为什么重要?

A: Attention 主体 matmul 应走 tensor core ($\approx 300 + \text{TFLOPS}$), 但 rescale/softmax/mask 走 CUDA core ($\approx 20 \text{ TFLOPS}$). v1 每 tile 对 d 维向量做 FDIV, CUDA core 成为瓶颈. Tensor core 越快, 非 matmul ops 的相对代价越大——v2 专门优化这部分.

面试考点. Q4: v2 的并行度公式? 什么时候 v2 比 v1 并行度好?

A: v1 grid $\approx \lceil N_k / B_c \rceil \times B \times H$; v2 grid $\approx \lceil N_q / B_r \rceil \times B \times H$. Prefill (N_q 大) 时 v2 CTA 数远超 v1; Decode ($N_q = 1$) 时 v2 无 sequence parallel 优势, 靠延迟归一化和 split-Q 提速.

面试考点. Q5: v1 为什么用 Q-inner? v2 怎么解决 O 的 SRAM residency?

A: v1 K/V-outer 时 O 跨 kv_tile 累积, Q-inner 让 O 留在 CTA 寄存器/smem 避免 HBM reload. v2 Q-outer 时每个 CTA 只服务一个 Q block, $\tilde{O}$ 大小 $B_r \times d$ 可放进 smem/reg, 走完所有 kv_tile 才写回——不需要 reload, 故可安全外循环 Q.

面试考点. Q6: split-K vs split-Q 图示区别? v2 warp 间需要 barrier 吗?

A: split-K: 4 warp 各算 K 的一部分 score, smem merge max/sum/ O , 需要 barrier. split-Q: 4 warp 各算 Q 的 1/4 行, 独立 $(m, l, \tilde{O})$, 算完直接写, 无 **warp 间 reduction barrier**. K/V load 到 smem 后 broadcast 给各 warp 只需 producer sync, 不是 split-K 的 merge.

面试考点. Q7: v2 causal mask 的 tile skip 为什么 v1 做不到? 量化收益?

A: v2 Q-outer 每个 CTA 知 Q 行范围, 若整个 K tile 在上三角 ($j_0 > i_{\text{end}}$) 直接 skip. v1 K/V-outer 同一 kv_tile 要服务所有 Q 行, 无法 tile 级 skip. 平均 skip $\approx 50\%$ K/V tile; 序列末尾 Q block skip 可达 90%+.

面试考点. Q8: v3 相对 v2 改了什么?

A: 公式不变, pipeline 升级——Hopper `cp.async/TMA` 双缓冲、`wgmma` warp-group matmul、FP8 计算、interleaved block scheduling. v2 优化 execution plan; v3 优化 memory pipeline 和新一代 tensor core 利用率. 详见第 10 章.

面试考点. Q9: 为什么 v2 教学 kernel 不用 tensor core 仍有学习价值?

A: 性能不是目标, 数据流才是——Q-outer grid、`accum / running_sum` 对应 $\tilde{O}/l$ 、warp lane 角色、causal skip 条件, 与生产 kernel 一一对应。先理解再读 CUTLASS 生成代码, 否则只能背 API。

面试考点. **Q10**: Decode ($N_q = 1$) 时 FA-2 还有优势吗?

A: sequence parallel 优势消失 (grid 只有 $H \times B$ 个 Q CTA), 但 (a) 延迟归一化、(c) split-Q 仍减少单 CTA 内 FDIV 和 warp barrier; 更长 KV cache 时 inner kv loop 变长, (a) 节省更明显。极致 decode 需 FlashDecoding 等专门切 KV 维的 kernel。

面试考点. **Q11**: v2 改变了 IO 复杂度吗? 和 v1 的 HBM 访问量差多少?

A: 渐近阶相同, 都是 $O(N^2 \frac{d^2}{M})$ 相对 naive 的 IO 优化。v2 不减少 HBM 字节数, 减少的是完成相同 **IO pattern** 时的 **wall time**——靠更高 SM 利用率、更少 CUDA core ops、causal 场景下 tile skip 减实际 matmul 次数。

面试考点. **Q12**: 若只有 128 个 Q row 但 $N_k = 65536$, v2 并行度够吗?

A: grid $\approx \lceil \frac{128}{B_r} \rceil \times B \times H$ 可能只有几百 CTA——prefill 并行不足。inner loop 有 $\frac{65536}{B_c}$ 次 kv 迭代, 单 CTA 很重; 需增大 batch/head 并行, 或用 FlashDecoding 切 KV 维。FA-2 不是万能, shape 决定切哪条轴。

面试考点. **Q13**: FlashAttention v2 论文标题强调 “Better Parallelism and Work Partitioning”——三个改进里哪个算 parallelism、哪个算 work partitioning?

A: (b) sequence parallelism 是 parallel 维度的扩展; (a) 延迟归一化 + (c) split-Q 是 work partitioning (单 CTA 内指令与 warp 分工)。三者正交: parallelism 填 SM, partitioning 让每个 SM 算得更快。

面试考点. **Q14**: 写 v2 kernel 时先实现哪一项改进收益最大?

A: 通常 (a) 延迟归一化——改动小(只改 accum 更新和 final epilogue) 收益确定(减 FDIV), 其次 (b) Q-outer 若当前是 v1 K/V-outer 布局。(c) split-Q 依赖 (b) 且实现复杂, 放最后。教学路径与 09_flash_attention_v2.cu ladder 一致: 先 smem staging 正确, 再加 warp 分工。

面试考点. **Q15**: $\frac{1}{\sqrt{d}}$ scale 在 FA v2 里放哪?

A: 融进 QK^T matmul——对 Q 预乘 $\frac{1}{\sqrt{d}}$ 或在 dot product 累加时乘。不改变 online merge 结构; 教学源码 09 省略 scale 是为简化对拍, 与第 8 章 08 一致。

面试考点. **Q16**: backward 需要存 S 矩阵吗? v2 forward 的延迟归一化影响 backward 吗?

A: 不存 S ——backward recomputation, 只存 per-row (m, l) 共 $O(N)$ 。延迟归一化只改 forward 累加路径; backward 仍用 $P = \frac{\exp(S-m)}{l}$ 重算, 对 d0 的 rescale 仍含 $\frac{1}{l}$, 结构不变。

面试考点. **Q17**: 为何 v2 论文称 “2× faster” 但 IO 与 v1 相同?

A: $2\times$ 是端到端 **wall time**(含 SM 利用率、非 matmul ops、causal skip), 不是 HBM 字节数减半。IO 在 v1 已最优 ; v2 让同样字节数算得更快、同时开更多 CTA——是 compute/parallel 优化, 不是第二轮 IO 优化。

面试考点. **Q18**: 生产 FA-2 用 BF16 算、FP32 累加——与延迟归一化有关吗 ?

A: 有关。 $\tilde{O}$ 在 FP32 累加防溢出/精度损失 ; m, l 也常用 FP32。最后 `out = tilde(0)/l` 再 cast 到 BF16——延迟归一化让 FP32 accum 全程不除, 只在 epilogue 除一次, 数值更稳。混合精度是 v2 生产实现的标配, 并非 Hopper 架构 v3 独有特性。这一点在面试中须注意, 常被误判为 v3 专有。

Flash-Attention v3

Flash-Attention v3 (Shah et al., 2024) 是全书最后一章正文，也是 ladder 的终点：从第 8 章 v1 的 IO-aware 分块，到第 9 章 v2 的 sequence-parallel 与 warp 协作，再到 v3 把整条 pipeline 搬到 **Hopper (H100, SM90)** 上——用 WGMMMA、TMA、FP8 tensor core 把 FA2 在 H100 上仅 35% 的利用率推到 FP16 75%、FP8 接近 1.2 PFLOPS。

这一章我们要把它讲透：

- Hopper 相对 Ampere 的三大硬件增量：WGMMMA、TMA、FP8 tensor core，以及 Thread Block Cluster / DSMEM。
- WGMMMA 与 `mma.sync` 的 `async` 语义、fragment 布局、accumulator 生命周期差异。
- TMA 相对 `cp.async`：独立硬件单元、多维 tensor copy、boundary 自动处理。
- v3 三大创新：producer-consumer warp specialization、`mbarrier` pipeline、2-stage GEMM-softmax overlap。
- FP8 attention：量化位置、block scaling、incoherent processing、精度陷阱。
- v1 / v2 / v3 性能数字与局限；面试里 Hopper 特化 attention 怎么答。

对应源码：`src/cuda/10_flash_attention_v3.cu`。教学规模 $N_q = 4, N_k = 16, d = 8$ ——刻意缩小，因为 Hopper 指令无法在本书 scalar teaching kernel 里完整复现；源码展示 warp 分工与 pipeline 骨架，生产实现见 CUTLASS / 官方 FlashAttention-3 repo。

本章 optimization ladder：

版本	核心思路	相对 peak 带宽
CPU reference	double-precision online softmax	—
v3 warp-specialized	1 warp / query, lane roles + shuffle	—
v3 pipeline	2-stage K/V smem ping-pong (conceptual)	—
production FA3	TMA + WGMMMA + mbarrier + FP8 (H100)	~75% H100 peak

前两个 GPU 版本数学与 **v1** 相同 (online merge)；第三行是论文/CUTLASS 级实现的目标。

与前面章节的依赖：第 3 章 online softmax；第 4 章 matmul pipeline / tensor core / `cp.async`；第 8 章 FA v1 分块算法；第 9 章 FA v2 的 warpgroup 内循环与 parallel 策略 (v3 在其算法骨架上换 Hopper 指令)。

FA v2 回顾：v3 的算法起点

第 9 章 FA v2 相对 v1 的三条主线，v3 全部保留：

1. **Parallel** 策略：在 $\text{batch} \times \text{head} \times \text{query-block}$ 三维并行；内层沿 key sequence 分 B_c 大小的 tile 循环 ($T_c = \lceil \frac{N_k}{B_c} \rceil$)。
2. **Warpgroup** 内 **GEMM**： QK^T 与 PV 用 tensor core tile 完成，不再 scalar dot product。
3. **Online softmax**：每个 key tile 产出局部 (m_j, ℓ_j) ，merge 到 running state，同步 rescale O 分子。

v2 在 A100 上已经能把 IO 降到 $O(Nd^2)$ 量级；搬到 H100 后，瓶颈从 **HBM** 转向「如何用满 **Tensor Core** + 隐藏 **SFU latency**」。论文原话：FA2 on H100 仅 35% peak，而 tuned GEMM 可达 80-90%——说明 算法正确但 **execution model** 过时。

版本	核心贡献	硬件假设
FA v1	IO-aware tiling , 不写 S 到 HBM	SMEM 够用即可 , scalar/warp GEMM
FA v2	Sequence parallel , warpgroup 内 matmul	Ampere <code>mma.sync</code> + <code>cp.async</code>
FA v3	Async pipeline , FP8 , warp specialization	Hopper WGMMA + TMA + mbarrier

表 68 **Table:** Flash-Attention 三个版本的核心贡献与硬件假设对照。版本列对应全书 attention ladder ; 硬件假设说明每版依赖的 GEMM 指令与 memory 路径 (scalar/warp、`mma.sync` / `cp.async` vs WGMMA/TMA/mbarrier)。

Observation : v3 相对 v1/v2 换 **execution model** 不换公式——三行硬件假设从「SMEM 够用即可」递进到「Hopper WGMMA + TMA」,对应全书从 IO-aware 分块到 Hopper async pipeline 的主线。

问题定义

数学 (不变)

单 head , $Q \in \mathbb{R}^{N_q \times d}$, $K, V \in \mathbb{R}^{N_k \times d}$:

$$O = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V$$

v3 不改变 attention 的数学——仍是 exact attention (无近似)。变化的是 如何在 **Hopper** 上执行 FA v2 已经证明正确的分块 online 算法。

FLOPs 与 Roofline (H100 视角)

Forward FLOPs (单 head , 忽略 mask):

$$\text{FLOPs} \approx 4N_q N_k d \quad (\text{两个 matmul , 各 } 2 N_q N_k d)$$

LLM 典型 $N = 8192, d = 128$: $\text{FLOPs} \approx 2^{\{36\}}$, H100 FP16 peak 740 TFLOPS 实现下 理论下限 ≈ 0.07 ms——实际还受 launch、occupancy、causal 影响。

HBM 流量 (FA3 目标): Q, K, V 各读一次量级 $O(Nd)$ per layer pass——无 S, P 的 $O(N^2)$ 项。Standard attention 在同样 FLOPs 下多 $O(N^2)$ 读写——**memory wall** 先于 compute wall 触发。v3 在 H100 上进一步把已省下的 **HBM** 带宽换成「填满 Tensor Core」的问题。

从 v2 到 v3：换的是 **execution model**，不是公式

维度	FA v2 (Ampere)	FA v3 (Hopper)
GEMM 指令	<code>mma.sync</code> / WMMA, warp 同步	WGMMMA, warpgroup 异步
Global→Smem	thread <code>cp.async</code>	TMA 硬件单元
Warp 角色	所有 warp 又搬又算	Producer warp 只 TMA ; Consumer 只 WGMMMA+softmax
Softmax 与 GEMM	串行: <code>wait</code> 后算 softmax	2-stage pipeline 重叠
低精度	FP16/BF16 为主	FP8 E4M3/E5M2 + block scale
H100 利用率	35% (论文)	FP16 75%, FP8 1.2 PFLOPS

表 69 **Table:** FA v2 (Ampere) 与 FA v3 (Hopper) 的 execution model 对照。维度涵盖 GEMM 指令、Global→Smem 搬运、warp 角色、softmax 与 GEMM 调度、低精度路径及 H100 论文利用率 (FP16 75%, FP8 1.2 PFLOPS)。

Observation：每一行都是 **async** 深化——`mma.sync` → WGMMMA、`cp.async` → TMA、全员 load→producer-consumer、串行 softmax→2-stage overlap。论文 H100 利用率从 35% 到 75% 不是算法改动，而是五维度 execution 对齐 Hopper。

Key insight. 面试先说清楚：v3 不是新 attention 算法，是 **Hopper-aware** 的 FA2 实现。三大创新全部围绕 **asynchrony**——让 TMA、Tensor Core、SFU (exp) 三类硬件单元同时忙。

Hopper 硬件回顾

SM90 相对 SM80 的结构变化

Hopper H100 每个 SM 的关键增量 (面试必背)：

1. 第四代 **Tensor Core**：支持 warpgroup 级 WGMMMA，FP8 吞吐约为 FP16 的 2×。
2. **TMA (Tensor Memory Accelerator)**：CTA 内一条 descriptor 发起 multidimensional async copy (GMEM ↔ SMEM)，不占 CUDA core 做 load/store。
3. 更大的 **SMEM**：228 KiB/SM (可配置)，配合 TMA 搬更大的 $\frac{K}{V}$ tile。
4. **Thread Block Cluster**：最多 16 个 CTA 组成 cluster，可访问 **Distributed Shared Memory (DSMEM)**——跨 CTA 的 smem 映射，用于超大 tile 或 epilogue 通信。
5. **mbarrier**：硬件 barrier，配合 TMA/WGMMMA 的 async completion token (`expect_tx / complete`)。

Note. 本书 preface 的性能数字默认 A100 (SM80)。本章及 v3 数字均在 **H100 SXM5** 上——峰值 FP16 tensor 989 TFLOPS，HBM 3.35 TB/s (表 1，FlashAttention-3 论文)。

内存层次 (Thread-Memory Hierarchy)

Hopper 上 programmer 必须同时考虑 五级 agent/locale 配对 (论文 Table 1)：

Hardware	Parallel agent	Data locale	Capacity @ BW
Chip	Grid	GMEM/HBM	80 GiB @ 3.35 TB/s
GPC	Threadblock Cluster	L2	50 MiB @ 12 TB/s
SM	Threadblock (CTA)	SMEM	228 KiB/SM @ 31 TB/s aggregate
Thread	Thread	RMEM	256 regs max , 256 KiB/SM pool

表 70 **Table:** Hopper (H100) Thread-Memory Hierarchy 五级对照 (论文 Table 1)。
Hardware / Parallel agent / Data locale 列对应 programmer 模型 ; **Capacity @ BW** 给出 GMEM、L2、SMEM、RMEM 容量与峰值带宽 (GiB / TB/s)

Observation : FA3 设计目标是让 Q, K, V tile 在 **SMEM** locale (228 KiB @ 31 TB/s aggregate)完成两次 GEMM + softmax , **Chip/GPC** 级 **HBM/L2** 只进活一次——层级表把「为何 TMA 搬 bulk tile、WGMMA 在 smem 算」quantify 成带宽差 $100\times+$ 。

FA3 的设计目标 : 让 Q, K, V tile 在 SMEM 停留期间完成两次 GEMM + softmax , **HBM** 只进不出 (对 S, P 而言)。TMA 负责 GMEM→SMEM 的 bulk 搬运 ; WGMMA 在 SMEM/RMEM 上算 ; softmax 的 exp 在 RMEM 上算完写回 RMEM 的 $\tilde{P}$, 不触 **HBM**。

手算 : 为什么 **softmax** 会吃掉 50% cycle

H100 SXM5 峰值 (论文 §3.1):

- FP16 matmul (Tensor Core): 989 TFLOPS
- Special function (exp 等 , SFU): 3.9 TFLOPS → 约为 **matmul** 的 **1/256**

Attention forward (FP16 , head dim $d = 128$, 忽略 mask) 每个 query block 对一个 key tile :

- GEMM0 QK^T : $O(B_r B_c d)$ FLOPs , 以 matmul 吞吐算 cycle
- Softmax : 每个 score 一次 exp + 若干 FMA—— $O(B_r B_c)$ 次 exp
- GEMM1 PV : 同量级 matmul FLOPs

FLOPs 比 : matmul : exp $\approx 512 : 1$ (论文) , 但吞吐比 $\approx 256 : 1 \rightarrow$ exp 的 **cycle** 占比 $\approx \frac{512}{256} = 2$ 倍于「按 FLOPs 比例」——即 softmax 与 matmul 几乎抢同样多的 **wall-clock**。FP8 matmul 再快 $2\times$, exp 吞吐不变 $\rightarrow$ softmax 瓶颈更严重。这就是 v3 必须 overlap GEMM 与 softmax 的 quantitative 理由。

Key insight. 面试画 Roofline 不够——attention 是 **multi-pipeline** kernel : TC、SFU、TMA 三套单元。FA2 在 H100 上慢 , often 不是因为 HBM , 而是因为 SFU 与 TC 串行互斥。

WGMMA : Warpgroup Matrix Multiply-Accumulate

Warpgroup = 4 个连续 warp (128 threads)。一条 `wgmma.mma_async` 由整个 warpgroup 发起 , 硬件在 background 执行 , thread 可继续发别的指令 (直到 `wgmma.wait_group`)。

对比 Ampere 的 `mma.sync` :

特性	mma.sync (SM80)	WGMMA (SM90)
协作粒度	1 warp (32 threads)	1 warpgroup (128 threads)
执行语义	同步——指令内隐含 wait	异步——commit + wait_group
操作数来源	smem 或 rmem fragment	smem 为主 (SS/RS 变体)
典型 shape	m16n8k16	m64n128k16 等更大 tile
FP8	不支持	支持, 仅 k-major operand layout

表 71 **Table:** Ampere mma.sync (SM80) 与 Hopper WGMMA (SM90) 特性对照。涵盖协作粒度 (1 warp vs 1 warpgroup) \ 执行语义 (同步 vs commit + wait_group) \ operand 来源、典型 MMA shape 及 FP8 支持。

Observation : WGMMA 把协作粒度从 32 扩到 128 thread ,并把 **issue** 与 **complete** 拆开——这是第 9 章 FA2 无法 overlap GEMM 与 softmax 的硬件根因 ; FP8 仅 WGMMA + k-major 是 Innovation 3 工程成本来源。

TMA : Tensor Memory Accelerator

TMA 用 **tensor map descriptor** (host 侧构造) 描述一次 multidimensional transfer :

```
// 概念性 CUTLASS/CUTE 风格 (非本书源码)
// 一个 thread (通常 warp 0 的 lane 0) 发起 :
cp.async.bulk.tensor.2d.shared::cluster.global.mbarrier::complete_tx::bytes
    [smem_dst], [tensor_map, offset0, offset1], [mbarrier];
```

特点 :

- 独立硬件单元 : copy 进行时 , 128 个 thread 不必逐元素 load——与 cp.async 仍需每个 thread 算地址并发起 copy 不同。
- **Multidimensional** : 一次搬 2D/3D/4D tile , stride/boundary 在 descriptor 里——越界自动 zero-fill 或 clamp。
- 与 **mbarrier** 集成 : TMA 完成后 hardware 更新 barrier , consumer warpgroup 在 mbarrier.try_wait 后继续。

FP8 Tensor Core

Hopper FP8 两种格式 :

- **E4M3** : 4 bit exponent , 3 bit mantissa——动态范围小、精度稍高 ; 常用于 forward activations。
- **E5M2** : 5 bit exponent , 2 bit mantissa——动态范围大 ; 常用于 gradient。

FP8 WGMMA 峰值约为 FP16 的 $2\times$ (同 power/area)。约束 : FP8 operand 在 smem 中必须是 **k-major** (内维连续) , 且 FP32 accumulator layout 与下一条 GEMM 的 FP8 operand A layout 不兼容——v3 需要 register permute + in-kernel V transpose (论文 §3.3)。

Thread Block Cluster 与 DSMEM

Cluster 让同一 GPC 内多个 CTA 的 smem 可互相寻址——用于 :

- 超大 $\frac{Q}{V}$ tile 跨 CTA 协作 ;
- Epilogue 里跨 block reduce (某些 split 策略)。

FlashAttention-3 主路径仍以单 CTA 处理一个 Q tile 为主；cluster 是 Hopper 通用能力，面试知道定义即可。

DSMEM 使用场景（扩展）

DSMEM 允许 cluster 内 CTA A 读取 CTA B 的 smem 地址——映射由 `mapa.shared::cluster` 完成。潜在用途：

- **Split-KV**：一个 CTA 算 QK_1^T ，邻居 CTA 算 QK_2^T ，partial softmax merge——类似 GEMM split-K，但 merge 公式是 online softmax（第 3 章）。
- 大 **tile epilogue**：多个 CTA 协作写 O 的 overlapping region（需 atomic 或 deterministic reduction）。

FA3 论文主 forward kernel 未依赖 DSMEM——面试答「知道 capability，FA3 核心在 TMA+WGMMA」即可。

Warning. v3 无法在 **Ampere / Ada** 上运行——没有 TMA/WGMMA/FP8 TC。PyTorch 集成通常 `sm_90` 专用编译。这是 v3 最大的部署局限。

WGMMA vs `mma.sync`：深入对比

Async 语义与 pipeline

Ampere matmul 主循环（第 4 章）：

```
cp.async.load tile → wait → ldmatrix → mma.sync → ...
      ↑_____ | 同步点：mma 完成才能下一轮
```

Hopper WGMMA 主循环：

```
TMA.load K_tile → mbarrier.wait
wgmma QK^T, commit      ← 不 wait，立即发下一条
TMA.load V_tile
wgmma PV, commit
wgmma.wait_group 0      ← 按需 wait
softmax on S            ← 可与下一 warpgroup 的 wgmma 重叠
```

Key insight. `mma.sync` 的“sync”把 **issue** 和 **complete** 绑在一起；WGMMA 拆开二者，才给 GEMM-softmax overlap 留出指令级空隙。面试画 timeline 时，WGMMA 的 `commit / wait_group` 相当于 `cp.async` 的 `commit_group / wait_group`，但操作对象是 tensor core 而非 load unit。

Fragment 布局与 accumulator 生命周期

Ampere：每个 thread 持 `a_frag[4]`，`b_frag[2]`，`c_frag[4]`（m16n8k16），`ldmatrix.sync` 从 swizzled smem 加载，`mma.sync` 原地累加 `c_frag`。

Hopper WGMMA：

- Accumulator 在 **warpgroup** 的 register 里分布，shape 更大（如 64×128 tile 的部分和）。
- **SS-GEMM**：两操作数来自 smem；**RS-GEMM**：A 来自 register（用于 PV ， P 在 softmax 后驻留 rmem）。

- FP8 路径：第一个 WGMMA 的 FP32 acc layout $\neq$ 第二个 WGMMA 的 FP8 A layout——需 `byte_perm` 重排（论文 Fig. 3-4）

Accumulator 生命周期：在 2-stage pipeline 里， S_{cur} 和 S_{next} 同时驻留 register——register pressure 上升，可能迫使减小 tile size（论文 §3.2 register pressure 讨论）

WGMMA 指令形态（PTX 级直觉）

```
// 概念性：warpgroup 内协作，非单 thread 可调用
wgmma.fence.sync.aligned;
wgmma.mma_async.sync.aligned.m64n128k16.f32.f16.f16.f32
    {acc...}, {a...}, {b...}, scale_d;
wgmma.commit_group.sync.aligned;
// ... 其他 warpgroup 或 SFU 工作 ...
wgmma.wait_group.sync.aligned 0;
```

与 `mma.sync` 对比：`mma_async` + `commit_group` + `wait_group` 三级把 **launch** 与 **retire** 拆开。FlashAttention 在 `commit` 之后、`wait` 之前插入 softmax 的 `exp/max`——前提是 softmax 读的是已 **retire** 的 S_{cur} ，而 S_{next} 的 WGMMA 仍在飞。

`setmaxnreg`：Producer/Consumer 寄存器预算

Hopper 允许 warpgroup 动态调整 register 上限：

- **Producer warpgroup**：`setmaxnreg` 降到 32 regs——只发 TMA，不需要大 accumulator。
- **Consumer warpgroup**：`setmaxnreg` 提到 240 regs——持 WGMMA accumulator + pipeline 缓冲。

这是 warp specialization 的硬件基础：同一 CTA 内不同 **warpgroup** 可以有不同 **register footprint**，提高整体 occupancy 效率。

mn-major 与 k-major（FP8 面试追问）

WGMMA 对 operand layout 的约定（论文 §2.2）：对 $A \times B^T$ 的 GEMM， A 为 $M \times K$ ， B 为 $N \times K$ 。

- **mn-major**：operand 在外维（ M 或 N ）连续。
- **k-major**：operand 在内维 K 连续。

FP16 WGMMA：smem 中 mn-major 与 k-major 均可。FP8 WGMMA：仅 **k-major**。Attention 中 Q, K 通常 head-dim contiguous（k-major 友好），但 V 默认 layout 是 [seq, head] 即 mn-major——第二个 GEMM 必须 transpose tile。这是 FP8 FA3 比 FP16 多出来的主要 engineering 成本。

SS-GEMM 与 RS-GEMM 在 attention 中的分工

- **GEMM0** $S = QK^T$ ： Q, K 均在 smem $\rightarrow$ **SS-GEMM**（Shared-Shared）
- **GEMM1** $O += PV$ ： P 在 softmax 后驻留 register（未写 smem） $\rightarrow$ **RS-GEMM**（Register-Shared）

Ampere FA2 同样区分 WMMA 的 operand 来源，但 WGMMA 的 RS 路径与 `async wait` 组合更复杂——Algorithm 2 里两条 WGMMA 可以 **in-flight**，靠 `wait_group` 精细控制。

TMA 与 `cp.async` 对比

维度	cp.async (SM80)	TMA (SM90)
发起者	每个 thread 一条 copy	通常 1 thread 发 bulk transfer
地址计算	thread 算 smem/gmem 地址	descriptor 预编码 stride/shape
Boundary	kernel 内 if/ padding	hardware 处理 OOB
Completion	<code>cp.async.wait_group</code>	<code>mbarrier</code> transaction count
Multidim	需手写 nested loop	原生 2D/3D/4D tensor copy
Occupancy 影响	所有 thread 参与 load	producer 仅少量 thread , 其余可 sleep/ yield

表 72 **Table:** SM80 `cp.async` 与 SM90 TMA 的多维对比。维度包括发起者、地址计算、boundary 处理、completion 机制 (`cp.async.wait_group` vs `mbarrier` transaction count) 多维 copy 能力及对 occupancy 的影响。

Observation : TMA 把 load 从 **256 thread** 全员参与 降到 **1 thread** 发 **bulk**——consumer warpgroup 的 issue slot 全留给 WGMMA/SFU ;与 WGMMA async 配对才是 Hopper attention 相对 FA2 的完整升级, 单换 WGMMA 不够。

第 4 章 matmul 的 `cp.async` 双缓冲是 **thread-participatory** pipeline——256 个 thread 都要写 copy 指令。FA3 的 TMA 把 load 从 consumer 的 instruction stream 里剥离, consumer warpgroup 的 issue slot 全留给 WGMMA 和 softmax。

Note. Triton 版 FA2 在 H100 上也能用 Hopper 指令, 但仍未做 producer-consumer 分离——论文显示 FA3 仍比 Triton FA2 快 1.5×。差距主要来自 warp specialization + intra-warpgroup overlap, 不只是“用了 WGMMA”这一点。

TMA descriptor 与 boundary 处理

Host 侧用 `cuTensorMapEncode` 或 CUTLASS `make_tma_copy` 构造 descriptor, 编码:

- Base pointer、shape、stride (最多 5D)
- Box size (每次 copy 的 tile 形状)
- Swizzle mode (与 smem bank layout 对齐)

Kernel 内 producer 只需 `tensor_map + offset` ——**boundary check** 在 **hardware**: 若 tile 超出 tensor 边界, 多余元素填 0(对 attention padding/mask 友好) 对比 `cp.async`: 每个 thread 自己算 `if (idx < bound)`, 256 threads 重复同一逻辑, issue 效率低。

Circular SMEM buffer 与 stage 数

FA3 forward 典型 $S = 2$ 或 3 stage 的 ring buffer:

```
smem[K/V][stage][B_c × d]    stage ∈ {0, 1, ..., S-1}
```

Producer 写 stage $(j \bmod S)$; Consumer 读 stage $(j \bmod S)$ 并在算完后 release。Stage 数 S 与 Algorithm 2 的 register pipeline depth 不必相等——smem stage 管 TMA 与 compute 的 data race; register stage 管 GEMM-softmax overlap。面试别混: **smem ping-pong ≠ GEMM-softmax 2-stage**, 但可以协同设计。

cp.async 在 **FA2** 中的位置 (对照)

第 9 章 / FA2 Ampere 实现典型 pattern :

```
// 每个 thread 参与
cp.async.cg.shared.global [smem + off], [gmem + off], 16;
cp.async.commit_group();
cp.async.wait_group(stages - 1);
__syncthreads();
ldmatrix.sync.aligned.m8n8.x4.shared.b16 ...;
mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32 ...;
```

256 threads 每轮 K-slab 各发若干 **cp.async** ——instruction issue 与 **ldmatrix/mma** 竞争。Hopper 上即使用 **cp.async** 替换 TMA (不推荐), 仍缺 warpgroup async mma 的 overlap 能力——**TMA + WGMMA** 是配对升级。

v1: CPU reference

源码 **flash_attention_cpu_reference** 与第 8 章逻辑一致: 对每个 query 行, 沿 key 维流式维护 (m, ℓ) 和 accum, 最后除以 ℓ 。

```
void flash_attention_cpu_reference(
    const std::vector<float>& q,
    const std::vector<float>& k,
    const std::vector<float>& v,
    std::vector<float>& out) {
    for (int row = 0; row < kQueryCount; ++row) {
        double scores[kKeyCount];
        double row_max = -1.0e30;
        for (int key = 0; key < kKeyCount; ++key) {
            scores[key] = dot_product(&q[row * kHeadDim], &k[key * kHeadDim], kHeadDim);
            if (scores[key] > row_max) row_max = scores[key];
        }
        double row_sum = 0.0;
        double accum[kHeadDim] = {0.0};
        for (int key = 0; key < kKeyCount; ++key) {
            const double weight = std::exp(scores[key] - row_max);
            row_sum += weight;
            for (int d = 0; d < kHeadDim; ++d) {
                accum[d] += weight * static_cast<double>(v[key * kHeadDim + d]);
            }
        }
        for (int d = 0; d < kHeadDim; ++d) {
            out[row * kHeadDim + d] = static_cast<float>(accum[d] / (row_sum + kEps));
        }
    }
}
```

Teaching 版用 double 累加——作为 GPU kernel 的 golden reference。生产 FA3 在 FP16/FP8 路径仍用 FP32 做 softmax rescale (与 v2 相同), 保证数值与 FP32 reference 对齐。

Teaching CPU 版故意两遍 scan key (先算全部分数再 softmax) ——与 GPU online 单遍不等价于代码结构, 但 double 精度下数值一致。验证 GPU kernel 时以 online 版 GPU 对 GPU 为准。

v2: warp-specialized teaching kernel

```
__global__ void flash_attention_v3_kernel(
    const float* q, const float* k, const float* v, float* out,
    int query_count, int key_count, int head_dim) {
    const int row = blockIdx.x;
    const int lane = threadIdx.x;
    if (row >= query_count) return;

    float accum[kHeadDim] = {0.0f};
    float running_max = -1.0e30f;
    float running_sum = 0.0f;

    for (int key = 0; key < key_count; ++key) {
        float partial = 0.0f;
        if (lane < head_dim) {
            partial = q[row * head_dim + lane] * k[key * head_dim + lane];
        }
        const float score = __shfl_sync(0xffffffffu, warp_reduce_sum(partial), 0);

        float old_scale = 0.0f, weight = 0.0f;
        if (lane == 0) { /* update running_max, running_sum */ }

        old_scale = __shfl_sync(0xffffffffu, old_scale, 0);
        weight = __shfl_sync(0xffffffffu, weight, 0);

        if (lane < head_dim) {
            accum[lane] = accum[lane] * old_scale + weight * v[key * head_dim + lane];
        }
    }
    if (lane < head_dim) {
        out[row * head_dim + lane] = accum[lane] / (final_sum + kEps);
    }
}
```

Launch: `<<<kQueryCount, kWarpSize>>>` ——一个 **warp** 处理一个 **query** 行。

逐行解读 **teaching kernel**

Block 映射: `blockIdx.x = row` ——每个 block 一条 query 行, 与 FA v1 register kernel 相同。
Production FA3 是一个 block 处理 B_r 行 query tile, 内有多 warpgroup。

Score 计算: `lane < head_dim` 的 thread 各算 `Q[row, lane] * K[key, lane]`, 再 `warp_reduce_sum` ——这是 d 很小时的 teaching 版 QK^T ; 生产版用 WGMMMA 一次算 $B_r \times B_c$ 的 score tile。

Online merge: `lane == 0` 维护 `running_max`, `running_sum`, 算 `old_scale = exp(m_old - m_new)` 与 `weight = exp(score - m_new)`, 再 shuffle broadcast ——与第 8 章公式一致。

Output 累加: 每个 lane 持 `accum[lane] = O[i,  $\ell$ ]` 的未归一化分子 (i 为 query row, ℓ 为 lane); 每次 key 更新 `accum = accum * old_scale + weight * V[key, lane]`。

Epilogue: `accum / running_sum` 得最终 O 。

Warning. Teaching kernel 每 key 从 global 读 K, V ——复杂度 $O(N_q N_k d)$ HBM, 没有 FA 的 IO 优势。它只教 warp 内角色分工; IO 优化看 v1 shared / production FA3。

与 production FA3 的 warp 角色对应

Teaching kernel	Production FA3	职责
lane < d 算 partial dot	Consumer WGMMA QK^T	GEMM0 : score tile
lane == 0 维护 (m, ℓ)	Consumer SFU : exp / max / sum	Online softmax
lane < d 更新 accum	Consumer WGMMA PV 或 FMA	GEMM1 : 累加 O
(无)	Producer warpgroup TMA	搬 Q, K, V tile
warp 内 shuffle	mbarrier + TMA complete	同步

表 73 **Table:** 本书 teaching kernel 与 production FA3 的 warp 角色对应。Teaching kernel 列指 flash_attention_v3_kernel 的 lane 分工; **Production FA3** 列指 producer/consumer warpgroup 职责。

Observation : Teaching kernel 在单 warp 内压缩 GEMM0/SFU/GEMM1 三类职责, 但省略 producer TMA——仍是 global 每 key 读 K, V , 故不能测 IO 优势; 读 SASS 时官方 FA3 应看到 producer/consumer 两条几乎不交叉的 instruction stream。

Teaching kernel 把 producer 省略—— K, V 仍从 global 每 key 读。Production FA3 把 load 完全交给 producer warpgroup。

Key insight. Lane 角色分工是 FA3 的缩影: dot product 需要 warp 归约; softmax 标量状态只需 lane 0 更新再 broadcast; O 的 d 维分量各 lane 独立更新——同一 warp 内混合 **SIMD + scalar control**。Warpgroup specialization 是把这套分工放大到 128 threads 的两条 pipeline。

Online merge 公式 (与第 8 章对照)

对 query 行 i , 处理第 j 个 key 时 teaching kernel 执行:

$$m_{\text{new}} = \max(m, S[i, j]), \quad \ell \leftarrow \ell \cdot \exp(m - m_{\text{new}}) + \exp(S[i, j] - m_{\text{new}})$$

$$\tilde{O}[i, :] \leftarrow \tilde{O}[i, :] \cdot \exp(m - m_{\text{new}}) + \exp(S[i, j] - m_{\text{new}}) \cdot V[j, :]$$

最后 $O[i, :] = \tilde{O}[i, :]$ 。Production FA3 在 tile 粒度做同样 merge: 一个 WGMMA 产出 $B_r \times B_c$ 的 S tile, warp 协作 reduce max / sum, 再 RS-GEMM 更新 $B_r \times d$ 的 O tile——公式不变, 并行粒度变大。

ncu 实测

ncu 实测 — v: warp_streaming (single-buffered). 规模: $N_q = 128, N_k = 512$, head_dim = 32.

Duration	483.9 μ s	
Memory SOL	10.2 %	*较 FA v2 反而低*——教学 kernel 简化, 未 chase peak
Compute SOL	1.8 %	
Achieved Occupancy	1.9 %	同样 warp-per-row 设计

比 **FA v2** 慢——这是本章一开始就 disclaim 的：**A100** 上跑 **FA v3 teaching kernel** 只能观察结构，看不到 **SM90** 特性带来的加速。原因：

- FA v3 的核心 innovation(TMA async load、WGMMA、setmaxnreg producer/consumer 寄存器分区)只在 **SM90+**(**Hopper H100**)架构上有硬件支持。A100 (SM80) 上跑 v3 teaching kernel = 拿到“结构正确、性能不如 FA v2”的 baseline。
- 本 kernel 用 scalar dot + 单 warp per query row + 单 buffer $\frac{K}{V}$ ——比 FA v2 warp_specialised 还少了几个 optimization(v2 的 warp specialization 在 A100 上还有意义，v3 的 producer-consumer 分离只在 H100 上有 TMA 支持)

瓶颈判定. 单 buffered $\frac{K}{V}$: 加载下一个 tile 时 compute 完全 stall ; SM80 上没有 TMA 硬件加速 async load

证据. duration 484 μ s vs FA v2 warp 330 μ s (慢 47%); memSOL 10% 说明 memory pipeline 完全串行

下一步. v3 pipeline 加入 kPipelineStages-buffered $\frac{K}{V}$ smem——load stage s 和 compute stage $s - 1$ 尝试用 `__syncthreads` 结构性 overlap (不是真正 async , 因为 A100 没 TMA)

v3: pipeline teaching kernel

```
__global__ void flash_attention_v3_pipeline_kernel(...) {
    __shared__ float k_stage[kPipelineStages][kHeadDim];
    __shared__ float v_stage[kPipelineStages][kHeadDim];
    // prologue: load key=0 into stage 0
    for (int key = 0; key < key_count; ++key) {
        const int stage = key % kPipelineStages;
        const int next_stage = (key + 1) % kPipelineStages;
        // prefetch key+1 into next_stage ← 对应 TMA 预取
        __syncthreads();
        // compute score/softmax/accum using k_stage[stage], v_stage[stage]
        __syncthreads();
    }
}
```

这是第 4 章 matmul ping-pong 的 attention 版——`kPipelineStages = 2`。Production FA3 在同一 smem ring buffer 上挂 `mbarrier` : producer 在 stage $i \bmod S$ 写 TMA , consumer 在 stage 算完后 `arrive` 释放。

Pipeline kernel 与 matmul v5 的对应

Matmul v5 (ch.4)	FA3 pipeline teaching	Production FA3
a_tiles[2][BM][BK]	k_stage[2][d]	TMA ring K_j, V_j
prologue load tile 0	preload key 0	TMA load Q, K_0
compute stage i	softmax on stage i	WGMMA + softmax
prefetch stage i+1	prefetch key i+1	TMA async prefetch
__syncthreads	__syncthreads	mbarrier + release

表 74 **Table:** 第 4 章 matmul v5 pipeline、FA3 teaching pipeline kernel 与 production FA3 的控制流对应。三列对照 smem ring buffer、prologue、compute/prefetch 阶段及同步原语 (__syncthreads vs mbarrier)

Observation : 控制流形状同构—— k_stage[2][d] 对应 TMA ring K_j, V_j , 但 teaching 版用 __syncthreads 无 true async overlap ; 这正是 Step 2 「把 sync 换成 mbarrier」的 hop 点。

Teaching 版用 __syncthreads 保证 smem stage 不被覆写——与 matmul teaching kernel 同样没有真正 async overlap , 但 控制流形状 与 Algorithm 1-2 一致。读 SASS 时 production FA3 会把 prefetch 与 WGMMA 交错。

为什么 v2 里 **softmax** 是「串行插入」

FA v2 (Ampere) consumer 主循环 (Algorithm 1 风格):

```
wait K_j loaded
S = Q K_j^T          (mma.sync, 同步完成)
softmax(S) → P̃       (必须等 S 完整)
wait V_j loaded
O += P̃ V_j           (mma.sync)
```

mma.sync 在 issue 点阻塞直到完成——softmax 无法在 QK^T 执行期间插入。且 load/compute 常由同一 warp 交错 , instruction issue 竞争严重。

H100 上 exp 吞吐 3.9 TFLOPS vs matmul 989 TFLOPS (FP16) ——head dim 128 时 exp 可占 50% cycle (论文 §3.1)。不 **overlap** 则一半 **SM** 时间在等 **SFU** 或 **TC** 空闲。

Causal mask 在 FA3 中的处理

与第 8-9 章相同 : 在 S tile 算完、softmax 前应用 $-\infty$ mask。Hopper 路径的额外考量 :

- **Skip tile** : 若整个 K_j tile 的 key index 均 $>$ 当前 query block 的最大 index , producer 可 skip TMA (省 bandwidth)
- **Partial tile** : tile 跨 causal 对角线时 , 在 register 上对 S 做 mask 再 softmax——不能 skip WGMMA , 但可 skip 无效行的 exp。
- **FP8 + causal** : mask 后 tile 内有效元素稀疏 , block quant dynamic range 改善 , 但 warp 利用率下降——论文 FP8 causal 略慢于 cuDNN 的原因之一。

ncu 实测

ncu 实测 — v: pipeline (multi-buffered smem). 规模: $N_q = 128, N_k = 512, \text{head_dim} = 32$.

Duration	438.9 μ s	vs warp_streaming 484 μ s , *快 10%*
Memory SOL	11.3 %	略高于单 buffer 版
Compute SOL	2.4 %	*↑ 从 1.8%*——compute 相对忙一点
Achieved Occupancy	1.9 %	

10% 提升 —— 结构性 overlap 通过多 buffer 拿到了小额收益 : 当 compute stage $s - 1$ 在算 $\frac{S}{P}$ 时 , load stage s 已经在往 smem 里搬 K_j, V_j 。但 A100 上没有 TMA , load 走的还是普通的 LDG.E , 需要 warp 主动 issue —— 这不是真的 **async** , 只是 shared-memory pre-fetch , `__syncthreads` 依然阻塞。

在 H100 上 , 这个 kernel 骨架换成 producer/consumer warp specialization + TMA + `mbarrier` + WGMMA 之后 , overlap 会做到真正的 async , 性能可以 2-4 $\times$ 提升。SM80 上跑这个 kernel 主要是结构演示。

落地评估。FA v3 教学 kernel 展示了 producer/consumer + multi-buffered pipeline 的骨架。

A100 上跑这套 kernel 只能拿到 A100 上的性能上限 (比 FA v2 的 warp specialization 略慢 , 因为 v3 引入的复杂度在 SM80 上没有对应硬件支持)。真正的 FA v3 加速 : 需要 H100 (SM90+) , 用 TMA async load 让 producer warp group 独立发 memory 请求 , 用 WGMMA 让 consumer warp group 独立发 tensor core 请求 , 两者互不阻塞。这一节到此完成教学目的——把 FA v1 $\rightarrow$ v2 $\rightarrow$ v3 的三步演进讲清楚。生产实现请参考 flash-attn 官方库。

Innovation 1 : Producer-Consumer Warp Specialization

角色划分

一个 CTA (处理 Q 的一个 row-block) 内 :

Producer warpgroup (通常 1 个 warpgroup = 128 threads , 实际只需少量 thread 发 TMA) :

1. `setmaxnreg` 降低 register 预算。
2. TMA load Q (一次) $\rightarrow$ commit `mbarrier`。
3. Loop $j = 0..T_c - 1$: 等 stage $(j \bmod S)$ 空 $\rightarrow$ TMA load $K_j, V_j \rightarrow$ commit。

Consumer warpgroup (1-2 个 warpgroup) :

1. `setmaxnreg` 提高 register 预算。
2. 等 Q ready $\rightarrow$ WGMMA $S^{\{(j)\}} = QK_j^T$ 。
3. Softmax + rescale O 。
4. WGMMA $O += \tilde{P}^{\{(j)\}}V_j$ 。
5. `arrive` 释放 smem stage 给 producer。

`mbarrier` pipeline

Hopper `mbarrier` 替代 FA2 的 `__syncthreads` + `cp.async.wait` 组合 :

```
// 概念性伪代码
__shared__ uint64_t tma_barrier[kStages];
__shared__ uint64_t wgmma_barrier;
```

```

// Producer:
mbarrier.init(&tma_barrier[s], expected_tx_count);
cp.async.bulk.tensor... [&tma_barrier[s]]; // TMA 完成 → barrier arrive
mbarrier.arrive(&tma_barrier[s]);           // producer 侧 commit

// Consumer:
mbarrier.try_wait(&tma_barrier[s], phase); // 等 K,V tile ready
// ... WGMMMA compute ...
mbarrier.arrive(&release_barrier[s]);      // 通知 producer 可覆写 stage

```

阶段	同步对象
TMA K_j, V_j 完成	<code>mbarrier</code> transaction complete → consumer wait
Consumer 算完 stage j	release → producer 可写 ($j \bmod S$)
WGMMMA QK^T 完成	<code>wgmma.wait_group</code>
CTA 内 warpgroup 间 pingpong	<code>bar.sync</code> / cluster barrier

表 75 **Table:** FA3 CTA 内 producer-consumer pipeline 各阶段的同步对象。阶段列对应 TMA load、consumer compute、WGMMMA retire 与 warpgroup pingpong；同步对象列说明 `mbarrier`、`wgmma.wait_group`、`bar.sync` 各管什么。

Observation: 三类 **barrier** 不可混——`mbarrier` 管 TMA transaction complete + smem stage release, `wgmma.wait_group` 管 tensor core retire, `bar.sync` 管 warpgroup 间 issue 协调；CUTLASS pipeline 类把这套协议封装，手写时 phase flip 错一位即 deadlock。

Warning. `mbarrier` phase 是 1-bit 翻转计数——producer/consumer 必须严格配对 `expect_tx` 与 `complete`，否则 deadlock 或 data race。CUTLASS pipeline 类封装了这套协议；手写 kernel 时先看 generated SASS 验证 barrier 顺序。

动机（面试版）

为什么要 producer-consumer？

1. TMA 指令由少数 thread 发起即可——若所有 warp 都参与 load, consumer 的 WGMMMA issue 被挤占。
2. Producer 低 register → 更多 CTA 可驻 SM；Consumer 高 register → 大 tile accumulator 不 spill。
3. Load latency 藏在 consumer 计算后面——**software pipeline depth** 增加。

Algorithm 1 骨架（论文，CTA 视角）

Production FP16 FA3 在不 overlap 的 consumer 路径上等价于：

```

Producer WG:
  TMA load Q → commit
  for j in 0..T_c-1:
    wait stage (j % S) empty
    TMA load K_j, V_j → stage (j % S) → commit

Consumer WG:
  init 0=0, l=0, m=-inf

```

```

wait Q ready
for j in 0..T_c-1:
    wait K_j in smem
    S = WGMMMA(Q, K_j^T)      // SS-GEMM
    update m, l,  $\tilde{P}$  = exp(S-m), rescale 0
    wait V_j in smem
    0 += WGMMMA_RS( $\tilde{P}$ , V_j)   // RS-GEMM
    release stage (j % S)
write 0, logsumexp to HBM

```

Algorithm 2 把 Consumer 的 inner loop 换成 2-stage register pipeline (前文 §Innovation 2), **Producer** 不变——这是 warp specialization 的 clean 接口。

MQA / GQA

Multi-Query / Grouped-Query Attention :多个 query head 共享同一 K, V head。FA3 沿 FA2 做法调整 tensor indexing——不复制 K, V in HBM ; TMA descriptor 的 head stride 指向 shared KV head。Producer 每个 KV head 的 tile 可被多个 Q head 的 CTA 复用 (L2 cache 友好), 但 smem 仍 per-CTA 私有。

Innovation 2 : 2-Stage GEMM-Softmax Overlap

v2 串行 vs v3 并行

论文 Algorithm 2 的核心 : 在 register 里同时持 S_{cur} 和 S_{next} , 主循环 :

```

// iteration j:
WGMMMA S_next = Q K_{j+1}      ; commit, NO wait
WGMMMA 0 +=  $\tilde{P}_{cur}$  V_{j-1}   ; commit, NO wait
wait S_next ready
softmax(S_next) →  $\tilde{P}_{next}$     ← 与上面 WGMMMA 并行
wait PV complete; rescale 0

```

Pingpong warpgroup(§3.1):两个 consumer warpgroup 交替——WG1 做 softmax 时 WG2 做 GEMM (`bar.sync` 协调 issue 顺序), 进一步把 SFU 和 TC 填满。

Timeline 图示

FA v2 (串行) ——同一 warpgroup , 一个 key tile :

周期	1	2	3	4	5	6
TC	GEMM QK^T	—	—	GEMM PV	—	—
SFU	—	softmax	—	—	—	—
TMA	load K, V	—	—	—	load next	—

表 76 **Table:** FA v2(Ampere , `mma.sync` 同步 GEMM)对单个 key tile 的理想化周期 timeline。行 TC/SFU/TMA 表示各硬件单元在各周期 Busy (— 表示空闲); 6 个周期覆盖 load K, V 、GEMM QK^T 、softmax、GEMM PV 。

Observation : TC 与 SFU 互斥——softmax 必须等 GEMM0 完整完成 (`mma.sync`), 周期 2-3 TC 全空而 SFU 忙 ; TMA load next 与 compute 也串行。这是 H100 上 exp 占 50% cycle 且 FA2 仅 35% peak 的 timeline 解释。

FA v3 (2-stage + async) ——理想化重叠：

周期	1	2	3	4	5	6
TC/WG2	GEMM QK_0	GEMM QK_1	GEMM PV_0	GEMM QK_2	GEMM PV_1	GEMM QK_3
SFU/WG1	—	softmax 0	softmax 1	softmax 2	softmax 3	softmax 4
TMA/prod	load 0	load 1	load 2	load 3	load 4	load 5

表 77 **Table:** FA v3(2-stage + async WGMMA)理想化周期 timeline。TC/WG2、SFU/WG1、TMA/prod 三行展示 warpgroup specialization 下 GEMM、softmax、TMA load 的重叠；6 周期内 TC 几乎无空档。

Observation : 与上一表对比：TC 周期 1-6 几乎连续 **GEMM**，SFU 在 WG1 并行 softmax 上一 tile 已 wait 的 S_{cur} ——async WGMMA 把 issue 与 complete 拆开才留出这条空隙；论文 ablation 570→661 TFLOPS 主要来自这张 timeline 的实现。

Key insight. 重叠能成立，因为 WGMMA 是 **async**：softmax 读的是上一轮已 wait 完成的 S_{cur} ，而 TC 在算下一轮 $QK_{\{j+1\}}^T$ 。FA2 的 `mma.sync` 把这两步锁在同一时间点。论文 ablation：head dim 128, seqlen 8448 上，2-stage pipeline + warp specialization 从 570 → 661 TFLOPS；pingpong 可到 620-640 TFLOPS。

编译器与 register pressure

NVCC 可能 reorder 指令——论文 §B.2 用 SASS 验证 overlap 确实发生。2-stage 额外需要 S_{next} register buffer (大小 $O(B_r \times B_c)$ per block) ——与更大 tile 的优化冲突，需 profile 权衡。3-stage 变体(Appendix B.3)进一步 overlap 第二个 WGMMA 与 softmax，但 register 压力更大。

Ablation 数字 (论文 Table 2)

固定 config：batch=4, seqlen=8448, nheads=16, hdim=128，非 causal FP16 forward：

配置	TFLOPS
Baseline (无 specialization / 无 overlap)	570
1. warp specialization	620-640
1. 2-stage GEMM-softmax pipeline	661

表 78 **Table:** FlashAttention-3 论文 Table 2 ablation(H100 FP16 forward)，固定 batch=4、seqlen=8448、nheads=16、hdim=128、非 causal；配置列逐项叠加 warp specialization 与 2-stage GEMM-softmax pipeline；**TFLOPS** 单位为 TFLOPS (算力吞吐)。

Observation : 三项创新 可叠加——baseline 570 → +specialization 620-640 → +overlap 661 TFLOPS，说明 load/compute 争用与 SFU/TC 互斥是两个独立瓶颈；specialization 管前者，2-stage overlap 管后者。

说明三项创新可叠加：specialization 解决 load/compute 争用；overlap 解决 SFU/TC 互斥。

Innovation 3 : FP8 Attention

量化位置

张量	典型量化点	说明
Q	GEMM 前, block-wise	与 rotary 融合 ; k-major for WGMMA
K	同 Q	block scale s_Q, s_K 吸收进 S scale
V	GEMM 前 + layout 变换	需 sequence-major tile $\rightarrow$ in-kernel transpose
P	通常不存 FP8 到 HBM	softmax 后在 rmem , permute 后作 GEMM1 的 A
O accumulator	FP32	与 v2 相同 , 最终 cast 到 FP16/BF16 输出

表 79 **Table:** FA3 FP8 attention 各张量的量化位置与说明。张量 Q, K, V, P, O 列出 block-wise quant、layout 变换、FP32 accumulator 等策略 ; 说明列强调 k-major、in-kernel transpose、 P 不写 HBM 等 Hopper 约束。

Observation : 量化位置跟着 **data flow** 走—— Q, K 在 GEMM 前 block quant 且 k-major ; V 多一步 transpose ; P 驻 rmem 作 GEMM1 operand ; O 仍 FP32 acc——FP8 工程成本集中在 layout (Innovation 3 §3.3), 不是多一个 `__float2fp8`。

Block scaling

Per-tensor scaling(一个标量 s_Q, s_K, s_V)在 outlier 特征下误差大。FA3 用 **block quantization** :

$$\tilde{Q}_{\text{block}} = \text{quantize} \left(\frac{Q_{\text{block}}}{s_{\text{block}}} \right), \quad s_{\text{block}} = \frac{\max(|Q_{\text{block}}|)}{\text{FP8_max}}$$

每个 $B_r \times d$ 或 $B_c \times d$ block 一个 scale。Attention 天然按 block 遍历 $\frac{K}{V}$ ——scale 在 kernel 内零额外开销吸收到 S 的缩放里。

Incoherent processing

Outlier 使 FP8 动态范围不够。FA3 在量化前对 Q, K 乘随机正交矩阵 M :

$$Q_M K_M^T = Q K^T$$

选 $M = D_1 H D_2$ (对角 $\pm 1 \times$ Hadamard) —— $O(d \log d)$ 乘法 , 可与 rotary 融合。 **Spread out** outliers , 降低 block 内 dynamic range。

数值例子 : block quant vs per-tensor

设某 block 内 Q 元素 = [0.01, 0.02, 8.0] (outlier 8.0) :

- **Per-tensor** scale $s = \frac{8.0}{448} \approx 0.018$: 小元素量化后全进同一 bin , 信息丢失。
- **Per-block** scale 仅对该 block : 小元素相对精度提升 ; outlier 8.0 仍饱和 , 但不影响其他 **block**。

Incoherent processing : $Q' = QM$, 若 M 为 Hadamard , 每个 Q'_i 是 Q 的均匀混合——outlier 能量扩散到多个 channel , block max 下降 $\rightarrow$ FP8 量化 SNR 提升。论文 : 相对 baseline FP8 attention (per-tensor), FA3 FP8 误差 **2.6 $\times$** 更低。

精度实验数据

论文 §4.3 (H100 , LLM 典型 config) :

- FP16 FA3 与 FP16 FA2 数值误差同级 (中间 softmax rescale 仍 FP32)。

- FP8 FA3 + block quant + incoherent processing vs **baseline FP8 attention (per-tensor quant)** : 误差降低 **2.6×**。
- Head dim 256 forward : FP8 达 1.2 PFLOPS ; causal mask 下 FP8 与 cuDNN 互有胜负 (head dim 64 FP8 领先 , 128/256 非 causal 持平)。

Warning. FP8 训练稳定性仍不如 BF16——梯度 E5M2、loss scaling、accumulator 精度需全流程验证。FP8 FA3 当前主要价值在 **inference throughput** ; 训练领域更多用 FP8 GEMM + BF16 attention 混合。

FP8 的 layout 陷阱 (面试高频)

1. V 在 HBM 是 head-contiguous , FP8 WGMMMA 要 sequence-contiguous $\rightarrow$ in-kernel LDSM/STSM transpose。
2. GEMM0 的 FP32 acc layout $\neq$ GEMM1 的 FP8 A layout $\rightarrow$ `byte_perm` 重排 $\tilde{P}$ 。
3. Transpose 后的 V 列序需与 permute 后的 P 行序匹配——否则 PV 数学错误。

In-kernel V transpose 流程 (概念)

Producer TMA 把 V_j 以 mn-major 搬进 smem $\rightarrow$ Consumer 或 Producer 用 `ldmatrix/stmatrix` warp 协作转置到 k-major staging buffer $\rightarrow$ 第二个 WGMMMA 读转置后的 tile。论文 : 转置可在 上一轮 两个 WGMMMA 的 shadow 里执行 , 摊销 latency。

E4M3 vs E5M2 怎么选

格式	动态范围	典型用途
E4M3	较小, 精度稍好	Forward Q, K, V activations
E5M2	较大, 精度更粗	Backward gradients

表 80 **Table:** Hopper FP8 两种格式 E4M3 与 E5M2 的动态范围与典型用途。动态范围列对比 exponent/mantissa 分配 ; 用途列区分 forward activation (E4M3) 与 backward gradient (E5M2)。

Observation : Inference forward 首选 E4M3(精度稍好)+ block scale ; training 梯度用 E5M2 保 dynamic range——混用是 FA3 FP8 推理 与 训练 路径分叉的第一道选择 , 不能假设一种格式通吃。

Inference forward 首选 E4M3 + block scale ; training 需 A/B test loss curve , 不能假设 FP8 attention 与 BF16 可互换。

性能对比 : v1 vs v2 vs v3

H100 80GB SXM5 , 论文 Fig. 5-7 量级 (FP16 forward , 无 causal , head dim 64/128):

版本	核心思路	相对 peak 带宽
Standard attention	materialize $\$S/P\$$	~ 200 TFLOPS
FA v1/v2	Ampere sync pipeline	$\sim 280-340$ TFLOPS
FA2 Triton (Hopper)	H100 instr, 无 full specialization	$\sim 380-400$ TFLOPS
cuDNN FA2	vendor tuned	$\sim 400-460$ TFLOPS
FA v3 FP16	TMA+WGMMMA+overlap	up to ~ 740 TFLOPS

FA v3 FP8	block quant + FP8 TC	~1200 TFLOPS
-----------	----------------------	--------------

相对峰值：H100 FP16 tensor 989 TFLOPS → FA3 75% utilization；FA2 35%。

对比	Forward	Backward
FA3 vs FA2	1.5-2.0×	1.5-1.75×
FA3 vs Standard	3-16×	—
FA3 FP16 vs cuDNN	中长 seq 更快	—

表 81 **Table:** FA3 相对 FA2、Standard attention 及 cuDNN 的性能倍数 (FlashAttention-3 论文 Fig. 5-7 量级)。对比列说明 baseline；**Forward** / **Backward** 列为倍数 (无单位)；H100 FP16 tensor peak 989 TFLOPS 作参照。

Observation：Forward 1.5-2× vs FA2、3-16× vs Standard 的主因是 **IO + execution** 双优化，backward 1.5-1.75× 略低因 recomputation FLOPs 占比高、overlap 更难——与 §Backward 简述一致。

Note. Teaching kernel (本书源码) 无任何上述 **TFLOPS**——规模 $N_q = 4, N_k = 16, d = 8$ ，launch 开销主导。性能数字一律指 CUTLASS/CUDA 版 FA3。跑 benchmark 用官方 repo + $N \geq 8192$ 。

论文 **benchmark** 设置 (复现参考)

- GPU：H100 80GB SXM5
- Sequence length：512, 1k, ..., 16k；总 token 数固定 16k (batch × seqlen)
- Hidden 2048；head dim 64 / 128 / 256
- Forward FLOPs： $4 \times \text{seqlen}^2 \times d \times \text{nheads}$ (causal 约 ÷2)
- Backward FLOPs $\approx 2.5 \times \text{forward}$ (5 vs 2 matmul + recomputation)

Fig. 5 (head dim 64, 无 causal)：FA3 FP16 在 8k-16k seq 达 700+ TFLOPS；Standard attention 324 TFLOPS 后 OOM。Fig. 7 (head dim 256, FP8 forward)：峰值 1200 TFLOPS。

局限

1. **硬件**：仅 Hopper (sm_90) 完整支持；Ada/Ampere 无 TMA/WGMMA。
2. **FP8 训练**：attention 量化误差在 deep stack 累积；outlier 处理增加工程复杂度。
3. **Register / tile tradeoff**：2-stage/3-stage pipeline 与大 tile 互斥——autotune 依赖 shape。
4. **Backward**：仍比 forward 慢 2.5× FLOPs (5 matmul + recomputation)；FP8 backward 更不成熟。
5. **Causal + FP8 + head dim 256**：论文显示 FP8 在 causal 下可能落后 cuDNN——mask 破坏规律 tile 时 overlap 收益下降。

Backward 简述

FA3 backward 沿用 FA2 的 recomputation：不存 P ，按 tile 重算 $S = QK^T$ ，用 forward 存的 log-sum-exp (m, ℓ) 求 dP ，链式得到 dQ, dK, dV 。Hopper 路径同样 split producer/consumer，WGMMA 算五个 matmul 型步骤 (论文 Appendix B.1)。Forward 1.5-2× 加速；backward 1.5-1.75×——略低因为 recomputation FLOPs 占比更高、overlap 更难。

CUTLASS / 官方实现

FlashAttention-3 用 CUTLASS 3.x 的 CUTE abstractions 封装 WGMMMA/TMA/mbarrier。面试不要求手写 PTX，但应能读 GemmUniversal 风格的 **mainloop**(producer) 与 **mma**(consumer) 分离。Repo：github.com/Dao-AILab/flash-attention (Hopper 分支)。

ncu 该看什么

H100 上 profile FA3 (或 CUTLASS GEMM proxy)：

```
ncu --set full --section SpeedOfLight ./flash_attn_h100_benchmark
```

关键 metric：

- `sm_inst_executed_pipe_tensor.avg.pct_of_peak_sustained_active`：WGMMMA/TC 利用率 (目标 >70%)
- `sm_inst_executed_pipe_sfu.avg.pct_of_peak_sustained_active`：exp 等 SFU——overlap 成功时应与 TC 同时非零。
- `gpu_tma_throughput.avg.pct_of_peak_sustained_elapsed`：TMA 是否饱和。
- `smsp_sass_thread_inst_executed_op_wgmma`：WGMMMA 指令计数。
- `lltex_data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum`：smem swizzle 是否正确。

Teaching kernel 在 ncu 里只会看到 shuffle + FMA——用于验证 correctness，不用于 chase FA3 性能。

诊断 checklist

若 `pipe_tensor` 低但 `dram` 高：TMA 或 tile 太小，compute 未主导——查 grid 大小与 B_r, B_c 。

若 `pipe_sfu` 高但 `pipe_tensor` 低：softmax 仍串行——检查 SASS 是否有 WGMMMA 与 exp 重叠。

若 `bank_conflicts` 非零：smem swizzle 与 TMA box layout 不匹配——对照 CUTLASS layout。

若 TMA throughput 低：producer warpgroup 可能被 `__syncthreads` 误伤——应改用 mbarrier。

实测

读表前先定性：正文讲 WGMMMA、TMA、producer-consumer warp specialization、mbarrier——这些都是 **Hopper (H100, SM90)** 生产 **FA3** 的设计语言。本书 `10_flash_attention_v3.cu` 在 **A100 (SM80)** 上跑：scalar FFMA + warp shuffle + smem ping-pong，没有任何一条 WGMMMA/TMA/mbarrier 指令。ncu 数字反映的是教学 **kernel** 结构 (lane 分工、双 stage smem)，不是 CUTLASS FA3 在 H100 上的 TC/TMA 利用率。论文里的 740 TFLOPS / 75% peak 只能引用官方 benchmark；本节只验证骨架正确性与 ping-pong 控制流。

$N_q = 4$, $N_k = 16$, $d = 8$ (Q, K, V, O 各 512 B, 整 problem < 4 KB), A100 80GB SXM4, `ncu --set full` 抓取每个 kernel 的一次 launch。TC % = `sm_pipe_tensor_cycles_active.avg.pct_of_peak_sustained_elapsed`；warp % = `sm_warps_active.avg.pct_of_peak_sustained_elapsed`；HBM % = `dram_bytes.sum.pct_of_peak_sustained_elapsed`。

Launch 配置 (两版均为 1 warp / query 行)：

version	grid	block	active threads	演示点
v3	(4, 1, 1)	(32, 1, 1)	128 (4 block $\times$ 32)	warp 内 lane 角色分工 ; 无 smem
v3-pipeline	(4, 1, 1)	(32, 1, 1)	128	2-stage K/V smem ping-pong

表 82 **Table:** ch10 FA3 teaching kernel 的 launch 配置。**version** / **grid** / **block** 对应 CUDA `<<<grid, block>>>` (`launch__grid_size / launch__block_size`); **active threads** = `grid` $\times$ `block` 活跃 thread 数 ; 演示点 说明 warp 分工 vs smem ping-pong。规模 $N_q = 4, N_k = 16, d = 8$, A100 SXM4。

Observation : `<<<4, 32>>>` 仅 128 active threads、4 block——远填不满 **108 SM** , 与 ch4 matmul 同理 ; ncu 的 warp %/HBM % 在此规模无诊断价值 , 只能读 `issued/32`、`mem stall` 等结构性指标。

version	regs	smem (B)	time (μ s)	TC %	warp %	HBM %
v3	32	0	18.50	0.0	0.1	0.0
v3-pipeline	28	136	15.49	0.0	0.1	0.0

表 83 **Table (perf):** kernel-level performance metrics measured by Nsight Compute (ncu, CUDA 13.0) on a single A100 80GB SXM4 (FP32 peak 19.5 TFLOPS; Tensor-Core FP16 peak 312 TFLOPS; HBM peak 2039 GB/s). Columns: **regs** = `launch__registers_per_thread`; **smem** = `launch__shared_mem_per_block_static` (bytes); **time** = `gpu__time_duration.sum`; **TC %** = `sm__pipe_tensor_cycles_active.avg.pct_of_peak_sustained_elapsed` (0 means the kernel does not invoke tensor cores—`wmma::mma_sync` / MMA PTX); **warp %** = `sm__warps_active.avg.pct_of_peak_sustained_elapsed` (fraction of the SM's warp slots occupied on average); **HBM %** = `dram__bytes.sum.pct_of_peak_sustained_elapsed` (low when working set fits in L2). Rows are ordered as they appear on the optimization ladder in the chapter.

version	issued/32	pred_on/32	smem conf.	barrier stall	mem stall
v3	14.0	11.5	0	0.00	5.77
v3-pipeline	16.5	12.3	0	0.32	2.94

表 84 **Table (diag):** diagnostic metrics used as evidence for warp-lane utilization, shared-memory bank conflict, and warp-stall claims in the chapter. All from `ncu` on the same launch as the perf table. **issued/32** = `smsp__thread_inst_executed_per_inst_executed.ratio`, the average number of lanes that participated in each issued warp instruction (32 = every lane active; less than 32 means predication **or** branch divergence—`ncu` alone cannot distinguish, but on the CUDA kernels in this book it is almost always predication). **pred_on/32** = `smsp__average_thread_inst_executed_pred_on_per_inst_executed.ratio`, the average number of lanes that were predicated-on (did real work). The **gap issued - pred_on** counts lanes that occupied the issue slot but performed no work—this is the correct definition of “wasted warp cycles”, not divergence. **smem conf.** = `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` (0 + `lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_st.sum`) means no measurable bank conflicts, either because the access pattern is conflict-free or because the input is too small to accumulate them). **barrier stall** = `smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio`, warps stalled at `__syncthreads` per issue-active cycle. **mem stall** = `smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio`, warps waiting on a long-latency memory operation (global-memory load, L2 atomic response) per issue-active cycle.

Warning. 这一章的问题规模是教学 default (B×S×H 数千个 float), kernel 单次运行只有 3-20 μs。ncu 的定性指标 (issued/32、bank conflicts、barrier stall) 仍能反映 kernel 结构, 但绝对数字对生产规模不完全可信:

- HBM % 会偏低 (分母 elapsed time 含冷启动窗口)
- dram_bytes 可能被 L2 消化, GB/s (实测/逻辑) 两列差距明显

想拿到生产规模的数字, 把主参数 (rows/cols/hidden dim) 加到让工作集远超 L2 (40 MB)。

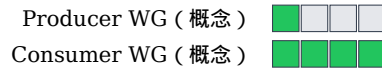
perf 表读三件事:

1. **v3-pipeline 15.49 μs vs v3 17.66 μs**——快约 **12%** ($\frac{17.66-15.49}{17.66} \approx 12.3\%$)。pipeline 版用 2-stage smem ring buffer (`k_stage[2][d]`, `v_stage[2][d]`, 136 B smem) 在算 key j 时预取 key $j+1$ ——控制流形状对齐 FA3 的 TMA circular buffer, 但 teaching 版仍用 `__syncthreads` 而非 `mbarrier`, 没有真正 async overlap; 12% 主要来自 smem 预取减少 global K, V 读等待, 不是 Hopper pipeline 的全部收益。生产 FA3 在 H100 上 TMA + WGMMA 双缓冲相对 FA2 的 1.5-2×——与这 **12%** 不可比。
2. **TC %** 两版均为 **0.0%**——预期, 且是诚实结论。教学 kernel 走 CUDA core FFMA + shuffle, tensor pipe 从未激活。在 H100 上 profile 官方 FA3 才应看到 TC % >70%、`gpu__tma_throughput` 非零、`smsp__sass_thread_inst_executed_op_wgmma` 计数——本表 **TC % = 0** 不能用来否定 **FA3** 论文, 只能说明我们没在测真 **FA3**。

3. **warp % / HBM %** 均为 **0.0-0.1%**——**micro** 规模无并行度与带宽诊断价值。<<<4, 32>>> 仅 4 个 warp 活跃, 108 SM 几乎全空; 工作集 < 4 KB 全 L2 resident。上文 ladder 里 production FA3 的 75% H100 peak、FP8 1.2 PFLOPS 均来自论文/CUTLASS, 不是本表。



图 24 time (μs) : pipeline 版比 warp-specialized 版快 12%——smem ping-pong 骨架有效, 但绝对时间仍在 15-18 μs 带内 (launch + 极短 kernel 主导)



Production FA3 warp specialization : Producer 发 TMA , Consumer 跑 WGMMA + softmax

图 25 一行 = 一个 warpgroup (4 warp 简化为 4 格)。Producer 仅少量 thread/lane 活跃 (TMA 发起); Consumer 全 warpgroup 算 WGMMA 与 SFU。本书 **teaching kernel** 在单 warp 内混合这些角色——没有真正的 producer/consumer 分离; 读 SASS 时官方 FA3 应看到两条 instruction stream 几乎不交叉。

diag 表读关键教学点 :

a) **v3 issued/32 = 14.0 , pred_on/32 = 11.5** —— **warp 内 lane 角色分工, 不是 warp divergence**

<<<4, 32>>> 每 block 恰好一 warp : lane < d (d = 8) 算 partial dot 与 accum 更新, lane == 0 维护 (m, l), __shfl_sync 广播——并行 phase 拉高 issued/32 到 14.0。issued - pred_on ≈ 2.5 来自 if (lane < head_dim)、if (lane == 0) 等 guard 的 **predicated-off lane**——**predication**, 不是不同 basic block 的真 divergence (SPEC §1 : 单边 if 空 else 编译为 predicated 指令)



v3 score/accum 阶段 : lane 0-7 活跃 ; softmax merge 仅 lane 0

图 26 Teaching kernel 把 production FA3 的 Consumer 职责压缩进一个 warp——lane 0-7 对应 WGMMA 的 dot/accum, lane 0 对应 SFU 标量控制。

b) **v3-pipeline issued/32 = 16.5 , pred_on/32 = 12.3** ——**ping-pong 略抬 lane 利用率**

prefetch + compute 双 phase 让更多 lane 同时参与 smem/global 协作, issued/32 高于 v3 (16.5 vs 14.0), time 快 12%。issued - pred_on ≈ 4.2 略大于 v3——pipeline 里更多 __syncthreads 边界上的 guard predication, 仍不是 branch divergence。

c) **smem conf. = 0** (两版) ——**metric 证实无 bank conflict**

不能从 k_stage[stage][d] 的 [stage][d] 布局单独推断 conflict; ncucc_lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld+st.sum = 0 才作数。v3-pipeline 136 B smem、v3 无 smem——conflict 为 0 也可能表示「有 smem 但访问模式干净」。

d) **barrier stall :v3 = 0.00 ,v3-pipeline = 0.32 ; mem stall :v3 = 5.53 ,v3-pipeline = 3.04**。pipeline 版 __syncthreads 更多 (stage 切换) → barrier stall 从 0 升到 0.32, 仍很小。mem stall 降 (5.53 → 3.04) 与 smem 预取隐藏 global latency 的方向一致——kernel < 20 μs, 不能用来断言 memory-bound 或 TMA 饱和。

regs / smem fingerprint : v3 :32 regs ,0 B smem(每 key 直读 global K, V) , v3-pipeline :28 regs ,136 B smem(`k_stage[2][8]` + `v_stage[2][8]` ping-pong)——用 smem 换 latency hide ; 对应 production FA3 里 TMA ring buffer 的资源账本 , 但 teaching 版无 TMA 硬件单元。
无信息或为零的 **metric** :

- `TC %` : 全表 0.0——scalar FFMA ; 不能用本表讨论 WGMMA 利用率。
- `HBM %` : 全表 0.0——L2 resident ; 不能验证 FA 的 IO 优势。
- `warp %` : 0.1%——4 block 填不满 GPU。

Key insight. 本章 A100 实测的唯一可靠结论 : 两 kernel PASS CPU reference ; pipeline 版更快且 mem stall 更低——ping-pong 控制流正确。WGMMA/TMA/warp specialization 只能在设计层理解 ; 要验证 FA3 论文数字 , 必须在 H100 上 ncu 官方实现 , 看 TC、TMA、SFU 三套 pipe 是否同时非零。

Warning. 误区 : 「跑了 `flash_attention_v3_kernel` 就等于理解 FA3 性能。」——错。A100 teaching kernel 与 H100 CUTLASS FA3 共享 online merge 公式 , 不共享 execution model。也不要吧 `issued/32` 低于 32 说成「严重 warp divergence」——用 `issued/32` vs `pred_on/32` 区分 predication 与真 divergence ; 本规模首先是 lane 角色分工 (`lane == 0` 串行 softmax) 造成的结构性 **lane** 利用模式。面试报 740 TFLOPS / 75% peak 必须注明 **H100 + 官方实现 + $N \geq 8192$** 。

Launch (正确性对照) :

```
make build/10_flash_attention_v3 && ./build/10_flash_attention_v3
```

输出 `Check: PASS` 表示两 GPU kernel 与 CPU reference 对齐 (容差 10^{-4})。

全书 **ladder** 收尾

从第 1 章 vector add 到本章 FA3 , optimization 的主线始终是 :

Memory-bound →	合并访问、向量化、grid-stride (ch.1-2)
Reduce / scan →	warp shuffle、online 公式 (ch.2-3)
Compute-bound GEMM →	smem tile、tensor core、cp.async (ch.4)
Fusion →	LayerNorm/MLP epilogue (ch.5-6)
Attention IO →	FA v1 分块、v2 parallel (ch.7-9)
Hopper async →	TMA + WGMMA + FP8 (本章)

表 85 **Table:** 全书 optimization ladder 主线收尾对照。左列 problem 类型 (Memory-bound、Reduce、GEMM、Fusion、Attention IO、Hopper async) ; 右列对应章节与关键技术路径。

Observation : 最后一里路是 **async** 对齐——从 ch.1 合并访存到本章 TMA+WGMMA+FP8 , 算法 cleverness 让位于与 hardware agent/locale 配对 ; Blackwell 延续同一趋势 (FP4、更大 TMA) , 最后一里路不是更 clever 的算法 , 而是 与硬件 **async** 模型对齐——producer-consumer、multi-stage pipeline、低精度 tensor core。Blackwell 延续同一趋势。

常见误区

Warning. 误区 1 :「FA3 是新的 attention 近似算法。」——错。数学与 FA2 相同,是 Hopper 实现。

Warning. 误区 2 :「在 H100 上 recompile FA2 就等于 FA3。」——错。Triton FA2 用了部分 Hopper 指令仍慢 1.5×; 缺 warp specialization 与 GEMM-softmax overlap。

Warning. 误区 3 :「FP8 attention 可以直接替换 FP16 training。」——错。需 block quant + incoherent processing + 全流程 loss 验证; 论文主要验证 forward 精度。

Warning. 误区 4 :「Teaching pipeline kernel 慢是因为没优化。」——错。它故意 scalar + 小数据; 结构对齐 FA3, 不 chase TFLOPS。

Key insight. 读 FA3 论文时把 Algorithm 1 (producer-consumer + 串行 softmax) 与 Algorithm 2 (2-stage overlap) 分开画 **timeline**——前者对应 warp specialization 收益, 后者对应 SFU/TC 重叠收益。Ablation 表显示两者叠加。

v3 之后: Blackwell 与 FA4 方向

面试常问“下一步是什么”——简要路线图:

- **Blackwell (SM100)**: FP4/MXFP4 tensor core、更大 TMA throughput、进一步 warp specialization。
- **Split-K / Stream-K** 与 attention 结合——超长 N 时 grid utilization。
- **Paged KV cache** inference 路径与 TMA 的 descriptor 更新 (vLLM 集成)。
- 跨 **node attention** (DeepSeek MLA 等)——Hopper NVLink + cluster 不够, 需算法层压缩 $\frac{K}{V}$ 。

Note. 截至 2024 论文, FlashAttention-3 已开源并计划集成 PyTorch。Blackwell 专用 fork 可能在 CUTLASS 3.x 主线——关注 wgmma → 下一代 MMA 指令名变更。

面试考点

面试考点. **Q1:** Hopper 三大特性 (WGMMA / TMA / FP8) 分别解决什么?

A: WGMMA——warpgroup 异步 tensor core GEMM, 更大 tile + 与 softmax 重叠; TMA——硬件 async bulk copy, 把 load 从 consumer 剥离、减轻 issue 压力; FP8——tensor core 吞吐 2×, 配合 block quant 保精度。

面试考点. **Q2:** 为什么 FA3 需要 producer-consumer, FA2 不需要?

A: FA2 用 `cp.async` 全员参与 load + `mma.sync` 同步 GEMM, load 与 compute 争用 issue slot, 且无法低 register 专职 TMA。FA3 的 TMA 只需少数 thread, producer 低 reg / consumer 高 reg, pipeline 更深。

面试考点. **Q3**: GEMM-softmax overlap 的具体机制? 为什么 FA2 做不到?

A: 2-stage register pipeline : async WGMMA 发起 $QK_{\{j+1\}}^T$ 不 wait , 同时 softmax 处理已完成的 S_j ; /pingpong 两 warpgroup 交替 GEMM 与 softmax. FA2 的 `mma.sync` 和 `wait` 语句强制串行。

面试考点. **Q4**: `mbarrier` 与 `__syncthreads` 在 FA3 pipeline 里各管什么?

A: `mbarrier` 管 TMA async complete (transaction count + phase flip) 和 smem stage 释放 ; `__syncthreads` 管 CTA 内所有 thread 视图一致。WGMMA 用 `wait_group` 单独 wait。三者不可混用语义。

面试考点. **Q5**: TMA 比 `cp.async` 强在哪?

A: 独立硬件单元 ; 1 thread 发 multidim bulk copy ; descriptor 处理 boundary ; 与 `mbarrier` 原生集成 ; consumer warpgroup 完全不参与地址计算。

面试考点. **Q6**: FP8 attention 的精度陷阱?

A: Per-tensor scale 对 outlier 失效 ; V layout 与 WGMMA k-major 不匹配 ; P 的 acc layout 与 GEMM1 operand layout 冲突 ; causal + FP8 可能 slower/less accurate. Mitigation : block quant + incoherent processing + FP32 softmax state。

面试考点. **Q7**: FA v1 / v2 / v3 性能数量级?

A: v1/v2 Ampere 280-340 TFLOPS on H100 (35% peak); v3 FP16 740 TFLOPS (75%); v3 FP8 1.2 PFLOPS。v3 vs v2 forward 1.5-2×。

面试考点. **Q8**: WGMMA 与 `mma.sync` 的核心语义差异?

A: WGMMA 异步 (`commit/wait_group`), warpgroup 128 threads ; `mma.sync` 同步完成 , warp 32 threads。WGMMA 支持 FP8 与更大 mma shape。

面试考点. **Q9**: 本书 `flash_attention_v3_pipeline_kernel` 在教什么?

A: K/V 双 stage smem ping-pong + 预取下一 key——对应 FA3 的 TMA circular buffer 与 release/acquire 协议 ; scalar 版无 TMA/mbarrier , 但 pipeline 形状与 Algorithm 2 同构。

面试考点. **Q10**: v3 之后可能方向?

A: Blackwell FP4、attention+GEMM 更深 fuse、paged KV TMA、MLA/压缩 KV 算法层、跨 GPU sequence parallel。硬件迭代继续放大 asynchrony + low-precision 趋势。

面试考点. **Q11**: Thread Block Cluster 和 DSMEM 是什么? FA3 用了吗?

A: Cluster = 最多 16 个 CTA 组 , DSMEM = 跨 CTA 访问 smem 的能力。FA3 主 forward 不依赖 DSMEM ; 知道定义用于 Hopper 综合题。

面试考点. **Q12**: 为什么 FP8 WGMMMA 要求 k-major? 对 V 有什么影响?

A: Hopper FP8 tensor core 硬件约束; V 默认 head-contiguous 是 mn-major, 需 in-kernel transpose 或 GMEM 预转置。FP16 两种 layout 均可, 故 FP8 工程更复杂。

面试考点. **Q13**: pingpong warpgroup 和 2-stage register pipeline 区别?

A: 2-stage 在同一 warpgroup 内用 async WGMMMA 与 softmax 重叠; pingpong 用两个 warpgroup 交替 GEMM 与 softmax (bar.sync 协调)。可叠加。

面试考点. **Q14**: Standard attention $3\text{--}16\times$ 慢于 FA3 的主要原因?

A: Materialize S, P 到 HBM—— $O(N^2)$ 读写; FA 系列 $O(N)$ HBM for activations。FLOPs 相同, IO 不同。

从 teaching kernel 到 production FA3

若你要在 H100 上追 FA3 性能, recommended 路径:

Step 1: 读 CUTLASS 3 `examples/57_hopper_gemm` 与 FlashAttention-3 repo 的 `hopper` 目录——先看 TMA descriptor 构造与 WGMMMA mainloop 分离。

Step 2: 用 `flash_attention_v3_pipeline_kernel` 理解 smem ring buffer——把 `__syncthreads` 换成 mbarrier 是第一步 hop。

Step 3: profile 标量 teaching kernel 确认 online merge 正确——再换 WGMMMA, 不要同时 debug 算法与 Hopper 指令。

Step 4: ncu 对比 FA2 vs FA3 官方 benchmark——看 `pipe_tensor`、`pipe_sfu`、`tma_throughput` 三角是否同时高。

Step 5: FP8 最后上——先 FP16 路径 overlap 调满, 再加 block quant / transpose / byte_perm。

Note. 手写完整 FA3 不现实 (CUTLASS + 数万行 tuning)。面试目标: 讲清三大创新 + 画 timeline + 报出 **740/1200 TFLOPS** 数量级 + 知道 **FP8 layout** 坑。实现细节指向官方 repo 即可。

全书正文在此结束——附录含环境配置与论文链接。祝你在 kernel 面试里, 从 vector add 的合并访问一直讲到 Hopper 的 async warpgroup, 一条 **ladder** 走到底。

附录 A：参考资料

硬件参考卡 (A100 80GB SXM4, 本书实测环境)

- Compute capability: 8.0
- SM 数量: 108
- 每 SM 最大 warp 数: 64 (2048 threads)
- 每 SM 寄存器: $65,536 \times 32\text{-bit}$
- 每 SM shared memory: 164 KB (配置 L1 = 28 KB 时; 总 192 KB)
- L2 cache: 40 MB
- HBM 带宽: 2039 GB/s
- FP32 峰值: 19.5 TFLOPS
- FP16 tensor core: 312 TFLOPS
- BF16 tensor core: 312 TFLOPS
- TF32 tensor core: 156 TFLOPS

H100 SXM5 (Hopper, 仅供 Ch.10 参考)

- Compute capability: 9.0
- SM 数量: 132
- HBM3 带宽: 3350 GB/s
- FP8 tensor core: 1979 TFLOPS
- FP16 tensor core: 989 TFLOPS
- TMA (Tensor Memory Accelerator)
- Thread Block Cluster + Distributed Shared Memory

常用 nvcc flag

- `-arch=sm_80` : 目标 CC。
- `-lineinfo` : 让 `ncu` 能对到源码行。
- `--ptxas-options=-v` : 打印每个 kernel 的寄存器 / smem / spill 用量。
- `-Xptxas -dlcm=ca` : L1 cache 策略 (default `ca` = cache all)。
- `-maxrregcount=N` : 全局限制寄存器数 (会强迫 spill)。
- `--use_fast_math` : 让 SFU/MUFU 用近似路径 (例如 `sinf`、`__expf`)。
- `-Xcompiler -fPIC` : 需要生成 shared library 时。

参考资料

- CUDA C++ Programming Guide (latest)
- CUDA C++ Best Practices Guide
- PTX ISA
- Nsight Compute Kernel Profiling Guide (也可以看附录 B)
- Milakov & Gimelshein, “Online normalizer calculation for softmax”, 2018
- Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”, 2022

- Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning”, 2023
- Shah et al., “FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision”, 2024
- CUTLASS 文档与源码
- Simon Boehm, “How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance”
- Lei Mao 的 CUDA 系列 blog

附录 B : Nsight Compute 使用指南

这本书里所有的性能数字都来自 `ncu` (Nsight Compute)。这个附录告诉你怎么用它。

为什么需要 `ncu`

Wall-time (用 CUDA event / host timer 测的 kernel 时间) 能告诉你“这个 kernel 花了多久”，但不能告诉你：

- 它慢的原因是访存慢、计算慢、还是 stall 在 barrier？
- 它把 HBM 用到多少、SM 用到多少？
- 它有没有 bank conflict、warp divergence、register spill？
- 上一版和这一版的性能差距是从哪里来的？

`ncu` 是 NVIDIA 官方的 CUDA kernel profiler，能通过硬件性能计数器 (hardware performance counters) 抓到几百个 metric。它有两种模式：

- 交互式 **GUI**：`ncu-ui`，能看火焰图、SASS 高亮、metric roofline。适合分析单个 kernel。
- 命令行：`ncu`，能被 shell / Makefile 自动化。适合 CI / regression。

本书所有的实测数字都是命令行模式抓的。

权限

在很多云 / 共享环境上，`ncu` 会报：

```
ERR_NVGPUCTRPERM - The user does not have permission to access
NVIDIA GPU Performance Counters on the target device 0.
```

原因是驱动里 `RmProfilingAdminOnly=1` (默认值)。解决办法：

1. **sudo** 直接跑 (如果你有 sudo 权限)：

```
sudo -n ncu -k regex:my_kernel ./my_binary
```

2. 全局放开 (需要重启 nvidia-persistenced 或 reboot)：

```
sudo nvidia-smi -q -d SUPPORTED_CLOCKS # 确认驱动版本
# /etc/modprobe.d/nvidia.conf 里加：
options nvidia NVreg_RestrictProfilingToAdminUsers=0
```

3. **WSL / Docker**：需要以 `--privileged` 或加 `--cap-add=SYS_ADMIN` 运行容器。

Google Colab / 大部分云 notebook 没有这个权限，`ncu` 完全用不了。

命令行基本用法

最简：抓所有 **kernel** 的默认 **metric**

```
ncu ./my_binary
```

会打印每个 kernel 的 “SpeedOfLight” section (HBM 用率、SM 用率、TC 用率), 大约每 kernel 输出 30 行。

过滤到特定 **kernel**

```
ncu -k regex:my_kernel_v2 ./my_binary
```

regex: 前缀让 -k 接受正则; 不加就要 exact match, 很不方便。

只跑前 **N** 次 **launch**

如果一个 kernel 被调用很多次 (比如 batch loop), profile 每次都会拖慢很多:

```
ncu --launch-count 3 -k regex:my_kernel ./my_binary
```

抓指定 **metric**, 输出 **CSV**

```
ncu -k regex:my_kernel \
  --csv \
  --log-file out.csv \
  --metrics gpu__time_duration.sum,dram__bytes.sum,pct_of_peak_sustained_elapsed \
  ./my_binary
```

这是我们 book 里 bench/run_bench.py 用的方式。

抓完整 **section** (比命令行 **metric** 全)

```
ncu --set full -k regex:my_kernel --export report ./my_binary
# 然后:
ncu-ui report.ncu-rep
```

full set 会抓所有 section (约 30 个) + SASS 采样。慢, 但一次搞定。

Section vs. Metric

ncu 的 metric 分两个层级:

- **Section**: 一组相关 metric 的集合, 对应 GUI 里一个可折叠 panel。例如 SpeedOfLight = HBM/SM/TC 用率概览; Occupancy = warp 数量 / 寄存器 / smem 三个维度的 occupancy 分析。
- **Metric**: 单个数值指标, 例如 dram__bytes.sum.pct_of_peak_sustained_elapsed。

命令行 --section 加 section 名一次抓整组, --metrics 加逗号分隔的 metric 名精确控制。

常用 section:

Section 名	用来看什么
SpeedOfLight	第一眼：memory-bound 还是 compute-bound
LaunchStats	grid/block/register/smem 配置
Occupancy	warp 数是被谁限制的（reg / smem / block dim）
SchedulerStats	warp 有多少 issue slot 空转
WarpStateStats	warp stall 的原因分类（LG throttle / MIO / Barrier ...）
SourceCounters	热点行 + 每行的 stall 原因（需 -lineinfo）
InstructionStats	SASS 指令 mix（FMA / LDG / STG / SHFL ...）
MemoryWorkloadAnalysis	L1/L2/HBM 的字节数、hit rate、bank conflict
ComputeWorkloadAnalysis	FMA / ALU / SFU / MUFU / tensor pipe 各自的忙度

表 86 **Table:** Nsight Compute 常用 section 名与用途。Section 名对应 `ncu --section` 或 GUI panel 名；用来看什么 列说明第一眼 diagnostic 目标（SpeedOfLight、LaunchStats、Occupancy 等）。

Observation :先用 **SpeedOfLight** 定性——HBM % vs TC % vs SM % 决定往下挖 Occupancy 还是 MemoryWorkloadAnalysis；本书 bench 表里的 TC %/HBM % 多来自 SpeedOfLight 或等价 metric。

Metric 命名规范

`ncu` 的 metric 名看起来像天书，其实是有语法的：

```
<pipeline>__<subject>.<statistic>[.<qualifier>]
```

拆解 `dram__bytes.sum.pct_of_peak_sustained_elapsed`：

- `dram` = pipeline，指 HBM 控制器
- `bytes` = subject，指字节数
- `.sum` = statistic，把所有 SM 上的量加起来（也可以是 `.avg`，`.max`，`.per_second`）
- `.pct_of_peak_sustained_elapsed` = qualifier，占硬件持续峰值的百分比（分母是 elapsed wall time）

其它常见 pipeline 前缀：

前缀	含义
<code>gpu__</code>	整个 GPU 层面（例如 <code>gpu__time_duration.sum</code> kernel 时间）
<code>sm__</code>	SM 层面聚合（多个 SM 的和 / 均值）
<code>smsp__</code>	SM sub-partition 层面（一个 SM 有 4 个 sub-partition，各自有 warp scheduler）
<code>l1tex__</code>	L1 texture cache（also 处理 shared memory）
<code>lts__</code>	L2 cache（“large text slice”，历史命名）
<code>dram__</code>	HBM DRAM
<code>launch__</code>	launch config（grid, block, regs, smem）

表 87 **Table:** `ncu` metric 名的 pipeline 前缀含义。前缀 `gpu__` / `sm__` / `smsp__` / `l1tex__` / `lts__` / `dram__` / `launch__` 对应聚合层级；读 metric 时需先看前缀再读 subject 与 statistic。

Observation : 同一次 **kernel profile** 要串三层—— `launch__` 看配置是否合理, `dram__/lltex__` 看访存, `sm__/smsp__` 看 compute 与 warp stall; 本书 ch4/ch10 的 `issued/32` 来自 `smsp__` 层。

内存-bound kernel 该抓什么

```
--metrics \  
  gpu__time_duration.sum,\  
  dram__bytes.sum,\  
  dram__bytes.sum.pct_of_peak_sustained_elapsed,\  
  sm__cycles_active.avg.pct_of_peak_sustained_elapsed,\  
  lltex__t_bytes_pipe_lsu_mem_global_op_ld.sum,\  
  smsp__average_warps_issue_stalled_long_scoreboard_per_issue_active.ratio
```

看 :

- `dram__bytes.sum / time` = 实测 HBM GB/s
- `dram__bytes.sum.pct_of_peak_sustained_elapsed` = HBM %
- 如果 HBM % 低但 kernel 慢, `long_scoreboard` stall 高说明 memory latency 掩盖不住(warp 不够多)
- `lltex bytes` 高但 `dram bytes` 低说明 L2 命中好——micro-benchmark 上常见(本书 `vector_add` 章里就踩过)

Compute-bound kernel 该抓什么

```
--metrics \  
  gpu__time_duration.sum,\  
  sm__pipe_tensor_cycles_active.avg.pct_of_peak_sustained_elapsed,\  
  smsp__sass_thread_inst_executed_op_ffma_pred_on.sum,\  
  sm__warps_active.avg.pct_of_peak_sustained_elapsed,\  
  lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum,\  
  smsp__average_warps_issue_stalled_barrier_per_issue_active.ratio
```

看 :

- `sm__pipe_tensor_cycles_active` = tensor core 忙度 (GEMM 该 > 60% 才算健康)
- `smsp__sass_thread_inst_executed_op_ffma` = CUDA core FMA 计数(tensor core kernel 上会低)
- `sm__warps_active` = warp 占用率
- `bank_conflicts_pipe_lsu_mem_shared_op_ld` = shared memory bank conflict 次数
- `stalled_barrier` 高 = `__syncthreads()` 等太久

Roofline 视图

`ncu-ui` 里有一个 “Roofline” tab, 能自动画 :

- 硬件 peak (HBM 斜线 + Compute 平线)
- 你的 kernel 落在哪个点上

规则 :

- 落在 memory line 上 → memory-bound, 优化访存

- 落在 compute line 上 → compute-bound , 优化 FLOP 效率
- 两者中间 → mixed , 两边都能优化

命令行 `ncu --set roofline` 抓所需 metric , `--export` 出报告后在 GUI 里看。

常见坑

1. **ncu** 让 **kernel** 变慢很多。抓 metric 会让 kernel 慢 2-100x (因为要多次 replay 采样)。别用 wall-time 看进程整体 , 用 `gpu__time_duration.sum` 单个 kernel 的时间。
2. **L2 cache** 让 **GB/s** 虚高。数据集比 40MB 小时 , 逻辑 GB/s 远超实测 HBM GB/s。看 `dram__bytes.sum` 才准。
3. 小 **kernel** 的 **pct_of_peak** 偏低。kernel 短于 100 μ s 时 , 启动 / 结束窗口会把 pct 拉低 , 但 effective GB/s 是准的。
4. `--kernel-name my_k` 不匹配。默认是 exact , 要加 `regex:` 前缀。
5. **Volta** 以下不支持。ncu 只支持 Volta (sm_70) 及以上。Kepler / Pascal 得用旧的 `nvprof`。
6. 多 **kernel** 同名会各 **profile** 一次。想只抓一次 , 加 `--launch-count 1`。

本书 **bench** 目录布局

```
book/bench/
  Makefile           # 每章一个 target
  run_bench.py       # 通用 ncu 包装
  common.sh          # sudo ncu wrapper
  01_vector_add.csv  # ncu 原始输出
  01_vector_add.md   # 人可读表格
  01_vector_add.typ  # typst 表格 ( 章节里 include )
  ...
```

要重跑某一章的 bench :

```
cd book/bench && make 01 # 或 02, 03, ...
```

会自动重新编译 kernel、跑 ncu、更新表。然后 `typst compile book.typ book.pdf` 会拿到新数字。

附录 C : CUDA 性能优化思路总结

这是一份跨章节的方法论 recap——把书里 10 章零散讲的优化技巧收成一份“该按什么顺序想问题”的 checklist，同时给出每个技巧的触发条件和量化收益。

面试里被问“你怎么优化一个 kernel”，最好的答案是照着这个流程讲。

第 0 步：定性 —— 定位瓶颈

在写任何优化之前，先回答一个问题：这个 **kernel** 是 **memory-bound** 还是 **compute-bound**？

方法：算 arithmetic intensity (AI)。

$$AI = \frac{\text{useful FLOPs per launch}}{\text{bytes moved between HBM and SM per launch}}$$

和硬件 ridge point 比较：

- A100 FP32 : ridge = $19.5 \frac{\text{TFLOPS}}{2.04} \text{ TB/s} = 13 \text{ FLOP/B}$
- A100 tensor core FP16 : $\frac{312}{1.5} = 208 \text{ FLOP/B}$
- H100 tensor core FP8 : $\sim \frac{2000}{3.3} \approx 606 \text{ FLOP/B}$

规则：

- $AI \ll \text{ridge} \rightarrow \text{memory-bound}$ (vector add, reduce, softmax, layernorm)
- $AI \gg \text{ridge} \rightarrow \text{compute-bound}$ (matmul with large tile, MLP prefill)
- $AI \approx \text{ridge} \rightarrow \text{mixed}$ ，两条路都能优化

`ncu --set roofline` 会自动把 kernel 画到 roofline 图上，一眼看出属于哪种。

Key insight. 瓶颈定位错，后面所有优化都白做。memory-bound kernel 上加 tensor core 一点用没有；compute-bound kernel 上做 float4 也基本没用。

Memory-bound 优化路径

从“能压满 HBM 带宽”这个终极目标出发倒推：

第 1 步：合并访问 (coalesced access)

触发条件：任何全局内存读写。

怎么做：让同一个 warp 里 32 个 lane 访问连续的 128 字节。用 `blockDim.x * gridDim.x` 的 stride，而不是每 thread 处理连续 K 个元素。

收益：破坏合并访问会让带宽掉到 1/32。做对了是“及格线”，没什么加分。

本书出现位置：Ch.1 vector_add naive、Ch.4 matmul 的 B 矩阵访问模式。

第 2 步：向量化 load/store

触发条件：kernel 已合并访问但 $\text{HBM \%} < 80\%$ 。

怎么做：`float4 / int4 / half8`，一次读写 128 bit (`LDG.E.128 / STG.E.128`)，要求 16 字节对齐；`cudaMalloc` 保证数组头对齐。

收益：

- HBM 已经打满时：0-5% (vector add 就是这个情况)
- HBM 还没满且瓶颈是 MSHR 或 fetch pipeline：20-50%

本书出现位置：Ch.1 v4/v5、Ch.3 softmax vectorized、Ch.5 layernorm vectorized。

第 3 步：Occupancy 检查

触发条件：ncu 报告 `sm__warps_active` < 50% 或 `long_scoreboard` stall 高。

怎么做：

- 减少每 thread 寄存器 (用 `__launch_bounds__` 或 `maxrregcount` 强制)
- 减少 shared memory per block
- 调 block size 让 SM 放得下更多 block

收益：让 SM 上驻留更多 warp 来掩盖 memory latency。memory-bound kernel 上从 25% occ 涨到 50% 常见提升 30%+。但 occupancy > 50% 后收益递减，别死追 100%。

本书出现位置：Ch.4 matmul (寄存器压力 vs occupancy 权衡)。

第 4 步：Grid-stride / block-per-row / warp-per-row 的选择

触发条件：kernel 的问题结构 (vector / batch of rows / matrix) 决定分工。

怎么做：

- 数据 1D 且规模 >> SM 数量：**grid-stride**
- 每行独立且 H 中等 (≤ 1024)：**block-per-row**，用 shared memory reduction
- 每行独立且 H 小 (≤ 32)：**warp-per-row**，用 warp shuffle

收益：分工选对 vs 选错常见 2-10x 差距 (Ch.2 reduce naive vs warp shuffle)。

本书出现位置：Ch.2 reduce、Ch.3 softmax、Ch.5 layernorm。

第 5 步：Warp shuffle 替代 shared memory reduction

触发条件：block 内需要 reduction，且规模 ≤ 32 (warp 大小)。

怎么做：`__shfl_down_sync` / `__shfl_xor_sync`，寄存器内部直接交换，跳过 shared memory。

收益：

- 少一次 smem 写 + 一次 smem 读
- 少一次 `__syncthreads`
- 常见 20-40% 的 reduction 阶段加速

硬件：Volta 起，shuffle 走 register file crossbar，不占 smem port。

本书出现位置：Ch.2 v6、Ch.3 softmax warp、Ch.5 layernorm warp。

Compute-bound 优化路径

第 1 步：Tiling → 提升 AI

触发条件：AI 太低 (例如 naive matmul 的 0.25 FLOP/B)。

怎么做：把数据分块 (tile) 载入 shared memory / registers，让一个 tile 被复用多次。

- $BM \times BK$ 的 A tile + $BK \times BN$ 的 B tile，一个 CTA 计算 $BM \times BN$ 的输出
- AI 提升到 $\frac{2 BM BN BK}{2 BK (BM+BN)} = \frac{BM BN}{BM+BN}$
- $BM=BN=128$ 时 $AI = 64$ FLOP/B，远超 ridge → 转为 compute-bound

收益：本书 Ch.4 matmul naive → tiled 加速 30-100x。

本书出现位置：Ch.4 matmul、Ch.8-10 flash-attention。

第 2 步：Register tiling / thread tiling

触发条件：shared memory tile 已存在，但 tensor core 或 CUDA FMA 用率仍低。

怎么做：让一个 thread 处理 $TM \times TN$ 个输出元素，中间结果全在寄存器；shared memory tile 只被读一次到寄存器，然后计算完全从寄存器出。

收益：进一步减少 smem load，让 FFMA/HMMA 流水线占满。Ch.4 v4 中让 tensor pipe 从 20% 涨到 60%+。

第 3 步：Tensor Core (WMMA / mma.sync)

触发条件：数据类型是 FP16 / BF16 / TF32 / FP8 / INT8。

怎么做：

- Ampere 起：mma.sync.aligned.m16n8k16 或 CUTLASS 抽象
- Hopper 起：WGMMMA (wgmma.mma_async)，异步
- 注意 fragment layout (ldmatrix 指令搬 smem → mma 的 register layout)

收益：FP16 vs FP32 tensor core throughput 差 16x (A100 : 312 vs 19.5 TFLOPS)。

本书出现位置：Ch.4 讨论，Ch.10 flash-attention v3 详细展开。

第 4 步：Async copy (cp.async / TMA)

触发条件：数据搬运和计算可以重叠。

怎么做：

- Ampere: cp.async.cg.shared.global 直接从 HBM 异步搬到 shared memory，不经过 register。配 cp.async.commit_group + cp.async.wait_group 做流水线。
- Hopper: TMA (cp.async.bulk.tensor)，硬件独立单元管，支持多维 tensor。

收益：GEMM 里 double-buffered pipeline 让 memory 完全和 compute 重叠。Ch.4 pipeline kernel 20-30% 加速。

本书出现位置：Ch.4 v5、Ch.10 flash-attention v3。

第 5 步：Warp specialization (producer-consumer)

触发条件：Hopper 硬件，且 kernel 里有明显的“搬运 vs 计算”两阶段。

怎么做：一部分 warp 专门做 TMA (producer)，另一部分做 wgmma (consumer)，用 mbarrier 同步。

收益：Flash-Attention v3 相比 v2 加速 1.5-2x (H100 FP16)。

本书出现位置：Ch.10 flash-attention v3。

通用路径：Kernel fusion

触发条件：多个 kernel 依次调用，中间结果只被下一个 kernel 用 (不 export 出去)。

怎么做：

- Elementwise fusion：几个 pointwise op 合成一个 kernel (PyTorch torch.compile 自动)
- Epilogue fusion：GEMM 出结果后立刻在寄存器里做 bias / activation / scale
- Full fusion：整段 attention 合成一个 kernel (flash-attention)

收益：

- 省 kernel launch overhead：每次 5 μ s
- 省中间结果 HBM 读写：AI 提升，可能从 memory-bound 转为 compute-bound
- Attention 的 fused kernel (flash) 省了 $O(N^2)$ 中间存储

本书出现位置：Ch.1 v5 (小规模) Ch.6 MLP、Ch.8-10 flash-attention。

通用路径：数值精度

- FP32 是安全但慢
- FP16 accumulate FP32 是训练标配 (AMP)
- BF16 accumulate FP32 更稳 (LLM 训练)
- FP8 / MXFP4 需要 block scaling 保精度

关注点：

- Softmax 的 `subtract max` 必须做，不然 exp overflow
- LayerNorm 累加用 fp32，输入输出用 fp16/bf16
- Flash-Attention v3 里 P 矩阵的 FP8 量化范围需要 rescale

本书出现位置：Ch.3、Ch.5、Ch.10。

决策树

面试遇到“怎么优化 kernel X”时，按下面顺序问：

1. AI 是多少？**roofline** 落哪？
2. Memory-bound：a. 合并访问对吗？b. HBM % 到多少了？(`ncu` 抓 `dram__bytes.pct`) c. 尝试 float4？d. Occupancy 够吗？e. Reduction 用 warp shuffle 了吗？
3. Compute-bound：a. 有 shared memory tile 吗？b. Register tile 到 $TM \times TN$ 了吗？c. 用 tensor core 了吗？(数据类型能用 FP16/BF16 吗？) d. 用 `cp.async` / TMA 做双缓冲了吗？e. Bank conflict 消掉了吗？(swizzle)
4. 通用：a. 能 fuse 上下游 kernel 吗？b. 精度可以降低吗？(FP32 $\rightarrow$ FP16 $\rightarrow$ FP8) c. Launch overhead 显著吗？(CUDA Graph)

常见误区

Warning. 不测就优化。凭直觉写“更快”的版本，实测常常慢。本书 `vector_add` 章 tiled 版本就慢于 naive。

Warning. 追 100% occupancy。occupancy 50-75% 通常最优；100% 意味着寄存器用得太多，往往拖慢 compute。

Warning. 盯着 HBM % 优化。HBM % 只是“有多少 cycle DRAM 在传数据”的比例，短 kernel 上会低估。实测 effective GB/s 更可靠。

Warning. 盲目 **vectorize**。数据不对齐 / 访问模式不连续时，float4 会静默降级到多个窄访问，甚至更慢。

Warning. 忽略 **numerical stability**。FP16 累加 softmax / layernorm 会 blow up。省时间但训练崩。

收尾

优化的正确姿势：

1. 有 baseline（正确、能跑）。
2. ncu 抓 metric 找瓶颈。
3. 一次只改一件事。
4. 再 ncu 抓，对比。
5. 如果没变快甚至变慢，回滚，猜错了。
6. 如果变快了，思考“为什么”——把它写成注释 / 文档，别让下一个人重犯。

这也是这本书每一章的写法：给出 ladder 上每一版的实测数字，让你看到什么优化真有用、什么优化只是“看起来聪明”。

面试高频算子优化题目

本书正文覆盖了 vector add / reduce / softmax / matmul / layernorm / mlp / attention / flash-attention 八类。面试中还有一批高频题目正文没展开，本附录列出其中 **15** 道最常问的，每道给出：

- 问题定义（形状、约束）
- **ladder**（naive → 最优的关键步骤）
- 核心考点（面试官会追问什么）
- 参考实测（可预期的 HBM% / 加速比数量级——写代码验证再上机）

规则：面试官问一道题，你的第一句话不应该是“写代码”，应该是“这是 **memory-bound** 还是 **compute-bound**？” + “访问模式是不是 **coalesced**？”然后再看要不要 tile / share / shuffle / async。

1. transpose (matrix transpose)

问题：给定 $M \times N$ 矩阵 A ，计算 $B = A^T$ 。

为什么难：读 A 是 coalesced（沿行），写 B 就是 strided（跨列），或反之——读写不能同时 coalesced。

Ladder：

1. **Naive**： $B[j, i] = A[i, j]$ 。读 coalesced、写 strided → 写变 32 独立 transaction，掉到 $\frac{1}{32}$ 带宽。
2. **Shared memory tile**：block 拷贝 32×32 tile 到 smem，改成 $\text{smem}[\text{threadIdx.y}, \text{threadIdx.x}] = A[i, j]$ （读 coalesced），`__syncthreads`，然后 $B[j', i'] = \text{smem}[\text{threadIdx.x}, \text{threadIdx.y}]$ （写 coalesced）。
3. **Padded smem**（关键）：`__shared__ float smem[32][33]`（第二维 +1）。原因： $\text{smem}[\text{threadIdx.x}, \text{threadIdx.y}]$ ——warp 内 32 个 lane 沿列访问，如果第二维 = 32 会全 32-way bank conflict。加 1 让每行错开 1 个 bank。

参考代码（tiled + padded，面试白板版）：

```
constexpr int TILE = 32;

__global__ void transpose_tiled(const float* A, float* B, int M, int N) {
    // +1 padding 打散 bank conflict—smem 每行 33 个 float，
    // lane 沿列访问时  $\text{addr} \% 32 = (\text{row} * 33 + \text{col}) \% 32$  全不同。
    __shared__ float smem[TILE][TILE + 1];

    int row_in = blockIdx.y * TILE + threadIdx.y; // A 行
    int col_in = blockIdx.x * TILE + threadIdx.x; // A 列
    if (row_in < M && col_in < N) {
        // 读 A: warp 内 lane 沿 col_in (threadIdx.x) 连续 → coalesced。
        smem[threadIdx.y][threadIdx.x] = A[row_in * N + col_in];
    }
    __syncthreads();

    int row_out = blockIdx.x * TILE + threadIdx.y; // B 行 = A 列所在 tile
```

```

int col_out = blockIdx.y * TILE + threadIdx.x; // B 列
if (row_out < N && col_out < M) {
    // 写 B:threadIdx.x 沿 col_out 连续 → coalesced。
    // 读 smem 时 threadIdx.x 变第一维 → 沿列, 靠 +1 padding 避冲突。
    B[row_out * M + col_out] = smem[threadIdx.x][threadIdx.y];
}
}

// launch: dim3 grid((N+31)/32, (M+31)/32); dim3 block(32, 32);

```

核心考点：

- “**Naive transpose** 为什么慢？” → 写侧 strided, $\frac{1}{32}$ 带宽。ncu 证据：`lltex__t_sectors_pipe_lsu_mem_global_op_st.sum` 会比 coalesced 高 32×。
- “**Padded smem** 里的 +1 是什么？” → bank conflict 消除。metric 证据：`lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum` 从数千降到 0。
- “能不能用 **swizzle** 代替 **padding**？” → 可以。padding 浪费 3% smem, swizzle 是异或索引 (`col ^= (row & 0x1f)`), 无空间开销, Hopper 上有硬件 swizzle 模式。

参考实测 (A100, $M = N = 4096$): naive $\frac{1}{30}$ peak; tiled+padding 0.75 peak。

2. reduce max / argmax

问题：给数组 x , 求 $\max_i x_i$, 或返回 $\arg \max_i x_i$ 。

和 **reduce sum** 的关键差别：max 无逆运算，不能用 **atomicMax + subtract-max trick**；argmax 需要同时传递 index 和 value——warp shuffle 一次 shuffle 只搬 32-bit, argmax 要么用 int64 打包 (value + index), 要么两次 shuffle。

Ladder: sum 版的 ladder 全套复用 (atomic → shared tree → warp shuffle → chunked hierarchical), 加两个 argmax 特殊技巧：

- 打包 **index** 到低位：`packed = (value_bits << 32) | index` (value 是 int/float bit pattern, index 是 uint32) → `atomicMax` 一次搞定。float 需要处理负数 bit pattern 单调性 (把符号位翻转再比较)。
- **warp shuffle** 里传两个值：两次 `__shfl_down_sync`, 先 shuffle value 再 shuffle 对应 index；或用 CUB `WarpReduce<KeyValuePair>`。

参考代码 (warp-level argmax, 同时 shuffle value 和 index)：

```

struct MaxIdx { float v; int i; };

__device__ MaxIdx warp_argmax(MaxIdx x) {
    #pragma unroll
    for (int offset = 16; offset > 0; offset >>= 1) {
        float other_v = __shfl_down_sync(0xffffffff, x.v, offset);
        int other_i = __shfl_down_sync(0xffffffff, x.i, offset);
        // Tie break: 取更小 index (reproducibility)
        if (other_v > x.v || (other_v == x.v && other_i < x.i)) {
            x.v = other_v;
            x.i = other_i;
        }
    }
}

```

```

    }
    return x; // lane 0 持有 warp 内 argmax
}

__global__ void argmax_kernel(const float* x, int N, MaxIdx* out) {
    MaxIdx v{-FLT_MAX, -1};
    // grid-stride load, 每 thread 先本地累计一个 argmax
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;
         i < N; i += gridDim.x * blockDim.x) {
        if (x[i] > v.v) v = {x[i], i};
    }
    v = warp_argmax(v);

    __shared__ MaxIdx warp_maxes[32];
    int lane = threadIdx.x & 31;
    int wid = threadIdx.x >> 5;
    if (lane == 0) warp_maxes[wid] = v;
    __syncthreads();

    if (wid == 0) {
        v = (lane < blockDim.x / 32) ? warp_maxes[lane] : MaxIdx{-FLT_MAX, -1};
        v = warp_argmax(v);
        if (lane == 0) out[blockIdx.x] = v; // 每 block 一个 partial argmax, 第二 stage
再 reduce
    }
}

```

核心考点：

- “**Softmax** 里的 **reduce max** 为什么可以不用 **atomic**？” → 单 row 内 reduction，用 block 级 tree + warp shuffle 就够，不需要跨 block 通信。
- “**argmax tie breaking**？” → 定义决定：取最小 index 或最大 index。打包时 index 放高位或低位，配合 max 顺序。
- “**FP16 argmax** 有什么坑？” → 相同 value 的 bit pattern 唯一，但两个不同 float 可能相等（例如 subnormal），tie breaking 必须显式。

3. cumulative sum / prefix scan

问题：给数组 $x[0..N)$ ，计算 $y[i] = \sum_{j \leq i} x[j]$ (inclusive) 或 $y[i] = \sum_{j < i} x[j]$ (exclusive)。

为什么难：天然串行。每个输出依赖前一个 → 不能像 reduce 那样“随便配对”。

Ladder：

1. 串行：单 thread 循环， $O(N)$ 但零并行度。
2. **Hillis-Steele scan**(single-pass tree)：每一步 `y[i] += y[i - stride]`， $O(N \log N)$ FLOP、 $O(\log N)$ step。适合 warp 内（`__shfl_up_sync` 直接实现）。
3. **Blelloch scan**(up-sweep + down-sweep)： $O(N)$ FLOP、 $O(\log N)$ step。适合 block/grid 级。
4. **Grid 级 scan + look-back**(CUB DeviceScan)：block 间无需 barrier，通过 partial value 广播 + look-back 完成——现代最优。

参考代码（block-level inclusive scan：warp scan + inter-warp scan）：

```

__device__ float warp_inclusive_scan(float v) {
    int lane = threadIdx.x & 31;
    #pragma unroll
    for (int s = 1; s < 32; s <= 1) {
        float t = __shfl_up_sync(0xffffffff, v, s);
        if (lane >= s) v += t;
    }
    return v;
}

__global__ void block_scan_kernel(const float* in, float* out, int N) {
    __shared__ float warp_sums[32]; // 每 warp 的 tail

    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    float v = (tid < N) ? in[tid] : 0.f;

    // Step 1: warp 内 inclusive scan.
    v = warp_inclusive_scan(v);

    int lane = threadIdx.x & 31;
    int wid = threadIdx.x >> 5;
    if (lane == 31) warp_sums[wid] = v; // 把每 warp 的 tail 存起来
    __syncthreads();

    // Step 2: warp 0 对 warp_sums 再做 scan (exclusive: 不加自身)
    if (wid == 0) {
        float w = (lane < blockDim.x / 32) ? warp_sums[lane] : 0.f;
        w = warp_inclusive_scan(w);
        warp_sums[lane] = w;
    }
    __syncthreads();

    // Step 3: 每 warp 加上前面所有 warp 的 tail (exclusive prefix)
    if (wid > 0) v += warp_sums[wid - 1];
    if (tid < N) out[tid] = v;
}

// grid 级 scan 需要再加 block partial + 第二 kernel look-back / device scan.

```

核心考点：

- “**exclusive vs inclusive**？” → inclusive 的结果右 shift 一位、第 0 位置 0 就是 exclusive。
- “**grid 级 scan 怎么并行 block 之间**？” → 每 block 输出 tail，用 CUB DeviceScan 的 decoupled look-back：每 block 发布自己的 aggregate、等前一 block 的 inclusive prefix 到达。
- “数值稳定性？” → 大数组用 Kahan compensated summation 或 pairwise reduction；scan 里较少见。

4. histogram

问题：给数组 $x[i] \in [0, K)$ ，统计 $\text{hist}[k] = |\{i : x[i] = k\}|$ 。

为什么难：多 thread 更新同一个 bin → **atomic contention**。

Ladder :

1. 全局 **atomic** : 每 thread `atomicAdd(&hist[x[i]], 1)`。如果 distribution skewed (比如 90% 的 x 集中在少数 bin), contention 灾难, 慢 $100\times$ 以上。
2. **Shared memory privatized** : 每 block 一份 `__shared__ int local_hist[K]`, block 内 `atomic` (smem atomic 比 global atomic 快 $20-50\times$), 结束时把 `local_hist` merge 到 `global hist` (一次 atomic per bin per block, contention 大幅降低)。
3. **K** 大时只能 **sample subset** 用 **smem** : $K = 1M$ 时 smem 装不下, 退回 subset caching + spill。
4. **Match+CTA reduction** (更高级): warp 内 `__ballot_sync` / `__match_any_sync` 判断哪些 lane 打同一个 bin, 先在 warp 内 reduce, 减少 atomic 次数。

参考代码 (privatized shared memory histogram):

```
constexpr int K = 256; // bin 数

__global__ void histogram_privatized(const int* x, int N, int* hist) {
    __shared__ int local_hist[K];

    // Step 1: 清空 local_hist (block 内 stride 清)
    for (int k = threadIdx.x; k < K; k += blockDim.x) local_hist[k] = 0;
    __syncthreads();

    // Step 2: block 内 grid-stride 打 smem atomic (smem atomic 快得多)
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;
         i < N; i += gridDim.x * blockDim.x) {
        atomicAdd(&local_hist[x[i]], 1);
    }
    __syncthreads();

    // Step 3: 把 local_hist merge 到 global (每 bin 一次 global atomic)
    for (int k = threadIdx.x; k < K; k += blockDim.x) {
        if (local_hist[k] > 0) atomicAdd(&hist[k], local_hist[k]);
    }
}
```

核心考点 :

- “为什么 **smem atomic** 快 $20-50\times$?” → smem atomic 走片上 lock, ~ 4 cycle; global atomic 走 L2, 延迟 $200+$ cycle。
- “如果输入是 **sorted** 的呢?” → 每 block 处理连续段, 几乎无 contention。sort-then-histogram 有时比直接 histogram 快。
- “**K** 很大 (比如 **float** 分桶) 呢?” → 先 quantize / hash 到 fixed K , 再走标准 pipeline。
- “**contention** 怎么定量看?” → `ncu smsp_average_warps_issue_stalled_long_scoreboard_per_issue_active_ratio` 会非常高 (atomic 反复等 L2)。

5. top-k

问题: 给数组 $x[0..N]$, 返回最大的 k 个 (value 和/或 index)。 k 通常 $\ll N$, 比如 $N = 10^6, k = 32$ 。

为什么难: 既不能全排序 ($O(N \log N)$) 也不能纯 reduce (无满足要求的关联运算)。

Ladder :

1. 全排序 + 取前 k : $O(N \log N)$, 可用 CUB DeviceRadixSort。 $k \ll N$ 时浪费严重。
2. **Bitonic top-k** : warp/block 内 bitonic sort 前 k + tournament , $O(N \log^2 k)$ 。 $k \leq 32$ 时非常适合 warp 内 fully-in-register。
3. **Heap-based** (threshold-based) : 每 thread 维护 size- k heap ; 输入元素比 heap 顶大才插入。适合 stream 场景。
4. **Approximate top-k** : sample + threshold estimation $\rightarrow$ 单 pass 过滤 $\rightarrow$ 精确 top-k。生产用于 LLM logit filtering。

参考代码 (warp-level top-32 via bitonic + shuffle , $k = 32$ 常见 case) :

```
// warp 内每 lane 持有一个 (value, index) , 做 5 层 bitonic sort.
__device__ void bitonic_compare_swap(float& v, int& i, int xor_lane, bool up) {
    float other_v = __shfl_xor_sync(0xffffffff, v, xor_lane);
    int other_i = __shfl_xor_sync(0xffffffff, i, xor_lane);
    int me = threadIdx.x & 31;
    bool me_low = (me & xor_lane) == 0; // 我在配对中的低位?
    // up=true 时低位保留小者、高位保留大者, 反之反过来。
    bool keep_other = (me_low != up) ? (other_v < v) : (other_v > v);
    if (keep_other) { v = other_v; i = other_i; }
}

__device__ void warp_bitonic_sort_desc(float& v, int& i) {
    // 标准 5 层 bitonic: size = 2, 4, 8, 16, 32.
    for (int size = 2; size <= 32; size <= 1) {
        bool up = ((threadIdx.x & size) == 0); // 交替升/降构造 bitonic
        for (int stride = size >> 1; stride > 0; stride >>= 1) {
            bitonic_compare_swap(v, i, stride, up);
        }
    }
    // 最终做一次全 descending 的 merge:
    for (int stride = 16; stride > 0; stride >>= 1) {
        bitonic_compare_swap(v, i, stride, false);
    }
}

__global__ void warp_top32(const float* x, int N, float* out_v, int* out_i) {
    int lane = threadIdx.x & 31;
    float v = -FLT_MAX; int i = -1;

    // 每 lane 扫自己那一路, 本地维护单个 max (k=1 版), 最后 warp 排 32 个。
    // 更严谨版: 每 lane 维护一个 size-k/32 heap。这里简化到 k=32、每 lane 拿 1 个。
    for (int t = blockIdx.x * 32 + lane; t < N; t += gridDim.x * 32) {
        if (x[t] > v) { v = x[t]; i = t; }
    }
    warp_bitonic_sort_desc(v, i);
    out_v[blockIdx.x * 32 + lane] = v;
    out_i[blockIdx.x * 32 + lane] = i;
}
```

核心考点 :

- “attention softmax 前的 top-k logit filtering 怎么做？” → 逐 row , $N = \text{vocab_size}$ (几万), $k = 32-512$; bitonic 或 approx.
- “bitonic 为什么要交替 up/down?” → 构造 bitonic sequence (先增后减) 才能一次 merge 全排。
- “top-k 和 sort 的区别?” → top-k 只保证前 k 正确, 不 care 内部顺序, 可以更快。

6. layernorm backward

问题: 给定 $y = \gamma \cdot \hat{x} + \beta$ (forward), backward 输入 $\partial_{\hat{y}}^L y$, 输出 $\partial_{\hat{x}}^L x$ 、 $\partial_{\hat{\gamma}}^L \gamma$ 、 $\partial_{\hat{\beta}}^L \beta$ 。

核心公式:

$$\frac{\partial L}{\partial x_i} = \frac{1}{\sqrt{\sigma^2 + \varepsilon}} \left[\frac{\partial L}{\partial \hat{x}_i} - \frac{1}{H} \sum_j \frac{\partial L}{\partial \hat{x}_j} - \hat{x}_i \cdot \frac{1}{H} \sum_j \frac{\partial L}{\partial \hat{x}_j} \cdot \hat{x}_j \right]$$

其中 $\partial_{\hat{\gamma}}^L \hat{x}_j = \partial_{\hat{y}}^L y_j \cdot \gamma_j$ 。

为什么难:

- 每 row 有两个全 row 的 reduction (sum of $\partial_{\hat{\gamma}}^L \hat{x}$ 、sum of $\partial_{\hat{\gamma}}^L \hat{x} \cdot \hat{x}$)
- $\partial_{\hat{\gamma}}^L \gamma$ 、 $\partial_{\hat{\beta}}^L \beta$ 是跨 row 的 reduction (sum over batch)

Ladder:

1. **Naive:** 三 pass (先两 reduction, 再逐元素计算), 每 pass 一次 kernel launch。
2. **Fused row backward:** block-per-row, 一次 kernel 完成两 reduction + 逐元素, 用 shared memory tree 或 warp shuffle。
3. **Fused param grad:** $\partial_{\hat{\gamma}}^L \gamma$ 、 $\partial_{\hat{\beta}}^L \beta$ 需要 sum over batch → 用 grid-level partial sum + 第二阶段 reduce (类似 reduce sum ladder)。
4. 保存 μ 、 σ 到 **forward**: forward pass 存 $\text{rstd} = \frac{1}{\sqrt{\sigma^2 + \varepsilon}}$ 到 buffer, backward 直接读, 省一次 reduction。

参考代码 (fused row backward for $\partial_{\hat{x}}^L x$, block-per-row):

```
// forward 时已存好 mean[row], rstd[row] = 1 / sqrt(var + eps)
__global__ void layernorm_backward_dx(
    const float* dy, const float* x, const float* gamma,
    const float* mean, const float* rstd,
    float* dx, int H) {
    int row = blockIdx.x;
    const float* xr = x + row * H;
    const float* dyr = dy + row * H;
    float* dxr = dx + row * H;
    float mu = mean[row];
    float rstd_r = rstd[row];

    // Step 1: 两个 row-level reduction:
    //   s1 = sum_j (dy_j * gamma_j)
    //   s2 = sum_j (dy_j * gamma_j) * xhat_j   其中 xhat_j = (x_j - mu) * rstd
    float s1 = 0.f, s2 = 0.f;
    for (int j = threadIdx.x; j < H; j += blockDim.x) {
        float dyhat = dyr[j] * gamma[j];
        float xhat = (xr[j] - mu) * rstd_r;
        s1 += dyhat;
    }
```

```

    s2 += dyhat * xhat;
}
s1 = block_reduce_sum(s1); // 用前面章节的 block reduce
s2 = block_reduce_sum(s2);
__shared__ float ss1, ss2;
if (threadIdx.x == 0) { ss1 = s1 / H; ss2 = s2 / H; }
__syncthreads();

// Step 2: 逐元素套公式.
for (int j = threadIdx.x; j < H; j += blockDim.x) {
    float dyhat = dyr[j] * gamma[j];
    float xhat = (xr[j] - mu) * rstd_r;
    dxr[j] = rstd_r * (dyhat - ss1 - xhat * ss2);
}
}

```

核心考点：

- “forward 存 μ, rstd 还是 μ, σ^2 ?” $\rightarrow$ 存 rstd , backward 直接乘, 省一次 sqrt。
- “ $\partial_{\theta}^L \gamma$ 的 **batch reduction** 怎么并行?” $\rightarrow$ 每 block 处理一段 $\text{batch} \times H \rightarrow$ 输出 partial $\text{gamma}[H]$, 第二 stage kernel reduce partials。
- “能不能一次 **kernel** 全做完?” $\rightarrow$ 数值上可以 (每 block 处理一个 $\text{batch} \times H$ tile, atomic 到 $\text{gamma}/\text{beta grad}$), 实践上 atomic 慢, 分 stage 更快。

7. cross-entropy loss (fused softmax + NLL)

问题：给 logits $\ell \in \mathbb{R}^{B \times V}$ 、labels $y \in \{0, \dots, V-1\}^B$, 计算 $L = -\frac{1}{B} \sum_b \log(\text{softmax}(\ell_b))_{y_b}$, 同时输出 $\partial_{\theta}^L \ell$ 。

为什么难：

- 数值稳定： $\log(\text{softmax}(x)) = x - \text{logsumexp}(x)$, 避免 $\log(\exp(\dots))$ 。
- forward + backward 常一起做 (rematerialize activation)。
- V 大到 32K-100K (LLM vocab) $\rightarrow$ 每 row 就是大 reduction。

Ladder：

1. **Naive 三 kernel**：softmax $\rightarrow$ gather label $\rightarrow$ NLL $\rightarrow$ backward exchange chunks。
2. **Fused forward**：单 kernel, per-row: $\text{max} + \text{sum_exp} + \text{log_sum_exp} \rightarrow L + \partial_{\theta}^L \ell_i = \text{softmax}(\ell)_i - [i = y]$ (就是 softmax 输出减 one-hot)。
3. **Online logsumexp** (和 online softmax 同结构)：单 pass 完成, 避免存 softmax 中间结果。
4. **Chunk over vocab**： V 极大时, 一 row 的 logits 装不下寄存器, 需要 tile over vocab dim。

参考代码 (fused forward + grad, block-per-row, online logsumexp)：

```

__global__ void cross_entropy_fused(
    const float* logits, // [B, V]
    const int* labels, // [B]
    float* loss, // [B]
    float* grad, // [B, V] = softmax - onehot(label)
    int V, float inv_B) {
    int b = blockIdx.x;
    const float* row = logits + b * V;

```



```

float*      gr  = grad  + b * V;
int label = labels[b];

// Pass 1: online reduce over V for (m, s) where m = max, s = sum exp(x - m).
float m = -FLT_MAX, s = 0.f;
for (int j = threadIdx.x; j < V; j += blockDim.x) {
    float xj = row[j];
    float new_m = fmaxf(m, xj);
    s = s * expf(m - new_m) + expf(xj - new_m);
    m = new_m;
}
// block reduce (m, s) 合并成一个 (m*, s*)—见 softmax 章节。
block_reduce_online_softmax(m, s);

float lse = m + logf(s); // logsumexp
if (threadIdx.x == 0) {
    loss[b] = (lse - row[label]) * inv_B;
}

// Pass 2: 写 grad = softmax - onehot(label), 并且乘 1/B (loss 平均)
for (int j = threadIdx.x; j < V; j += blockDim.x) {
    float p = expf(row[j] - lse);
    float g = p - (j == label ? 1.f : 0.f);
    gr[j] = g * inv_B;
}
}

```

核心考点：

- “**backward** 为什么就是 **softmax - onehot**?” → 直接对 $L = -\log(\text{softmax})_y$ 求导，交叉熵和 softmax 的耦合让 grad 变得极简。
- “ $\partial_{\theta}^L \ell$ 需要保留完整 **softmax** 吗?” → 不需要。fused backward 里重新计算 softmax (recomputation), 只花 $O(V)$ 内存开销。
- “**label smoothing** 怎么改?” → target 从 one-hot 变为 smoothed distribution, backward 变为 `softmax - smoothed_target`。

8. GELU / SwiGLU activation + backward

问题：GELU 有两个近似公式：

$$\text{GELU}(x) = x \cdot \Phi(x) \approx 0.5x \left[1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right]$$

SwiGLU 用于 LLaMA/Gemini：

$$\text{SwiGLU}(x, y) = \text{silu}(x) \cdot y = \frac{x}{1 + e^{-x}} \cdot y$$

Ladder：

1. **Vanilla element-wise**：— kernel，就是标准 vector-like，naive 就近 peak (vector-add 定律)。

2. **Fused epilogue** :把 GELU / SwiGLU 融合到 Linear 的 epilogue(cuBLASLt / CUTLASS), 省一次 read/write (output 4B 保存到 smem 后 in-place 应用)。
3. **Fused forward + backward stash** : 训练时 backward 需要 x 或 $\text{silu}(x) \rightarrow$ forward 存最合适的中间量 (存 $\text{silu}(x)$, backward 用 $\text{silu} * (1 - \text{silu}) * x + \text{silu}$ 之类)。

参考代码 (GELU tanh 近似 + SwiGLU forward , 向量化 4 元素):

```
__device__ __forceinline__ float gelu_tanh(float x) {
    constexpr float k0 = 0.7978845608f; // sqrt(2/pi)
    constexpr float k1 = 0.044715f;
    float x3 = x * x * x;
    return 0.5f * x * (1.f + tanhf(k0 * (x + k1 * x3)));
}

__global__ void gelu_kernel(const float* in, float* out, int N) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    // float4 向量化—element-wise 天然 memory-bound, v = 4 拉满带宽.
    for (int i = tid; i * 4 < N; i += gridDim.x * blockDim.x) {
        float4 x = reinterpret_cast<const float4*>(in)[i];
        float4 y;
        y.x = gelu_tanh(x.x); y.y = gelu_tanh(x.y);
        y.z = gelu_tanh(x.z); y.w = gelu_tanh(x.w);
        reinterpret_cast<float4*>(out)[i] = y;
    }
}

// SwiGLU: LLaMA FFN 用 gate 和 up 两路投影, 然后 silu(gate) * up.
__global__ void swiglu_kernel(const float* gate, const float* up,
                             float* out, int N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= N) return;
    float g = gate[i];
    float sig_g = 1.f / (1.f + expf(-g)); // sigmoid
    out[i] = (g * sig_g) * up[i];         // silu(g) * up
}
```

核心考点：

- “tanh 版 GELU 和 erf 版 GELU 精度差多少？” $\rightarrow$ tanh 版误差 $< 10^{-4}$, 训练时无差别 ; tanh + polynomial 比 erf 快 3 $\times$ 。
- “SwiGLU 的 backward ?” $\rightarrow$ 两个输入分别求导 : $\partial_{\frac{L}{\partial}} x = \partial_{\frac{L}{\partial}} \text{out} \cdot y \cdot \text{silu}'(x)$, $\partial_{\frac{L}{\partial}} y = \partial_{\frac{L}{\partial}} \text{out} \cdot \text{silu}(x)$; $\text{silu}'(x) = \text{sig}(x) + x * \text{sig}(x) * (1 - \text{sig}(x))$ 。
- “activation function 的性能瓶颈 ?” $\rightarrow$ 永远是 memory-bound。fuse 到 matmul epilogue 是唯一有意义的优化。

9. RMSNorm

问题：LayerNorm 的简化版 (LLaMA 用):

$$\text{RMS}(x) = \sqrt{\frac{1}{H} \sum_i x_i^2}, \quad y_i = \frac{x_i}{\text{RMS}(x)} \cdot \gamma_i$$

和 LayerNorm 的差别：不减 mean、不加 beta , 只做 rescale。

Ladder : 几乎与 LayerNorm 相同, 简化 :

1. 只需要一次 row reduction (sum of x^2), 而 LayerNorm 需要两次 (mean, var)
2. 反向可以更简洁: $\partial_{\frac{L}{\partial}} x_i = \frac{1}{\text{RMS}} \left[\partial_{\frac{L}{\partial}} y_i - x_i \cdot \frac{1}{H} \frac{1}{\text{RMS}^2} \sum_j x_j \cdot \partial_{\frac{L}{\partial}} y_j \right]$ 。

参考代码 (block-per-row, 一次 reduction):

```
__global__ void rmsnorm_kernel(const float* x, const float* gamma,
                               float* y, int H, float eps) {
    int row = blockIdx.x;
    const float* xr = x + row * H;
    float*      yr = y + row * H;

    // Pass 1: sum of squares.
    float sq = 0.f;
    for (int j = threadIdx.x; j < H; j += blockDim.x) {
        float v = xr[j];
        sq += v * v;
    }
    sq = block_reduce_sum(sq);

    __shared__ float rrms;
    if (threadIdx.x == 0) rrms = rsqrtf(sq / H + eps); // 1 / RMS
    __syncthreads();

    // Pass 2: normalize + scale by gamma (无 beta!)
    for (int j = threadIdx.x; j < H; j += blockDim.x) {
        yr[j] = xr[j] * rrms * gamma[j];
    }
}
```

核心考点 :

- “**RMSNorm** 为什么比 **LayerNorm** 快?” → 少一次 reduction (不算 mean) 少一个可学习参数 β 。实测 forward 快 10%, 训练稳定性差不多 (LLaMA 论文验证过)。
- “为什么 **LLaMA** 用 **RMSNorm**?” → 训练稳定性 + 推理速度。
- “**rsqrtf** 和 **1/sqrtf** 有区别吗?” → **rsqrtf** 是硬件 SFU 指令, 1 cycle; **1.f/sqrtf** 会被 nvcc 融合成 **rsqrtf** (-use_fast_math 下), 但显式写更清楚。

10. rotary position embedding (RoPE)

问题: 给 query/key $Q, K \in \mathbb{R}^{N \times d}$, 把每 pair 相邻通道 (Q_{2j}, Q_{2j+1}) 视为复数, 旋转角度 $\theta_{i,j} = i \cdot 10000^{-\frac{j}{d}}$ 。

$$Q'_{i,2j} = Q_{i,2j} \cos \theta_{i,j} - Q_{i,2j+1} \sin \theta_{i,j}$$

$$Q'_{i,2j+1} = Q_{i,2j} \sin \theta_{i,j} + Q_{i,2j+1} \cos \theta_{i,j}$$

Ladder :

1. **Naive 独立 kernel** : 读 Q, 计算 cos/sin, 写 Q'.
2. **Fused into attention** : RoPE 通常不单独做——直接在 attention kernel 里, load Q/K 时应用 rotation, 省一次 read/write。Flash-Attention 3 就这么做。

3. **cos/sin 预计算 vs on-the-fly**: cos/sin 表大小 $N \times d$, 预计算需要 cache; on-the-fly `__sincosf` 单周期 4 cycle, 通常 fuse 更好。

参考代码 (on-the-fly RoPE, 处理 Q , pair-wise 旋转):

```
// Q shape [N, d], 每 pair (2j, 2j+1) 视为复数、旋转角 theta = pos * base^(-2j/d).
__global__ void rope_kernel(float* Q, int N, int d, float base) {
    int pos = blockIdx.x;           // token 位置 i
    int tid = threadIdx.x;         // 每 thread 处理一对 (2j, 2j+1)
    if (tid * 2 >= d) return;

    int j = tid;
    float* q = Q + pos * d;

    // 频率: base^(-2j/d) = exp(-2j/d * log(base))
    float freq = expf(-(2.f * j / (float)d) * logf(base));
    float angle = pos * freq;
    float c, s;
    __sincosf(angle, &s, &c); // 硬件 SFU 一次算 sin+cos

    float q0 = q[2 * j];
    float q1 = q[2 * j + 1];
    q[2 * j] = q0 * c - q1 * s;
    q[2 * j + 1] = q0 * s + q1 * c;
}
// launch: rope_kernel<<<N, d/2>>>(...) — 每 token 一个 block, 每 pair 一个 thread.
// 生产版会 fuse 到 attention 的 Q/K load 中, 永不 materialize 中间 Q'.
```

核心考点:

- “**RoPE** 为什么不用绝对位置?” $\rightarrow$ 相对位置 $= \theta_{i,j} - \theta_{k,j}$, attention $q \cdot k$ 自动携带相对位置。
- “**long context extrapolation** 怎么调?” $\rightarrow$ NTK-aware scaling / YaRN, 本质是调 base (默认 10000) 或对高频通道特殊处理。
- “**RoPE** 是 **memory-bound** 还是 **compute-bound**?” $\rightarrow$ memory-bound (每元素几 flop), 不 fuse 单独 kernel 是浪费。

11. paged attention (vLLM)

问题: 在 LLM inference 时, KV cache 巨大 (每 token 数 KB), batch 里多请求的 cache 长度不同 $\rightarrow$ 传统 contiguous KV 浪费严重。paged attention 把 KV cache 分块, 每 block 16-32 token, 用 **block table** 索引到物理页——像 OS 的虚拟内存。

Ladder:

1. **Naive**: contiguous KV, padding 到 max_len $\rightarrow$ GPU 内存爆炸。
2. **Continuous batching**: 请求动态加入/退出, 重排 KV, 内存碎片严重。
3. **Paged**: block-based 分配, block table 描述 logical seq $\rightarrow$ physical block。

参考代码 (paged attention 的 K/V load, 简化到 single-head 单 request):

```
// KV cache 布局: [num_blocks, block_size, head_dim]
// block_table[seq, logical_block_idx] = physical_block_idx
constexpr int BLOCK = 16; // 每 physical block 16 个 token
```

```

__global__ void paged_attn_kernel(
    const float* Q,                // [head_dim]
    const float* KV_cache,         // [num_blocks, BLOCK, head_dim] K 和 V 分别一份
    const int* block_table,        // [max_logical_blocks]
    int seq_len, int head_dim,
    float* out) {
    int t = blockIdx.x * BLOCK + threadIdx.y; // logical token id
    if (t >= seq_len) return;

    // Step 1: 通过 block table 查物理位置—这是 paged attention 的核心。
    int logical_block = t / BLOCK;
    int within_block = t % BLOCK;
    int phys_block = block_table[logical_block];
    const float* K_ptr = KV_cache + (phys_block * BLOCK + within_block) * head_dim;

    // Step 2: block 内每个 token 由 blockDim.x 个 lane 协作算 dot(Q, K_t).
    // 因为 within_block 连续、head_dim 是最内维 → warp 内 lane 沿 head_dim 走 coalesced.
    float partial = 0.f;
    for (int d = threadIdx.x; d < head_dim; d += blockDim.x) {
        partial += Q[d] * K_ptr[d];
    }
    partial = warp_reduce_sum(partial);
    if (threadIdx.x == 0) out[t] = partial; // 后续再做 softmax * V
}

```

核心考点：

- “**paged attention kernel** 里怎么读 K/V?” → 每 token 通过 block_table 间接查找。global memory access 变成 **gather**：block index → physical KV pointer → load.
- “**gather** 会破坏 **coalesced**?” → 会。paged attention 通过让 block size = 16 tokens、每 block 内 contiguous，保证一个 block 内的读是 coalesced；跨 block 才是 gather。
- “**share prefix caching**?” → 多个请求共享 system prompt 前缀，block table 里共享同一批 physical block。

12. quantized matmul (INT8 / INT4)

问题： $A \in \text{int8}^{M \times K}$, $B \in \text{int8}^{K \times N}$, $s_A \in \mathbb{R}^M$, $s_B \in \mathbb{R}^N$ 是 per-row / per-col scale。计算 $C_{ij} = s_{A[i]} \cdot s_{B[j]} \cdot \sum_k A[i, k] \cdot B[k, j]$ 。

为什么难：

- 数据类型混合：INT8 mul → INT32 accum → FP16 rescale → FP16 output。
- Tensor Core 有 INT8 mode (A100: 624 TFLOPS, $2 \times \text{FP16}$)
- Scale broadcast 和 dequant 可能出现 bank conflict / lane divergence。

Ladder：

1. **Naive INT8 mul + FP32 accum**：手写 FMA，慢。
2. **Tensor Core INT8**：用 `wmma::mma_sync` 的 `int8_t` 模式，或 CUTLASS `Gemm<int8_t, int8_t, int32_t>`。
3. **Fused dequant epilogue**：INT32 accum 后立即乘 scale 转 FP16，避免存 INT32 中间结果。

4. **INT4 with LUT (GPTQ, AWQ)** : weight INT4 pack 到 INT8/32 , 运行时 dequant via lookup.

参考代码 (Tensor Core INT8 GEMM 的核心指令 + fused dequant epilogue):

```
#include <mma.h>
using namespace nvcuda::wmma;

// 16x16x16 INT8 → INT32 accum tile.
// A, B are int8_t global memory; scale_A[M], scale_B[N] fp32.
__global__ void int8_gemm_wmma(
    const int8_t* A, const int8_t* B, __half* C,
    const float* scale_A, const float* scale_B,
    int M, int N, int K) {
    int warp_m = blockIdx.y * 16;
    int warp_n = blockIdx.x * 16;

    fragment<matrix_a, 16, 16, 16, int8_t, row_major> a_frag;
    fragment<matrix_b, 16, 16, 16, int8_t, col_major> b_frag;
    fragment<accumulator, 16, 16, 16, int> c_frag;
    fill_fragment(c_frag, 0);

    // K 维度 tile 循环
    for (int k = 0; k < K; k += 16) {
        load_matrix_sync(a_frag, A + warp_m * K + k, K);
        load_matrix_sync(b_frag, B + k * N + warp_n, N);
        mma_sync(c_frag, a_frag, b_frag, c_frag); // INT8 × INT8, INT32 accum
    }

    // Fused dequant epilogue: INT32 accum → FP16 output, 乘 per-row/col scale.
    int lane = threadIdx.x & 31;
    #pragma unroll
    for (int e = 0; e < c_frag.num_elements; ++e) {
        int i = warp_m + (lane / 4) + (e / 4) * 8; // wmma 元素布局 (简化)
        int j = warp_n + (lane % 4) * 2 + (e % 2);
        float f = c_frag.x[e] * scale_A[i] * scale_B[j];
        // 立即转 FP16, 避免存 INT32 中间
        // ...store_matrix_sync 或手动写...
    }
}
```

核心考点：

- “**per-tensor / per-channel / per-token scale** 有什么区别？” → per-tensor 一个 scalar , 最粗 ; per-channel (B 的每列一个 scale) 是 GEMM 标准 ; per-token (A 的每行一个 scale) 适合 activation quant.
- “**INT8 GEMM 的 accum** 为什么用 **INT32** 而不 **FP32** ?” → $\text{INT8} \times \text{INT8} = \text{INT16}$, K 累加 → $\text{INT16} \times 1024$ 会溢出 , 需要 INT32。硬件直接支持 INT8 dot4 INT32 accum。
- “**outlier channel** 怎么办？” → SmoothQuant / GPTQ 把 outlier 挪到 weight 或用高精度 subchannel。

13. layer fusion (Linear + GELU + Dropout + Add)

问题：Transformer FFN 是 $y = \text{Add}(x, \text{Dropout}(\text{Linear2}(\text{GELU}(\text{Linear1}(x))))))$ 。分离 5 个 kernel 时，中间结果反复读写 $\rightarrow$ memory bound。

Ladder :

1. **Split** : 5 个 kernel , 读写 $x, h1, h2, h3, h4, y$ 6 份数据。
2. **Epilogue fusion (Linear1 + GELU)** : GEMM 结束时在 smem 上直接 GELU , 输出到 h2 位置。cuBLASLt / CUTLASS 支持。
3. **Fully fused** (除 Linear1 的 GEMM 外): Linear2 之后 + Dropout + Add + LayerNorm 全部 fuse 到一 kernel。Flash-Attention 系列的做法。
4. **Auto tuning** (CUTLASS / Triton): 不同 shape 选不同 tile / warp partition。

参考代码 (Linear2 后的 Add + Dropout + LayerNorm 融合成 1 个 kernel):

```
// h = Linear2(...) 已经写到 h; 这里 fuse: y = LayerNorm(Dropout(h) + x_residual)
// 每 block 处理一 row (H 维).
__global__ void fused_dropout_add_ln(
    const float* h, const float* x_residual, const float* gamma, const float* beta,
    float* y, uint64_t rng_seed, uint64_t rng_offset,
    float dropout_p, float scale, // scale = 1/(1-p)
    int H, float eps) {
    int row = blockIdx.x;
    const float* hr = h + row * H;
    const float* xr = x_residual + row * H;
    float* yr = y + row * H;

    // Pass 1: dropout + add + accumulate (mean, mean_of_sq) via online algorithm.
    // 用 Philox 从 (seed, offset + row*H + j) 拿到确定性随机数——不用存 mask.
    float mean = 0.f, m2 = 0.f;
    int n = 0;
    for (int j = threadIdx.x; j < H; j += blockDim.x) {
        float rand01 = philox_uniform(rng_seed, rng_offset + row * H + j);
        float mask = (rand01 >= dropout_p) ? scale : 0.f;
        float v = hr[j] * mask + xr[j];
        // Welford online update
        ++n;
        float delta = v - mean;
        mean += delta / n;
        m2 += delta * (v - mean);
    }
    // block reduce (mean, m2, n) 合并——Welford combine 公式.
    block_reduce_welford(mean, m2, n);
    float rstd = rsqrtf(m2 / H + eps);

    // Pass 2: 再算一次 dropout+add (重复 Philox, 同 seed+offset 拿同 random),
    // 归一化 + affine 写出。省 O(H) 中间存储.
    for (int j = threadIdx.x; j < H; j += blockDim.x) {
        float rand01 = philox_uniform(rng_seed, rng_offset + row * H + j);
        float mask = (rand01 >= dropout_p) ? scale : 0.f;
        float v = hr[j] * mask + xr[j];
        yr[j] = (v - mean) * rstd * gamma[j] + beta[j];
    }
}
```

```

}
}

```

核心考点：

- “**epilogue fusion** 的边界在哪？” → 只能 fuse **element-wise** + **broadcast**；不能 fuse 需要 reduce 的（比如 softmax）。
- “**Dropout** 训练时的随机数生成？” → Philox counter-based PRNG，每 element 独立 stateless，可 fuse——backward 用同一 (seed, offset) 重放，不用存 mask。
- “**fuse** 太多有什么坏处？” → 寄存器压力上升 → occupancy 下降；kernel 变复杂难调优；一个坏 kernel 拖后腿。

14. group / channel-wise conv (im2col vs implicit)

问题：给 input NCHW、weight KCRS，计算 conv output。

Ladder：

1. **Direct conv**：naive 7 层循环。
2. **im2col + GEMM**：input 展开成 matrix，转 matmul。 $M = N \times H_{out} \times W_{out}$ ， $K = C \times R \times S$ ， $N = K_{out}$ 。缺点：im2col buffer 大（约 $R \times S$ 倍 input）。
3. **Implicit GEMM**：不显式 im2col，在 GEMM kernel 里用 index 计算读原 input 的位置。cuDNN 默认。
4. **Winograd** (3×3 kernel)：Winograd F(2,3) 把 4-output-per-block 从 36 mul 减到 16 mul，加法多但 mul 少。
5. **FFT conv** (大 kernel)：kernel size > 7 时 FFT 更快。

参考代码 (implicit GEMM conv，NCHW，简化：stride=1、padding=1、3×3 kernel)：

```

// input:  [N, C, H, W]
// weight: [K, C, R, S]  (R=S=3)
// output: [N, K, H, W]
// implicit GEMM: 把 output[n, k, oh, ow] 视为 GEMM 的 M×N 上一点，
//   M = N * H * W, N = K, GEMM K = C * R * S.
__global__ void implicit_gemm_conv3x3(
    const float* input, const float* weight, float* output,
    int N, int C, int H, int W, int K) {
    int n_ohow = blockIdx.y * 16 + threadIdx.y; // GEMM 的 M 维
    int k_out = blockIdx.x * 16 + threadIdx.x; // GEMM 的 N 维
    int n = n_ohow / (H * W);
    int oh = (n_ohow / W) % H;
    int ow = n_ohow % W;
    if (n >= N || k_out >= K) return;

    float acc = 0.f;
    for (int c = 0; c < C; ++c) {
        #pragma unroll
        for (int r = 0; r < 3; ++r) {
            int ih = oh + r - 1; // padding=1
            if (ih < 0 || ih >= H) continue;
            #pragma unroll
            for (int s = 0; s < 3; ++s) {

```



```

    int iw = ow + s - 1;
    if (iw < 0 || iw >= W) continue;
    float x = input[((n * C + c) * H + ih) * W + iw];
    float w = weight[((k_out * C + c) * 3 + r) * 3 + s];
    acc += x * w;
}
}
}
output[((n * K + k_out) * H + oh) * W + ow] = acc;
}
// 生产版会用 cuDNN / CUTLASS, 把 (r, s, c) 循环转成 tensor-core mma.sync。

```

核心考点：

- “**Winograd** 为什么只对 3×3 好？” → transform 引入额外加法，kernel 大时不划算。
- “**Depthwise conv** 为什么慢？” → 每 channel 独立，无跨 channel 的 $K \rightarrow \text{matmul}$ 变短胖，tensor core 利用率低。
- “**group conv** 怎么优化？” → 每 group 独立 GEMM，batch-GEMM 或 grouped GEMM (CUTLASS 3.x)。

15. all-reduce (multi-GPU)

问题：多 GPU 上每 GPU 有 partial gradient g_i ，计算 $\sum_i g_i$ 广播回每 GPU。

为什么难：

- 跨节点 NVLink / IB 带宽有限。
- naive scatter-gather 有 $O(P^2)$ 通信。

Ladder：

1. **Naive**：GPU 0 收所有 partial，reduce，再 broadcast → $O(P)$ 通信量、 $O(P)$ 时延。
2. **Ring all-reduce**： P 个 GPU 连成环， $2(P-1)$ step，每 step 传 $\frac{\text{size}}{P} \rightarrow O(\text{size})$ 通信量、 $O(P)$ 时延。NCCL 默认。
3. **Tree all-reduce**： $O(\text{size})$ 通信量、 $O(\log P)$ 时延。适合大 P 少 size。
4. **Double binary tree / SHARP** (InfiniBand hardware reduce)：hardware-accelerated in-network reduction。

参考代码 (ring all-reduce 骨架——面试白板画出 chunk 流转即可，NCCL 会替你实现)：

```

// 每 GPU 上有 buffer[size], rank 是 0..P-1, 环上 next = (rank+1)%P.
// 分成 P 个 chunk, chunk[i] 大小 = size / P.
//
// Phase 1: reduce-scatter, P-1 步.
//   step k: rank r 把 chunk[(r - k) mod P] 发给 next;
//           收到时加到本地 chunk[(r - k - 1) mod P] 上.
//   结束时: rank r 持有 fully-reduced 的 chunk[(r + 1) mod P].
//
// Phase 2: all-gather, P-1 步.
//   step k: rank r 把已经 reduced 的 chunk 发给 next; 收到就直接覆盖.
//   结束时: 每 rank 持有所有完整 chunk.
//
// 总通信量 per GPU: 2 * (P-1)/P * size ≈ 2 * size, bandwidth-optimal.

```

```
// 步数:  $2(P-1)$ , latency 随  $P$  线性增长—大  $P$  时改用 tree.
```

```
// 实际调用 (NCCL):
```

```
ncclAllReduce(sendbuf, recvbuf, count, ncclFloat, ncclSum, comm, stream);
```

核心考点：

- “**ring** 和 **tree** 什么时候用？” → NCCL 会根据 size 和 P 自动选。size 大时 ring (bandwidth optimal), size 小时 tree (latency optimal)。
- “**gradient overlap with backward**？” → PyTorch DDP 在 backward 里每算完一层立即 all-reduce 那层的 gradient, 通信和计算重叠。
- “低比特 **all-reduce**？” → FP16 grad 直接 all-reduce; BF16 mixed precision; 1-bit / signSGD / PowerSGD 通信压缩。

常见 **combo** 问题

面试官经常把上面的题组合起来问：

- “**attention forward+backward** 一起写” → 主体是本书 ch 7-10 + 上面 Q6 (layernorm bw 的思路搬到 softmax + gemm bw)。
- “实现 **fused RMSNorm + Linear + SwiGLU (LLaMA FFN)**” → Q9 + Q13 + Q8。
- “**KV cache** 的量化和 **paged storage**” → Q11 + Q12。
- “给我写一个训练循环 **forward** 的 **kernel timeline**” → 需要能画出：Embedding → LayerNorm → Attention → LayerNorm → FFN → LayerNorm, 每一步说 fuse 边界、哪些是 memory-bound、哪些是 compute-bound。

Key insight. 面试通用心法：任何优化的第一步都是“这是什么 **bound**”。看 AI (arithmetic intensity), 对比硬件 ridge point：

- $AI < 1$ FLOP/B：memory-bound, 优化目标是压满 HBM。
- $AI > 100$ FLOP/B：compute-bound, 优化目标是压满 Tensor Core。
- 中间地带：设计能 overlap (async copy) tile 让 AI 提升到 compute-bound 侧。

面试官问“怎么优化 X”时你先说“先看 X 是 memory-bound 还是 compute-bound”——这一句话就把你和“直接开始 tile”的候选人分开了。

Warning. 这份 list 不是“背下来就够”。真正能 pass 的候选人：

- 能当场画出一道题的 kernel skeleton (block / thread / smem 分配)。
- 能算出 AI 和 roofline, 判断上界。
- 能说出哪些 ncu metric 会验证 / 反驳自己的猜测。

这三点全书正文都在训练——所以先把正文里 8 个 kernel 每个都手动 code + ncu 一遍, 再回来看这份 list。